



Compassse

Multi-Tenant School Management Platform

Complete System & API Documentation

Backend API **v1**

Laravel 12 · PHP 8.2 · stanc/tenancy

Generated 2026-09-06

Table of Contents

System Overview

- What is Compasse?
- Architecture
- User Roles
- Subscription Plans & Module Gating
- Request Flow
- Public (No-Auth) Endpoints
- Authentication Endpoints
- Key Directories
- Environment Variables
- Frontend Integration — Tenancy Bootstrapping
- Common Error Responses

Super Admin

- Who is the Super Admin?
- User Story
- Authentication
- Platform Dashboard
- Tenant Management
- School Management
- Access to School Data — Implementation Pattern
- What Super Admin Cannot Do
- Security Notes
- Frontend Integration — Super Admin Portal

Subscription Management

- Overview
- User Stories
- Plans
- Subscription Lifecycle
- API Endpoints
- Full Request / Response Examples
- Module Access Check (HTTP 403 Response)
- Caching Details
- Frontend Integration — Subscription & Module Gating

School Fees & Payments

- Overview
- User Stories
- Fee Model
- Payment Model
- API Endpoints
- Full Request / Response Examples — Fees
- Full Request / Response Examples — Payments
- Full Request / Response Examples — Expenses
- Business Rules

Payroll & Pay Stubs

- Overview
- User Stories
- Payroll Model
- API Endpoints
- Full Request / Response Examples
- Business Rules
- Typical Monthly Payroll Workflow
- Frontend Integration — Payroll Listing and Pay Stub Modal
- Bulk Payroll Creation (Batch Flow)

Inventory Management

- Overview
- User Stories
- Models
- API Endpoints
- Request / Response Examples
- Business Rules
- Frontend Integration

Transport Management

- Overview
- User Stories
- Models
- API Endpoints
- Request / Response Examples
- Business Rules
- Frontend Integration

Hostel Management

- Overview
- User Stories
- Models
- API Endpoints
- Request / Response Examples
- Business Rules
- Frontend Integration

Health Management

- Overview
- User Stories
- Models
- API Endpoints
- Request / Response Examples
- Business Rules
- Frontend Integration

Academic Management

- Overview
- User Stories
- Data Hierarchy
- API Endpoints
- Request / Response Examples
- Student Management
- Guardian Management
- Teacher Management
- School Structure Endpoints
- Business Rules
- Frontend Integration

Assessment, CBT & Results

- Overview
- User Stories
- Exam Workflow
- API Endpoints
- Complete Request / Response Examples
- Analytics
- Question Bank
- Frontend Integration
- Business Rules

Communication — Messaging, Notifications, SMS & Email

- Overview
- User Stories
- Internal Messages
- Notifications
- SMS
- Email
- Guardian Communication
- Frontend Integration

Attendance Management

- Overview
- User Stories
- Attendance Model
- API Endpoints
- Complete Request / Response Examples
- Business Rules
- Frontend Integration

Events & School Calendar

- Overview
- User Stories
- Models
- API Endpoints
- Complete Request / Response Examples
- Business Rules

Frontend Integration

API Reference

Base URL
Authentication
HTTP Status Codes
Pagination
Full Route Map
Common Request Headers
Validation Errors
Module Denied
Frontend Integration

Frontend Integration Guide — Multi-Tenant Architecture

How Multi-Tenancy Works
Base URL Strategy
Required Headers (all authenticated requests)
Authentication Flow
Super Admin Authentication
Module-Based UI Gating
Pagination
File Uploads
Error Handling Reference
React Query / SWR Integration
Real-Time Notifications (WebSocket / Polling)
Multi-School Support (within one tenant)
Landing Page (Public)
Environment Variables (Frontend)
React Native / Mobile
TypeScript Types Reference

School Customization — Landing Page & Branding

Overview
User Stories
Templates
API Endpoints
Request & Response Examples
Configurable Settings Reference
Business Rules
Frontend Integration

Admin Dashboard API Documentation

Table of Contents
Authentication
Dashboard Overview
School Management
User Management
Profile Picture Management
Academic Setup

Student Management
Teacher Management
Staff & Guardian Management
Settings
Subscriptions & Modules
Summary

Principal Dashboard API Documentation

Authentication
Dashboard Overview
School Management
Approval Management
Academic Oversight
Disciplinary Management
Communication
Summary

HOD (Head of Department) Dashboard API Documentation

Authentication
Dashboard Overview
Department Management
Teacher Management
Subject Management
Summary

Teacher Dashboard API Documentation

Table of Contents
Authentication
Dashboard Overview
My Classes & Students
Exams & CBT
Question Bank
Grading & Results
Assignments
Attendance
Timetable
Communication
Profile Management
Summary

Student Dashboard API Documentation

Table of Contents
Authentication
Dashboard Overview
My Profile
My Classes & Subjects
Assignments
Exams & CBT
My Grades & Results

- My Attendance
- My Timetable
- My Guardians
- Communication
- Profile Management
- Summary

Parent/Guardian Dashboard API Documentation

- Table of Contents
- Authentication
- Dashboard Overview
- My Children
- Child Performance
- Attendance Monitoring
- Assignments & Homework
- Exam Results
- Fee Management
- Communication
- Profile Management
- Summary

Finance/Accountant Dashboard API Documentation

- Table of Contents
- Authentication
- Dashboard Overview
- Fee Management
- Payment Processing
- Expense Management
- Payroll Management
- Financial Reports
- Budget Management
- Invoice Generation
- Summary

Librarian Dashboard API Documentation

- Authentication
- Dashboard Overview
- Book Management
- Member Management
- Reports
- Summary

Nurse/Medical Dashboard API Documentation

- Authentication
- Dashboard Overview
- Medical Records
- Medication Management
- Vaccination Management
- Medical Supplies

[Summary](#)

Driver Dashboard API Documentation

[Authentication](#)

[Dashboard Overview](#)

[My Route Management](#)

[Trip Management](#)

[Vehicle Management](#)

[Summary](#)

Security Dashboard API Documentation

[Authentication](#)

[Dashboard Overview](#)

[Visitor Management](#)

[Gate Pass Management](#)

[Incident Management](#)

[Vehicle Management](#)

[Patrol Management](#)

[Lost and Found](#)

[Summary](#)



PART I

Platform Guide

Architecture, every functional module, workflows, and the full REST API surface.

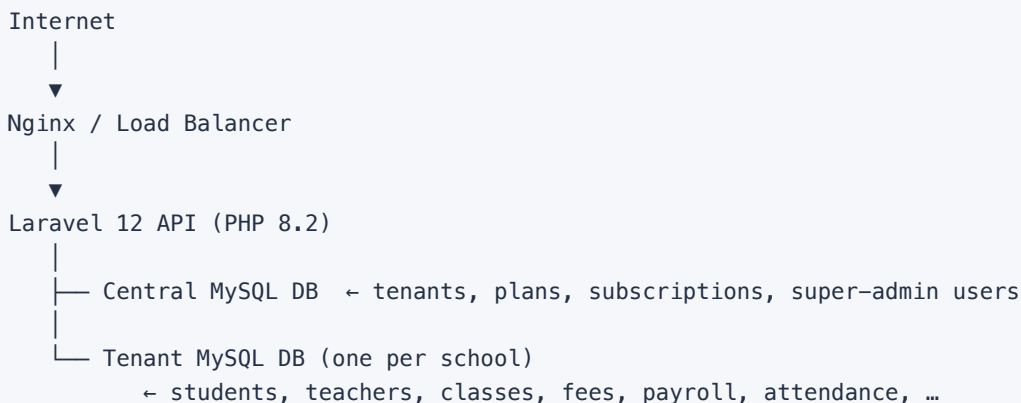
System Overview

Base URL: `https://{subdomain}.compasse.net/api/v1/` **Auth:** `Authorization: Bearer {token}` required on all protected endpoints **Module gate:** None — system-level endpoints

What is Compasse?

Compasse is a **multi-tenant, multi-database SaaS school management platform**. Each school (tenant) runs in a completely isolated MySQL database. The central database tracks tenants, plans, and subscriptions. All school data — students, staff, fees, exams, attendance — lives in the tenant's own database.

Architecture



Multi-Tenancy (stanc/tenancy v3)

- Tenant is identified by **subdomain** (e.g. `greenfield.compasse.net`)
- `TenantMiddleware` resolves the subdomain → looks up tenant record → switches the active DB connection to the tenant's database
- All tenant-scoped routes require this middleware
- Sanctum tokens are stored **per-tenant** — a token from one school cannot authenticate at another

Authentication

- **Laravel Sanctum** token-based auth
- Token is created on login and returned to the client
- All protected routes require `Authorization: Bearer {token}` header
- Throttle: 5 login attempts per minute per IP

User Roles

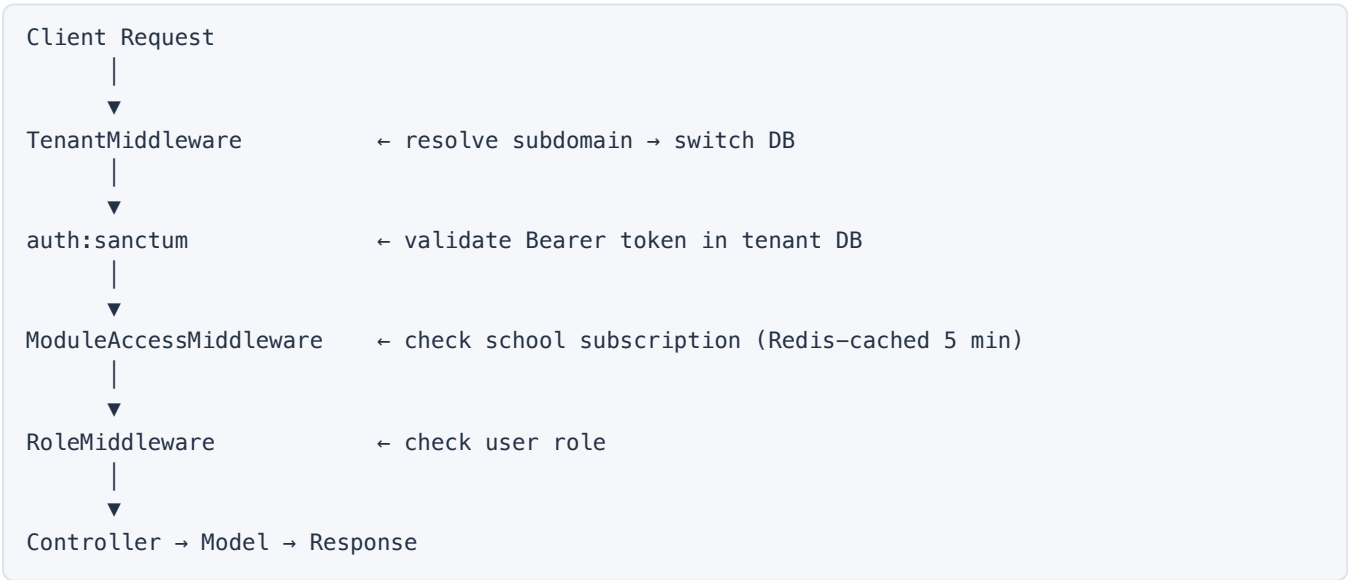
Role	Scope	Description
super_admin	Central DB	Platform owner. Full access to all tenants and schools
admin	Tenant DB	School administrator. Full access to their school
principal	Tenant DB	Academic head. Read/write on academic operations
vice_principal	Tenant DB	Deputy academic head
teacher	Tenant DB	Manages classes, grades, assignments
student	Tenant DB	Read-only access to own data
guardian	Tenant DB	Read-only access to ward's data
accountant	Tenant DB	Full access to financial module
librarian	Tenant DB	Access to library module
nurse	Tenant DB	Access to health module
driver	Tenant DB	Access to transport module
staff	Tenant DB	General non-teaching staff
security	Tenant DB	Access to secure pickup verification

Subscription Plans & Module Gating

Every school must have an active subscription to use module-gated features. The subscription defines which **modules** are enabled. Routes protected by `ModuleAccessMiddleware` return HTTP 403 if the school's plan does not include that module.

Module Slug	Feature
academic_management	Classes, subjects, academic years, terms
student_management	Student enrollment, profiles
teacher_management	Teacher profiles, assignments
cbt	Computer-based testing, exams, results, report cards
fee_management	School fees, payments, receipts, payroll
attendance_management	Student and staff attendance
transport_management	Vehicles, drivers, routes, secure pickup
hostel_management	Rooms, allocations, maintenance
health_management	Health records, appointments, medications
inventory_management	Categories, items, checkout/return
event_management	Events, school calendar
livestream	Live classes
sms_integration	SMS notifications
email_integration	Email notifications

Request Flow



Middleware Chain — Code Reference

TenantMiddleware (`app/Http/Middleware/TenantMiddleware.php`):

```
// Resolves subdomain from header, URL, or request host
$subdomain = $request->header('X-Subdomain')
?? $request->route('subdomain')
?? explode('.', $request->getHost())[0];

$tenant = Tenant::where('id', $subdomain)->firstOrFail();
tenancy()->initialize($tenant);
```

ModuleAccessMiddleware (app/Http/Middleware/ModuleAccessMiddleware.php):

```
// Cache key: sub:school:{id}:module:{slug}
$hasAccess = Cache::tags(["school:{$schoolId}:subscription"])
->remember("sub:school:{$schoolId}:module:{$module}", 300, fn() =>
    SubscriptionService::schoolHasModule($schoolId, $module)
);

if (!$hasAccess) {
    return response()->json([
        'error' => 'Module access denied',
        'message' => "This school does not have access to the {$module} module. Please
upgrade your subscription.",
        'module' => $module,
        'upgrade_required' => true,
    ], 403);
}
```

RoleMiddleware (app/Http/Middleware/RoleMiddleware.php):

```
// Usage in routes: ->middleware('role:admin,accountant')
$allowedRoles = explode(',', $role);
if (!in_array(auth()->user()->role, $allowedRoles)) {
    return response()->json(['error' => 'Unauthorized role'], 403);
}
```

Public (No-Auth) Endpoints

These endpoints require no token. They are used to bootstrap the frontend before a user logs in.

Health Check

```
GET /api/v1/health
```

Request:

```
GET /api/v1/health HTTP/1.1
Host: compasse.net
Accept: application/json
```

Response 200 OK :

```
{
  "status": "ok",
  "timestamp": "2026-03-30T10:15:00Z",
  "version": "1.0.0"
}
```

Database Health (Super Admin Only)

```
GET /api/v1/health/db
Authorization: Bearer {superadmin_token}
```

Response 200 OK :

```
{
  "status": "ok",
  "central_db": "connected",
  "queue": "running",
  "cache": "connected",
  "timestamp": "2026-03-30T10:15:02Z"
}
```

Response 500 Internal Server Error (degraded):

```
{
  "status": "degraded",
  "central_db": "connected",
  "queue": "not_running",
  "cache": "disconnected",
  "timestamp": "2026-03-30T10:15:02Z"
}
```

Subdomain / Tenant Lookup

Used by the SPA to verify that a school exists before showing the login form.

```
GET /api/v1/schools/by-subdomain/{subdomain}
```

Request:

```
GET /api/v1/schools/by-subdomain/greenfield HTTP/1.1
Host: compasse.net
Accept: application/json
```

Response 200 OK :

```
{
  "exists": true,
  "tenant": {
    "id": "greenfield",
    "name": "Greenfield Academy",
    "subdomain": "greenfield",
    "status": "active",
    "logo_url": "https://cdn.compassse.net/schools/greenfield/logo.png",
    "primary_color": "#2E7D32",
    "address": "14 Victoria Island, Lagos",
    "phone": "+2348012345678",
    "email": "info@greenfieldacademy.edu.ng"
  }
}
```

Response 404 Not Found :

```
{
  "exists": false,
  "message": "School not found"
}
```

Response 403 Forbidden (suspended tenant):

```
{
  "exists": true,
  "status": "suspended",
  "message": "This school account has been suspended. Please contact support."
}
```

Public School Info

Returns publicly visible school information (used for landing pages / login pages).

```
GET /api/v1/public/{subdomain}
```

Request:

```
GET /api/v1/public/greenfield HTTP/1.1
Host: compassse.net
Accept: application/json
```

Response 200 OK :

```
{
  "school": {
    "name": "Greenfield Academy",
    "subdomain": "greenfield",
    "logo_url": "https://cdn.compassse.net/schools/greenfield/logo.png",
    "cover_image_url": "https://cdn.compassse.net/schools/greenfield/cover.jpg",
    "primary_color": "#2E7D32",
    "tagline": "Nurturing Excellence",
    "address": "14 Victoria Island, Lagos",
    "phone": "+2348012345678",
    "email": "info@greenfieldacademy.edu.ng",
    "website": "https://greenfieldacademy.edu.ng"
  }
}
```

Tenant Verify (POST)

Alternative subdomain verification accepting JSON body.

```
POST /api/v1/tenants/verify
Content-Type: application/json
```

Request Body:

```
{
  "subdomain": "greenfield"
}
```

Response 200 OK :

```
{
  "valid": true,
  "tenant_id": "greenfield",
  "school_name": "Greenfield Academy",
  "status": "active"
}
```

Response 422 Unprocessable Entity :

```
{
  "valid": false,
  "message": "The subdomain field is required."
}
```

Authentication Endpoints

Login (Tenant User)

POST /api/v1/auth/login

Request:

```
POST /api/v1/auth/login HTTP/1.1
Host: greenfield.compassse.net
Content-Type: application/json
Accept: application/json
X-Subdomain: greenfield
```

Request Body:

```
{
  "email": "admin@greenfieldacademy.edu.ng",
  "password": "SecureP@ss123"
}
```

Response 200 OK :

```
{
  "token": "3|abc123xyz456tokenvalue",
  "token_type": "Bearer",
  "user": {
    "id": 1,
    "name": "Mrs. Adaobi Nwosu",
    "email": "admin@greenfieldacademy.edu.ng",
    "role": "admin",
    "school_id": 1,
    "avatar_url": null,
    "created_at": "2025-09-01T08:00:00Z"
  },
  "school": {
    "id": 1,
    "name": "Greenfield Academy",
    "subdomain": "greenfield",
    "logo_url": "https://cdn.compassse.net/schools/greenfield/logo.png"
  }
}
```

Response 401 Unauthorized :

```
{
  "message": "Invalid credentials"
}
```

Response 429 Too Many Requests (throttle):

```
{
  "message": "Too many login attempts. Please try again in 60 seconds."
}
```

Logout

```
POST /api/v1/auth/logout
Authorization: Bearer {token}
X-Subdomain: greenfield
```

Response 200 OK :

```
{
  "message": "Logged out successfully"
}
```

Get Authenticated User

```
GET /api/v1/auth/me
Authorization: Bearer {token}
X-Subdomain: greenfield
```

Response 200 OK :

```
{
  "user": {
    "id": 1,
    "name": "Mrs. Adaobi Nwosu",
    "email": "admin@greenfieldacademy.edu.ng",
    "role": "admin",
    "school_id": 1,
    "avatar_url": null
  }
}
```

Key Directories

```
app/  
  Http/  
    Controllers/      ← All API controllers  
    Middleware/       ← TenantMiddleware, ModuleAccessMiddleware, RoleMiddleware  
  Models/            ← Eloquent models  
  Services/          ← TenantService, SubscriptionService  
  Jobs/              ← SendEmailJob, SendSMSJob (queued)  
  
database/  
  migrations/        ← Central DB migrations  
  migrations/tenant/ ← Per-tenant DB migrations (run on tenant creation)  
  
routes/  
  api.php            ← All API routes  
  
config/  
  tenancy.php        ← stancl/tenancy configuration  
  queue.php          ← Queue driver config (Redis recommended)
```

Environment Variables

Variable	Example Value	Description
APP_NAME	Compassse	Application name
APP_ENV	production	Environment (local , production)
APP_KEY	base64:...	Laravel encryption key
APP_URL	https://compasse.net	Central app URL
APP_DEBUG	false	Debug mode (always false in prod)
DB_CONNECTION	mysql	Central DB driver
DB_HOST	127.0.0.1	Central DB host
DB_PORT	3306	Central DB port
DB_DATABASE	compasse_central	Central DB name
DB_USERNAME	compasse_user	Central DB username
DB_PASSWORD	secret	Central DB password
TENANCY_DB_PREFIX	tenant_	Tenant DB name prefix (e.g. tenant_greenfield)
QUEUE_CONNECTION	redis	Queue driver (redis recommended in prod)
CACHE_STORE	redis	Cache driver (required for module-access caching)
REDIS_HOST	127.0.0.1	Redis host
REDIS_PORT	6379	Redis port
SESSION_DRIVER	redis	Session driver
MAIL_MAILER	smtp	Mail driver
MAIL_HOST	smtp.mailgun.org	SMTP host
MAIL_PORT	587	SMTP port
MAIL_USERNAME	postmaster@...	SMTP username
MAIL_PASSWORD	secret	SMTP password
MAIL_FROM_ADDRESS	noreply@compasse.net	Default sender address
SERVICES_SMS_PROVIDER	termii	SMS provider (twilio , vonage , termii , log)
SERVICES_SMS_SENDER_ID	Compassse	SMS sender ID
SERVICES_SMS_API_KEY	...	SMS provider API key
SANCTUM_STATEFUL_DOMAINS	compasse.net , *.compasse.net	Stateful domains for Sanctum

Frontend Integration — Tenancy Bootstrapping

This section explains how a React/Next.js SPA should handle the multi-tenant context.

Step 1 — Detect the Subdomain

```
// utils/tenancy.ts

export function getSubdomain(): string | null {
  if (typeof window === 'undefined') return null;

  const hostname = window.location.hostname; // e.g. "greenfield.compassse.net"
  const parts = hostname.split('.');

  // "greenfield.compassse.net" → ["greenfield", "compassse", "com"]
  // "compassse.net" → ["compassse", "com"] (no subdomain → central/super-admin)
  if (parts.length >= 3 && parts[0] !== 'www') {
    return parts[0]; // "greenfield"
  }
  return null;
}
```

Step 2 — Verify the School Exists

Before rendering the login form, call the public endpoint:

```
// hooks/useSchoolBootstrap.ts
import { useEffect, useState } from 'react';

export function useSchoolBootstrap(subdomain: string) {
  const [school, setSchool] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    fetch(`https://compassse.net/api/v1/schools/by-subdomain/${subdomain}`)
      .then(res => res.json())
      .then(data => {
        if (data.exists && data.tenant.status === 'active') {
          setSchool(data.tenant);
        } else if (data.status === 'suspended') {
          setError('This school account is suspended.');
```

Step 3 — Login and Store the Token

Tokens are per-tenant. Store them with a subdomain-specific key to support users who access multiple schools.

```
// services/auth.ts

const BASE_URL = (subdomain: string) =>
  `https://compasse.net/api/v1`;

const TOKEN_KEY = (subdomain: string) =>
  `compasse_token_${subdomain}`;

export async function login(subdomain: string, email: string, password: string) {
  const res = await fetch(`${BASE_URL(subdomain)}/auth/login`, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'Accept': 'application/json',
      'X-Subdomain': subdomain,
    },
    body: JSON.stringify({ email, password }),
  });

  if (!res.ok) {
    const err = await res.json();
    throw new Error(err.message || 'Login failed');
  }

  const data = await res.json();

  // Store token scoped to this school's subdomain
  localStorage.setItem(TOKEN_KEY(subdomain), data.token);
  localStorage.setItem(`compasse_user_${subdomain}`, JSON.stringify(data.user));
  localStorage.setItem(`compasse_school_${subdomain}`, JSON.stringify(data.school));

  return data;
}

export function getToken(subdomain: string): string | null {
  return localStorage.getItem(TOKEN_KEY(subdomain));
}

export function logout(subdomain: string) {
  localStorage.removeItem(TOKEN_KEY(subdomain));
  localStorage.removeItem(`compasse_user_${subdomain}`);
  localStorage.removeItem(`compasse_school_${subdomain}`);
}
```

Step 4 — Attach Headers to All API Calls

```
// services/api.ts

export function createApiClient(subdomain: string) {
  const token = localStorage.getItem(`compasse_token_${subdomain}`);

  return {
    get: (path: string) =>
      fetch(`https://compasse.net/api/v1${path}`, {
        headers: {
          'Authorization': `Bearer ${token}`,
          'X-Subdomain': subdomain,
          'Accept': 'application/json',
        },
      }).then(handleResponse),

    post: (path: string, body: object) =>
      fetch(`https://compasse.net/api/v1${path}`, {
        method: 'POST',
        headers: {
          'Authorization': `Bearer ${token}`,
          'X-Subdomain': subdomain,
          'Content-Type': 'application/json',
          'Accept': 'application/json',
        },
        body: JSON.stringify(body),
      }).then(handleResponse),

    put: (path: string, body: object) =>
      fetch(`https://compasse.net/api/v1${path}`, {
        method: 'PUT',
        headers: {
          'Authorization': `Bearer ${token}`,
          'X-Subdomain': subdomain,
          'Content-Type': 'application/json',
          'Accept': 'application/json',
        },
        body: JSON.stringify(body),
      }).then(handleResponse),

    delete: (path: string) =>
      fetch(`https://compasse.net/api/v1${path}`, {
        method: 'DELETE',
        headers: {
          'Authorization': `Bearer ${token}`,
          'X-Subdomain': subdomain,
          'Accept': 'application/json',
        },
      }).then(handleResponse),
  };
}

async function handleResponse(res: Response) {
  if (res.status === 401) {
    // Token expired or invalid – redirect to login
    const subdomain = getSubdomain();
```

```

    if (subdomain) logout(subdomain);
    window.location.href = '/login';
    return;
  }
  if (res.status === 403) {
    const data = await res.json();
    if (data.upgrade_required) {
      // Dispatch a global event to show upgrade modal
      window.dispatchEvent(new CustomEvent('module:upgrade-required', { detail: data }));
    }
    throw data;
  }
  if (!res.ok) throw await res.json();
  return res.json();
}

```

Step 5 — Full Bootstrapping Flow (App Entry Point)

```

// app/[school]/layout.tsx (Next.js App Router example)

export default async function SchoolLayout({
  params,
  children,
}: {
  params: { school: string };
  children: React.ReactNode;
}) {
  // Verify school exists server-side
  const res = await fetch(
    `https://compasse.net/api/v1/schools/by-subdomain/${params.school}`
  );
  const data = await res.json();

  if (!data.exists || data.tenant?.status !== 'active') {
    notFound();
  }

  return (
    <SchoolProvider school={data.tenant}>
      {children}
    </SchoolProvider>
  );
}

```

Token Storage Summary

Key	Value	Purpose
compasse_token_{subdomain}	Bearer token string	Auth token for that school
compasse_user_{subdomain}	JSON user object	Cached logged-in user info
compasse_school_{subdomain}	JSON school object	Cached school branding/info
compasse_modules_{subdomain}	JSON array of module slugs	Cached enabled modules (set on login)
compasse_superadmin_token	Bearer token string	Super admin token (central domain only)

Common Error Responses

HTTP Status	Meaning	When It Occurs
400	Bad Request	Malformed request body
401	Unauthenticated	Missing or invalid/expired token
403	Forbidden	Wrong role, or module not in subscription
404	Not Found	Resource does not exist
409	Conflict	Duplicate record (e.g. same payroll month/year)
422	Validation Error	Request fields failed validation
429	Too Many Requests	Login throttle exceeded
500	Server Error	Unexpected backend error

Validation Error Shape (422):

```
{
  "message": "The given data was invalid.",
  "errors": {
    "email": ["The email field is required."],
    "password": ["The password must be at least 8 characters."]
  }
}
```

Module Access Denied Shape (403):

```
{
  "error": "Module access denied",
  "message": "This school does not have access to the hostel_management module. Please upgrade your subscription.",
  "module": "hostel_management",
  "upgrade_required": true
}
```

Super Admin

Base URL: `https://compasse.net/api/v1/` **Auth:** Authorization: Bearer {token} required on all protected endpoints **Module gate:** None — super admin operates on the central database, not a tenant

Who is the Super Admin?

The super admin is the **platform owner**. They operate at the central-database level and have full visibility and control over every tenant (school) on the platform. Super admin accounts are stored in the **central database**, not in any tenant database.

User Story

As a super admin, I want to manage all schools on the platform — create new tenants, view their usage, suspend or activate them, manage subscription plans, and see platform-wide analytics — all from a single dashboard without needing to log into each school separately.

Authentication

Super admins log in via the central auth endpoint. Their token is issued against the **central database**. They do **not** use a tenant subdomain and do **not** send an `X-Subdomain` header.

Login

```
POST /api/v1/auth/login
```

Request:

```
POST /api/v1/auth/login HTTP/1.1
Host: compasse.net
Content-Type: application/json
Accept: application/json
```

Request Body:

```
{
  "email": "owner@compasse.net",
  "password": "SuperSecureP@ss!"
}
```

Response 200 OK :

```
{
  "token": "1|superadmintoken9876xyz",
  "token_type": "Bearer",
  "user": {
    "id": 1,
    "name": "Platform Owner",
    "email": "owner@compasse.net",
    "role": "super_admin",
    "created_at": "2025-01-01T00:00:00Z"
  }
}
```

Response 401 Unauthorized :

```
{
  "message": "Invalid credentials"
}
```

Response 429 Too Many Requests :

```
{
  "message": "Too many login attempts. Please try again in 60 seconds."
}
```

All super-admin-only routes are protected by `['auth:sanctum', 'role:super_admin']`.

Platform Dashboard

Get Super Admin Dashboard

```
GET /api/v1/dashboard/super-admin
Authorization: Bearer {superadmin_token}
```

Response 200 OK :

```
{
  "overview": {
    "total_tenants": 42,
    "active_tenants": 38,
    "suspended_tenants": 4,
    "total_schools": 45,
    "total_students": 18340,
    "total_teachers": 1250,
    "total_revenue_ngn": 6750000
  },
  "recent_tenants": [
    {
      "id": "springfield",
      "name": "Springfield Secondary School",
      "status": "active",
      "plan": "Pro",
      "created_at": "2026-03-20T11:00:00Z"
    },
    {
      "id": "lakeside",
      "name": "Lakeside International",
      "status": "active",
      "plan": "Enterprise",
      "created_at": "2026-03-15T09:30:00Z"
    }
  ],
  "expiring_subscriptions": [
    {
      "tenant_id": "pearlbridge",
      "school_name": "Pearl Bridge Academy",
      "plan": "Starter",
      "end_date": "2026-04-05",
      "days_remaining": 6
    }
  ],
  "revenue_by_month": [
    { "month": "2026-01", "amount": 1200000 },
    { "month": "2026-02", "amount": 1350000 },
    { "month": "2026-03", "amount": 1500000 }
  ]
}
```

Tenant Management

Full CRUD on tenants. Each tenant = one school on its own isolated database.

List All Tenants

```
GET /api/v1/tenants
Authorization: Bearer {superadmin_token}
```

Query Parameters:

Parameter	Type	Description
status	string	Filter by active , suspended , trial
plan	string	Filter by plan slug
search	string	Search by name or subdomain
per_page	integer	Items per page (default 20)
page	integer	Page number

Response 200 OK :

```
{
  "data": [
    {
      "id": "greenfield",
      "name": "Greenfield Academy",
      "subdomain": "greenfield",
      "status": "active",
      "db_name": "tenant_greenfield",
      "plan": {
        "name": "Pro",
        "slug": "pro"
      },
      "subscription_end_date": "2026-04-17",
      "school_count": 1,
      "created_at": "2025-09-01T08:00:00Z"
    },
    {
      "id": "lakeside",
      "name": "Lakeside International",
      "subdomain": "lakeside",
      "status": "active",
      "db_name": "tenant_lakeside",
      "plan": {
        "name": "Enterprise",
        "slug": "enterprise"
      },
      "subscription_end_date": "2027-01-01",
      "school_count": 2,
      "created_at": "2025-06-15T10:00:00Z"
    }
  ],
  "meta": {
    "current_page": 1,
    "per_page": 20,
    "total": 42,
    "last_page": 3
  }
}
```

Create Tenant

Provisions a new school — creates the tenant record, a dedicated MySQL database, runs all tenant migrations, creates the school record, and generates the admin user account.

```
POST /api/v1/tenants
Authorization: Bearer {superadmin_token}
Content-Type: application/json
```

Request Body:

```
{
  "subdomain": "springfield",
  "name": "Springfield Secondary School",
  "email": "admin@springfield.edu.ng",
  "phone": "+2348098765432",
  "address": "12 Ajah Road, Lagos",
  "plan_id": 2
}
```

What happens internally:

1. Validates subdomain uniqueness
2. Creates tenant record in central DB
3. Provisions new MySQL database `tenant_springfield`
4. Runs all tenant migrations on the new database
5. Creates the school record inside the tenant DB
6. Creates the school admin user with a generated secure password
7. Sends welcome email to admin (if email integration configured)

Response `201 Created` :

```
{
  "message": "Tenant created successfully",
  "tenant": {
    "id": "springfield",
    "name": "Springfield Secondary School",
    "subdomain": "springfield",
    "status": "active",
    "db_name": "tenant_springfield",
    "created_at": "2026-03-30T14:00:00Z"
  },
  "school": {
    "id": 1,
    "name": "Springfield Secondary School",
    "email": "admin@springfield.edu.ng"
  },
  "admin_credentials": {
    "email": "admin@springfield.edu.ng",
    "password": "Xk9$mP2qRv7T"
  }
}
```

Security Note: `admin_credentials.password` is shown **once only** and is never stored in plaintext. Display it immediately to the super admin and instruct them to share it securely with the school admin.

Response 422 Unprocessable Entity (subdomain taken):

```
{
  "message": "The given data was invalid.",
  "errors": {
    "subdomain": ["The subdomain has already been taken."]
  }
}
```

Get Tenant Details

```
GET /api/v1/tenants/{id}
Authorization: Bearer {superadmin_token}
```

Example: `GET /api/v1/tenants/greenfield`

Response 200 OK :

```
{
  "tenant": {
    "id": "greenfield",
    "name": "Greenfield Academy",
    "subdomain": "greenfield",
    "status": "active",
    "db_name": "tenant_greenfield",
    "created_at": "2025-09-01T08:00:00Z",
    "updated_at": "2026-03-01T10:00:00Z"
  },
  "subscription": {
    "plan": "Pro",
    "status": "active",
    "end_date": "2026-04-17",
    "days_remaining": 18
  }
}
```

Update Tenant

```
PUT /api/v1/tenants/{id}
Authorization: Bearer {superadmin_token}
Content-Type: application/json
```

Request Body:

```
{
  "name": "Greenfield Academy (Updated)",
  "status": "active"
}
```

Response 200 OK :

```
{
  "message": "Tenant updated successfully",
  "tenant": {
    "id": "greenfield",
    "name": "Greenfield Academy (Updated)",
    "status": "active",
    "updated_at": "2026-03-30T15:00:00Z"
  }
}
```

Delete Tenant

Deletes the tenant record and **drops the entire tenant database**. This is irreversible.

```
DELETE /api/v1/tenants/{id}
Authorization: Bearer {superadmin_token}
```

Response 200 OK :

```
{
  "message": "Tenant and associated database deleted successfully"
}
```

Response 409 Conflict (active subscription exists):

```
{
  "message": "Cannot delete a tenant with an active subscription. Cancel the subscription first."
}
```

Get Tenant Stats

Switches to the tenant DB to count students, teachers, and classes.

```
GET /api/v1/tenants/{id}/stats
Authorization: Bearer {superadmin_token}
```

Response 200 OK :


```
{
  "tenant_id": "greenfield",
  "stats": {
    "students": 842,
    "teachers": 64,
    "classes": 18,
    "staff": 30,
    "academic_years": 2,
    "active_academic_year": "2025/2026"
  }
}
```

School Management

Super admins can view, update, suspend, and activate any school across all tenants.

List All Schools

```
GET /api/v1/schools
Authorization: Bearer {superadmin_token}
```

Query Parameters:

Parameter	Type	Description
tenant_id	string	Filter by tenant subdomain
status	string	Filter by active , suspended
search	string	Search by school name

Response 200 OK :

```
{
  "data": [
    {
      "id": 1,
      "tenant_id": "greenfield",
      "name": "Greenfield Academy",
      "email": "info@greenfieldacademy.edu.ng",
      "phone": "+2348012345678",
      "status": "active",
      "student_count": 842,
      "teacher_count": 64,
      "created_at": "2025-09-01T08:00:00Z"
    }
  ],
  "meta": {
    "current_page": 1,
    "per_page": 20,
    "total": 45
  }
}
```

Get School Details + Stats

```
GET /api/v1/admin/schools/{id}
Authorization: Bearer {superadmin_token}
```

Response 200 OK :

```
{
  "school": {
    "id": 1,
    "tenant_id": "greenfield",
    "name": "Greenfield Academy",
    "email": "info@greenfieldacademy.edu.ng",
    "phone": "+2348012345678",
    "address": "14 Victoria Island, Lagos",
    "logo_url": "https://cdn.compassse.net/schools/greenfield/logo.png",
    "status": "active",
    "created_at": "2025-09-01T08:00:00Z"
  },
  "stats": {
    "students": 842,
    "teachers": 64,
    "classes": 18,
    "pending_fees": 1240000,
    "subscription_status": "active",
    "subscription_end_date": "2026-04-17"
  }
}
```

Get School Dashboard

```
GET /api/v1/admin/schools/{id}/dashboard
Authorization: Bearer {superadmin_token}
```

Response 200 OK :

```
{
  "school_id": 1,
  "name": "Greenfield Academy",
  "dashboard": {
    "students": 842,
    "teachers": 64,
    "classes": 18,
    "attendance_rate_today": 91.4,
    "fees_collected_this_month": 4500000,
    "fees_outstanding": 1240000,
    "recent_registrations": 5,
    "upcoming_events": 2
  }
}
```

Get School Usage Statistics

```
GET /api/v1/admin/schools/{id}/stats
Authorization: Bearer {superadmin_token}
```

Response 200 OK :

```
{
  "school_id": 1,
  "usage": {
    "students": 842,
    "teachers": 64,
    "staff": 30,
    "classes": 18,
    "subjects": 72,
    "academic_years": 2,
    "fee_records": 3600,
    "payments_recorded": 2901,
    "payroll_records": 180,
    "attendance_records": 42000,
    "storage_used_mb": 1245
  }
}
```

Update School

```
PUT /api/v1/admin/schools/{id}
Authorization: Bearer {superadmin_token}
Content-Type: application/json
```

Request Body:

```
{
  "name": "Greenfield Academy",
  "email": "info@greenfieldacademy.edu.ng",
  "phone": "+2348012345678",
  "address": "14 Victoria Island, Lagos",
  "website": "https://greenfieldacademy.edu.ng"
}
```

Response 200 OK :

```
{
  "message": "School updated successfully",
  "school": {
    "id": 1,
    "name": "Greenfield Academy",
    "email": "info@greenfieldacademy.edu.ng",
    "updated_at": "2026-03-30T15:30:00Z"
  }
}
```

Suspend School

Blocks all login and API access for users of that school.

```
POST /api/v1/admin/schools/{id}/suspend
Authorization: Bearer {superadmin_token}
Content-Type: application/json
```

Request Body:

```
{
  "reason": "Outstanding subscription payment"
}
```

Response 200 OK :

```
{
  "message": "School suspended successfully",
  "school_id": 1,
  "status": "suspended",
  "reason": "Outstanding subscription payment"
}
```

Activate School

Re-activates a previously suspended school.

```
POST /api/v1/admin/schools/{id}/activate
Authorization: Bearer {superadmin_token}
```

Response 200 OK :

```
{
  "message": "School activated successfully",
  "school_id": 1,
  "status": "active"
}
```

Send Email to School Admin

```
POST /api/v1/admin/schools/{id}/send-email
Authorization: Bearer {superadmin_token}
Content-Type: application/json
```

Request Body:

```
{
  "subject": "Subscription Renewal Reminder",
  "message": "Your Compasse Pro subscription expires in 7 days. Please renew to avoid service interruption.",
  "type": "notification"
}
```

Response 200 OK :

```
{
  "message": "Email sent successfully",
  "recipient": "admin@greenfieldacademy.edu.ng"
}
```

Reset School Admin Password

Generates a new secure password for the school admin and emails it to them.

```
POST /api/v1/admin/schools/{id}/reset-admin-password
Authorization: Bearer {superadmin_token}
```

Response 200 OK :

```
{
  "message": "Admin password reset successfully. New credentials sent to
admin@greenfieldacademy.edu.ng",
  "email": "admin@greenfieldacademy.edu.ng"
}
```

Get User Count

```
GET /api/v1/admin/schools/{id}/users-count
Authorization: Bearer {superadmin_token}
```

Response 200 OK :

```
{
  "school_id": 1,
  "users": {
    "total": 970,
    "admin": 1,
    "principal": 1,
    "vice_principal": 1,
    "teacher": 64,
    "student": 842,
    "guardian": 45,
    "accountant": 2,
    "staff": 10,
    "librarian": 1,
    "nurse": 1,
    "driver": 2
  }
}
```

Get Activity Logs

```
GET /api/v1/admin/schools/{id}/activity-logs
Authorization: Bearer {superadmin_token}
```

Query Parameters:

Parameter	Type	Description
per_page	integer	Items per page (default 20)
page	integer	Page number
from	date	Start date filter (YYYY-MM-DD)
to	date	End date filter (YYYY-MM-DD)

Response 200 OK :

```
{
  "data": [
    {
      "id": 5501,
      "user": "Mrs. Adaobi Nwosu",
      "role": "admin",
      "action": "student.created",
      "description": "Created student record for John Okafor (JS1A)",
      "ip_address": "105.112.44.8",
      "created_at": "2026-03-30T09:14:22Z"
    },
    {
      "id": 5500,
      "user": "Mr. Emeka Eze",
      "role": "accountant",
      "action": "payment.recorded",
      "description": "Recorded ₦50,000 payment for Jane Doe (Fee ID: 12)",
      "ip_address": "105.112.44.9",
      "created_at": "2026-03-30T08:45:10Z"
    }
  ],
  "meta": {
    "current_page": 1,
    "per_page": 20,
    "total": 5501
  }
}
```

Delete School

```
DELETE /api/v1/schools/{id}
Authorization: Bearer {superadmin_token}
```

Response 200 OK :

```
{
  "message": "School deleted successfully"
}
```

Access to School Data — Implementation Pattern

Yes — super admin has access to all school information, but the access pattern is intentional:

- For **tenant-level data** (subdomain, DB name, status) → central DB, direct access
- For **school-level data** (students, teachers, fees) → the system switches to the tenant's database using `tenancy()->initialize($tenant)` before querying

The `TenantService::getTenantStats()` method demonstrates this pattern:

```
tenancy()->initialize($tenant);
try {
    $stats = [
        'students' => Student::count(),
        'teachers' => Teacher::count(),
        'classes' => SchoolClass::count(),
    ];
} finally {
    tenancy()->end(); // always restores central DB context
}
```

Super admin cannot accidentally leak data between tenants because the `tenancy()->end()` call in a `finally` block ensures the DB connection is always restored.

What Super Admin Cannot Do

- Log in as a school user (no impersonation endpoint exists)
 - Read raw student records, exam results, or financial records directly — these require switching to the tenant DB and are not exposed through super-admin endpoints
 - Bypass the subscription check for individual schools (they can only change the plan/subscription)
-

Security Notes

- Super admin role is **not assignable** via the public registration endpoint (`super_admin` is excluded from the `role` validation whitelist)
 - Super admin accounts should be created via `php artisan tinker` or a seeder, never via API
 - The `/api/v1/health/db` diagnostic endpoint is restricted to `super_admin` only
 - All super-admin routes are wrapped in `['auth:sanctum', 'role:super_admin']` middleware
-

Frontend Integration — Super Admin Portal

The super admin portal operates on the **central domain** (`compasse.net`), not a subdomain. There is no `X-Subdomain` header required.

Login Flow

```
// services/superAdminAuth.ts

const SUPER_ADMIN_TOKEN_KEY = 'compassse_superadmin_token';

export async function superAdminLogin(email: string, password: string) {
  const res = await fetch('https://compasse.net/api/v1/auth/login', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'Accept': 'application/json',
      // No X-Subdomain header – this is the central domain
    },
    body: JSON.stringify({ email, password }),
  });

  if (!res.ok) {
    const err = await res.json();
    throw new Error(err.message || 'Login failed');
  }

  const data = await res.json();

  if (data.user.role !== 'super_admin') {
    throw new Error('Access denied. Super admin credentials required.');
```

```
  }

  localStorage.setItem(SUPER_ADMIN_TOKEN_KEY, data.token);
  localStorage.setItem('compasse_superadmin_user', JSON.stringify(data.user));

  return data;
}

export function getSuperAdminToken(): string | null {
  return localStorage.getItem(SUPER_ADMIN_TOKEN_KEY);
}

export function superAdminLogout() {
  localStorage.removeItem(SUPER_ADMIN_TOKEN_KEY);
  localStorage.removeItem('compasse_superadmin_user');
}
```

API Client for Super Admin

```
// services/superAdminApi.ts

export function createSuperAdminApiClient() {
  const token = localStorage.getItem('compasse_superadmin_token');

  const headers = {
    'Authorization': `Bearer ${token}`,
    'Content-Type': 'application/json',
    'Accept': 'application/json',
    // No X-Subdomain – central database operations
  };

  return {
    get: (path: string) =>
      fetch(`https://compasse.net/api/v1${path}`, { headers })
        .then(handleSuperAdminResponse),

    post: (path: string, body: object) =>
      fetch(`https://compasse.net/api/v1${path}`, {
        method: 'POST',
        headers,
        body: JSON.stringify(body),
      }).then(handleSuperAdminResponse),

    put: (path: string, body: object) =>
      fetch(`https://compasse.net/api/v1${path}`, {
        method: 'PUT',
        headers,
        body: JSON.stringify(body),
      }).then(handleSuperAdminResponse),

    delete: (path: string) =>
      fetch(`https://compasse.net/api/v1${path}`, {
        method: 'DELETE',
        headers,
      }).then(handleSuperAdminResponse),
  };
}

async function handleSuperAdminResponse(res: Response) {
  if (res.status === 401) {
    superAdminLogout();
    window.location.href = '/admin/login';
    return;
  }
  if (!res.ok) throw await res.json();
  return res.json();
}
```

Dashboard Usage Example

```
// pages/admin/dashboard.tsx

import { createSuperAdminApiClient } from '@services/superAdminApi';

export default function SuperAdminDashboard() {
  const [stats, setStats] = useState(null);
  const api = createSuperAdminApiClient();

  useEffect(() => {
    api.get('/dashboard/super-admin').then(setStats);
  }, []);

  return (
    <div>
      <h1>Platform Overview</h1>
      <p>Total Tenants: {stats?.overview.total_tenants}</p>
      <p>Active Tenants: {stats?.overview.active_tenants}</p>
      <p>Total Students: {stats?.overview.total_students}</p>
    </div>
  );
}
```

Displaying the One-Time Admin Password After Tenant Creation

```
// After POST /api/v1/tenants
const result = await api.post('/tenants', formData);

// Show once – never store
showModal({
  title: 'Tenant Created Successfully',
  body: `
    School admin credentials (SHOW ONCE – copy before closing):
    Email: ${result.admin_credentials.email}
    Password: ${result.admin_credentials.password}
  `,
  onClose: () => {
    // Clear from state immediately after user dismisses
    clearSensitiveData();
  }
});
```

Subscription Management

Base URL: `https://{subdomain}.compasse.net/api/v1/` **Auth:** `Authorization: Bearer {token}` required on all protected endpoints **Module gate:** None — all authenticated tenant users can check subscription status

Overview

Every school must subscribe to a plan to access module-gated features. Plans define which modules are enabled and what usage limits apply (max students, storage, etc.). Subscriptions are stored in the **tenant database** and managed via `SubscriptionService`.

Module access decisions are **Redis-cached for 5 minutes** — changes to a school's plan take effect within 5 minutes system-wide.

User Stories

As a super admin, I want to create and manage subscription plans so that schools can choose the tier that fits their needs.

As a school admin, I want to view my current subscription, see which modules I have access to, and upgrade my plan if needed.

As the system, I want to block access to premium features when a school's subscription is expired or cancelled, returning a clear 403 response with an upgrade prompt.

Plans

Plans are created by super admins and stored in the central database.

Plan Fields

Field	Type	Description
name	string	Plan display name (e.g. “Starter”, “Pro”, “Enterprise”)
slug	string	Unique identifier (e.g. starter , pro)
price	decimal	Price in the plan’s currency
billing_cycle	enum	monthly , quarterly , yearly
currency	string	e.g. NGN , USD
trial_days	integer	Free trial days (0 = no trial)
features	JSON array	Module slugs included in this plan
limits	JSON object	Usage limits e.g. {"students": 500, "storage": 5000}
is_active	boolean	Whether plan is available for purchase
is_popular	boolean	Highlighted in pricing UI

Example Plan Object

```
{
  "id": 2,
  "name": "Pro",
  "slug": "pro",
  "price": 25000,
  "billing_cycle": "monthly",
  "currency": "NGN",
  "trial_days": 14,
  "is_active": true,
  "is_popular": true,
  "features": [
    "academic_management",
    "student_management",
    "teacher_management",
    "cbt",
    "fee_management",
    "attendance_management",
    "transport_management",
    "hostel_management",
    "health_management",
    "inventory_management",
    "event_management",
    "sms_integration",
    "email_integration"
  ],
  "limits": {
    "students": 2000,
    "teachers": 200,
    "storage": 50000
  }
}
```

Subscription Lifecycle



Status Rules

Status	isActive()	isTrial()	Module Access
active + end_date future	Yes	No	Full plan modules
active + trial_end_date future	Yes	Yes	Full plan modules
active + end_date past	No	No	None — denied
cancelled	No	No	None — denied
No record	—	—	None — denied

No subscription = no access. There is no free tier bypass.

API Endpoints

All endpoints require `tenant` + `auth:sanctum` middleware. No additional module gate.

Method	Endpoint	Description
GET	/api/v1/subscriptions/plans	List all active plans
GET	/api/v1/subscriptions/modules	List all available modules
GET	/api/v1/subscriptions/status	Current school subscription status
GET	/api/v1/subscriptions/school/modules	Modules enabled for this school
GET	/api/v1/subscriptions/school/limits	Usage limits for this school
GET	/api/v1/subscriptions/modules/{module}/access	Check single module access
GET	/api/v1/subscriptions/features/{feature}/access	Check single feature access
GET	/api/v1/subscriptions	List subscriptions for this school
GET	/api/v1/subscriptions/{id}	Get subscription details
POST	/api/v1/subscriptions/create	Create subscription
PUT	/api/v1/subscriptions/{id}/upgrade	Upgrade to new plan
POST	/api/v1/subscriptions/{id}/renew	Renew subscription
DELETE	/api/v1/subscriptions/{id}/cancel	Cancel subscription

Full Request / Response Examples

List All Active Plans

```
GET /api/v1/subscriptions/plans
Authorization: Bearer {token}
X-Subdomain: greenfield
```

Response 200 OK :

```
{
  "plans": [
    {
      "id": 1,
      "name": "Starter",
      "slug": "starter",
      "price": 8000,
      "billing_cycle": "monthly",
      "currency": "NGN",
      "trial_days": 7,
      "is_popular": false,
      "features": [
        "academic_management",
        "student_management",
        "teacher_management",
        "fee_management"
      ],
      "limits": {
        "students": 300,
        "teachers": 30,
        "storage": 5000
      }
    },
    {
      "id": 2,
      "name": "Pro",
      "slug": "pro",
      "price": 25000,
      "billing_cycle": "monthly",
      "currency": "NGN",
      "trial_days": 14,
      "is_popular": true,
      "features": [
        "academic_management",
        "student_management",
        "teacher_management",
        "cbt",
        "fee_management",
        "attendance_management",
        "transport_management",
        "hostel_management",
        "health_management",
        "inventory_management",
        "event_management",
        "sms_integration",
        "email_integration"
      ],
      "limits": {
        "students": 2000,
        "teachers": 200,
        "storage": 50000
      }
    },
    {
      "id": 3,
      "name": "Enterprise",
      "slug": "enterprise",
```



```
"price": 60000,
"billing_cycle": "monthly",
"currency": "NGN",
"trial_days": 30,
"is_popular": false,
"features": [
  "academic_management",
  "student_management",
  "teacher_management",
  "cbt",
  "fee_management",
  "attendance_management",
  "transport_management",
  "hostel_management",
  "health_management",
  "inventory_management",
  "event_management",
  "livestream",
  "sms_integration",
  "email_integration"
],
"limits": {
  "students": -1,
  "teachers": -1,
  "storage": -1
}
}
```

"students": -1 means unlimited.

List All Available Modules

```
GET /api/v1/subscriptions/modules
Authorization: Bearer {token}
X-Subdomain: greenfield
```

Response 200 OK :

```
{
  "modules": [
    { "slug": "academic_management", "name": "Academic Management", "description": "Classes, subjects, academic years, terms" },
    { "slug": "student_management", "name": "Student Management", "description": "Student enrollment and profiles" },
    { "slug": "teacher_management", "name": "Teacher Management", "description": "Teacher profiles and class assignments" },
    { "slug": "cbt", "name": "CBT & Exams", "description": "Computer-based testing, results, report cards" },
    { "slug": "fee_management", "name": "Fee Management", "description": "School fees, payments, payroll, receipts" },
    { "slug": "attendance_management", "name": "Attendance", "description": "Student and staff attendance tracking" },
    { "slug": "transport_management", "name": "Transport", "description": "Vehicles, drivers, routes, secure pickup" },
    { "slug": "hostel_management", "name": "Hostel", "description": "Rooms, allocations, maintenance" },
    { "slug": "health_management", "name": "Health", "description": "Health records, appointments, medications" },
    { "slug": "inventory_management", "name": "Inventory", "description": "Categories, items, checkout and return" },
    { "slug": "event_management", "name": "Events", "description": "Events and school calendar" },
    { "slug": "livestream", "name": "Livestream", "description": "Live classes and virtual sessions" },
    { "slug": "sms_integration", "name": "SMS Notifications", "description": "SMS alerts and notifications" },
    { "slug": "email_integration", "name": "Email Notifications", "description": "Email alerts and notifications" }
  ]
}
```

Check Subscription Status

```
GET /api/v1/subscriptions/status
Authorization: Bearer {token}
X-Subdomain: greenfield
```

Response 200 OK (active subscription):

```
{
  "status": "active",
  "plan": "Pro",
  "plan_slug": "pro",
  "plan_id": 2,
  "features": [
    "academic_management",
    "student_management",
    "teacher_management",
    "cbt",
    "fee_management",
    "attendance_management",
    "transport_management",
    "hostel_management",
    "health_management",
    "inventory_management",
    "event_management",
    "sms_integration",
    "email_integration"
  ],
  "limits": {
    "students": 2000,
    "teachers": 200,
    "storage": 50000
  },
  "start_date": "2026-03-01",
  "end_date": "2026-04-17",
  "days_remaining": 18,
  "is_trial": false,
  "trial_end_date": null,
  "auto_renew": true,
  "next_billing_date": "2026-04-17"
}
```

Response 200 OK (trial subscription):

```
{
  "status": "active",
  "plan": "Pro",
  "plan_slug": "pro",
  "plan_id": 2,
  "features": ["academic_management", "fee_management"],
  "limits": { "students": 2000 },
  "start_date": "2026-03-25",
  "end_date": "2026-04-25",
  "days_remaining": 26,
  "is_trial": true,
  "trial_end_date": "2026-04-08",
  "auto_renew": false,
  "next_billing_date": null
}
```

Response 200 OK (no subscription):

```
{
  "status": "none",
  "plan": null,
  "message": "This school has no active subscription. Please subscribe to access features."
}
```

Get School's Enabled Modules

```
GET /api/v1/subscriptions/school/modules
Authorization: Bearer {token}
X-Subdomain: greenfield
```

Response 200 OK :

```
{
  "school_id": 1,
  "plan": "Pro",
  "modules": [
    "academic_management",
    "student_management",
    "teacher_management",
    "cbt",
    "fee_management",
    "attendance_management",
    "transport_management",
    "hostel_management",
    "health_management",
    "inventory_management",
    "event_management",
    "sms_integration",
    "email_integration"
  ],
  "cached_at": "2026-03-30T10:00:00Z",
  "cache_ttl_seconds": 300
}
```

Get School Usage Limits

```
GET /api/v1/subscriptions/school/limits
Authorization: Bearer {token}
X-Subdomain: greenfield
```

Response 200 OK :

```
{
  "school_id": 1,
  "plan": "Pro",
  "limits": {
    "students": {
      "max": 2000,
      "current": 842,
      "remaining": 1158,
      "percent_used": 42.1
    },
    "teachers": {
      "max": 200,
      "current": 64,
      "remaining": 136,
      "percent_used": 32.0
    },
    "storage_mb": {
      "max": 50000,
      "current": 1245,
      "remaining": 48755,
      "percent_used": 2.5
    }
  }
}
```

Check Single Module Access

```
GET /api/v1/subscriptions/modules/{module}/access
Authorization: Bearer {token}
X-Subdomain: greenfield
```

Example: GET /api/v1/subscriptions/modules/hostel_management/access

Response 200 OK (access granted):

```
{
  "module": "hostel_management",
  "has_access": true,
  "plan": "Pro"
}
```

Response 200 OK (access denied):

```
{
  "module": "livestream",
  "has_access": false,
  "plan": "Pro",
  "message": "The livestream module is not included in your current plan. Upgrade to Enterprise to access it.",
  "upgrade_required": true
}
```

Check Single Feature Access

```
GET /api/v1/subscriptions/features/{feature}/access
Authorization: Bearer {token}
X-Subdomain: greenfield
```

Response 200 OK :

```
{
  "feature": "sms_integration",
  "has_access": true
}
```

List Subscriptions for School

```
GET /api/v1/subscriptions
Authorization: Bearer {token}
X-Subdomain: greenfield
```

Response 200 OK :

```
{
  "data": [
    {
      "id": 3,
      "plan_id": 2,
      "plan_name": "Pro",
      "status": "active",
      "start_date": "2026-03-01",
      "end_date": "2026-04-17",
      "payment_method": "bank_transfer",
      "auto_renew": true,
      "created_at": "2026-03-01T09:00:00Z"
    },
    {
      "id": 2,
      "plan_id": 1,
      "plan_name": "Starter",
      "status": "cancelled",
      "start_date": "2025-09-01",
      "end_date": "2026-02-28",
      "payment_method": "bank_transfer",
      "auto_renew": false,
      "created_at": "2025-09-01T08:00:00Z"
    }
  ]
}
```

Get Subscription Details

```
GET /api/v1/subscriptions/{id}
Authorization: Bearer {token}
X-Subdomain: greenfield
```

Response 200 OK :

```
{
  "subscription": {
    "id": 3,
    "school_id": 1,
    "plan_id": 2,
    "plan": {
      "name": "Pro",
      "slug": "pro",
      "price": 25000,
      "billing_cycle": "monthly",
      "currency": "NGN"
    },
    "status": "active",
    "start_date": "2026-03-01",
    "end_date": "2026-04-17",
    "trial_end_date": null,
    "is_trial": false,
    "payment_method": "bank_transfer",
    "auto_renew": true,
    "notes": null,
    "created_at": "2026-03-01T09:00:00Z",
    "updated_at": "2026-03-01T09:00:00Z"
  }
}
```

Create Subscription

```
POST /api/v1/subscriptions/create
Authorization: Bearer {token}
X-Subdomain: greenfield
Content-Type: application/json
```

Request Body:

```
{
  "plan_id": 2,
  "payment_method": "bank_transfer",
  "auto_renew": true,
  "start_date": "2026-04-01"
}
```

Response 201 Created :

```
{
  "message": "Subscription created successfully",
  "subscription": {
    "id": 4,
    "school_id": 1,
    "plan_id": 2,
    "plan_name": "Pro",
    "status": "active",
    "start_date": "2026-04-01",
    "end_date": "2026-05-01",
    "is_trial": false,
    "trial_end_date": null,
    "payment_method": "bank_transfer",
    "auto_renew": true,
    "created_at": "2026-03-30T16:00:00Z"
  }
}
```

Response 409 Conflict (active subscription already exists):

```
{
  "message": "This school already has an active subscription. Please upgrade or wait for renewal."
}
```

Upgrade Plan

```
PUT /api/v1/subscriptions/{id}/upgrade
Authorization: Bearer {token}
X-Subdomain: greenfield
Content-Type: application/json
```

Request Body:

```
{
  "plan_id": 3
}
```

Response 200 OK :

```
{
  "message": "Subscription upgraded successfully",
  "subscription": {
    "id": 3,
    "plan_id": 3,
    "plan_name": "Enterprise",
    "status": "active",
    "end_date": "2026-04-30",
    "upgraded_at": "2026-03-30T16:05:00Z"
  }
}
```


Response 422 Unprocessable Entity (downgrade attempt):

```
{
  "message": "You cannot downgrade to a lower-tier plan through this endpoint. Cancel and create a new subscription."
}
```

Renew Subscription

Extends the subscription end date by one billing cycle.

```
POST /api/v1/subscriptions/{id}/renew
Authorization: Bearer {token}
X-Subdomain: greenfield
Content-Type: application/json
```

Request Body:

```
{
  "payment_method": "bank_transfer",
  "payment_reference": "TRF-2026-03-RENEW"
}
```

Response 200 OK :

```
{
  "message": "Subscription renewed successfully",
  "subscription": {
    "id": 3,
    "plan_name": "Pro",
    "status": "active",
    "previous_end_date": "2026-04-17",
    "new_end_date": "2026-05-17",
    "renewed_at": "2026-03-30T16:10:00Z"
  }
}
```

Cancel Subscription

```
DELETE /api/v1/subscriptions/{id}/cancel
Authorization: Bearer {token}
X-Subdomain: greenfield
Content-Type: application/json
```

Request Body:

```
{
  "immediate": false,
  "reason": "Switching to a different provider"
}
```

Field	Description
immediate	true = cancel now (access ends immediately). false = cancel at period end.
reason	Optional cancellation reason

Response 200 OK (cancel at period end):

```
{
  "message": "Subscription cancelled. Access will continue until 2026-04-17.",
  "subscription": {
    "id": 3,
    "status": "cancelled",
    "access_until": "2026-04-17",
    "cancelled_at": "2026-03-30T16:15:00Z"
  }
}
```

Response 200 OK (immediate cancellation):

```
{
  "message": "Subscription cancelled immediately. Access has been revoked.",
  "subscription": {
    "id": 3,
    "status": "cancelled",
    "access_until": "2026-03-30",
    "cancelled_at": "2026-03-30T16:15:00Z"
  }
}
```

Module Access Check (HTTP 403 Response)

When a school tries to access a gated route without the required module:

```
HTTP 403 Forbidden
{
  "error": "Module access denied",
  "message": "This school does not have access to the hostel_management module. Please upgrade your subscription.",
  "module": "hostel_management",
  "upgrade_required": true
}
```

Caching Details

`SubscriptionService` uses Laravel Cache (Redis) to avoid a DB hit on every request:

Cache Key	TTL	Invalidated On
<code>sub:school:{id}:module:{slug}</code>	5 min	create / upgrade / renew / cancel
<code>sub:school:{id}:modules</code>	5 min	same
<code>sub:school:{id}:status</code>	1 min	same

Redis tags (`school:{id}:subscription`) are used when the cache driver supports them (Redis), giving instant full invalidation. File/database drivers rely on TTL expiry.

Frontend Integration — Subscription & Module Gating

Step 1 — Fetch and Cache Modules on Login

Immediately after a successful login, fetch the school's enabled modules and cache them. This allows the frontend to conditionally show/hide UI sections without a round-trip on every navigation.

```

// services/modules.ts

const MODULES_KEY = (subdomain: string) => `compass_modules_${subdomain}`;

export async function fetchAndCacheModules(subdomain: string, token: string) {
  const res = await fetch('https://compasse.net/api/v1/subscriptions/school/modules', {
    headers: {
      'Authorization': `Bearer ${token}`,
      'X-Subdomain': subdomain,
      'Accept': 'application/json',
    },
  });

  if (!res.ok) return [];

  const data = await res.json();
  const modules: string[] = data.modules;

  // Cache in localStorage – refresh on every login
  localStorage.setItem(MODULES_KEY(subdomain), JSON.stringify(modules));
  localStorage.setItem(`${MODULES_KEY(subdomain)}_at`, Date.now().toString());

  return modules;
}

export function getCachedModules(subdomain: string): string[] {
  const raw = localStorage.getItem(MODULES_KEY(subdomain));
  return raw ? JSON.parse(raw) : [];
}

export function hasModule(subdomain: string, moduleSlug: string): boolean {
  return getCachedModules(subdomain).includes(moduleSlug);
}

```

Step 2 — Protect Navigation Items

```
// components/Sidebar.tsx

import { hasModule } from '@services/modules';
import { getSubdomain } from '@utils/tenancy';

const subdomain = getSubdomain()!;

const navItems = [
  { label: 'Dashboard', path: '/dashboard', module: null },
  { label: 'Students', path: '/students', module: 'student_management' },
  { label: 'Fees', path: '/fees', module: 'fee_management' },
  { label: 'Payroll', path: '/payroll', module: 'fee_management' },
  { label: 'CBT / Exams', path: '/cbt', module: 'cbt' },
  { label: 'Attendance', path: '/attendance', module: 'attendance_management' },
  { label: 'Transport', path: '/transport', module: 'transport_management' },
  { label: 'Hostel', path: '/hostel', module: 'hostel_management' },
  { label: 'Health', path: '/health', module: 'health_management' },
  { label: 'Livestream', path: '/livestream', module: 'livestream' },
];

export function Sidebar() {
  return (
    <nav>
      {navItems
        .filter(item => !item.module || hasModule(subdomain, item.module))
        .map(item => (
          <NavLink key={item.path} to={item.path}>{item.label}</NavLink>
        ))
      }
    </nav>
  );
}
```

Step 3 — Handle the 403 Upgrade Prompt

Listen for the global upgrade event (emitted by the API client on any `403 upgrade_required` response) and show an upgrade modal.

```
// components/UpgradeModal.tsx

import { useEffect, useState } from 'react';

export function UpgradeModal() {
  const [upgradeData, setUpgradeData] = useState<{
    module: string;
    message: string;
  } | null>(null);

  useEffect(() => {
    const handler = (e: CustomEvent) => {
      setUpgradeData(e.detail);
    };
    window.addEventListener('module:upgrade-required', handler as EventListener);
    return () => window.removeEventListener('module:upgrade-required', handler as
EventListener);
  }, []);

  if (!upgradeData) return null;

  return (
    <Modal onClose={() => setUpgradeData(null)}>
      <h2>Upgrade Required</h2>
      <p>{upgradeData.message}</p>
      <p>
        Module: <strong>{upgradeData.module}</strong>
      </p>
      <Button onClick={() => navigate('/settings/subscription')}>
        View Plans & Upgrade
      </Button>
    </Modal>
  );
}
```

Step 4 — Subscription Settings Page

```
// pages/settings/subscription.tsx

import { createApiClient } from '@services/api';
import { getSubdomain } from '@utils/tenancy';

export default function SubscriptionPage() {
  const subdomain = getSubdomain();
  const api = createApiClient(subdomain);
  const [status, setStatus] = useState(null);
  const [plans, setPlans] = useState([]);

  useEffect(() => {
    Promise.all([
      api.get('/subscriptions/status'),
      api.get('/subscriptions/plans'),
    ]).then(([statusData, plansData]) => {
      setStatus(statusData);
      setPlans(plansData.plans);
    });
  }, []);

  const handleUpgrade = async (planId: number, subscriptionId: number) => {
    await api.put(`/subscriptions/${subscriptionId}/upgrade`, { plan_id: planId });
    // Refresh modules cache after upgrade
    const token = localStorage.getItem(`comparse_token_${subdomain}`);
    await fetchAndCacheModules(subdomain, token!);
    alert('Plan upgraded! New features are now available.');
```

```
  };

  return (
    <div>
      <h1>Subscription</h1>
      {status && (
        <div>
          <p>Current Plan: <strong>{status.plan}</strong></p>
          <p>Status: <strong>{status.status}</strong></p>
          <p>Expires: <strong>{status.end_date}</strong></p>
          <p>Days Remaining: <strong>{status.days_remaining}</strong></p>
        </div>
      )}
      <h2>Available Plans</h2>
      {plans.map(plan => (
        <PlanCard
          key={plan.id}
          plan={plan}
          isCurrent={plan.slug === status?.plan_slug}
          onUpgrade={() => handleUpgrade(plan.id, status.subscription_id)}
        />
      ))}
    </div>
  );
}
```

Module Cache Invalidation Strategy

After any subscription change (upgrade, renew, cancel), always re-fetch and re-cache the modules list:

```
// After any subscription mutation
await api.put(`/subscriptions/${id}/upgrade`, { plan_id: newPlanId });

// Invalidate and refresh the module cache
const token = localStorage.getItem(`compasse_token_${subdomain}`);
await fetchAndCacheModules(subdomain, token!);

// Optionally reload the page to reflect new module access in nav
window.location.reload();
```


School Fees & Payments

Base URL: `https://{subdomain}.compasse.net/api/v1/financial/` **Auth:** Authorization: Bearer {token} required on all protected endpoints **Module gate:** `fee_management`

Overview

The fee management module handles school fee creation, student billing, payment recording, and receipt generation. It is gated by the `fee_management` module.

User Stories

As a school admin / accountant, I want to create fee structures per class and term so students are automatically billed the correct amount.

As an accountant, I want to record payments made by students or guardians, automatically update the balance, and generate a receipt.

As a school admin, I want to see which students have outstanding balances so I can follow up before exams.

As a parent / guardian, I want to view my child's fee balance and payment history.

Fee Model

Field	Type	Description
school_id	FK	Owning school
student_id	FK	Student being billed (nullable for class-wide fees)
class_id	FK	Target class
academic_year_id	FK	Academic year
term_id	FK	Term
fee_type	string	e.g. tuition , bus , hostel , uniform , exam
amount	decimal	Total fee amount
amount_paid	decimal	Total amount paid so far
balance	decimal	Outstanding balance
due_date	date	Payment deadline
status	enum	unpaid , partial , paid , waived , overdue
description	string	Additional details

Computed Properties (model methods)

- getRemainingAmount() — returns balance if set, otherwise amount – amount_paid
- getStats() — returns { total_amount, amount_paid, balance, payment_percent, is_overdue }

Payment Model

Field	Type	Description
school_id	FK	Owning school
student_id	FK	Student
fee_id	FK	Fee being paid
amount	decimal	Amount paid in this transaction
payment_method	enum	cash , bank_transfer , card , mobile_money , cheque
payment_date	date	Date payment was received
reference	string	Bank ref / receipt number
received_by	FK → User	Staff who recorded the payment
status	enum	pending , confirmed , failed , refunded
notes	string	Optional notes

API Endpoints

Required module: fee_management Base path: /api/v1/financial/

Fees

Method	Endpoint	Description
GET	/fees	List fees (filter: class_id, student_id, status, fee_type)
POST	/fees	Create a fee record
GET	/fees/{id}	Get fee details + payment stats
PUT	/fees/{id}	Update fee
DELETE	/fees/{id}	Delete fee (blocked if payments exist)
POST	/fees/{id}/pay	Record a payment against this fee
GET	/fees/student/{student_id}	All fees for a student
GET	/fees/structure	Fee structure summary by class/term
POST	/fees/structure	Bulk-create fees for an entire class
PUT	/fees/structure/{id}	Update a fee structure entry

Payments

Method	Endpoint	Description
GET	/payments	List all payments
POST	/payments	Record a payment
GET	/payments/{id}	Get payment details
GET	/payments/student/{student_id}	Payments by student
GET	/payments/receipt/{id}	Generate receipt data

Full Request / Response Examples — Fees

List Fees

```
GET /api/v1/financial/fees
Authorization: Bearer {token}
X-Subdomain: greenfield
```

Query Parameters:

Parameter	Type	Description
class_id	integer	Filter by class
student_id	integer	Filter by student
status	string	unpaid , partial , paid , waived , overdue
fee_type	string	tuition , bus , hostel , uniform , exam
term_id	integer	Filter by term
academic_year_id	integer	Filter by academic year
per_page	integer	Items per page (default 20)
page	integer	Page number

Response 200 OK :

```
{
  "data": [
    {
      "id": 11,
      "student_id": 102,
      "student_name": "John Okafor",
      "class_id": 3,
      "class_name": "JSS 1A",
      "academic_year": "2025/2026",
      "term": "Second Term",
      "fee_type": "tuition",
      "amount": "75000.00",
      "amount_paid": "75000.00",
      "balance": "0.00",
      "due_date": "2026-03-30",
      "status": "paid",
      "description": "Second term tuition fee",
      "created_at": "2026-01-15T09:00:00Z"
    },
    {
      "id": 12,
      "student_id": 103,
      "student_name": "Jane Doe",
      "class_id": 3,
      "class_name": "JSS 1A",
      "academic_year": "2025/2026",
      "term": "Second Term",
      "fee_type": "tuition",
      "amount": "75000.00",
      "amount_paid": "50000.00",
      "balance": "25000.00",
      "due_date": "2026-04-30",
      "status": "partial",
      "description": "Second term tuition fee",
      "created_at": "2026-01-15T09:00:00Z"
    }
  ],
  "meta": {
    "current_page": 1,
    "per_page": 20,
    "total": 156
  },
  "summary": {
    "total_fees": 156,
    "total_amount": 11700000,
    "total_paid": 9200000,
    "total_outstanding": 2500000,
    "unpaid_count": 22,
    "partial_count": 31,
    "paid_count": 103
  }
}
```

Create a Fee

```
POST /api/v1/financial/fees
Authorization: Bearer {token}
X-Subdomain: greenfield
Content-Type: application/json
```

Request Body (student-specific fee):

```
{
  "student_id": 103,
  "class_id": 3,
  "academic_year_id": 1,
  "term_id": 2,
  "fee_type": "tuition",
  "amount": 75000,
  "due_date": "2026-04-30",
  "description": "Second term tuition fee"
}
```

Request Body (class-wide fee — no student_id):

```
{
  "class_id": 3,
  "academic_year_id": 1,
  "term_id": 2,
  "fee_type": "tuition",
  "amount": 75000,
  "due_date": "2026-04-30",
  "description": "Second term tuition fee for JSS 1A"
}
```

Response 201 Created :

```
{
  "message": "Fee created successfully",
  "fee": {
    "id": 12,
    "student_id": 103,
    "student_name": "Jane Doe",
    "class_id": 3,
    "class_name": "JSS 1A",
    "academic_year": "2025/2026",
    "term": "Second Term",
    "fee_type": "tuition",
    "amount": "75000.00",
    "amount_paid": "0.00",
    "balance": "75000.00",
    "due_date": "2026-04-30",
    "status": "unpaid",
    "description": "Second term tuition fee",
    "created_at": "2026-03-30T14:00:00Z"
  }
}
```

Response 422 Unprocessable Entity :

```
{
  "message": "The given data was invalid.",
  "errors": {
    "amount": ["The amount must be at least 1."],
    "due_date": ["The due date must be a valid date in YYYY-MM-DD format."]
  }
}
```

Get Fee Details + Stats

```
GET /api/v1/financial/fees/{id}
Authorization: Bearer {token}
X-Subdomain: greenfield
```

Response 200 OK :

```
{
  "fee": {
    "id": 12,
    "student_id": 103,
    "student_name": "Jane Doe",
    "class_id": 3,
    "class_name": "JSS 1A",
    "academic_year": "2025/2026",
    "term": "Second Term",
    "fee_type": "tuition",
    "amount": "75000.00",
    "amount_paid": "50000.00",
    "balance": "25000.00",
    "due_date": "2026-04-30",
    "status": "partial",
    "description": "Second term tuition fee"
  },
  "stats": {
    "total_amount": 75000,
    "amount_paid": 50000,
    "balance": 25000,
    "payment_percent": 66.7,
    "is_overdue": false,
    "days_until_due": 31
  },
  "payments": [
    {
      "id": 45,
      "amount": "50000.00",
      "payment_method": "bank_transfer",
      "payment_date": "2026-03-28",
      "reference": "TRX-2026-001234",
      "received_by": "Mr. Adeyemi",
      "status": "confirmed",
      "created_at": "2026-03-28T10:00:00Z"
    }
  ]
}
```

Update Fee

```
PUT /api/v1/financial/fees/{id}
Authorization: Bearer {token}
X-Subdomain: greenfield
Content-Type: application/json
```

Request Body:

```
{
  "amount": 80000,
  "due_date": "2026-05-15",
  "description": "Second term tuition fee (revised)"
}
```


Response 200 OK :

```
{
  "message": "Fee updated successfully",
  "fee": {
    "id": 12,
    "amount": "80000.00",
    "balance": "30000.00",
    "due_date": "2026-05-15",
    "status": "partial",
    "updated_at": "2026-03-30T15:00:00Z"
  }
}
```

Delete Fee

```
DELETE /api/v1/financial/fees/{id}
Authorization: Bearer {token}
X-Subdomain: greenfield
```

Response 200 OK :

```
{
  "message": "Fee deleted successfully"
}
```

Response 409 Conflict (payments exist):

```
{
  "message": "Cannot delete this fee because payment records exist against it. Waive the fee instead."
}
```

Record Payment Against a Fee

```
POST /api/v1/financial/fees/{id}/pay
Authorization: Bearer {token}
X-Subdomain: greenfield
Content-Type: application/json
```

Request Body:

```
{
  "amount": 50000,
  "payment_method": "bank_transfer",
  "payment_date": "2026-03-28",
  "reference": "TRX-2026-001234",
  "notes": "First installment"
}
```

Response 200 OK :

```
{
  "message": "Payment recorded",
  "payment": {
    "id": 45,
    "fee_id": 12,
    "student_id": 103,
    "amount": "50000.00",
    "payment_method": "bank_transfer",
    "payment_date": "2026-03-28",
    "reference": "TRX-2026-001234",
    "received_by": 1,
    "status": "confirmed",
    "notes": "First installment",
    "created_at": "2026-03-28T10:00:00Z"
  },
  "fee": {
    "id": 12,
    "amount": "75000.00",
    "amount_paid": "50000.00",
    "balance": "25000.00",
    "status": "partial"
  }
}
```

Response 422 Unprocessable Entity (overpayment):

```
{
  "message": "Payment amount exceeds the remaining balance of ₦25,000.00"
}
```

Get All Fees for a Student

```
GET /api/v1/financial/fees/student/{student_id}
Authorization: Bearer {token}
X-Subdomain: greenfield
```

Query Parameters:

Parameter	Type	Description
academic_year_id	integer	Filter by academic year
term_id	integer	Filter by term
status	string	Filter by status

Response 200 OK :

```
{
  "student": {
    "id": 103,
    "name": "Jane Doe",
    "class": "JSS 1A",
    "admission_number": "GFA/2024/0103"
  },
  "fees": [
    {
      "id": 12,
      "fee_type": "tuition",
      "term": "Second Term",
      "academic_year": "2025/2026",
      "amount": "75000.00",
      "amount_paid": "50000.00",
      "balance": "25000.00",
      "due_date": "2026-04-30",
      "status": "partial",
      "is_overdue": false
    },
    {
      "id": 8,
      "fee_type": "tuition",
      "term": "First Term",
      "academic_year": "2025/2026",
      "amount": "75000.00",
      "amount_paid": "75000.00",
      "balance": "0.00",
      "due_date": "2025-12-15",
      "status": "paid",
      "is_overdue": false
    }
  ],
  "summary": {
    "total_owed": 150000,
    "total_paid": 125000,
    "total_outstanding": 25000,
    "overdue_count": 0
  }
}
```

Get Fee Structure

Returns the fee structure summary by class and term.

```
GET /api/v1/financial/fees/structure
Authorization: Bearer {token}
X-Subdomain: greenfield
```

Query Parameters: `academic_year_id`, `term_id`

Response 200 OK:

```
{
  "structures": [
    {
      "id": 1,
      "class_id": 3,
      "class_name": "JSS 1A",
      "term": "Second Term",
      "academic_year": "2025/2026",
      "fee_type": "tuition",
      "amount": "75000.00",
      "due_date": "2026-04-30",
      "student_count": 42,
      "total_billed": 3150000,
      "total_collected": 2850000,
      "total_outstanding": 300000,
      "collection_rate_percent": 90.5
    }
  ]
}
```

Bulk-Create Fees for an Entire Class

Creates one fee record per student currently enrolled in the given class.

```
POST /api/v1/financial/fees/structure
Authorization: Bearer {token}
X-Subdomain: greenfield
Content-Type: application/json
```

Request Body:

```
{
  "class_id": 3,
  "academic_year_id": 1,
  "term_id": 2,
  "fee_type": "tuition",
  "amount": 75000,
  "due_date": "2026-04-30",
  "description": "Second term tuition for JSS 1A"
}
```

Response 201 Created :

```
{
  "message": "Fee structure created. 42 fee records generated.",
  "structure": {
    "id": 1,
    "class_id": 3,
    "class_name": "JSS 1A",
    "term": "Second Term",
    "fee_type": "tuition",
    "amount": "75000.00",
    "due_date": "2026-04-30",
    "students_billed": 42,
    "total_amount": 3150000
  }
}
```

Update Fee Structure Entry

```
PUT /api/v1/financial/fees/structure/{id}
Authorization: Bearer {token}
X-Subdomain: greenfield
Content-Type: application/json
```

Request Body:

```
{
  "amount": 80000,
  "due_date": "2026-05-15"
}
```

Response 200 OK :

```
{
  "message": "Fee structure updated. 42 fee records updated.",
  "structure": {
    "id": 1,
    "amount": "80000.00",
    "due_date": "2026-05-15",
    "updated_at": "2026-03-30T16:00:00Z"
  }
}
```

Full Request / Response Examples — Payments

List All Payments

```
GET /api/v1/financial/payments
Authorization: Bearer {token}
X-Subdomain: greenfield
```

Query Parameters:

Parameter	Type	Description
student_id	integer	Filter by student
fee_type	string	Filter by fee type
payment_method	string	Filter by method
status	string	pending , confirmed , failed , refunded
from_date	date	Start date (YYYY-MM-DD)
to_date	date	End date (YYYY-MM-DD)
per_page	integer	Items per page

Response 200 OK :

```
{
  "data": [
    {
      "id": 45,
      "fee_id": 12,
      "student_id": 103,
      "student_name": "Jane Doe",
      "class": "JSS 1A",
      "fee_type": "tuition",
      "amount": "50000.00",
      "payment_method": "bank_transfer",
      "payment_date": "2026-03-28",
      "reference": "TRX-2026-001234",
      "received_by": "Mr. Adeyemi",
      "status": "confirmed",
      "created_at": "2026-03-28T10:00:00Z"
    }
  ],
  "meta": {
    "current_page": 1,
    "per_page": 20,
    "total": 2901
  },
  "summary": {
    "total_payments": 2901,
    "total_amount_collected": 9200000,
    "by_method": {
      "bank_transfer": 7500000,
      "cash": 1200000,
      "card": 500000,
      "mobile_money": 0,
      "cheque": 0
    }
  }
}
```

Record a Payment (Direct)

```
POST /api/v1/financial/payments
Authorization: Bearer {token}
X-Subdomain: greenfield
Content-Type: application/json
```

Request Body:

```
{
  "fee_id": 12,
  "student_id": 103,
  "amount": 25000,
  "payment_method": "cash",
  "payment_date": "2026-03-30",
  "reference": "CASH-2026-0030",
  "notes": "Final installment – balance cleared"
}
```

Response 201 Created :

```
{
  "message": "Payment recorded successfully",
  "payment": {
    "id": 46,
    "fee_id": 12,
    "student_id": 103,
    "student_name": "Jane Doe",
    "amount": "25000.00",
    "payment_method": "cash",
    "payment_date": "2026-03-30",
    "reference": "CASH-2026-0030",
    "received_by": 1,
    "received_by_name": "Mr. Adeyemi",
    "status": "confirmed",
    "notes": "Final installment – balance cleared",
    "created_at": "2026-03-30T16:30:00Z"
  },
  "fee": {
    "id": 12,
    "amount": "75000.00",
    "amount_paid": "75000.00",
    "balance": "0.00",
    "status": "paid"
  }
}
```

Get Payment Details

```
GET /api/v1/financial/payments/{id}
Authorization: Bearer {token}
X-Subdomain: greenfield
```

Response 200 OK :

```
{
  "payment": {
    "id": 45,
    "fee_id": 12,
    "student": {
      "id": 103,
      "name": "Jane Doe",
      "admission_number": "GFA/2024/0103",
      "class": "JSS 1A"
    },
    "fee": {
      "id": 12,
      "fee_type": "tuition",
      "term": "Second Term",
      "academic_year": "2025/2026",
      "total_amount": "75000.00"
    },
    "amount": "50000.00",
    "payment_method": "bank_transfer",
    "payment_date": "2026-03-28",
    "reference": "TRX-2026-001234",
    "received_by": "Mr. Adeyemi",
    "status": "confirmed",
    "notes": "First installment",
    "created_at": "2026-03-28T10:00:00Z"
  }
}
```

Get Payments by Student

```
GET /api/v1/financial/payments/student/{student_id}
Authorization: Bearer {token}
X-Subdomain: greenfield
```

Response 200 OK :


```
{
  "student": {
    "id": 103,
    "name": "Jane Doe",
    "admission_number": "GFA/2024/0103",
    "class": "JSS 1A"
  },
  "payments": [
    {
      "id": 46,
      "fee_type": "tuition",
      "term": "Second Term",
      "amount": "25000.00",
      "payment_method": "cash",
      "payment_date": "2026-03-30",
      "reference": "CASH-2026-0030",
      "status": "confirmed"
    },
    {
      "id": 45,
      "fee_type": "tuition",
      "term": "Second Term",
      "amount": "50000.00",
      "payment_method": "bank_transfer",
      "payment_date": "2026-03-28",
      "reference": "TRX-2026-001234",
      "status": "confirmed"
    }
  ],
  "total_paid": 75000
}
```

Get Fee Receipt

```
GET /api/v1/financial/payments/receipt/{id}
Authorization: Bearer {token}
X-Subdomain: greenfield
```

Response 200 OK :

```
{
  "receipt_id": "RCP-0000045",
  "school": {
    "name": "Greenfield Academy",
    "address": "14 Victoria Island, Lagos",
    "phone": "+2348012345678",
    "logo_url": "https://cdn.compassse.net/schools/greenfield/logo.png"
  },
  "student": {
    "name": "Jane Doe",
    "admission_number": "GFA/2024/0103",
    "class": "JSS 1A"
  },
  "fee": {
    "type": "tuition",
    "term": "Second Term",
    "academic_year": "2025/2026",
    "total_fee": 75000,
    "amount_paid_before": 0,
    "amount_paid_this_receipt": 50000,
    "cumulative_paid": 50000,
    "outstanding_after": 25000
  },
  "payment": {
    "id": 45,
    "amount": 50000,
    "payment_method": "bank_transfer",
    "payment_date": "2026-03-28",
    "reference": "TRX-2026-001234",
    "received_by": "Mr. Adeyemi",
    "status": "confirmed"
  },
  "issued_at": "2026-03-28T10:05:00Z"
}
```

Full Request / Response Examples — Expenses

Base path: `/api/v1/financial/expenses`

List Expenses

```
GET /api/v1/financial/expenses
Authorization: Bearer {token}
X-Subdomain: greenfield
```

Query Parameters:

Parameter	Type	Description
category	string	e.g. utilities , maintenance , supplies , salaries
status	string	pending , approved , paid
from_date	date	Start date
to_date	date	End date
per_page	integer	Items per page

Response 200 OK :

```
{
  "data": [
    {
      "id": 14,
      "description": "Generator fuel – March 2026",
      "amount": "45000.00",
      "category": "utilities",
      "date": "2026-03-25",
      "payment_method": "cash",
      "vendor": "Akin Petroleum",
      "receipt_number": "APL-20260325",
      "status": "paid",
      "approved_by": "Mrs. Adaobi Nwosu",
      "created_at": "2026-03-25T11:00:00Z"
    },
    {
      "id": 15,
      "description": "Classroom chairs replacement – Block B",
      "amount": "120000.00",
      "category": "maintenance",
      "date": "2026-03-28",
      "payment_method": "bank_transfer",
      "vendor": "Lagos Furniture Ltd",
      "receipt_number": null,
      "status": "approved",
      "approved_by": "Mrs. Adaobi Nwosu",
      "created_at": "2026-03-28T09:00:00Z"
    }
  ],
  "meta": {
    "current_page": 1,
    "per_page": 20,
    "total": 38
  },
  "summary": {
    "total_expenses": 38,
    "total_amount": 1450000,
    "by_status": {
      "pending": 5,
      "approved": 8,
      "paid": 25
    },
    "by_category": {
      "utilities": 320000,
      "maintenance": 580000,
      "supplies": 210000,
      "salaries": 340000
    }
  }
}
```

Record Expense

```
POST /api/v1/financial/expenses
Authorization: Bearer {token}
X-Subdomain: greenfield
Content-Type: application/json
```

Request Body:

```
{
  "description": "Internet subscription – April 2026",
  "amount": 35000,
  "category": "utilities",
  "date": "2026-04-01",
  "payment_method": "bank_transfer",
  "vendor": "Spectranet Nigeria",
  "receipt_number": "SPN-APR2026-00892"
}
```

Response 201 Created :

```
{
  "message": "Expense recorded successfully",
  "expense": {
    "id": 16,
    "description": "Internet subscription – April 2026",
    "amount": "35000.00",
    "category": "utilities",
    "date": "2026-04-01",
    "payment_method": "bank_transfer",
    "vendor": "Spectranet Nigeria",
    "receipt_number": "SPN-APR2026-00892",
    "status": "pending",
    "approved_by": null,
    "created_by": "Mr. Emeka Eze",
    "created_at": "2026-03-30T16:45:00Z"
  }
}
```

Get Expense Details

```
GET /api/v1/financial/expenses/{id}
Authorization: Bearer {token}
X-Subdomain: greenfield
```

Response 200 OK :

```
{
  "expense": {
    "id": 14,
    "description": "Generator fuel – March 2026",
    "amount": "45000.00",
    "category": "utilities",
    "date": "2026-03-25",
    "payment_method": "cash",
    "vendor": "Akin Petroleum",
    "receipt_number": "APL-20260325",
    "status": "paid",
    "approved_by": "Mrs. Adaobi Nwosu",
    "approved_at": "2026-03-25T12:00:00Z",
    "created_by": "Mr. Emeka Eze",
    "created_at": "2026-03-25T11:00:00Z",
    "updated_at": "2026-03-25T12:00:00Z"
  }
}
```

Update Expense

```
PUT /api/v1/financial/expenses/{id}
Authorization: Bearer {token}
X-Subdomain: greenfield
Content-Type: application/json
```

Request Body (approve and mark paid):

```
{
  "status": "paid",
  "payment_method": "bank_transfer",
  "receipt_number": "LFL-20260328-REF"
}
```

Response 200 OK :

```
{
  "message": "Expense updated successfully",
  "expense": {
    "id": 15,
    "status": "paid",
    "approved_by": "Mrs. Adaobi Nwosu",
    "receipt_number": "LFL-20260328-REF",
    "updated_at": "2026-03-30T17:00:00Z"
  }
}
```

Delete Expense

```
DELETE /api/v1/financial/expenses/{id}
Authorization: Bearer {token}
X-Subdomain: greenfield
```

Response 200 OK :

```
{
  "message": "Expense deleted successfully"
}
```

Response 409 Conflict (expense already paid):

```
{
  "message": "Cannot delete a paid expense. Reverse it manually if needed."
}
```

Business Rules

1. **Duplicate protection** — A fee cannot be deleted if any payment records exist against it
2. **Auto status update** — When `amount_paid >= amount` , fee status is set to `paid` ; partial payment → `partial`
3. **Overdue detection** — `getStats()` returns `is_overdue: true` if `due_date` is past and `balance > 0`
4. **Class-wide billing** — `POST /fees/structure` creates one fee record per student in the class automatically
5. **School scoping** — All fee queries are scoped to the current tenant's school; cross-school access is not possible
6. **Overpayment blocked** — Payment amount cannot exceed the remaining balance
7. **Expense delete guard** — Expenses with `status=paid` cannot be deleted

Frontend Integration — Fee Payment Flow

This section explains the complete flow from fetching a student's fees to recording a payment and refreshing the balance.

Step 1 — Check Module Access on Route Mount

```
// pages/fees/index.tsx
import { hasModule } from '@services/modules';
import { getSubdomain } from '@utils/tenancy';

const subdomain = getSubdomain(!);

if (!hasModule(subdomain, 'fee_management')) {
  return <UpgradePrompt module="fee_management" />;
}
```

Step 2 — Fetch Outstanding Fees for a Student

```
// hooks/useStudentFees.ts
import { createApiClient } from '@services/api';

export function useStudentFees(studentId: number) {
  const subdomain = getSubdomain(!);
  const api = createApiClient(subdomain);
  const [fees, setFees] = useState([]);
  const [summary, setSummary] = useState(null);
  const [loading, setLoading] = useState(true);

  const fetchFees = async () => {
    setLoading(true);
    const data = await api.get(`/financial/fees/student/${studentId}`);
    setFees(data.fees);
    setSummary(data.summary);
    setLoading(false);
  };

  useEffect(() => { fetchFees(); }, [studentId]);

  return { fees, summary, loading, refetch: fetchFees };
}
```


Step 3 — Display Outstanding Fees

```
// components/StudentFeeList.tsx

export function StudentFeeList({ studentId }: { studentId: number }) {
  const { fees, summary, loading, refetch } = useStudentFees(studentId);

  if (loading) return <Spinner />;

  const outstanding = fees.filter(f => f.status !== 'paid' && f.status !== 'waived');

  return (
    <div>
      <div className="summary-bar">
        <span>Total Outstanding: ₦{summary.total_outstanding.toLocaleString()}</span>
      </div>
      {outstanding.map(fee => (
        <FeeRow
          key={fee.id}
          fee={fee}
          onPaymentComplete={refetch}
        />
      ))}
    </div>
  );
}
```

Step 4 — Record Payment

```
// components/PaymentModal.tsx

async function handlePayment(feeId: number, amount: number, method: string, reference: string) {
  const subdomain = getSubdomain();
  const api = createApiClient(subdomain);

  try {
    const result = await api.post(`/financial/fees/${feeId}/pay`, {
      amount,
      payment_method: method,
      payment_date: new Date().toISOString().split('T')[0],
      reference,
    });

    // Payment recorded – show receipt and refresh fee balance
    showReceipt(result.payment.id);
    onPaymentComplete(); // triggers useStudentFees refetch

    toast.success(`Payment of ₦${amount.toLocaleString()} recorded successfully`);
  } catch (err) {
    toast.error(err.message || 'Payment failed');
  }
}
```

Step 5 — Fetch and Display Receipt

```
async function showReceipt(paymentId: number) {
  const subdomain = getSubdomain();
  const api = createApiClient(subdomain);

  const data = await api.get(`/financial/payments/receipt/${paymentId}`);

  openModal(
    <ReceiptView receipt={data} />,
    { title: `Receipt ${data.receipt_id}`, printable: true }
  );
}
```

Full Payment Flow Summary

1. User navigates to student profile → Fees tab
GET /financial/fees/student/{id}
→ Display list of fees, highlight outstanding ones
2. Accountant clicks "Record Payment" on a fee row
→ Open payment modal with fee details pre-filled
3. Accountant enters amount, method, and reference
POST /financial/fees/{id}/pay
→ Response includes updated fee balance
4. On success:
 - a. Refresh the fee list (refetch step 1)
 - b. Fetch receipt: GET /financial/payments/receipt/{payment_id}
 - c. Display printable receipt modal

Payroll & Pay Stubs

Base URL: `https://{subdomain}.compasse.net/api/v1/financial/` **Auth:** Authorization: Bearer {token} required on all protected endpoints **Module gate:** `fee_management`

Overview

The payroll module manages staff salary records, allowances, deductions, and net pay calculation. It generates pay stubs that staff can download or view. Payroll is gated by the `fee_management` module.

User Stories

As a school admin / accountant, I want to create monthly payroll records for each staff member specifying their basic salary, allowances, and deductions so the system calculates net pay automatically.

As a staff member, I want to view my pay stub for any given month showing my earnings breakdown, deductions, and net salary.

As a school admin, I want to mark payrolls as paid once bank transfers are confirmed so we have an accurate payment trail.

As an accountant, I want to filter payroll by month, year, or staff member to prepare financial reports.

Payroll Model

Field	Type	Description
school_id	FK	Owning school
staff_id	FK → users	Staff member receiving pay
academic_year_id	FK	Academic year (optional)
month	integer	1–12
year	integer	4-digit year
basic_salary	decimal	Base gross salary
allowances	decimal	Housing, transport, and other allowances
deductions	decimal	Tax, pension, loans, etc.
net_salary	decimal	basic_salary + allowances – deductions (auto-calculated)
payment_date	date	Date salary was disbursed
payment_method	enum	bank_transfer , cash , cheque
status	enum	pending , paid , cancelled
processed_by	FK → users	Who created/approved the payroll
notes	string	Any additional remarks

Net salary is always recalculated whenever `basic_salary` , `allowances` , or `deductions` change.

API Endpoints

Required module: `fee_management` **Base path:** `/api/v1/financial/payroll`

Method	Endpoint	Description
GET	<code>/payroll</code>	List payrolls (filter: <code>staff_id</code> , <code>month</code> , <code>year</code> , <code>status</code>)
POST	<code>/payroll</code>	Create payroll record
GET	<code>/payroll/{id}</code>	Get payroll details
PUT	<code>/payroll/{id}</code>	Update payroll (salary components or status)
DELETE	<code>/payroll/{id}</code>	Delete payroll (blocked if <code>status=paid</code>)
GET	<code>/payroll/{id}/pay-stub</code>	Generate pay stub for this record

Full Request / Response Examples

List Payrolls

```
GET /api/v1/financial/payroll
Authorization: Bearer {token}
X-Subdomain: greenfield
```

Query Parameters:

Parameter	Type	Description
staff_id	integer	Filter by staff member
month	integer	Filter by month (1–12)
year	integer	Filter by year (e.g. 2026)
status	string	pending , paid , cancelled
per_page	integer	Items per page (default 20)
page	integer	Page number

Example — list March 2026 payroll:

```
GET /api/v1/financial/payroll?month=3&year=2026
Authorization: Bearer {token}
X-Subdomain: greenfield
```

Response 200 OK :

```
{
  "data": [
    {
      "id": 22,
      "staff_id": 14,
      "staff_name": "Mr. Chukwuemeka Obi",
      "staff_role": "teacher",
      "department": "Mathematics",
      "month": 3,
      "year": 2026,
      "period": "March 2026",
      "basic_salary": "120000.00",
      "allowances": "30000.00",
      "deductions": "18000.00",
      "net_salary": "132000.00",
      "payment_date": "2026-03-29",
      "payment_method": "bank_transfer",
      "status": "paid",
      "processed_by": "Mrs. Adaobi Nwosu",
      "notes": "March 2026 salary",
      "created_at": "2026-03-25T09:00:00Z"
    },
    {
      "id": 23,
      "staff_id": 15,
      "staff_name": "Miss Ngozi Eze",
      "staff_role": "teacher",
      "department": "English",
      "month": 3,
      "year": 2026,
      "period": "March 2026",
      "basic_salary": "95000.00",
      "allowances": "20000.00",
      "deductions": "14250.00",
      "net_salary": "100750.00",
      "payment_date": null,
      "payment_method": "bank_transfer",
      "status": "pending",
      "processed_by": "Mrs. Adaobi Nwosu",
      "notes": null,
      "created_at": "2026-03-25T09:05:00Z"
    }
  ],
  "meta": {
    "current_page": 1,
    "per_page": 20,
    "total": 64
  },
  "summary": {
    "period": "March 2026",
    "total_staff": 64,
    "total_gross": 7800000,
    "total_deductions": 975000,
    "total_net": 6825000,
    "paid_count": 48,
    "pending_count": 16,
    "cancelled_count": 0
  }
}
```

```
}  
}
```

Create Payroll

```
POST /api/v1/financial/payroll  
Authorization: Bearer {token}  
X-Subdomain: greenfield  
Content-Type: application/json
```

Request Body:

```
{  
  "staff_id": 14,  
  "month": 3,  
  "year": 2026,  
  "basic_salary": 120000,  
  "allowances": 30000,  
  "deductions": 18000,  
  "payment_method": "bank_transfer",  
  "notes": "March 2026 salary"  
}
```

Field	Required	Notes
staff_id	Yes	Must be a valid user in this school
month	Yes	1-12
year	Yes	4-digit year
basic_salary	Yes	Must be > 0
allowances	No	Defaults to 0
deductions	No	Defaults to 0
payment_method	No	Defaults to bank_transfer
notes	No	Optional remarks

Response 201 Created :

```
{
  "message": "Payroll created successfully",
  "payroll": {
    "id": 22,
    "staff_id": 14,
    "staff_name": "Mr. Chukwuemeka Obi",
    "month": 3,
    "year": 2026,
    "period": "March 2026",
    "basic_salary": "120000.00",
    "allowances": "30000.00",
    "deductions": "18000.00",
    "net_salary": "132000.00",
    "payment_date": null,
    "payment_method": "bank_transfer",
    "status": "pending",
    "processed_by": 1,
    "processed_by_name": "Mrs. Adaobi Nwosu",
    "notes": "March 2026 salary",
    "created_at": "2026-03-25T09:00:00Z"
  }
}
```

Response 409 Conflict (duplicate month/year for same staff):

```
{
  "message": "A payroll record already exists for Mr. Chukwuemeka Obi for March 2026. Use the update endpoint to modify it."
}
```

Response 422 Unprocessable Entity :

```
{
  "message": "The given data was invalid.",
  "errors": {
    "staff_id": ["The selected staff member does not belong to this school."],
    "month": ["The month must be between 1 and 12."]
  }
}
```

Get Payroll Details

```
GET /api/v1/financial/payroll/{id}
Authorization: Bearer {token}
X-Subdomain: greenfield
```

Response 200 OK :


```
{
  "payroll": {
    "id": 22,
    "staff": {
      "id": 14,
      "name": "Mr. Chukwuemeka Obi",
      "email": "c.obi@greenfieldacademy.edu.ng",
      "role": "teacher",
      "department": "Mathematics",
      "bank_name": "First Bank Nigeria",
      "bank_account_number": "3012345678"
    },
    "school_id": 1,
    "month": 3,
    "year": 2026,
    "period": "March 2026",
    "basic_salary": "120000.00",
    "allowances": "30000.00",
    "deductions": "18000.00",
    "net_salary": "132000.00",
    "payment_date": "2026-03-29",
    "payment_method": "bank_transfer",
    "status": "paid",
    "processed_by": {
      "id": 1,
      "name": "Mrs. Adaobi Nwosu"
    },
    "notes": "March 2026 salary",
    "created_at": "2026-03-25T09:00:00Z",
    "updated_at": "2026-03-29T14:00:00Z"
  }
}
```

Update Payroll

Used to adjust salary components while status is `pending` , or to mark a record as paid after bank transfer.

```
PUT /api/v1/financial/payroll/{id}
Authorization: Bearer {token}
X-Subdomain: greenfield
Content-Type: application/json
```

Request Body — adjust deductions:

```
{
  "deductions": 20000,
  "notes": "Pension deduction updated per March schedule"
}
```

Response 200 OK :

```
{
  "message": "Payroll updated successfully",
  "payroll": {
    "id": 22,
    "basic_salary": "120000.00",
    "allowances": "30000.00",
    "deductions": "20000.00",
    "net_salary": "130000.00",
    "status": "pending",
    "updated_at": "2026-03-26T10:00:00Z"
  }
}
```

Request Body — mark as paid:

```
{
  "status": "paid",
  "payment_date": "2026-03-29",
  "payment_method": "bank_transfer"
}
```

Response 200 OK :

```
{
  "message": "Payroll marked as paid",
  "payroll": {
    "id": 22,
    "status": "paid",
    "payment_date": "2026-03-29",
    "payment_method": "bank_transfer",
    "updated_at": "2026-03-29T14:00:00Z"
  }
}
```

Response 422 Unprocessable Entity (attempt to update a paid record's salary):

```
{
  "message": "Salary components cannot be changed after a payroll is marked as paid. Cancel and recreate if correction is needed."
}
```

Delete Payroll

```
DELETE /api/v1/financial/payroll/{id}
Authorization: Bearer {token}
X-Subdomain: greenfield
```

Response 200 OK :

```
{
  "message": "Payroll record deleted successfully"
}
```

Response 409 Conflict (cannot delete paid record):

```
{
  "message": "Cannot delete a paid payroll record. Cancel it first if a correction is needed."
}
```

Get Pay Stub

Returns the full pay stub data for a given payroll record. This is used to render a printable/downloadable pay stub in the frontend.

```
GET /api/v1/financial/payroll/{id}/pay-stub
Authorization: Bearer {token}
X-Subdomain: greenfield
```

Response 200 OK (full pay stub JSON):

```
{
  "pay_stub": {
    "stub_id": "PS-2026-0022",
    "generated_at": "2026-03-30T17:00:00Z",

    "school": {
      "name": "Greenfield Academy",
      "address": "14 Victoria Island, Lagos",
      "phone": "+2348012345678",
      "email": "info@greenfieldacademy.edu.ng",
      "logo_url": "https://cdn.compass.net/schools/greenfield/logo.png"
    },

    "employee": {
      "id": 14,
      "name": "Mr. Chukwuemeka Obi",
      "email": "c.obi@greenfieldacademy.edu.ng",
      "role": "Teacher",
      "department": "Mathematics",
      "employment_type": "full_time",
      "bank_name": "First Bank Nigeria",
      "bank_account_number": "3012345678"
    },

    "period": {
      "month": 3,
      "year": 2026,
      "label": "March 2026",
      "payment_date": "2026-03-29"
    },

    "earnings": {
      "basic_salary": 120000,
      "allowances": 30000,
      "gross_salary": 150000
    },

    "deductions": {
      "total": 18000,
      "breakdown": {
        "tax_paye": 12000,
        "pension": 6000,
        "loan_repayment": 0,
        "other": 0
      }
    },

    "net_salary": 132000,

    "payment_info": {
      "method": "bank_transfer",
      "status": "paid",
      "reference": null
    },

    "processed_by": "Mrs. Adaobi Nwosu",
    "notes": "March 2026 salary"
  }
}
```

```
}  
}
```

Response 403 Forbidden (staff accessing another staff's stub):

```
{  
  "message": "You do not have permission to view this pay stub."  
}
```

Response 404 Not Found :

```
{  
  "message": "Payroll record not found."  
}
```

Business Rules

1. **Duplicate prevention** — The system blocks creating a second payroll record for the same `staff_id + month + year` combination
 2. **Delete guard** — Paid payrolls cannot be deleted (status= `paid`); only `pending` and `cancelled` records can be removed
 3. **Auto net salary** — `net_salary` is always `basic_salary + allowances - deductions`; it cannot be set manually
 4. **Salary lock on paid** — Salary component fields (`basic_salary` , `allowances` , `deductions`) cannot be changed once status is `paid`
 5. **Processed by** — `processed_by` is auto-set to the authenticated user on creation
 6. **School scoping** — All payroll queries are scoped to the current tenant's school
 7. **Staff access** — Staff members (role: `teacher` , `staff` , etc.) can only view their own pay stubs via `GET /payroll/{id}/pay-stub`; they cannot list all payroll records or see other staff pay data
-

Typical Monthly Payroll Workflow

1. Accountant creates payroll record for each staff member
POST /payroll { staff_id, month, year, basic_salary, allowances, deductions }
→ status = pending, net_salary auto-calculated
2. Admin reviews and optionally adjusts deductions
PUT /payroll/{id} { deductions: 20000 }
→ net_salary recalculated automatically
3. Bank transfers are made externally (outside the system)
4. Accountant marks each record as paid
PUT /payroll/{id} { "status": "paid", "payment_date": "2026-03-29" }
5. Staff view their pay stub
GET /payroll/{id}/pay-stub
→ Full pay stub JSON returned for rendering

Frontend Integration — Payroll Listing and Pay Stub Modal

Step 1 — Check Module Access on Route Mount

```
// pages/payroll/index.tsx
import { hasModule } from '@services/modules';
import { getSubdomain } from '@utils/tenancy';

export default function PayrollPage() {
  const subdomain = getSubdomain();

  if (!hasModule(subdomain, 'fee_management')) {
    return <UpgradePrompt module="fee_management" />;
  }

  return <PayrollList subdomain={subdomain} />;
}
```

Step 2 — Payroll List with Month/Year Selectors

```
// components/PayrollList.tsx
import { useState, useEffect } from 'react';
import { createApiClient } from '@services/api';

const MONTHS = [
  { value: 1, label: 'January' }, { value: 2, label: 'February' },
  { value: 3, label: 'March' }, { value: 4, label: 'April' },
  { value: 5, label: 'May' }, { value: 6, label: 'June' },
  { value: 7, label: 'July' }, { value: 8, label: 'August' },
  { value: 9, label: 'September' }, { value: 10, label: 'October' },
  { value: 11, label: 'November' }, { value: 12, label: 'December' },
];

export function PayrollList({ subdomain }: { subdomain: string }) {
  const currentDate = new Date();
  const [month, setMonth] = useState(currentDate.getMonth() + 1);
  const [year, setYear] = useState(currentDate.getFullYear());
  const [payrolls, setPayrolls] = useState([]);
  const [summary, setSummary] = useState(null);
  const [loading, setLoading] = useState(true);
  const [selectedPayroll, setSelectedPayroll] = useState(null); // for pay stub modal

  const api = createApiClient(subdomain);

  useEffect(() => {
    setLoading(true);
    api.get(`/financial/payroll?month=${month}&year=${year}&per_page=100`)
      .then(data => {
        setPayrolls(data.data);
        setSummary(data.summary);
      })
      .finally(() => setLoading(false));
  }, [month, year]);

  // Generate year options – current year back 3 years
  const yearOptions = Array.from({ length: 4 }, (_, i) => year - i);

  return (
    <div>
      <div className="filters">
        <select value={month} onChange={e => setMonth(Number(e.target.value))}>
          {MONTHS.map(m => (
            <option key={m.value} value={m.value}>{m.label}</option>
          ))}
        </select>
        <select value={year} onChange={e => setYear(Number(e.target.value))}>
          {yearOptions.map(y => (
            <option key={y} value={y}>{y}</option>
          ))}
        </select>
      </div>

      {summary && (
        <div className="payroll-summary">
          <span>Total Gross: $${summary.total_gross.toLocaleString()}</span>
        </div>
      )}
    </div>
  );
}
```


Step 3 — Mark as Paid

```
async function markAsPaid(payloadId: number) {
  const subdomain = getSubdomain(!);
  const api = createApiClient(subdomain);

  const today = new Date().toISOString().split('T')[0];

  try {
    await api.put(`/financial/payroll/${payloadId}`, {
      status: 'paid',
      payment_date: today,
    });

    toast.success('Payroll marked as paid');
    // Re-fetch payrolls for the current month/year
    refetchPayrolls();
  } catch (err) {
    toast.error(err.message || 'Failed to update payroll status');
  }
}
```

Step 4 — Pay Stub Modal

```
// components/PayStubModal.tsx
import { useEffect, useState } from 'react';
import { createApiClient } from '@services/api';

export function PayStubModal({
  payrollId,
  onClose,
}): {
  payrollId: number;
  onClose: () => void;
} {
  const subdomain = getSubdomain();
  const api = createApiClient(subdomain);
  const [stub, setStub] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    api.get(`/financial/payroll/${payrollId}/pay-stub`)
      .then(data => setStub(data.pay_stub))
      .finally(() => setLoading(false));
  }, [payrollId]);

  if (loading) return <Modal onClose={onClose}><Spinner /></Modal>;

  return (
    <Modal onClose={onClose} printable>
      <div className="pay-stub" id="pay-stub-printable">
        {/* School Header */}
        <div className="stub-header">
          <img src={stub.school.logo_url} alt="School logo" />
          <h2>{stub.school.name}</h2>
          <p>{stub.school.address}</p>
        </div>

        <h3>PAY STUB — {stub.period.label}</h3>
        <p>Stub ID: {stub.stub_id}</p>

        {/* Employee Info */}
        <section>
          <h4>Employee Details</h4>
          <table>
            <tbody>
              <tr><td>Name</td><td>{stub.employee.name}</td></tr>
              <tr><td>Role</td><td>{stub.employee.role}</td></tr>
              <tr><td>Department</td><td>{stub.employee.department}</td></tr>
              <tr><td>Bank</td><td>{stub.employee.bank_name}</td></tr>
              <tr><td>Account</td><td>{stub.employee.bank_account_number}</td></tr>
            </tbody>
          </table>
        </section>

        {/* Earnings */}
        <section>
          <h4>Earnings</h4>
          <table>
```

```

        <tbody>
          <tr><td>Basic Salary</td><td>₹{stub.earnings.basic_salary.toLocaleString()}</td>
</tr>

          <tr><td>Allowances</td><td>₹{stub.earnings.allowances.toLocaleString()}</td></tr>
          <tr className="total"><td>Gross Salary</td><td>₹
{stub.earnings.gross_salary.toLocaleString()}</td></tr>
        </tbody>
      </table>
</section>

    {/* Deductions */}
    <section>
      <h4>Deductions</h4>
      <table>
        <tbody>
          <tr><td>PAYE Tax</td><td>₹{stub.deductions.breakdown.tax_paye.toLocaleString()}
</td></tr>

          <tr><td>Pension</td><td>₹{stub.deductions.breakdown.pension.toLocaleString()}</td>
</tr>

          {stub.deductions.breakdown.loan_repayment > 0 && (
            <tr><td>Loan Repayment</td><td>₹
{stub.deductions.breakdown.loan_repayment.toLocaleString()}</td></tr>
          )}
          <tr className="total"><td>Total Deductions</td><td>₹
{stub.deductions.total.toLocaleString()}</td></tr>
        </tbody>
      </table>
    </section>

    {/* Net Pay */}
    <div className="net-pay">
      <strong>NET PAY: ₹{stub.net_salary.toLocaleString()}</strong>
    </div>

    <div className="stub-footer">
      <p>Payment Method: {stub.payment_info.method.replace('_', ' ')}</p>
      <p>Payment Date: {stub.period.payment_date ?? 'Pending'}</p>
      <p>Processed by: {stub.processed_by}</p>
      <p>Generated: {new Date(stub.generated_at).toLocaleString()}</p>
    </div>
  </div>

  <button onClick={() => window.print()}>Print Pay Stub</button>
</Modal>
);
}

```

Step 5 — Create Payroll Form

```
// components/CreatePayrollForm.tsx

async function handleCreatePayroll(formData: {
  staffId: number;
  month: number;
  year: number;
  basicSalary: number;
  allowances: number;
  deductions: number;
  notes: string;
}) {
  const subdomain = getSubdomain(!);
  const api = createApiClient(subdomain);

  try {
    const result = await api.post('/financial/payroll', {
      staff_id: formData.staffId,
      month: formData.month,
      year: formData.year,
      basic_salary: formData.basicSalary,
      allowances: formData.allowances,
      deductions: formData.deductions,
      notes: formData.notes,
    });

    // Show calculated net salary to user before closing form
    toast.success(
      `Payroll created. Net salary: ₦${Number(result.payroll.net_salary).toLocaleString()}`
    );
    onSuccess();
  } catch (err) {
    if (err.message?.includes('already exists')) {
      toast.error(err.message); // Duplicate month/year
    } else {
      toast.error('Failed to create payroll record');
    }
  }
}
```

Pay Stub Print Styles (CSS)

```
/* Only shown when printing */
@media print {
  body * { visibility: hidden; }
  #pay-stub-printable,
  #pay-stub-printable * { visibility: visible; }
  #pay-stub-printable { position: fixed; top: 0; left: 0; width: 100%; }
  button { display: none !important; }
}
```

Bulk Payroll Creation (Batch Flow)

For schools with many staff, create payroll for all staff members at once:

```
// Fetch all active staff first
const staffList = await api.get('/staff?per_page=200&status=active');

// Create payroll records in sequence (or parallel in batches)
const results = await Promise.allSettled(
  staffList.data.map((staff: any) =>
    api.post('/financial/payroll', {
      staff_id: staff.id,
      month: selectedMonth,
      year: selectedYear,
      basic_salary: staff.basic_salary, // pre-loaded from staff profile
      allowances: staff.allowances ?? 0,
      deductions: staff.deductions ?? 0,
    })
  )
);

const succeeded = results.filter(r => r.status === 'fulfilled').length;
const failed = results.filter(r => r.status === 'rejected').length;

toast.info(`Payroll created for ${succeeded} staff members. ${failed} failed (duplicates or errors).`);
```

Inventory Management

Base URL: `https://{subdomain}.compasse.net/api/v1/` **Auth:** `Authorization: Bearer {token}`
required on all protected endpoints **Module gate:** `inventory_management`

Overview

The inventory module tracks school assets and consumables — books, lab equipment, sports gear, stationery, furniture — including purchase history, checkout loans, and returns.

User Stories

As a school admin, I want to categorise all school assets and track quantities so I always know what we have in stock.

As a librarian / store keeper, I want to check out items to students or staff and record when items are returned so nothing goes missing.

As an admin, I want to be alerted when item quantities fall below the minimum threshold so I can reorder before stock runs out.

As an accountant, I want to record new purchases and disposals so the asset register stays accurate.

Models

InventoryCategory

Field	Type	Description
id	integer	Auto-increment primary key
name	string	Category name (e.g. “Lab Equipment”, “Stationery”)
description	string null	Optional description
color	string	UI colour hex for display (e.g. #4CAF50)
created_at	timestamp	Creation timestamp
updated_at	timestamp	Last update timestamp

InventoryItem

Field	Type	Description
id	integer	Auto-increment primary key
category_id	integer	Parent category FK
name	string	Item name
sku	string	Stock-keeping unit (unique per school)
quantity	integer	Current quantity in stock
unit	string	Unit of measurement (pcs , kg , litres , etc.)
min_quantity	integer	Low-stock threshold
unit_price	decimal	Cost per unit
location	string null	Physical storage location
supplier	string null	Supplier name
status	enum	active , inactive , discontinued

isLowStock() returns true when quantity <= min_quantity .

InventoryTransaction

Field	Type	Description
id	integer	Auto-increment primary key
item_id	integer	Item involved
type	enum	purchase , checkout , return , adjustment , disposal
quantity	integer	Units involved
remaining_quantity	integer	Stock level after this transaction (snapshot)
borrower_id	integer null	Who borrowed (for checkout)
borrower_type	string null	Polymorphic: App\Models\Student , App\Models\Staff , App\Models\Teacher
borrower_name	string null	Display name of borrower
purpose	string null	Reason for checkout/adjustment
expected_return_date	date null	When item should be returned
returned_at	timestamp null	Actual return timestamp
notes	string null	Additional notes
status	enum	completed , checked_out , returned
recorded_by	integer	FK → User who recorded this

API Endpoints

Base path: /api/v1/inventory/

Categories

Method	Endpoint	Description
GET	/categories	List categories (with item count)
POST	/categories	Create category
GET	/categories/{id}	Get single category
PUT	/categories/{id}	Update category
DELETE	/categories/{id}	Delete (blocked if items exist in category)

Items

Method	Endpoint	Description
GET	/items	List items (filter: category_id , status , low_stock=1 , search)
POST	/items	Create item
GET	/items/{id}	Get item + transaction history + low stock flag
PUT	/items/{id}	Update item
DELETE	/items/{id}	Delete (blocked if transaction history exists)

Transactions

Method	Endpoint	Description
GET	/transactions	List transactions (filter: item_id , type , status , borrower_id)
POST	/transactions	Record purchase / adjustment / disposal
GET	/transactions/{id}	Get transaction details
PUT	/transactions/{id}	Update notes/purpose
DELETE	/transactions/{id}	Delete transaction record
POST	/transactions/checkout	Checkout item to a borrower
POST	/transactions/{id}/return	Return a checked-out item

Request / Response Examples

Categories

CREATE CATEGORY

Request


```
POST /api/v1/inventory/categories HTTP/1.1
Host: greenfield.compassse.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
Content-Type: application/json
```

```
{
  "name": "Lab Equipment",
  "description": "Science laboratory tools and apparatus",
  "color": "#2196F3"
}
```

Response HTTP 201 Created

```
{
  "message": "Category created successfully",
  "category": {
    "id": 3,
    "name": "Lab Equipment",
    "description": "Science laboratory tools and apparatus",
    "color": "#2196F3",
    "items_count": 0,
    "created_at": "2026-03-30T08:15:00Z",
    "updated_at": "2026-03-30T08:15:00Z"
  }
}
```

LIST CATEGORIES

Request

```
GET /api/v1/inventory/categories HTTP/1.1
Host: greenfield.compassse.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
```

Response HTTP 200 OK

```
{
  "data": [
    {
      "id": 1,
      "name": "Stationery",
      "description": "Pens, notebooks, markers",
      "color": "#FF9800",
      "items_count": 12
    },
    {
      "id": 2,
      "name": "Sports Equipment",
      "description": "Football, basketball, nets",
      "color": "#4CAF50",
      "items_count": 8
    },
    {
      "id": 3,
      "name": "Lab Equipment",
      "description": "Science laboratory tools and apparatus",
      "color": "#2196F3",
      "items_count": 0
    }
  ],
  "meta": {
    "total": 3
  }
}
```

UPDATE CATEGORY

Request

```
PUT /api/v1/inventory/categories/3 HTTP/1.1
Host: greenfield.compass.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
Content-Type: application/json
```

```
{
  "name": "Laboratory Equipment",
  "color": "#1565C0"
}
```

Response HTTP 200 OK

```
{
  "message": "Category updated successfully",
  "category": {
    "id": 3,
    "name": "Laboratory Equipment",
    "description": "Science laboratory tools and apparatus",
    "color": "#1565C0",
    "items_count": 0
  }
}
```

DELETE CATEGORY

Request

```
DELETE /api/v1/inventory/categories/3 HTTP/1.1
Host: greenfield.compassse.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
```

Response HTTP 200 OK

```
{
  "message": "Category deleted successfully"
}
```

Blocked response (category has items) HTTP 422 Unprocessable Entity

```
{
  "message": "Cannot delete category with existing items. Remove or reassign items first."
}
```

Items

CREATE ITEM

Request

```
POST /api/v1/inventory/items HTTP/1.1
Host: greenfield.compassse.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
Content-Type: application/json
```

```
{
  "category_id": 2,
  "name": "Bunsen Burner",
  "sku": "LAB-BB-001",
  "quantity": 20,
  "unit": "pcs",
  "min_quantity": 5,
  "unit_price": 4500,
  "location": "Science Lab Store",
  "supplier": "EduSupply Ltd",
  "status": "active"
}
```

Response HTTP 201 Created

```
{
  "message": "Item created successfully",
  "item": {
    "id": 7,
    "category_id": 2,
    "category": {
      "id": 2,
      "name": "Lab Equipment",
      "color": "#2196F3"
    },
    "name": "Bunsen Burner",
    "sku": "LAB-BB-001",
    "quantity": 20,
    "unit": "pcs",
    "min_quantity": 5,
    "unit_price": "4500.00",
    "location": "Science Lab Store",
    "supplier": "EduSupply Ltd",
    "status": "active",
    "is_low_stock": false,
    "created_at": "2026-03-30T09:00:00Z"
  }
}
```

LIST ITEMS

Request

```
GET /api/v1/inventory/items?category_id=2&status=active HTTP/1.1
Host: greenfield.compass.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
```

Response HTTP 200 OK

```
{
  "data": [
    {
      "id": 7,
      "name": "Bunsen Burner",
      "sku": "LAB-BB-001",
      "category": { "id": 2, "name": "Lab Equipment" },
      "quantity": 20,
      "unit": "pcs",
      "min_quantity": 5,
      "unit_price": "4500.00",
      "location": "Science Lab Store",
      "status": "active",
      "is_low_stock": false
    },
    {
      "id": 8,
      "name": "Test Tubes (pack of 10)",
      "sku": "LAB-TT-010",
      "category": { "id": 2, "name": "Lab Equipment" },
      "quantity": 3,
      "unit": "pack",
      "min_quantity": 5,
      "unit_price": "1200.00",
      "location": "Science Lab Store",
      "status": "active",
      "is_low_stock": true
    }
  ],
  "meta": {
    "total": 2,
    "per_page": 15,
    "current_page": 1,
    "last_page": 1
  }
}
```

LIST LOW-STOCK ITEMS

Request

```
GET /api/v1/inventory/items?low_stock=1 HTTP/1.1
Host: greenfield.compass.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
```

Response HTTP 200 OK

```
{
  "data": [
    {
      "id": 8,
      "name": "Test Tubes (pack of 10)",
      "sku": "LAB-TT-010",
      "quantity": 3,
      "min_quantity": 5,
      "unit": "pack",
      "is_low_stock": true,
      "category": { "id": 2, "name": "Lab Equipment" }
    },
    {
      "id": 11,
      "name": "A4 Printer Paper",
      "sku": "STN-PP-A4",
      "quantity": 2,
      "min_quantity": 10,
      "unit": "ream",
      "is_low_stock": true,
      "category": { "id": 1, "name": "Stationery" }
    }
  ],
  "meta": {
    "total": 2
  }
}
```

GET SINGLE ITEM

Request

```
GET /api/v1/inventory/items/7 HTTP/1.1
Host: greenfield.compass.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
```

Response HTTP 200 OK

```
{
  "data": {
    "id": 7,
    "name": "Bunsen Burner",
    "sku": "LAB-BB-001",
    "category": { "id": 2, "name": "Lab Equipment", "color": "#2196F3" },
    "quantity": 18,
    "unit": "pcs",
    "min_quantity": 5,
    "unit_price": "4500.00",
    "location": "Science Lab Store",
    "supplier": "EduSupply Ltd",
    "status": "active",
    "is_low_stock": false,
    "recent_transactions": [
      {
        "id": 18,
        "type": "checkout",
        "quantity": 2,
        "remaining_quantity": 18,
        "borrower_name": "Emeka Okafor",
        "purpose": "Lab practical – SS2B",
        "status": "checked_out",
        "created_at": "2026-03-28T10:00:00Z"
      }
    ]
  }
}
```

UPDATE ITEM

Request

```
PUT /api/v1/inventory/items/7 HTTP/1.1
Host: greenfield.compass.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
Content-Type: application/json

{
  "min_quantity": 8,
  "location": "Physics Lab Store",
  "unit_price": 4800
}
```

Response HTTP 200 OK

```
{
  "message": "Item updated successfully",
  "item": {
    "id": 7,
    "name": "Bunsen Burner",
    "min_quantity": 8,
    "location": "Physics Lab Store",
    "unit_price": "4800.00",
    "is_low_stock": false
  }
}
```

DELETE ITEM

Request

```
DELETE /api/v1/inventory/items/7 HTTP/1.1
Host: greenfield.compass.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
```

Response (no transaction history) HTTP 200 OK

```
{
  "message": "Item deleted successfully"
}
```

Blocked response (has transaction history) HTTP 422 Unprocessable Entity

```
{
  "message": "Cannot delete item with existing transaction history."
}
```

Transactions

RECORD A PURCHASE (INCREASES STOCK)

Request

```
POST /api/v1/inventory/transactions HTTP/1.1
Host: greenfield.compass.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
Content-Type: application/json

{
  "item_id": 7,
  "type": "purchase",
  "quantity": 10,
  "purpose": "Restocking after term break",
  "notes": "Purchased from EduSupply Ltd, invoice #INV-2026-0112"
}
```

Response HTTP 201 Created


```
{
  "message": "Transaction recorded successfully",
  "transaction": {
    "id": 22,
    "item_id": 7,
    "item": { "name": "Bunsen Burner", "sku": "LAB-BB-001" },
    "type": "purchase",
    "quantity": 10,
    "remaining_quantity": 28,
    "purpose": "Restocking after term break",
    "notes": "Purchased from EduSupply Ltd, invoice #INV-2026-0112",
    "status": "completed",
    "recorded_by": { "id": 3, "name": "Mr. Segun Adeyemi" },
    "created_at": "2026-03-30T11:00:00Z"
  }
}
```

CHECKOUT ITEM TO A BORROWER

Request

```
POST /api/v1/inventory/transactions/checkout HTTP/1.1
Host: greenfield.compass.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
Content-Type: application/json
```

```
{
  "item_id": 7,
  "quantity": 2,
  "borrower_id": 45,
  "borrower_type": "App\\Models\\Student",
  "borrower_name": "Emeka Okafor",
  "purpose": "Lab practical – SS2B",
  "expected_return_date": "2026-04-10"
}
```

Response HTTP 201 Created

```
{
  "message": "Item checked out successfully",
  "transaction": {
    "id": 18,
    "item_id": 7,
    "item": {
      "id": 7,
      "name": "Bunsen Burner",
      "quantity": 18,
      "is_low_stock": false
    },
    "type": "checkout",
    "quantity": 2,
    "remaining_quantity": 18,
    "borrower_id": 45,
    "borrower_type": "App\\Models\\Student",
    "borrower_name": "Emeka Okafor",
    "purpose": "Lab practical – SS2B",
    "expected_return_date": "2026-04-10",
    "returned_at": null,
    "status": "checked_out",
    "recorded_by": { "id": 3, "name": "Mr. Segun Adeyemi" },
    "created_at": "2026-03-28T10:00:00Z"
  }
}
```

Insufficient stock HTTP 422 Unprocessable Entity

```
{
  "message": "Insufficient stock. Available: 1, Requested: 2"
}
```

RETURN A CHECKED-OUT ITEM

Request

```
POST /api/v1/inventory/transactions/18/return HTTP/1.1
Host: greenfield.compass.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
Content-Type: application/json
```

```
{
  "notes": "Returned in good condition"
}
```

Response HTTP 200 OK

```
{
  "message": "Item returned successfully",
  "transaction": {
    "id": 18,
    "type": "checkout",
    "quantity": 2,
    "remaining_quantity": 18,
    "borrower_name": "Emeka Okafor",
    "status": "returned",
    "returned_at": "2026-04-09T14:30:00Z",
    "notes": "Returned in good condition",
    "item": {
      "id": 7,
      "name": "Bunsen Burner",
      "quantity": 20
    }
  }
}
```

RECORD AN ADJUSTMENT

Request

```
POST /api/v1/inventory/transactions HTTP/1.1
Host: greenfield.compass.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
Content-Type: application/json

{
  "item_id": 7,
  "type": "adjustment",
  "quantity": -3,
  "purpose": "Damaged during storage – written off after audit",
  "notes": "Three units found cracked during term-end stock count"
}
```

Response HTTP 201 Created

```
{
  "message": "Transaction recorded successfully",
  "transaction": {
    "id": 25,
    "type": "adjustment",
    "quantity": -3,
    "remaining_quantity": 17,
    "purpose": "Damaged during storage – written off after audit",
    "status": "completed",
    "created_at": "2026-03-30T14:00:00Z"
  }
}
```

LIST TRANSACTIONS

Request

```
GET /api/v1/inventory/transactions?item_id=7&type=checkout&status=checked_out HTTP/1.1
Host: greenfield.compass.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
```

Response HTTP 200 OK

```
{
  "data": [
    {
      "id": 18,
      "type": "checkout",
      "quantity": 2,
      "remaining_quantity": 18,
      "borrower_name": "Emeka Okafor",
      "borrower_type": "App\\Models\\Student",
      "purpose": "Lab practical – SS2B",
      "expected_return_date": "2026-04-10",
      "returned_at": null,
      "status": "checked_out",
      "item": { "id": 7, "name": "Bunsen Burner" },
      "created_at": "2026-03-28T10:00:00Z"
    }
  ],
  "meta": {
    "total": 1,
    "per_page": 15,
    "current_page": 1
  }
}
```

Business Rules

1. **Atomic quantity changes** — All checkout and return operations use `DB::transaction()` to prevent race conditions.
 2. **Insufficient stock guard** — Checkout is rejected if `item.quantity < requested_quantity`.
 3. **Return guard** — Only open `checkout` transactions (`status = checked_out`) can be returned.
 4. **Category guard** — A category cannot be deleted while it has items.
 5. **Transaction history guard** — An item cannot be deleted if it has any transaction records.
 6. **Snapshot quantity** — `remaining_quantity` is recorded on every transaction as a point-in-time snapshot, enabling a full audit trail even if the item is later adjusted.
 7. **Negative adjustment** — Adjustments accept negative `quantity` values for write-offs and disposals.
-

Frontend Integration

How the Frontend Handles Tenancy

The inventory module lives under the authenticated tenant context. When the frontend makes API calls:

1. **Subdomain routing** — The app is loaded at `https://{school}.compasse.net` . All API requests go to `https://{school}.compasse.net/api/v1/inventory/...` . The subdomain is part of the origin URL so no extra header is needed.
2. **Bearer token** — After login, the token is stored (e.g. in `localStorage` or a secure cookie). Every request includes `Authorization: Bearer {token}` .
3. **Module gate check** — On app load, the frontend checks the school's enabled modules. If `inventory_management` is not in the list, the Inventory nav item is hidden entirely.

Inventory Dashboard — Low-Stock Alerts

```
GET /api/v1/inventory/items?low_stock=1
```

The dashboard fetches low-stock items on load and displays a dismissible alert banner:

```
"5 items are running low — [View All]"
```

The alert card shows `item.name` , `item.quantity / item.min_quantity` , and the category colour badge.

Checkout Form with Borrower Search

The checkout form works in two steps:

1. **Search borrower** — A debounced input calls `GET /api/v1/students?search={query}` (or `staff/teachers`). The user selects a result which populates `borrower_id` , `borrower_type` , and `borrower_name` in the form state.
2. **Submit checkout** — `POST /api/v1/inventory/transactions/checkout` with the assembled payload. On `HTTP 201` , a success toast is shown and the item's quantity badge updates in real time.

Return Confirmation

The active-checkouts table has a "Return" button per row. Clicking it opens a confirmation modal:

```
"Confirm return of 2 × Bunsen Burner from Emeka Okafor?"
```

On confirmation the frontend calls `POST /api/v1/inventory/transactions/{id}/return` . On success, the row moves from "Checked Out" to "Returned" with the `returned_at` timestamp displayed.

Transport Management

Base URL: `https://{subdomain}.compasse.net/api/v1/` **Auth:** Authorization: Bearer {token}
required on all protected endpoints **Module gate:** `transport_management`

Overview

The transport module manages school vehicles, drivers, bus routes, student route assignments, and a secure pickup system for verifying authorised persons collecting students.

User Stories

As a school admin, I want to register all school vehicles and their insurance/service dates so I can track compliance.

As a transport coordinator, I want to create bus routes, assign a vehicle and driver, and enroll students on specific routes so parents know which bus their child rides.

As a security officer / driver, I want to verify the identity of a person collecting a student by entering their unique pickup code so only authorised persons can take students.

As a parent, I want to register an authorised pickup person with their photo and relationship, and get a pickup code to give them.

Models

Vehicle

Field	Type	Description
id	integer	Auto-increment primary key
make	string	Manufacturer (e.g. "Toyota")
model	string	Model name (e.g. "Coaster")
year	integer	Year of manufacture
plate_number	string	Unique registration number
capacity	integer	Passenger capacity
type	enum	bus , van , car , minibus
status	enum	active , inactive , maintenance
insurance_expiry	date null	Insurance expiry date
last_service_date	date null	Last maintenance date

Driver

Field	Type	Description
id	integer	Auto-increment primary key
user_id	integer null	Linked user account (optional)
name	string	Full name
license_number	string	Driver's licence number (unique per school)
license_expiry	date null	Licence expiry date
phone	string	Contact number
status	enum	active , inactive , suspended

TransportRoute

Field	Type	Description
id	integer	Auto-increment primary key
vehicle_id	integer	Assigned vehicle FK
driver_id	integer	Assigned driver FK
name	string	Route display name
route_code	string	Short unique code (e.g. RT-IKJ-01)
start_point	string	Origin terminal location
end_point	string	Destination terminal location
stops	JSON	Array of intermediate stop names
distance_km	decimal null	Route distance in kilometres
fare	decimal null	Student fare amount
morning_pickup_time	string	HH:MM format
afternoon_dropoff_time	string	HH:MM format
status	enum	active , inactive

Students are linked to routes via `student_transport_routes` pivot with `pickup_stop` and `dropoff_stop` .

SecurePickup

Field	Type	Description
id	integer	Auto-increment primary key
student_id	integer	Student who can be collected
authorized_name	string	Authorised person's full name
authorized_phone	string	Their phone number
relationship	string	e.g. Mother , Uncle , Guardian
authorized_photo	string null	Photo URL (uploaded via separate file endpoint)
pickup_code	string	Unique 8-character uppercase code (auto-generated)
status	enum	active , inactive

API Endpoints

Base path: `/api/v1/transport/`

Vehicles

Method	Endpoint	Description
GET	/vehicles	List vehicles (filter: status , type , search)
POST	/vehicles	Register vehicle
GET	/vehicles/{id}	Get vehicle + assigned routes
PUT	/vehicles/{id}	Update vehicle
DELETE	/vehicles/{id}	Delete (blocked if on active route)

Drivers

Method	Endpoint	Description
GET	/drivers	List drivers (filter: status , search)
POST	/drivers	Register driver
GET	/drivers/{id}	Get driver + assigned routes
PUT	/drivers/{id}	Update driver
DELETE	/drivers/{id}	Delete (blocked if on active route)

Routes

Method	Endpoint	Description
GET	/routes	List routes (filter: status , driver_id , vehicle_id)
POST	/routes	Create route
GET	/routes/{id}	Get route + vehicle + driver + students
PUT	/routes/{id}	Update route
DELETE	/routes/{id}	Delete (blocked if students assigned)
GET	/routes/{id}/students	List students on this route
POST	/routes/{id}/students	Assign student to route
DELETE	/routes/{id}/students	Remove student from route

Secure Pickup

Method	Endpoint	Description
GET	/secure-pickup	List authorised pickups (filter: <code>student_id</code> , <code>status</code>)
POST	/secure-pickup	Add authorised pickup person (code auto-generated)
GET	/secure-pickup/{id}	Get pickup auth details
PUT	/secure-pickup/{id}	Update pickup details
DELETE	/secure-pickup/{id}	Remove authorisation
POST	/secure-pickup/verify	Verify a pickup code at the gate

Request / Response Examples

Vehicles

REGISTER A VEHICLE

Request

```
POST /api/v1/transport/vehicles HTTP/1.1
Host: greenfield.compassse.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
Content-Type: application/json

{
  "make": "Toyota",
  "model": "Coaster",
  "year": 2022,
  "plate_number": "LSD-456AB",
  "capacity": 35,
  "type": "bus",
  "status": "active",
  "insurance_expiry": "2027-01-15",
  "last_service_date": "2026-01-10"
}
```

Response HTTP 201 Created

```
{
  "message": "Vehicle registered successfully",
  "vehicle": {
    "id": 3,
    "make": "Toyota",
    "model": "Coaster",
    "year": 2022,
    "plate_number": "LSD-456AB",
    "capacity": 35,
    "type": "bus",
    "status": "active",
    "insurance_expiry": "2027-01-15",
    "last_service_date": "2026-01-10",
    "created_at": "2026-03-30T09:00:00Z"
  }
}
```

LIST VEHICLES

Request

```
GET /api/v1/transport/vehicles?status=active HTTP/1.1
Host: greenfield.compassse.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
```

Response HTTP 200 OK

```
{
  "data": [
    {
      "id": 3,
      "make": "Toyota",
      "model": "Coaster",
      "year": 2022,
      "plate_number": "LSD-456AB",
      "capacity": 35,
      "type": "bus",
      "status": "active",
      "insurance_expiry": "2027-01-15"
    },
    {
      "id": 4,
      "make": "Mercedes",
      "model": "Sprinter",
      "year": 2021,
      "plate_number": "LSD-789CD",
      "capacity": 18,
      "type": "van",
      "status": "active",
      "insurance_expiry": "2026-08-30"
    }
  ],
  "meta": { "total": 2 }
}
```

UPDATE A VEHICLE

Request

```
PUT /api/v1/transport/vehicles/3 HTTP/1.1
Host: greenfield.compassse.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
Content-Type: application/json

{
  "status": "maintenance",
  "last_service_date": "2026-03-28"
}
```

Response HTTP 200 OK

```
{
  "message": "Vehicle updated successfully",
  "vehicle": {
    "id": 3,
    "plate_number": "LSD-456AB",
    "status": "maintenance",
    "last_service_date": "2026-03-28"
  }
}
```

DELETE A VEHICLE

Request

```
DELETE /api/v1/transport/vehicles/3 HTTP/1.1
Host: greenfield.compassse.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
```

Response HTTP 200 OK

```
{
  "message": "Vehicle deleted successfully"
}
```

Blocked response (vehicle on active route) HTTP 422 Unprocessable Entity

```
{
  "message": "Cannot delete vehicle assigned to an active route."
}
```

Drivers

REGISTER A DRIVER

Request

```
POST /api/v1/transport/drivers HTTP/1.1
Host: greenfield.compass.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
Content-Type: application/json
```

```
{
  "name": "Mr. Biodun Adekunle",
  "license_number": "LAG-DL-2019-00451",
  "license_expiry": "2028-06-30",
  "phone": "08034567890",
  "status": "active"
}
```

Response HTTP 201 Created

```
{
  "message": "Driver registered successfully",
  "driver": {
    "id": 2,
    "name": "Mr. Biodun Adekunle",
    "license_number": "LAG-DL-2019-00451",
    "license_expiry": "2028-06-30",
    "phone": "08034567890",
    "status": "active",
    "created_at": "2026-03-30T09:30:00Z"
  }
}
```

LIST DRIVERS

Request

```
GET /api/v1/transport/drivers?status=active HTTP/1.1
Host: greenfield.compass.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
```

Response HTTP 200 OK

```
{
  "data": [
    {
      "id": 2,
      "name": "Mr. Biodun Adekunle",
      "license_number": "LAG-DL-2019-00451",
      "license_expiry": "2028-06-30",
      "phone": "08034567890",
      "status": "active",
      "routes_count": 1
    }
  ],
  "meta": { "total": 1 }
}
```

UPDATE A DRIVER

Request

```
PUT /api/v1/transport/drivers/2 HTTP/1.1
Host: greenfield.compass.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
Content-Type: application/json

{
  "phone": "08099887766",
  "license_expiry": "2029-06-30"
}
```

Response HTTP 200 OK

```
{
  "message": "Driver updated successfully",
  "driver": {
    "id": 2,
    "name": "Mr. Biodun Adekunle",
    "phone": "08099887766",
    "license_expiry": "2029-06-30"
  }
}
```

DELETE A DRIVER

Request

```
DELETE /api/v1/transport/drivers/2 HTTP/1.1
Host: greenfield.compass.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
```

Blocked response (driver on active route) HTTP 422 Unprocessable Entity

```
{
  "message": "Cannot delete driver assigned to an active route."
}
```

Routes

CREATE A ROUTE

Request

```
POST /api/v1/transport/routes HTTP/1.1
Host: greenfield.compass.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
Content-Type: application/json
```

```
{
  "name": "Ikeja Express",
  "route_code": "RT-IKJ-01",
  "vehicle_id": 3,
  "driver_id": 2,
  "start_point": "Greenfield Academy, Lekki",
  "end_point": "Ikeja Under Bridge",
  "stops": ["Ajah", "Sangotedo", "Oregun"],
  "distance_km": 42.5,
  "fare": 15000,
  "morning_pickup_time": "06:30",
  "afternoon_dropoff_time": "15:00",
  "status": "active"
}
```

Response HTTP 201 Created

```
{
  "message": "Route created successfully",
  "route": {
    "id": 5,
    "name": "Ikeja Express",
    "route_code": "RT-IKJ-01",
    "vehicle": { "id": 3, "make": "Toyota", "model": "Coaster", "plate_number": "LSD-456AB" },
    "driver": { "id": 2, "name": "Mr. Biodun Adekunle" },
    "start_point": "Greenfield Academy, Lekki",
    "end_point": "Ikeja Under Bridge",
    "stops": ["Ajah", "Sangotedo", "Oregun"],
    "distance_km": "42.50",
    "fare": "15000.00",
    "morning_pickup_time": "06:30",
    "afternoon_dropoff_time": "15:00",
    "status": "active",
    "students_count": 0
  }
}
```

LIST ROUTES

Request

```
GET /api/v1/transport/routes?status=active HTTP/1.1
Host: greenfield.compass.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
```

Response HTTP 200 OK

```
{
  "data": [
    {
      "id": 5,
      "name": "Ikeja Express",
      "route_code": "RT-IKJ-01",
      "driver": { "name": "Mr. Biodun Adekunle" },
      "vehicle": { "plate_number": "LSD-456AB", "capacity": 35 },
      "morning_pickup_time": "06:30",
      "afternoon_dropoff_time": "15:00",
      "status": "active",
      "students_count": 24,
      "fare": "15000.00"
    }
  ],
  "meta": { "total": 1 }
}
```

UPDATE A ROUTE

Request

```
PUT /api/v1/transport/routes/5 HTTP/1.1
Host: greenfield.compass.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
Content-Type: application/json

{
  "morning_pickup_time": "06:15",
  "fare": 16000,
  "stops": ["Ajah", "Sangotedo", "Magodo", "Oregun"]
}
```

Response HTTP 200 OK

```
{
  "message": "Route updated successfully",
  "route": {
    "id": 5,
    "morning_pickup_time": "06:15",
    "fare": "16000.00",
    "stops": ["Ajah", "Sangotedo", "Magodo", "Oregun"]
  }
}
```

ASSIGN STUDENT TO ROUTE

Request


```
POST /api/v1/transport/routes/5/students HTTP/1.1
Host: greenfield.compass.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
Content-Type: application/json
```

```
{
  "student_id": 101,
  "pickup_stop": "Ajah",
  "dropoff_stop": "Ikeja Under Bridge"
}
```

Response HTTP 201 Created

```
{
  "message": "Student assigned to route successfully",
  "assignment": {
    "route_id": 5,
    "student_id": 101,
    "student": {
      "id": 101,
      "name": "Chisom Okonkwo",
      "admission_number": "GFA/2024/0101",
      "class": "JSS 2",
      "arm": "A"
    },
    "pickup_stop": "Ajah",
    "dropoff_stop": "Ikeja Under Bridge"
  }
}
```

Duplicate enrollment HTTP 422 Unprocessable Entity

```
{
  "message": "Student is already assigned to this route."
}
```

REMOVE STUDENT FROM ROUTE

Request

```
DELETE /api/v1/transport/routes/5/students HTTP/1.1
Host: greenfield.compass.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
Content-Type: application/json
```

```
{
  "student_id": 101
}
```

Response HTTP 200 OK

```
{
  "message": "Student removed from route successfully"
}
```

LIST STUDENTS ON A ROUTE

Request

```
GET /api/v1/transport/routes/5/students HTTP/1.1
Host: greenfield.compass.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
```

Response HTTP 200 OK

```
{
  "route": {
    "id": 5,
    "name": "Ikeja Express",
    "morning_pickup_time": "06:15"
  },
  "students": [
    {
      "id": 101,
      "name": "Chisom Okonkwo",
      "admission_number": "GFA/2024/0101",
      "class": "JSS 2A",
      "pickup_stop": "Ajah",
      "dropoff_stop": "Ikeja Under Bridge"
    },
    {
      "id": 87,
      "name": "David Afolabi",
      "admission_number": "GFA/2024/0087",
      "class": "SS1B",
      "pickup_stop": "Sangotedo",
      "dropoff_stop": "Ikeja Under Bridge"
    }
  ],
  "meta": { "total": 24 }
}
```

Secure Pickup

ADD AUTHORISED PICKUP PERSON

Request

```
POST /api/v1/transport/secure-pickup HTTP/1.1
Host: greenfield.compass.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
Content-Type: application/json
```

```
{
  "student_id": 101,
  "authorized_name": "Mrs. Grace Okonkwo",
  "authorized_phone": "08012345678",
  "relationship": "Mother",
  "status": "active"
}
```

Response HTTP 201 Created

```
{
  "message": "Secure pickup authorization created",
  "pickup": {
    "id": 14,
    "student_id": 101,
    "student": { "id": 101, "name": "Chisom Okonkwo" },
    "authorized_name": "Mrs. Grace Okonkwo",
    "authorized_phone": "08012345678",
    "relationship": "Mother",
    "authorized_photo": null,
    "pickup_code": "XKQR7MNP",
    "status": "active",
    "created_at": "2026-03-30T10:00:00Z"
  }
}
```

LIST AUTHORISED PICKUPS

Request

```
GET /api/v1/transport/secure-pickup?student_id=101 HTTP/1.1
Host: greenfield.compass.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
```

Response HTTP 200 OK

```
{
  "data": [
    {
      "id": 14,
      "student": { "id": 101, "name": "Chisom Okonkwo" },
      "authorized_name": "Mrs. Grace Okonkwo",
      "authorized_phone": "08012345678",
      "relationship": "Mother",
      "authorized_photo": null,
      "pickup_code": "XKQR7MNP",
      "status": "active"
    }
  ],
  "meta": { "total": 1 }
}
```

UPDATE PICKUP AUTH

Request

```
PUT /api/v1/transport/secure-pickup/14 HTTP/1.1
Host: greenfield.compassse.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
Content-Type: application/json

{
  "authorized_phone": "08099001122",
  "status": "active"
}
```

Response HTTP 200 OK

```
{
  "message": "Secure pickup updated successfully",
  "pickup": {
    "id": 14,
    "authorized_name": "Mrs. Grace Okonkwo",
    "authorized_phone": "08099001122",
    "pickup_code": "XKQR7MNP",
    "status": "active"
  }
}
```

DELETE PICKUP AUTH

Request

```
DELETE /api/v1/transport/secure-pickup/14 HTTP/1.1
Host: greenfield.compassse.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
```

Response HTTP 200 OK

```
{
  "message": "Pickup authorization removed successfully"
}
```

VERIFY PICKUP CODE AT GATE

This endpoint is accessible by security staff (reduced auth scope — no admin privileges required, but still within the tenant context).

Request

```
POST /api/v1/transport/secure-pickup/verify HTTP/1.1
Host: greenfield.compassse.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
Content-Type: application/json

{
  "pickup_code": "XKQR7MNP"
}
```

Response HTTP 200 OK – Verified

```
{
  "verified": true,
  "pickup": {
    "id": 14,
    "authorized_name": "Mrs. Grace Okonkwo",
    "authorized_phone": "08012345678",
    "relationship": "Mother",
    "authorized_photo": "https://cdn.compassse.net/tenants/greenfield/pickups/14.jpg",
    "status": "active"
  },
  "student": {
    "id": 101,
    "name": "Chisom Okonkwo",
    "admission_number": "GFA/2024/0101",
    "class": "JSS 2",
    "arm": "A",
    "photo": "https://cdn.compassse.net/tenants/greenfield/students/101.jpg"
  }
}
```

Invalid code HTTP 404 Not Found

```
{
  "verified": false,
  "message": "Invalid or inactive pickup code."
}
```

Business Rules

1. **Active route guard** — A vehicle or driver cannot be deleted while assigned to an active route.

2. **Student assignment guard** — A route cannot be deleted while students are enrolled on it.
 3. **Duplicate enrollment** — A student cannot be assigned to the same route twice.
 4. **Pickup code uniqueness** — Codes are randomly generated as 8-character uppercase strings and checked for collisions before saving.
 5. **Verify endpoint scope** — Security staff can POST to `/secure-pickup/verify` without full admin auth; the endpoint only requires a valid tenant Sanctum token, not a specific role.
 6. **Inactive pickup** — If a pickup record's `status` is `inactive`, the verify endpoint returns the same `404` response as an invalid code.
-

Frontend Integration

How the Frontend Handles Tenancy

All transport API calls are made to `https://{school}.compasse.net/api/v1/transport/...`. The subdomain resolves the tenant automatically. Every request carries `Authorization: Bearer {token}`. The `transport_management` module flag is checked on app boot; if absent the Transport section is hidden from the sidebar.

Gate Security App Flow

The gate security feature is designed to be used on a tablet or phone held by security staff at the school entrance. The flow is:

1. **Enter code or scan QR** — The security staff opens the Secure Pickup screen. They either:
 - Type the 8-character `pickup_code` manually, or
 - Scan a QR code (which encodes the `pickup_code`) using the device camera.
2. **POST verify** — The app immediately calls:

```
POST /api/v1/transport/secure-pickup/verify
{ "pickup_code": "XKQR7MNP" }
```

3. **Show result** — On success the screen displays:
 - **Authorised person:** `authorized_name`, `relationship`, `authorized_photo` (large, easy to visually confirm)
 - **Student:** `student.name`, `student.photo`, `class/arm`
4. **Not found** — If the API returns `404`, the screen shows a large red “NOT AUTHORISED” warning.
5. **No network** — A cached list of pickup codes for that day can be pre-fetched at shift start (`GET /api/v1/transport/secure-pickup?status=active`) and stored locally for offline fallback.

Transport Admin Views

- **Vehicle list** — Table with `status` badges (Active / Maintenance / Inactive). Clicking a vehicle opens its assigned routes.

- **Route map view** — The frontend renders the `stops` JSON array as a sequential list of stops with morning/afternoon times.
- **Student route assignment** — The “Assign Student” form includes a student search autocomplete (calls `GET /api/v1/students?search=...`), a stop selector pre-populated from the route’s `stops` array, and submits via `POST /api/v1/transport/routes/{id}/students`.

Hostel Management

Base URL: `https://{subdomain}.compasse.net/api/v1/` **Auth:** `Authorization: Bearer {token}`
required on all protected endpoints **Module gate:** `hostel_management`

Overview

The hostel module handles boarding student accommodation — room setup, student allocation, vacating, and maintenance requests.

User Stories

As a school admin, I want to create room records with capacity and pricing so I can manage the boarding house efficiently.

As a hostel master, I want to allocate students to specific rooms and track occupancy in real time.

As a hostel master, I want to vacate a student from their room (e.g. after term end) so the bed is freed for another student.

As any staff member, I want to report maintenance issues in a room — plumbing, electrical, furniture — so they get tracked and resolved.

As an admin, I want to see which rooms have available beds so I can quickly allocate new boarding students.

Models

HostelRoom

Field	Type	Description
id	integer	Auto-increment primary key
room_number	string	Room identifier (e.g. B204)
block	string	Block / wing name (e.g. Block B)
floor	string	Floor level (e.g. 2nd Floor)
type	enum	single , double , triple , dormitory
capacity	integer	Maximum beds
occupied_count	integer	Current occupants (auto-managed, default 0)
price_per_term	decimal	Fee charged per term
amenities	JSON	Array of amenity strings (e.g. ["fan", "wardrobe", "toilet"])
status	enum	available , occupied , maintenance , closed

Computed accessor: `available_beds = max(0, capacity - occupied_count)`

HostelAllocation

Field	Type	Description
id	integer	Auto-increment primary key
room_id	integer	Room assigned FK
student_id	integer	Student allocated FK
academic_year_id	integer	Academic year FK
term_id	integer	Term FK
allocated_at	date	Date allocated
vacated_at	date null	Date vacated (null if still active)
status	enum	active , vacated , transferred
amount_paid	decimal null	Payment for this allocation
payment_status	enum	paid , partial , unpaid
notes	string null	Additional notes

HostelMaintenance

Field	Type	Description
id	integer	Auto-increment primary key
room_id	integer	Affected room FK
title	string	Short issue title
description	string	Full description of the problem
reported_by	integer	FK → User who reported
assigned_to	integer null	FK → User (maintenance staff)
priority	enum	low , medium , high , urgent
status	enum	pending , in_progress , completed , cancelled
cost	decimal null	Repair cost
resolution_notes	string null	Notes on how it was resolved
completed_at	timestamp null	Auto-set when status transitions to completed

API Endpoints

Base path: /api/v1/hostel/

Rooms

Method	Endpoint	Description
GET	/rooms	List rooms (filter: status , type , block , available_only=1)
POST	/rooms	Create room
GET	/rooms/{id}	Get room + active allocations + maintenance history
PUT	/rooms/{id}	Update room
DELETE	/rooms/{id}	Delete (blocked if active allocations exist)

Allocations

Method	Endpoint	Description
GET	/allocations	List allocations (filter: status , room_id , student_id , term_id)
POST	/allocations	Allocate student to room
GET	/allocations/{id}	Get allocation details
PUT	/allocations/{id}	Update payment info or notes
DELETE	/allocations/{id}	Delete allocation record
POST	/allocations/{id}/vacate	Vacate student from room

Maintenance

Method	Endpoint	Description
GET	/maintenance	List requests (filter: status , priority , room_id , assigned_to)
POST	/maintenance	Report a maintenance issue
GET	/maintenance/{id}	Get issue details
PUT	/maintenance/{id}	Update (assign staff, change status, add resolution)
DELETE	/maintenance/{id}	Delete request

Request / Response Examples

Rooms

CREATE A ROOM

Request

```
POST /api/v1/hostel/rooms HTTP/1.1
Host: greenfield.compassse.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
Content-Type: application/json

{
  "room_number": "B204",
  "block": "Block B",
  "floor": "2nd Floor",
  "type": "double",
  "capacity": 2,
  "price_per_term": 85000,
  "amenities": ["fan", "wardrobe", "study desk", "reading lamp"],
  "status": "available"
}
```

Response HTTP 201 Created

```
{
  "message": "Room created successfully",
  "room": {
    "id": 8,
    "room_number": "B204",
    "block": "Block B",
    "floor": "2nd Floor",
    "type": "double",
    "capacity": 2,
    "occupied_count": 0,
    "available_beds": 2,
    "price_per_term": "85000.00",
    "amenities": ["fan", "wardrobe", "study desk", "reading lamp"],
    "status": "available",
    "created_at": "2026-03-30T08:00:00Z"
  }
}
```

LIST ROOMS

Request

```
GET /api/v1/hostel/rooms?block=Block+B&status=available HTTP/1.1
Host: greenfield.compassse.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
```

Response HTTP 200 OK

```
{
  "data": [
    {
      "id": 8,
      "room_number": "B204",
      "block": "Block B",
      "floor": "2nd Floor",
      "type": "double",
      "capacity": 2,
      "occupied_count": 1,
      "available_beds": 1,
      "price_per_term": "85000.00",
      "amenities": ["fan", "wardrobe", "study desk", "reading lamp"],
      "status": "available"
    },
    {
      "id": 9,
      "room_number": "B205",
      "block": "Block B",
      "floor": "2nd Floor",
      "type": "single",
      "capacity": 1,
      "occupied_count": 0,
      "available_beds": 1,
      "price_per_term": "95000.00",
      "amenities": ["ac", "wardrobe", "private toilet"],
      "status": "available"
    }
  ],
  "meta": {
    "total": 2,
    "per_page": 15,
    "current_page": 1
  }
}
```

LIST AVAILABLE ROOMS ONLY

Request

```
GET /api/v1/hostel/rooms?available_only=1 HTTP/1.1
Host: greenfield.compassse.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
```

Returns only rooms where `available_beds > 0`.

GET SINGLE ROOM

Request

```
GET /api/v1/hostel/rooms/8 HTTP/1.1
Host: greenfield.compassse.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
```

Response HTTP 200 OK

```
{
  "data": {
    "id": 8,
    "room_number": "B204",
    "block": "Block B",
    "floor": "2nd Floor",
    "type": "double",
    "capacity": 2,
    "occupied_count": 1,
    "available_beds": 1,
    "price_per_term": "85000.00",
    "amenities": ["fan", "wardrobe", "study desk", "reading lamp"],
    "status": "available",
    "active_allocations": [
      {
        "id": 11,
        "student": { "id": 55, "name": "Amara Eze", "admission_number": "GFA/2024/0055" },
        "allocated_at": "2026-01-08",
        "status": "active",
        "payment_status": "paid"
      }
    ],
    "maintenance_history": [
      {
        "id": 5,
        "title": "Broken ceiling fan",
        "status": "completed",
        "priority": "medium",
        "completed_at": "2026-02-15T14:00:00Z"
      }
    ]
  }
}
```

UPDATE A ROOM

Request

```
PUT /api/v1/hostel/rooms/8 HTTP/1.1
Host: greenfield.compassse.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
Content-Type: application/json

{
  "price_per_term": 90000,
  "amenities": ["fan", "wardrobe", "study desk", "reading lamp", "charging station"],
  "status": "available"
}
```

Response HTTP 200 OK

```
{
  "message": "Room updated successfully",
  "room": {
    "id": 8,
    "room_number": "B204",
    "price_per_term": "90000.00",
    "amenities": ["fan", "wardrobe", "study desk", "reading lamp", "charging station"],
    "status": "available"
  }
}
```

DELETE A ROOM

Request

```
DELETE /api/v1/hostel/rooms/8 HTTP/1.1
Host: greenfield.compassse.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
```

Blocked response (has active allocations) HTTP 422 Unprocessable Entity

```
{
  "message": "Cannot delete room with active student allocations."
}
```

Allocations

ALLOCATE STUDENT TO ROOM

Request

```
POST /api/v1/hostel/allocations HTTP/1.1
Host: greenfield.compassse.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
Content-Type: application/json
```

```
{
  "room_id": 8,
  "student_id": 55,
  "academic_year_id": 1,
  "term_id": 2,
  "allocated_at": "2026-01-08",
  "amount_paid": 85000,
  "payment_status": "paid"
}
```

Response HTTP 201 Created

```
{
  "message": "Student allocated to room successfully",
  "allocation": {
    "id": 11,
    "room": {
      "id": 8,
      "room_number": "B204",
      "block": "Block B",
      "capacity": 2,
      "occupied_count": 2,
      "available_beds": 0
    },
    "student": {
      "id": 55,
      "name": "Amara Eze",
      "admission_number": "GFA/2024/0055"
    },
    "academic_year_id": 1,
    "term_id": 2,
    "allocated_at": "2026-01-08",
    "vacated_at": null,
    "status": "active",
    "amount_paid": "85000.00",
    "payment_status": "paid"
  }
}
```

On success, `rooms.occupied_count` is incremented atomically inside a `DB::transaction()`.

Room full HTTP 422 Unprocessable Entity

```
{
  "message": "Room is at full capacity. No available beds."
}
```

Student already allocated HTTP 422 Unprocessable Entity

```
{
  "message": "Student already has an active hostel allocation."
}
```

LIST ALLOCATIONS

Request

```
GET /api/v1/hostel/allocations?status=active&room_id=8 HTTP/1.1
Host: greenfield.compass.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
```

Response HTTP 200 OK


```
{
  "data": [
    {
      "id": 11,
      "room": { "id": 8, "room_number": "B204", "block": "Block B" },
      "student": { "id": 55, "name": "Amara Eze" },
      "allocated_at": "2026-01-08",
      "vacated_at": null,
      "status": "active",
      "payment_status": "paid",
      "amount_paid": "85000.00"
    }
  ],
  "meta": { "total": 1 }
}
```

UPDATE ALLOCATION (PAYMENT INFO)

Request

```
PUT /api/v1/hostel/allocations/11 HTTP/1.1
Host: greenfield.compassse.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
Content-Type: application/json

{
  "payment_status": "paid",
  "amount_paid": 90000,
  "notes": "Balance of 5,000 received on 2026-03-01"
}
```

Response HTTP 200 OK

```
{
  "message": "Allocation updated successfully",
  "allocation": {
    "id": 11,
    "payment_status": "paid",
    "amount_paid": "90000.00",
    "notes": "Balance of 5,000 received on 2026-03-01"
  }
}
```

VACATE A STUDENT

Request

```
POST /api/v1/hostel/allocations/11/vacate HTTP/1.1
Host: greenfield.compassse.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
Content-Type: application/json

{
  "vacated_at": "2026-03-28"
}
```

Response HTTP 200 OK

```
{
  "message": "Student vacated from room successfully",
  "allocation": {
    "id": 11,
    "status": "vacated",
    "vacated_at": "2026-03-28",
    "room": {
      "id": 8,
      "room_number": "B204",
      "occupied_count": 1,
      "available_beds": 1
    }
  }
}
```

On success, `rooms.occupied_count` is decremented atomically inside a `DB::transaction()` .

Maintenance

REPORT A MAINTENANCE ISSUE

Request

```
POST /api/v1/hostel/maintenance HTTP/1.1
Host: greenfield.compass.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
Content-Type: application/json

{
  "room_id": 8,
  "title": "Broken ceiling fan",
  "description": "The fan in room B204 stopped working. Possible motor fault. Students are complaining about the heat.",
  "priority": "medium"
}
```

Response HTTP 201 Created

```
{
  "message": "Maintenance request submitted successfully",
  "maintenance": {
    "id": 5,
    "room": { "id": 8, "room_number": "B204", "block": "Block B" },
    "title": "Broken ceiling fan",
    "description": "The fan in room B204 stopped working. Possible motor fault. Students are complaining about the heat.",
    "priority": "medium",
    "status": "pending",
    "reported_by": { "id": 4, "name": "Mr. Chukwuemeka Obi" },
    "assigned_to": null,
    "cost": null,
    "resolution_notes": null,
    "completed_at": null,
    "created_at": "2026-03-30T12:00:00Z"
  }
}
```

LIST MAINTENANCE REQUESTS

Request

```
GET /api/v1/hostel/maintenance?status=pending&priority=urgent HTTP/1.1
Host: greenfield.compass.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
```

Response HTTP 200 OK

```
{
  "data": [
    {
      "id": 7,
      "room": { "room_number": "A101", "block": "Block A" },
      "title": "Burst water pipe",
      "priority": "urgent",
      "status": "pending",
      "reported_by": { "name": "Miss Fatima Sule" },
      "assigned_to": null,
      "created_at": "2026-03-30T07:30:00Z"
    }
  ],
  "meta": { "total": 1 }
}
```

ASSIGN MAINTENANCE STAFF (MOVE TO IN PROGRESS)

Request

```
PUT /api/v1/hostel/maintenance/5 HTTP/1.1
Host: greenfield.compass.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
Content-Type: application/json
```

```
{
  "assigned_to": 7,
  "status": "in_progress"
}
```

Response HTTP 200 OK

```
{
  "message": "Maintenance request updated successfully",
  "maintenance": {
    "id": 5,
    "status": "in_progress",
    "assigned_to": { "id": 7, "name": "Mr. Kola Maintenance" },
    "completed_at": null
  }
}
```

MARK AS COMPLETED (AUTO-SETS COMPLETED_AT)

Request

```
PUT /api/v1/hostel/maintenance/5 HTTP/1.1
Host: greenfield.compass.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
Content-Type: application/json
```

```
{
  "status": "completed",
  "cost": 12000,
  "resolution_notes": "Fan motor replaced with a new unit. Tested and confirmed working."
}
```

Response HTTP 200 OK

```
{
  "message": "Maintenance request updated successfully",
  "maintenance": {
    "id": 5,
    "title": "Broken ceiling fan",
    "status": "completed",
    "cost": "12000.00",
    "resolution_notes": "Fan motor replaced with a new unit. Tested and confirmed working.",
    "assigned_to": { "id": 7, "name": "Mr. Kola Maintenance" },
    "completed_at": "2026-03-30T16:45:00Z"
  }
}
```

`completed_at` is set automatically by the model when `status` transitions to `completed` .

Business Rules

1. **Capacity guard** — Allocation is rejected if `room.occupied_count >= room.capacity`.
 2. **Duplicate active allocation** — A student cannot have two active hostel allocations simultaneously.
 3. **Atomic occupancy** — `occupied_count` increments on allocation and decrements on vacate, both inside `DB::transaction()`.
 4. **Delete cascade** — A room cannot be deleted while it has active allocations.
 5. **Auto timestamp** — `completed_at` is set automatically when a maintenance record's status changes to `completed`.
 6. **Available beds accessor** — `available_beds` is computed as `max(0, capacity - occupied_count)` and is included in all room responses.
-

Frontend Integration

How the Frontend Handles Tenancy

Hostel API calls are made to `https://{school}.compasse.net/api/v1/hostel/...` with `Authorization: Bearer {token}`. The tenant is resolved by subdomain. If the school does not have `hostel_management` in its enabled modules list, the Hostel section is hidden from the navigation.

Room Availability Grid

The hostel dashboard shows a grid of room cards, one per room, colour-coded by status:

- **Green** — `available` (has free beds)
- **Yellow** — `available` but `occupied_count = capacity - 1` (nearly full)
- **Orange** — `status = occupied` (full)
- **Red** — `status = maintenance` or `closed`

Data source: `GET /api/v1/hostel/rooms` — fetched on page load and refreshed after every allocation or vacate action.

Each room card displays:

- Room number + block + floor
- Occupancy bar: `{occupied_count} / {capacity}` with a visual progress bar
- Amenity chips: icons for `fan`, `ac`, `wardrobe`, etc.
- `price_per_term` formatted as currency

Allocation Form with Capacity Indicator

When the hostel master clicks “Allocate Student” on a room card, a slide-over panel opens:

1. **Room summary** at top: room number, capacity, `available_beds` (live from the most recent GET response)
2. **Student search** — debounced autocomplete calls `GET /api/v1/students?search={query}`. If the selected student already has an active allocation, a warning is shown before submission.

3. **Academic year / term selectors** — dropdowns populated from `GET /api/v1/academic-years` and `/terms`.
4. **Payment fields** — `amount_paid` and `payment_status` (paid / partial / unpaid).
5. On submit, calls `POST /api/v1/hostel/allocations`. On success, the room card's occupancy bar updates immediately.

Maintenance Tracker

The Maintenance section shows a Kanban-style board with four columns: **Pending**, **In Progress**, **Completed**, **Cancelled**.

- Staff can drag a card to change its status; the frontend calls `PUT /api/v1/hostel/maintenance/{id}` with the new `status`.
- When a card is moved to **Completed**, a form pops up requesting `cost` and `resolution_notes`.
- Completed cards show the `completed_at` timestamp and the assigned staff name.
- Filter buttons at the top allow filtering by `priority` (Low / Medium / High / Urgent) and by `block`.

Health Management

Base URL: `https://{subdomain}.compasse.net/api/v1/` **Auth:** `Authorization: Bearer {token}`
required on all protected endpoints **Module gate:** `health_management`

Overview

The health module maintains medical records for students — physical measurements, allergies, immunisations, doctor appointments, and active medications.

User Stories

As a school nurse, I want to create and update each student's health record including blood group, allergies, and known medical conditions so I can respond quickly to emergencies.

As a nurse, I want to schedule health appointments for students and record diagnoses and prescriptions after visits.

As a nurse, I want to track current medications so I know which students need to receive daily doses.

As a parent, I want to view my child's health record and any upcoming appointments.

As a nurse, I want the BMI to be calculated automatically from height and weight.

Models

HealthRecord

One record per student (unique constraint on `student_id`).

Field	Type	Description
id	integer	Auto-increment primary key
student_id	integer	Student FK (unique)
blood_group	string null	e.g. 0+ , AB- , B+
height_cm	decimal null	Height in centimetres
weight_kg	decimal null	Weight in kilograms
allergies	JSON null	Array of allergy strings
medical_conditions	JSON null	Array of condition strings
current_medications	JSON null	Array of medication name strings (informational snapshot)
immunization_records	JSON null	Array of { vaccine, date, dose } objects
emergency_contact_name	string null	Emergency contact full name
emergency_contact_phone	string null	Emergency contact phone number
family_doctor_name	string null	Family doctor name
family_doctor_phone	string null	Family doctor phone
last_checkup_date	date null	Date of last physical exam
notes	string null	General nurse notes

Computed accessor: `bmi = weight_kg / (height_m)2` rounded to 1 decimal. Returns `null` if either `height_cm` or `weight_kg` is missing.

HealthAppointment

Field	Type	Description
id	integer	Auto-increment primary key
student_id	integer	Patient FK
doctor_name	string	Attending doctor / nurse name
appointment_date	date	Date of appointment
appointment_time	string	HH:MM format
reason	string	Purpose of visit
status	enum	scheduled , completed , cancelled , no_show
diagnosis	string null	Doctor's diagnosis (filled after appointment)
prescription	string null	Prescribed treatment / instructions
follow_up_date	date null	Next recommended appointment date
created_by	integer	FK → User (nurse who scheduled; auto-set)

Medication

Field	Type	Description
id	integer	Auto-increment primary key
student_id	integer	Student receiving medication FK
name	string	Medication name
dosage	string	e.g. 500mg , 100mcg
frequency	string	e.g. Twice daily after meals
start_date	date	Start date
end_date	date null	End date (null = indefinite / ongoing)
prescribed_by	string null	Prescribing doctor name
reason	string null	Condition being treated
side_effects	string null	Known side effects
status	enum	active , completed , discontinued

API Endpoints

Base path: /api/v1/health/

Health Records

Method	Endpoint	Description
GET	/records	List records (filter: student_id , blood_group)
POST	/records	Create health record (one per student)
GET	/records/{id}	Get record including computed BMI
PUT	/records/{id}	Update record
DELETE	/records/{id}	Delete record

Appointments

Method	Endpoint	Description
GET	/appointments	List appointments (filter: student_id , status , date_from , date_to)
POST	/appointments	Schedule an appointment
GET	/appointments/{id}	Get appointment details
PUT	/appointments/{id}	Update (add diagnosis, prescription, set status, follow-up)
DELETE	/appointments/{id}	Delete appointment

Medications

Method	Endpoint	Description
GET	/medications	List medications (filter: <code>student_id</code> , <code>status</code>)
POST	/medications	Prescribe a medication
GET	/medications/{id}	Get medication details
PUT	/medications/{id}	Update medication (status, dosage, end date)
DELETE	/medications/{id}	Delete medication record

Request / Response Examples

Health Records

CREATE HEALTH RECORD

Request

```
POST /api/v1/health/records HTTP/1.1
Host: greenfield.compassse.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
Content-Type: application/json

{
  "student_id": 42,
  "blood_group": "O+",
  "height_cm": 162,
  "weight_kg": 58,
  "allergies": ["Penicillin", "Peanuts"],
  "medical_conditions": ["Mild asthma"],
  "immunization_records": [
    { "vaccine": "Hepatitis B", "date": "2010-03-15", "dose": "3rd" },
    { "vaccine": "Yellow Fever", "date": "2018-09-01", "dose": "1st" }
  ],
  "emergency_contact_name": "Mrs. Ngozi Bello",
  "emergency_contact_phone": "08098765432",
  "family_doctor_name": "Dr. Adaeze Nwachukwu",
  "family_doctor_phone": "08011223344",
  "last_checkup_date": "2025-09-01",
  "notes": "Student uses an inhaler. Keep one in the sick bay."
}
```

Response HTTP 201 Created

```
{
  "message": "Health record created successfully",
  "record": {
    "id": 31,
    "student_id": 42,
    "student": {
      "id": 42,
      "name": "Adaora Bello",
      "admission_number": "GFA/2024/0042",
      "class": "SS2",
      "arm": "B"
    },
    "blood_group": "O+",
    "height_cm": "162.00",
    "weight_kg": "58.00",
    "bmi": 22.1,
    "allergies": ["Penicillin", "Peanuts"],
    "medical_conditions": ["Mild asthma"],
    "immunization_records": [
      { "vaccine": "Hepatitis B", "date": "2010-03-15", "dose": "3rd" },
      { "vaccine": "Yellow Fever", "date": "2018-09-01", "dose": "1st" }
    ],
    "emergency_contact_name": "Mrs. Ngozi Bello",
    "emergency_contact_phone": "08098765432",
    "family_doctor_name": "Dr. Adaeze Nwachukwu",
    "family_doctor_phone": "08011223344",
    "last_checkup_date": "2025-09-01",
    "notes": "Student uses an inhaler. Keep one in the sick bay.",
    "created_at": "2026-03-30T09:00:00Z"
  }
}
```

Duplicate record HTTP 422 Unprocessable Entity

```
{
  "message": "A health record already exists for this student."
}
```

GET HEALTH RECORD (WITH BMI)

Request

```
GET /api/v1/health/records/31 HTTP/1.1
Host: greenfield.compass.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
```

Response HTTP 200 OK

```
{
  "data": {
    "id": 31,
    "student": {
      "id": 42,
      "name": "Adaora Bello",
      "admission_number": "GFA/2024/0042"
    },
    "blood_group": "O+",
    "height_cm": "162.00",
    "weight_kg": "58.00",
    "bmi": 22.1,
    "allergies": ["Penicillin", "Peanuts"],
    "medical_conditions": ["Mild asthma"],
    "current_medications": ["Salbutamol Inhaler"],
    "immunization_records": [
      { "vaccine": "Hepatitis B", "date": "2010-03-15", "dose": "3rd" },
      { "vaccine": "Yellow Fever", "date": "2018-09-01", "dose": "1st" }
    ],
    "emergency_contact_name": "Mrs. Ngozi Bello",
    "emergency_contact_phone": "08098765432",
    "family_doctor_name": "Dr. Adaeze Nwachukwu",
    "family_doctor_phone": "08011223344",
    "last_checkup_date": "2025-09-01",
    "notes": "Student uses an inhaler. Keep one in the sick bay.",
    "updated_at": "2026-03-30T09:00:00Z"
  }
}
```

UPDATE HEALTH RECORD

Request

```
PUT /api/v1/health/records/31 HTTP/1.1
Host: greenfield.compass.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
Content-Type: application/json

{
  "height_cm": 164,
  "weight_kg": 60,
  "last_checkup_date": "2026-03-30",
  "notes": "Student uses an inhaler. Keep one in the sick bay. Weight has increased since last check."
}
```

Response HTTP 200 OK

```
{
  "message": "Health record updated successfully",
  "record": {
    "id": 31,
    "height_cm": "164.00",
    "weight_kg": "60.00",
    "bmi": 22.3,
    "last_checkup_date": "2026-03-30"
  }
}
```

Appointments

SCHEDULE AN APPOINTMENT

Request

```
POST /api/v1/health/appointments HTTP/1.1
Host: greenfield.compass.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
Content-Type: application/json
```

```
{
  "student_id": 42,
  "doctor_name": "Dr. Adaeze Nwachukwu",
  "appointment_date": "2026-04-05",
  "appointment_time": "10:00",
  "reason": "Routine asthma review and inhaler technique assessment"
}
```

Response HTTP 201 Created

```
{
  "message": "Appointment scheduled successfully",
  "appointment": {
    "id": 9,
    "student": {
      "id": 42,
      "name": "Adaora Bello",
      "class": "SS2B"
    },
    "doctor_name": "Dr. Adaeze Nwachukwu",
    "appointment_date": "2026-04-05",
    "appointment_time": "10:00",
    "reason": "Routine asthma review and inhaler technique assessment",
    "status": "scheduled",
    "diagnosis": null,
    "prescription": null,
    "follow_up_date": null,
    "created_by": { "id": 5, "name": "Nurse Amaka" },
    "created_at": "2026-03-30T10:00:00Z"
  }
}
```

LIST APPOINTMENTS

Request

```
GET /api/v1/health/appointments?student_id=42&status=scheduled HTTP/1.1
Host: greenfield.compassse.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
```

Response HTTP 200 OK

```
{
  "data": [
    {
      "id": 9,
      "student": { "id": 42, "name": "Adaora Bello" },
      "doctor_name": "Dr. Adaeze Nwachukwu",
      "appointment_date": "2026-04-05",
      "appointment_time": "10:00",
      "reason": "Routine asthma review",
      "status": "scheduled",
      "follow_up_date": null
    }
  ],
  "meta": { "total": 1 }
}
```

LIST APPOINTMENTS BY DATE RANGE

Request

```
GET /api/v1/health/appointments?date_from=2026-04-01&date_to=2026-04-30 HTTP/1.1
Host: greenfield.compassse.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
```

RECORD APPOINTMENT OUTCOME (COMPLETED)

Request

```
PUT /api/v1/health/appointments/9 HTTP/1.1
Host: greenfield.compassse.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
Content-Type: application/json

{
  "status": "completed",
  "diagnosis": "Mild persistent asthma – well-controlled on current regimen",
  "prescription": "Continue Salbutamol Inhaler 100mcg. 2 puffs before exercise. Increase to 4 puffs if symptoms worsen.",
  "follow_up_date": "2026-07-05"
}
```

Response HTTP 200 OK

```
{
  "message": "Appointment updated successfully",
  "appointment": {
    "id": 9,
    "status": "completed",
    "diagnosis": "Mild persistent asthma – well-controlled on current regimen",
    "prescription": "Continue Salbutamol Inhaler 100mcg. 2 puffs before exercise. Increase to 4 puffs if symptoms worsen.",
    "follow_up_date": "2026-07-05",
    "student": { "id": 42, "name": "Adaora Bello" }
  }
}
```

MARK APPOINTMENT AS NO-SHOW

Request

```
PUT /api/v1/health/appointments/9 HTTP/1.1
Host: greenfield.compass.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
Content-Type: application/json
```

```
{
  "status": "no_show"
}
```

Response HTTP 200 OK

```
{
  "message": "Appointment updated successfully",
  "appointment": {
    "id": 9,
    "status": "no_show"
  }
}
```

Medications

PRESCRIBE A MEDICATION

Request

```
POST /api/v1/health/medications HTTP/1.1
Host: greenfield.compass.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
Content-Type: application/json
```

```
{
  "student_id": 42,
  "name": "Salbutamol Inhaler",
  "dosage": "100mcg",
  "frequency": "As needed – max 4 puffs daily. 2 puffs before exercise.",
  "start_date": "2026-01-10",
  "end_date": null,
  "prescribed_by": "Dr. Adaeze Nwachukwu",
  "reason": "Mild persistent asthma",
  "side_effects": "Mild tremor or palpitations at high doses",
  "status": "active"
}
```

Response HTTP 201 Created

```
{
  "message": "Medication added successfully",
  "medication": {
    "id": 17,
    "student_id": 42,
    "student": { "id": 42, "name": "Adaora Bello" },
    "name": "Salbutamol Inhaler",
    "dosage": "100mcg",
    "frequency": "As needed – max 4 puffs daily. 2 puffs before exercise.",
    "start_date": "2026-01-10",
    "end_date": null,
    "prescribed_by": "Dr. Adaeze Nwachukwu",
    "reason": "Mild persistent asthma",
    "side_effects": "Mild tremor or palpitations at high doses",
    "status": "active",
    "created_at": "2026-03-30T11:00:00Z"
  }
}
```

LIST MEDICATIONS

Request

```
GET /api/v1/health/medications?student_id=42&status=active HTTP/1.1
Host: greenfield.compass.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
```

Response HTTP 200 OK


```
{
  "data": [
    {
      "id": 17,
      "student": { "id": 42, "name": "Adaora Bello" },
      "name": "Salbutamol Inhaler",
      "dosage": "100mcg",
      "frequency": "As needed – max 4 puffs daily",
      "start_date": "2026-01-10",
      "end_date": null,
      "prescribed_by": "Dr. Adaeze Nwachukwu",
      "reason": "Mild persistent asthma",
      "status": "active"
    }
  ],
  "meta": { "total": 1 }
}
```

UPDATE MEDICATION STATUS (DISCONTINUE)

Request

```
PUT /api/v1/health/medications/17 HTTP/1.1
Host: greenfield.compassse.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
Content-Type: application/json

{
  "status": "discontinued",
  "end_date": "2026-03-30"
}
```

Response HTTP 200 OK

```
{
  "message": "Medication updated successfully",
  "medication": {
    "id": 17,
    "name": "Salbutamol Inhaler",
    "status": "discontinued",
    "end_date": "2026-03-30"
  }
}
```

MARK MEDICATION AS COMPLETED (COURSE FINISHED)

Request

```
PUT /api/v1/health/medications/17 HTTP/1.1
Host: greenfield.compassse.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
Content-Type: application/json
```

```
{
  "status": "completed",
  "end_date": "2026-04-10"
}
```

Response HTTP 200 OK

```
{
  "message": "Medication updated successfully",
  "medication": {
    "id": 17,
    "name": "Salbutamol Inhaler",
    "status": "completed",
    "end_date": "2026-04-10"
  }
}
```

Business Rules

1. **One record per student** — Creating a second health record for the same student is rejected with HTTP 422 .
2. **BMI auto-calculation** — Computed on-the-fly from `height_cm` and `weight_kg` ; returns `null` if either field is missing.
3. **Appointment created_by** — Auto-set to the authenticated nurse/user on creation; not exposed as a writable field.
4. **Date filters** — Appointment listing supports `date_from` and `date_to` query parameters to power calendar views.
5. **Medication default status** — Medications default to `active` if no `status` is provided on creation.
6. **Medication lifecycle** — Valid status transitions are: `active` → `completed` (course finished), `active` → `discontinued` (stopped early).

Frontend Integration

How the Frontend Handles Tenancy

All health API calls go to `https://{school}.compassse.net/api/v1/health/...` with `Authorization: Bearer {token}` . The `health_management` module flag is checked on boot; if not enabled the Health section is hidden from navigation. Health data is sensitive — the frontend should never cache it in a shared location (use in-memory state only).

Student Health Card

The student health card is a dedicated panel within the student profile page (or accessible directly from the nurse's dashboard by searching for a student). It displays:

- **Vitals row:** Blood group badge (e.g. `O+`), Height, Weight, and computed BMI with a colour indicator:
 - BMI < 18.5 — blue “Underweight”
 - BMI 18.5–24.9 — green “Normal”
 - BMI 25–29.9 — yellow “Overweight”
 - BMI ≥ 30 — red “Obese”
- **Allergies list** — Rendered as red warning chips (e.g. `Penicillin`, `Peanuts`). Highly visible.
- **Active Medications** — Each active medication is shown as a card with name, dosage, frequency, and prescribing doctor.
- **Known Conditions** — Bulleted list of `medical_conditions`.
- **Emergency Contact** — Name + phone, with a tel-link for one-tap calling.
- **Last Checkup Date** — Displayed with a warning if older than 6 months.

Data source: `GET /api/v1/health/records?student_id={id}` (returns the single record or empty array).

Appointment Calendar

The appointments page uses a calendar view (monthly grid). Data is fetched per month:

```
GET /api/v1/health/appointments?date_from=2026-04-01&date_to=2026-04-30
```

Each day with an appointment shows a coloured dot. Clicking a day opens a list of that day's appointments. Each appointment card shows:

- Student name + class
- Time + doctor name
- Status badge (Scheduled / Completed / Cancelled / No-Show)
- An “Update” button that opens a form pre-filled with `diagnosis`, `prescription`, and `follow_up_date` fields.

Upcoming follow-ups (appointments where `follow_up_date` is within 7 days) are surfaced as a banner at the top of the page.

Academic Management

Base URL: `https://{subdomain}.compasse.net/api/v1/` **Auth:** `Authorization: Bearer {token}`
required on all protected endpoints **Module gate:** `academic_management` (students:
`student_management`, teachers: `teacher_management`)

Overview

The academic module is the structural backbone of the system. It manages the academic calendar, class organisation, subjects, departments, and student/teacher assignment.

User Stories

As a school admin, I want to set up academic years and terms so all activity (attendance, exams, fees) is properly dated.

As an admin, I want to create classes, arms (streams), and subjects so teachers and students can be assigned correctly.

As a principal, I want to view the organogram of teachers and classes in my school.

As a teacher, I want to see which classes and subjects I'm responsible for.

Data Hierarchy

```
School
├── AcademicYear (e.g. "2025/2026")
│   └── Term (e.g. "First Term", "Second Term")
School
├── Department (e.g. "Science", "Arts")
│   └── Subject (e.g. "Chemistry", "Literature")
School
├── Class (e.g. "JSS 1", "SS 2")
│   └── Arm (e.g. "A", "B", "Gold", "Silver")
│       └── Students
```

API Endpoints

Base path: `/api/v1/` (all require `academic_management` module unless noted)

Academic Years

Method	Endpoint	Description
GET	<code>/academic-years</code>	List academic years
POST	<code>/academic-years</code>	Create academic year
GET	<code>/academic-years/{id}</code>	Get details + terms
PUT	<code>/academic-years/{id}</code>	Update
DELETE	<code>/academic-years/{id}</code>	Delete (blocked if has terms)

Terms

Method	Endpoint	Description
GET	<code>/terms</code>	List terms (filter: <code>academic_year_id</code> , <code>status</code>)
POST	<code>/terms</code>	Create term
GET	<code>/terms/{id}</code>	Get term
PUT	<code>/terms/{id}</code>	Update term
DELETE	<code>/terms/{id}</code>	Delete term

Departments

Method	Endpoint	Description
GET	<code>/departments</code>	List departments
POST	<code>/departments</code>	Create department
GET	<code>/departments/{id}</code>	Get department + subjects
PUT	<code>/departments/{id}</code>	Update
DELETE	<code>/departments/{id}</code>	Delete (blocked if has subjects)

Classes

Method	Endpoint	Description
GET	/classes	List classes
POST	/classes	Create class
GET	/classes/{id}	Get class + arm details
PUT	/classes/{id}	Update
DELETE	/classes/{id}	Delete (blocked if has students)
GET	/classes/{id}/students	Students in this class (all arms)

Arms (Streams)

Method	Endpoint	Description
GET	/arms	List all arms
POST	/arms	Create arm
GET	/arms/{id}	Get arm
PUT	/arms/{id}	Update arm
DELETE	/arms/{id}	Delete arm (blocked if has students)
POST	/arms/assign-to-class	Assign arm to a class
POST	/arms/remove-from-class	Remove arm from class
GET	/arms/class/{classId}	Arms belonging to a class
GET	/arms/{armId}/students	Students in a specific arm

Subjects

Method	Endpoint	Description
GET	/subjects	List subjects (filter: department_id , class_id)
POST	/subjects	Create subject
GET	/subjects/{id}	Get subject
PUT	/subjects/{id}	Update subject
DELETE	/subjects/{id}	Delete subject

Request / Response Examples

Academic Years

CREATE ACADEMIC YEAR

Request

```
POST /api/v1/academic-years HTTP/1.1
Host: greenfield.compassse.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
Content-Type: application/json

{
  "name": "2025/2026",
  "start_date": "2025-09-01",
  "end_date": "2026-07-31",
  "is_current": true
}
```

Response HTTP 201 Created

```
{
  "message": "Academic year created successfully",
  "academic_year": {
    "id": 3,
    "name": "2025/2026",
    "start_date": "2025-09-01",
    "end_date": "2026-07-31",
    "is_current": true,
    "terms_count": 0,
    "created_at": "2026-03-30T08:00:00Z"
  }
}
```

LIST ACADEMIC YEARS

Request

```
GET /api/v1/academic-years HTTP/1.1
Host: greenfield.compassse.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
```

Response HTTP 200 OK

```
{
  "data": [
    {
      "id": 3,
      "name": "2025/2026",
      "start_date": "2025-09-01",
      "end_date": "2026-07-31",
      "is_current": true,
      "terms_count": 2
    },
    {
      "id": 2,
      "name": "2024/2025",
      "start_date": "2024-09-01",
      "end_date": "2025-07-31",
      "is_current": false,
      "terms_count": 3
    }
  ],
  "meta": { "total": 2 }
}
```

Terms

CREATE A TERM

Request

```
POST /api/v1/terms HTTP/1.1
Host: greenfield.compassse.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
Content-Type: application/json
```

```
{
  "academic_year_id": 3,
  "name": "Second Term",
  "start_date": "2026-01-06",
  "end_date": "2026-04-10",
  "is_current": true
}
```

Response HTTP 201 Created


```
{
  "message": "Term created successfully",
  "term": {
    "id": 7,
    "academic_year_id": 3,
    "academic_year": { "id": 3, "name": "2025/2026" },
    "name": "Second Term",
    "start_date": "2026-01-06",
    "end_date": "2026-04-10",
    "is_current": true,
    "created_at": "2026-03-30T08:15:00Z"
  }
}
```

Departments

CREATE A DEPARTMENT

Request

```
POST /api/v1/departments HTTP/1.1
Host: greenfield.compass.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
Content-Type: application/json
```

```
{
  "name": "Science",
  "description": "Biology, Chemistry, Physics and Mathematics department",
  "head_of_department_id": 12
}
```

Response HTTP 201 Created

```
{
  "message": "Department created successfully",
  "department": {
    "id": 2,
    "name": "Science",
    "description": "Biology, Chemistry, Physics and Mathematics department",
    "head_of_department": {
      "id": 12,
      "name": "Mr. Emeka Okonkwo"
    },
    "subjects_count": 0,
    "created_at": "2026-03-30T08:20:00Z"
  }
}
```

GET DEPARTMENT WITH SUBJECTS

Request

```
GET /api/v1/departments/2 HTTP/1.1
Host: greenfield.compass.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
```

Response HTTP 200 OK

```
{
  "data": {
    "id": 2,
    "name": "Science",
    "description": "Biology, Chemistry, Physics and Mathematics department",
    "head_of_department": { "id": 12, "name": "Mr. Emeka Okonkwo" },
    "subjects": [
      { "id": 5, "name": "Biology", "code": "BIO" },
      { "id": 6, "name": "Chemistry", "code": "CHE" },
      { "id": 7, "name": "Physics", "code": "PHY" }
    ]
  }
}
```

Classes

CREATE A CLASS

Request

```
POST /api/v1/classes HTTP/1.1
Host: greenfield.compass.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
Content-Type: application/json
```

```
{
  "name": "JSS 2",
  "level": "junior",
  "description": "Junior Secondary School Year 2"
}
```

Response HTTP 201 Created

```
{
  "message": "Class created successfully",
  "class": {
    "id": 4,
    "name": "JSS 2",
    "level": "junior",
    "description": "Junior Secondary School Year 2",
    "arms_count": 0,
    "students_count": 0,
    "created_at": "2026-03-30T08:30:00Z"
  }
}
```

GET CLASS WITH ARMS

Request

```
GET /api/v1/classes/4 HTTP/1.1
Host: greenfield.compass.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
```

Response HTTP 200 OK

```
{
  "data": {
    "id": 4,
    "name": "JSS 2",
    "level": "junior",
    "arms": [
      { "id": 7, "name": "A", "students_count": 38 },
      { "id": 8, "name": "B", "students_count": 35 }
    ],
    "students_count": 73
  }
}
```

Arms

CREATE AN ARM

Request

```
POST /api/v1/arms HTTP/1.1
Host: greenfield.compass.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
Content-Type: application/json
```

```
{
  "name": "A",
  "class_id": 4
}
```

Response HTTP 201 Created

```
{
  "message": "Arm created successfully",
  "arm": {
    "id": 7,
    "name": "A",
    "class_id": 4,
    "class": { "id": 4, "name": "JSS 2" },
    "students_count": 0,
    "created_at": "2026-03-30T08:35:00Z"
  }
}
```

ASSIGN ARM TO CLASS

Request

```
POST /api/v1/arms/assign-to-class HTTP/1.1
Host: greenfield.compassse.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
Content-Type: application/json

{
  "arm_id": 7,
  "class_id": 4
}
```

Response HTTP 200 OK

```
{
  "message": "Arm assigned to class successfully",
  "arm": { "id": 7, "name": "A" },
  "class": { "id": 4, "name": "JSS 2" }
}
```

REMOVE ARM FROM CLASS

Request

```
POST /api/v1/arms/remove-from-class HTTP/1.1
Host: greenfield.compassse.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
Content-Type: application/json

{
  "arm_id": 7,
  "class_id": 4
}
```

Response HTTP 200 OK

```
{
  "message": "Arm removed from class successfully"
}
```

LIST ARMS FOR A CLASS

Request

```
GET /api/v1/arms/class/4 HTTP/1.1
Host: greenfield.compassse.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
```

Response HTTP 200 OK

```
{
  "class": { "id": 4, "name": "JSS 2" },
  "arms": [
    { "id": 7, "name": "A", "students_count": 38 },
    { "id": 8, "name": "B", "students_count": 35 }
  ]
}
```

Subjects

CREATE A SUBJECT

Request

```
POST /api/v1/subjects HTTP/1.1
Host: greenfield.compass.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
Content-Type: application/json
```

```
{
  "name": "Chemistry",
  "code": "CHE",
  "department_id": 2,
  "class_id": 4,
  "description": "General Chemistry for JSS 2"
}
```

Response HTTP 201 Created

```
{
  "message": "Subject created successfully",
  "subject": {
    "id": 6,
    "name": "Chemistry",
    "code": "CHE",
    "department": { "id": 2, "name": "Science" },
    "class": { "id": 4, "name": "JSS 2" },
    "description": "General Chemistry for JSS 2",
    "created_at": "2026-03-30T08:45:00Z"
  }
}
```

Student Management

Required module: student_management

Student Endpoints

Method	Endpoint	Description
GET	/students	List students (filter: class_id , arm_id , status , search)
POST	/students	Enroll student (admission number auto-generated)
GET	/students/{id}	Get student profile
PUT	/students/{id}	Update student
DELETE	/students/{id}	Deactivate student
GET	/students/{id}/attendance	Student attendance records
GET	/students/{id}/results	Student exam results
GET	/students/{id}/assignments	Student assignments
GET	/students/{id}/subjects	Student subjects
POST	/students/generate-admission-number	Preview admission number
POST	/students/generate-credentials	Preview auto-generated email/username

Student Enrollment

FULL ENROLLMENT REQUEST (WITH GUARDIAN)

Request

```
POST /api/v1/students HTTP/1.1
Host: greenfield.compass.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
Content-Type: application/json
```

```
{
  "first_name": "Chukwuemeka",
  "last_name": "Obi",
  "middle_name": "Divine",
  "date_of_birth": "2012-07-14",
  "gender": "male",
  "class_id": 4,
  "arm_id": 7,
  "academic_year_id": 3,
  "term_id": 7,
  "religion": "Christianity",
  "state_of_origin": "Anambra",
  "nationality": "Nigerian",
  "address": "15 Adeola Odeku Street, Victoria Island, Lagos",
  "phone": "08011223344",
  "blood_group": "A+",
  "genotype": "AA",
  "profile_photo": null,
  "status": "active",
  "guardians": [
    {
      "first_name": "Obinna",
      "last_name": "Obi",
      "email": "obinna.obi@example.com",
      "phone": "08099887766",
      "relationship": "Father",
      "occupation": "Engineer",
      "address": "15 Adeola Odeku Street, Victoria Island, Lagos",
      "is_primary": true
    },
    {
      "first_name": "Ngozi",
      "last_name": "Obi",
      "email": "ngozi.obi@example.com",
      "phone": "08033445566",
      "relationship": "Mother",
      "occupation": "Teacher",
      "address": "15 Adeola Odeku Street, Victoria Island, Lagos",
      "is_primary": false
    }
  ]
}
```

Response HTTP 201 Created

```

{
  "message": "Student enrolled successfully",
  "student": {
    "id": 103,
    "admission_number": "GFA/2026/0103",
    "first_name": "Chukwuemeka",
    "last_name": "Obi",
    "middle_name": "Divine",
    "full_name": "Chukwuemeka Divine Obi",
    "date_of_birth": "2012-07-14",
    "gender": "male",
    "class": { "id": 4, "name": "JSS 2" },
    "arm": { "id": 7, "name": "A" },
    "academic_year": { "id": 3, "name": "2025/2026" },
    "term": { "id": 7, "name": "Second Term" },
    "status": "active",
    "guardians": [
      {
        "id": 55,
        "full_name": "Obinna Obi",
        "relationship": "Father",
        "phone": "08099887766",
        "is_primary": true
      },
      {
        "id": 56,
        "full_name": "Ngozi Obi",
        "relationship": "Mother",
        "phone": "08033445566",
        "is_primary": false
      }
    ]
  },
  "login_credentials": {
    "email": "chukwuemeka.obi@greenfield.edu.ng",
    "username": "chukwuemeka.obi",
    "password": "GFA@2026103",
    "note": "Student must change password on first login"
  }
}

```

WHAT HAPPENS DURING ENROLLMENT

1. All fields are validated.
2. Admission number is auto-generated in school-specific format (e.g. `GFA/2026/0103`).
3. School email is auto-generated (`firstname.lastname@schooldomain.com`).
4. A User account is created with a default password.
5. Guardian accounts are created if provided (up to 2), each with their own User login.
6. Guardians are attached to the student with `relationship` and `is_primary` flag.
7. Response includes `login_credentials` for handover to the parent.

LIST STUDENTS

Request


```
GET /api/v1/students?class_id=4&arm_id=7 HTTP/1.1
Host: greenfield.compass.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
```

Response HTTP 200 OK

```
{
  "data": [
    {
      "id": 103,
      "admission_number": "GFA/2026/0103",
      "full_name": "Chukwuemeka Divine Obi",
      "date_of_birth": "2012-07-14",
      "gender": "male",
      "class": { "id": 4, "name": "JSS 2" },
      "arm": { "id": 7, "name": "A" },
      "status": "active"
    }
  ],
  "meta": {
    "total": 38,
    "per_page": 15,
    "current_page": 1,
    "last_page": 3
  }
}
```

GET STUDENT ATTENDANCE

Request

```
GET /api/v1/students/103/attendance?term_id=7 HTTP/1.1
Host: greenfield.compass.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
```

Response HTTP 200 OK

```
{
  "student": { "id": 103, "full_name": "Chukwuemeka Divine Obi" },
  "term": { "id": 7, "name": "Second Term" },
  "summary": {
    "total_school_days": 52,
    "days_present": 48,
    "days_absent": 4,
    "attendance_percentage": 92.3
  },
  "records": [
    {
      "date": "2026-03-30",
      "status": "present",
      "remark": null
    },
    {
      "date": "2026-03-29",
      "status": "absent",
      "remark": "Sick"
    }
  ]
}
```

GET STUDENT RESULTS

Request

```
GET /api/v1/students/103/results?term_id=7 HTTP/1.1
Host: greenfield.compass.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
```

Response HTTP 200 OK

```
{
  "student": { "id": 103, "full_name": "Chukwuemeka Divine Obi" },
  "term": { "id": 7, "name": "Second Term" },
  "results": [
    {
      "subject": { "id": 6, "name": "Chemistry", "code": "CHE" },
      "ca_score": 28,
      "exam_score": 64,
      "total_score": 92,
      "grade": "A",
      "remark": "Excellent"
    },
    {
      "subject": { "id": 7, "name": "Physics", "code": "PHY" },
      "ca_score": 22,
      "exam_score": 58,
      "total_score": 80,
      "grade": "B",
      "remark": "Very Good"
    }
  ],
  "summary": {
    "total_subjects": 8,
    "average_score": 74.5,
    "class_position": 3,
    "class_size": 38
  }
}
```

GET STUDENT SUBJECTS

Request

```
GET /api/v1/students/103/subjects HTTP/1.1
Host: greenfield.compass.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
```

Response HTTP 200 OK

```
{
  "student": { "id": 103, "full_name": "Chukwuemeka Divine Obi" },
  "subjects": [
    { "id": 6, "name": "Chemistry", "code": "CHE", "teacher": { "name": "Mr. Emeka Okonkwo" } },
    { "id": 7, "name": "Physics", "code": "PHY", "teacher": { "name": "Mrs. Funke Adesanya" } }
  ]
}
```

Guardian Management

Required module: student_management

Guardian Endpoints

Method	Endpoint	Description
GET	/guardians	List guardians (filter: student_id , search)
POST	/guardians	Create guardian
GET	/guardians/{id}	Get guardian + linked students
PUT	/guardians/{id}	Update guardian
DELETE	/guardians/{id}	Delete guardian
POST	/guardians/{id}/assign-student	Assign student to guardian
POST	/guardians/{id}/remove-student	Remove student from guardian

CREATE GUARDIAN (STANDALONE)

Request

```
POST /api/v1/guardians HTTP/1.1
Host: greenfield.compass.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
Content-Type: application/json
```

```
{
  "first_name": "Chidi",
  "last_name": "Okafor",
  "email": "chidi.okafor@example.com",
  "phone": "08055667788",
  "relationship": "Uncle",
  "occupation": "Businessman",
  "address": "22 Broad Street, Lagos Island"
}
```

Response HTTP 201 Created

```
{
  "message": "Guardian created successfully",
  "guardian": {
    "id": 57,
    "first_name": "Chidi",
    "last_name": "Okafor",
    "full_name": "Chidi Okafor",
    "email": "chidi.okafor@example.com",
    "phone": "08055667788",
    "relationship": "Uncle",
    "occupation": "Businessman",
    "address": "22 Broad Street, Lagos Island",
    "students": []
  }
}
```

ASSIGN STUDENT TO GUARDIAN

Request

```
POST /api/v1/guardians/57/assign-student HTTP/1.1
Host: greenfield.compassse.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
Content-Type: application/json
```

```
{
  "student_id": 103,
  "relationship": "Uncle",
  "is_primary": false
}
```

Response HTTP 200 OK

```
{
  "message": "Student assigned to guardian successfully",
  "guardian": { "id": 57, "full_name": "Chidi Okafor" },
  "student": { "id": 103, "full_name": "Chukwuemeka Divine Obi" },
  "relationship": "Uncle",
  "is_primary": false
}
```

REMOVE STUDENT FROM GUARDIAN

Request

```
POST /api/v1/guardians/57/remove-student HTTP/1.1
Host: greenfield.compassse.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
Content-Type: application/json
```

```
{
  "student_id": 103
}
```

Response HTTP 200 OK

```
{
  "message": "Student removed from guardian successfully"
}
```

Teacher Management

Required module: `teacher_management`

Teacher Endpoints

Method	Endpoint	Description
GET	/teachers	List teachers (filter: department_id , status , search)
POST	/teachers	Create teacher (with user account)
GET	/teachers/{id}	Get teacher profile
PUT	/teachers/{id}	Update teacher
DELETE	/teachers/{id}	Delete teacher
GET	/teachers/{id}/classes	Classes assigned to this teacher
GET	/teachers/{id}/subjects	Subjects taught by this teacher
GET	/teachers/{id}/students	All students across the teacher's classes

CREATE TEACHER

Request

```
POST /api/v1/teachers HTTP/1.1
Host: greenfield.compass.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
Content-Type: application/json
```

```
{
  "first_name": "Emeka",
  "last_name": "Okonkwo",
  "email": "emeka.okonkwo@greenfield.edu.ng",
  "phone": "08077889900",
  "department_id": 2,
  "qualification": "B.Sc Chemistry, PGDE",
  "date_of_joining": "2020-09-01",
  "status": "active"
}
```

Response HTTP 201 Created

```
{
  "message": "Teacher created successfully",
  "teacher": {
    "id": 12,
    "staff_id": "GFA/TCH/0012",
    "full_name": "Emeka Okonkwo",
    "email": "emeka.okonkwo@greenfield.edu.ng",
    "phone": "08077889900",
    "department": { "id": 2, "name": "Science" },
    "qualification": "B.Sc Chemistry, PGDE",
    "date_of_joining": "2020-09-01",
    "status": "active"
  },
  "login_credentials": {
    "email": "emeka.okonkwo@greenfield.edu.ng",
    "password": "GFA@TCH0012",
    "note": "Teacher must change password on first login"
  }
}
```

GET TEACHER'S CLASSES

Request

```
GET /api/v1/teachers/12/classes HTTP/1.1
Host: greenfield.compass.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
```

Response HTTP 200 OK

```
{
  "teacher": { "id": 12, "full_name": "Emeka Okonkwo" },
  "classes": [
    {
      "id": 4,
      "name": "JSS 2",
      "arms": [
        { "id": 7, "name": "A", "students_count": 38 },
        { "id": 8, "name": "B", "students_count": 35 }
      ],
      "role": "Class Teacher"
    },
    {
      "id": 5,
      "name": "JSS 3",
      "arms": [
        { "id": 9, "name": "A", "students_count": 40 }
      ],
      "role": "Subject Teacher"
    }
  ]
}
```

GET TEACHER'S SUBJECTS

Request

```
GET /api/v1/teachers/12/subjects HTTP/1.1
Host: greenfield.compassse.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
```

Response HTTP 200 OK

```
{
  "teacher": { "id": 12, "full_name": "Emeka Okonkwo" },
  "subjects": [
    { "id": 6, "name": "Chemistry", "code": "CHE", "class": { "name": "JSS 2" } },
    { "id": 6, "name": "Chemistry", "code": "CHE", "class": { "name": "JSS 3" } }
  ]
}
```

GET TEACHER'S STUDENTS

Request

```
GET /api/v1/teachers/12/students HTTP/1.1
Host: greenfield.compassse.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
```

Response HTTP 200 OK

```
{
  "teacher": { "id": 12, "full_name": "Emeka Okonkwo" },
  "students": [
    {
      "id": 103,
      "full_name": "Chukwuemeka Divine Obi",
      "admission_number": "GFA/2026/0103",
      "class": "JSS 2",
      "arm": "A"
    }
  ],
  "meta": { "total": 113 }
}
```

School Structure Endpoints

Get School Organogram

```
GET /api/v1/schools/{id}/organogram HTTP/1.1
Host: greenfield.compassse.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
```

Returns the full school org chart: departments → teachers, classes → arms → students.

Response HTTP 200 OK


```
{
  "school": { "id": 1, "name": "Greenfield Academy" },
  "departments": [
    {
      "id": 2,
      "name": "Science",
      "head": { "id": 12, "name": "Mr. Emeka Okonkwo" },
      "teachers": [
        { "id": 12, "name": "Mr. Emeka Okonkwo", "subjects": ["Chemistry"] }
      ]
    }
  ],
  "classes": [
    {
      "id": 4,
      "name": "JSS 2",
      "arms": [
        { "id": 7, "name": "A", "students_count": 38, "class_teacher": { "name": "Mr. Emeka Okonkwo" } }
      ]
    }
  ]
}
```

Get School Stats

```
GET /api/v1/schools/{id}/stats HTTP/1.1
Host: greenfield.compassse.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
```

Response HTTP 200 OK

```
{
  "students": 412,
  "teachers": 28,
  "classes": 9,
  "arms": 18,
  "subjects": 42,
  "departments": 4,
  "active_terms": 1,
  "current_academic_year": "2025/2026"
}
```

Get Current School

```
GET /api/v1/schools/me HTTP/1.1
Host: greenfield.compassse.net
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
```

Returns the current authenticated school admin's school profile.

Business Rules

1. **Admission number** — Auto-generated per school on student creation; format is configured per school (e.g. `{prefix}/{year}/{sequence}`).
 2. **Email auto-generation** — Student school email is generated from `firstname.lastname@schooldomain.com` . Collisions are resolved by appending a number.
 3. **Guardian limit** — A student can have multiple guardians but only one marked `is_primary = true` .
 4. **Current academic year / term** — Only one academic year and one term per school can be `is_current = true` at a time.
 5. **Cascade delete protection** — Classes cannot be deleted with active students; departments cannot be deleted with subjects.
 6. **Teacher module independence** — `teacher_management` is a separate module gate from `academic_management` . A school can have academic structure without teacher management enabled, but teacher sub-resources require the teacher module.
-

Frontend Integration

How the Frontend Handles Tenancy

All academic API calls go to `https://{school}.compasse.net/api/v1/...` with `Authorization: Bearer {token}` . Three module flags control visibility:

- `academic_management` — academic years, terms, classes, arms, subjects, departments
- `student_management` — student enrollment, profiles, sub-resources, guardians
- `teacher_management` — teacher profiles and sub-resources

The frontend checks all three on app boot and conditionally renders the relevant nav sections.

Academic Year Selector

The academic year selector is a global dropdown in the top navigation bar. It is populated from `GET /api/v1/academic-years` . When the user changes the selected year, a React/Vue context value (or Pinia/Redux store) is updated.

Effect on all views: Every view that lists students, results, attendance, or fees reads the selected `academic_year_id` from context and appends it as a query parameter on all API calls. This means switching the year filter immediately changes all data displayed across the app without a full page reload.

The current academic year (`is_current: true`) is selected by default on app load.

Class → Arm Drill-Down Navigation

The class structure is navigated in two levels:

1. **Class list** — `GET /api/v1/classes` — shown as a grid of cards (one per class). Each card shows the class name and total student count.
2. **Arm list** — On clicking a class, the frontend calls `GET /api/v1/arms/class/{classId}` and shows a list of arms with their student counts.

3. **Student list** — On clicking an arm, the frontend calls `GET /api/v1/students?class_id={id}&arm_id={id}` and renders the student table with search and filter controls.

Breadcrumb navigation is shown throughout: `Classes > JSS 2 > Arm A > Students` .

This drill-down pattern ensures the frontend never loads all students at once — it always scopes to a specific class and arm before listing students.

Assessment, CBT & Results

Base URL: `https://{subdomain}.compasse.net/api/v1/` **Auth:** `Authorization: Bearer {token}`
required on all protected endpoints **Module gate:** `cbt`

Overview

The assessment module covers Computer-Based Testing (CBT), assignments, continuous assessment (CA), psychomotor assessment, grading systems, result generation, report cards, scoreboards, and student promotion. All endpoints are scoped to the authenticated school's tenant database.

User Stories

As a teacher, I want to create exams with questions and set a time limit so students take tests on their devices.

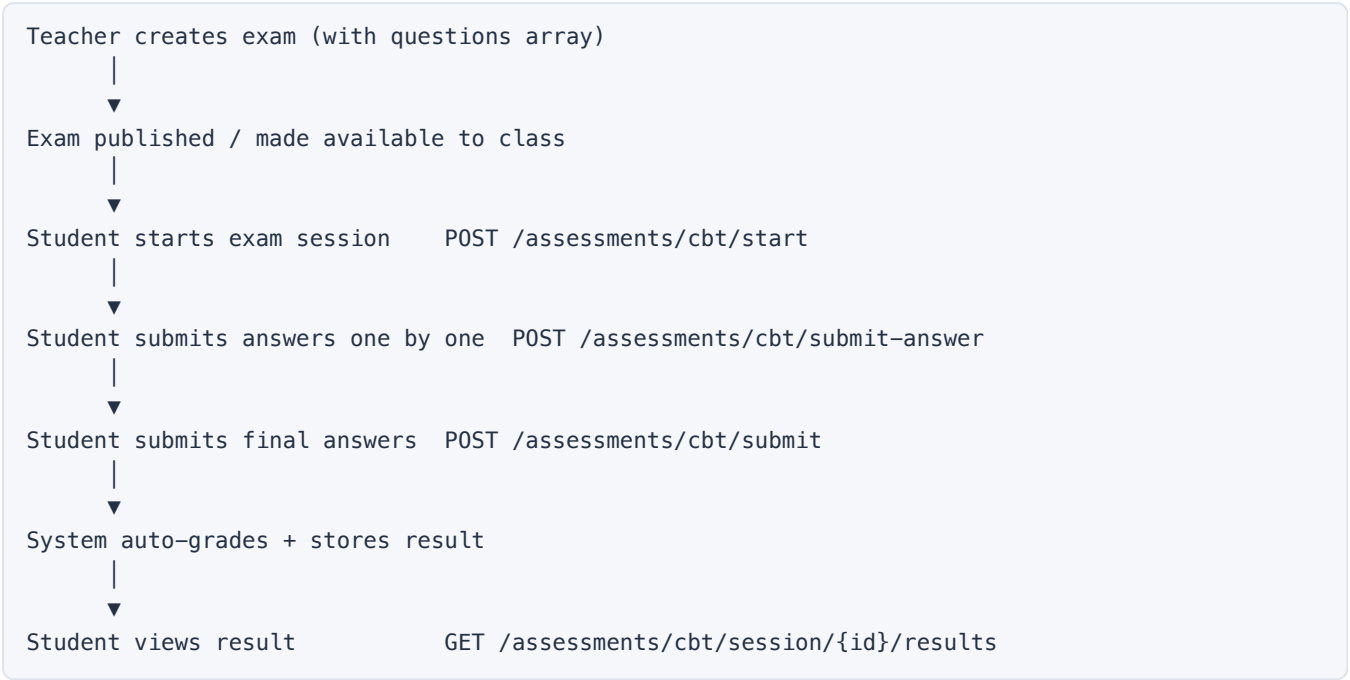
As a student, I want to start a CBT exam, answer questions one at a time, and see my result immediately after submission.

As a teacher, I want to create assignments, receive submissions, and grade each one.

As a principal, I want to generate end-of-term results that combine CA scores and exam scores, produce report cards, and bulk-email them to parents.

As an admin, I want to promote students who pass to the next class at the end of the year, and graduate SS3 students.

Exam Workflow



API Endpoints

Base path: /api/v1/assessments/

Exams

Method	Endpoint	Description
GET	/exams	List exams
POST	/exams	Create exam with questions array
GET	/exams/{id}	Get exam details
PUT	/exams/{id}	Update exam
DELETE	/exams/{id}	Delete exam

CBT Sessions

Method	Endpoint	Description
POST	/cbt/start	Start a CBT session
POST	/cbt/submit-answer	Submit a single answer during session
POST	/cbt/submit	Submit all answers and get score
GET	/cbt/{exam}/questions	Get questions for an exam
GET	/cbt/attempts/{attempt}/status	Check session status
GET	/cbt/session/{id}/results	Get session result

Assignments

Method	Endpoint	Description
GET	/assignments	List assignments
POST	/assignments	Create assignment
GET	/assignments/{id}	Get assignment details
PUT	/assignments/{id}	Update assignment
DELETE	/assignments/{id}	Delete assignment
GET	/assignments/{id}/submissions	List submissions + statistics
POST	/assignments/{id}/submit	Student submits assignment
PUT	/assignments/{id}/grade	Teacher grades submission

Continuous Assessment

Method	Endpoint	Description
GET	/continuous-assessments	List CA entries
POST	/continuous-assessments	Create CA
PUT	/continuous-assessments/{id}	Update CA
DELETE	/continuous-assessments/{id}	Delete CA
POST	/continuous-assessments/{id}/record-scores	Bulk record scores
GET	/continuous-assessments/{id}/scores	Get scores for CA
GET	/continuous-assessments/student/{id}/scores	All CA scores for a student

Psychomotor Assessment

Method	Endpoint	Description
GET	/psychomotor-assessments/{studentId}/{termId}/{yearId}	Get student's psychomotor report
POST	/psychomotor-assessments	Record single assessment
POST	/psychomotor-assessments/bulk	Bulk record for entire class
GET	/psychomotor-assessments/class/{classId}	Class psychomotor report
DELETE	/psychomotor-assessments/{id}	Delete record

Grading System

Method	Endpoint	Description
GET	/grading-systems	List grading systems
GET	/grading-systems/default	Get default grading system
POST	/grading-systems	Create grading system
PUT	/grading-systems/{id}	Update grading system
DELETE	/grading-systems/{id}	Delete grading system
POST	/grading-systems/calculate-grade	Calculate grade for a raw score

Results

Method	Endpoint	Description
POST	/results/generate	Generate results for a term + class
GET	/results/student/{id}/{termId}/{yearId}	Get full student result sheet
GET	/results/class/{classId}	Class result summary
POST	/results/{id}/comments	Add teacher / principal comment
POST	/results/{id}/approve	Approve a result
POST	/results/publish	Publish results to students and parents

Scoreboard

Method	Endpoint	Description
GET	/scoreboards/class/{classId}	Class rankings
GET	/scoreboards/top-performers	Top students school-wide
GET	/scoreboards/subject/{subjectId}/toppers	Subject top scorers
GET	/scoreboards/class-comparison	Cross-class comparison
POST	/scoreboards/refresh	Manually refresh cached scoreboard

Report Cards

Method	Endpoint	Description
GET	/report-cards/{student}/{term}/{year}	Get report card data (JSON)
GET	/report-cards/{student}/{term}/{year}/pdf	Download PDF (binary)
GET	/report-cards/{student}/{term}/{year}/print	Printable HTML version
POST	/report-cards/bulk-download	Bulk PDF for entire class
POST	/report-cards/{student}/{term}/{year}/email	Email report card to parent

Promotion & Graduation

Method	Endpoint	Description
GET	/promotions	List all promotions
POST	/promotions/promote	Promote a single student
POST	/promotions/bulk-promote	Promote an entire class
POST	/promotions/auto-promote	Auto-promote based on result data
POST	/promotions/graduate	Graduate SS3 / final-year students
GET	/promotions/statistics	Promotion statistics
DELETE	/promotions/{id}	Undo a promotion

Complete Request / Response Examples

Create Exam (with questions array)

POST /api/v1/assessments/exams

Authorization: Bearer {token}

Content-Type: application/json

```
{
  "title": "Mathematics Mid-Term Exam",
  "subject_id": 4,
  "class_id": 7,
  "term_id": 2,
  "academic_year_id": 3,
  "duration_minutes": 60,
  "total_marks": 40,
  "pass_mark": 20,
  "instructions": "Answer all questions. No calculators allowed.",
  "shuffle_questions": true,
  "shuffle_options": true,
  "show_result_immediately": true,
  "status": "draft",
  "questions": [
    {
      "question_text": "What is the value of x if  $2x + 4 = 10$ ?",
      "question_type": "multiple_choice",
      "marks": 2,
      "order": 1,
      "options": [
        { "option_text": "2", "is_correct": false },
        { "option_text": "3", "is_correct": true },
        { "option_text": "4", "is_correct": false },
        { "option_text": "5", "is_correct": false }
      ]
    },
    {
      "question_text": "The square root of 144 is ____.",
      "question_type": "multiple_choice",
      "marks": 2,
      "order": 2,
      "options": [
        { "option_text": "10", "is_correct": false },
        { "option_text": "11", "is_correct": false },
        { "option_text": "12", "is_correct": true },
        { "option_text": "14", "is_correct": false }
      ]
    },
    {
      "question_text": "Pi is approximately equal to 3.14159.",
      "question_type": "true_false",
      "marks": 1,
      "order": 3,
      "options": [
        { "option_text": "True", "is_correct": true },
        { "option_text": "False", "is_correct": false }
      ]
    }
  ]
}
```

```
]
}
```

Response 201 Created :

```
{
  "message": "Exam created successfully",
  "data": {
    "id": 18,
    "title": "Mathematics Mid-Term Exam",
    "subject_id": 4,
    "class_id": 7,
    "term_id": 2,
    "academic_year_id": 3,
    "duration_minutes": 60,
    "total_marks": 40,
    "pass_mark": 20,
    "status": "draft",
    "questions_count": 3,
    "created_at": "2026-03-30T08:00:00.000000Z"
  }
}
```

List Exams

```
GET /api/v1/assessments/exams?class_id=7&term_id=2&status=published
Authorization: Bearer {token}
```

Response 200 OK :

```
{
  "data": [
    {
      "id": 18,
      "title": "Mathematics Mid-Term Exam",
      "subject": { "id": 4, "name": "Mathematics" },
      "class": { "id": 7, "name": "JSS 1A" },
      "duration_minutes": 60,
      "total_marks": 40,
      "status": "published",
      "questions_count": 40,
      "created_at": "2026-03-30T08:00:00.000000Z"
    }
  ],
  "current_page": 1,
  "last_page": 1,
  "per_page": 15,
  "total": 1
}
```

Get Exam Details

```
GET /api/v1/assessments/exams/18
Authorization: Bearer {token}
```

Response 200 OK :

```
{
  "data": {
    "id": 18,
    "title": "Mathematics Mid-Term Exam",
    "subject": { "id": 4, "name": "Mathematics" },
    "class": { "id": 7, "name": "JSS 1A" },
    "term": { "id": 2, "name": "Second Term" },
    "academic_year": { "id": 3, "name": "2025/2026" },
    "duration_minutes": 60,
    "total_marks": 40,
    "pass_mark": 20,
    "status": "published",
    "shuffle_questions": true,
    "shuffle_options": true,
    "show_result_immediately": true,
    "instructions": "Answer all questions. No calculators allowed.",
    "questions": [
      {
        "id": 101,
        "question_text": "What is the value of x if  $2x + 4 = 10$ ?",
        "question_type": "multiple_choice",
        "marks": 2,
        "order": 1,
        "options": [
          { "id": 1, "option_text": "2", "is_correct": false },
          { "id": 2, "option_text": "3", "is_correct": true },
          { "id": 3, "option_text": "4", "is_correct": false },
          { "id": 4, "option_text": "5", "is_correct": false }
        ]
      }
    ]
  },
  "created_at": "2026-03-30T08:00:00.000000Z"
}
```

CBT Session Flow

STEP 1 — START SESSION

```
POST /api/v1/assessments/cbt/start
Authorization: Bearer {token}
Content-Type: application/json
```

```
{
  "exam_id": 18
}
```

Response 201 Created :

```
{
  "message": "Exam session started",
  "data": {
    "session_id": 55,
    "exam_id": 18,
    "student_id": 210,
    "started_at": "2026-03-30T09:00:00.000000Z",
    "expires_at": "2026-03-30T10:00:00.000000Z",
    "duration_minutes": 60,
    "total_questions": 40,
    "questions": [
      {
        "id": 101,
        "question_text": "What is the value of x if  $2x + 4 = 10$ ?",
        "question_type": "multiple_choice",
        "marks": 2,
        "order": 1,
        "options": [
          { "id": 1, "option_text": "2" },
          { "id": 2, "option_text": "3" },
          { "id": 3, "option_text": "4" },
          { "id": 4, "option_text": "5" }
        ]
      }
    ]
  }
}
```

Note: Correct answer flags (`is_correct`) are stripped from the student-facing question list.

STEP 2 — SUBMIT SINGLE ANSWER (PER QUESTION)

```
POST /api/v1/assessments/cbt/submit-answer
Authorization: Bearer {token}
Content-Type: application/json

{
  "session_id": 55,
  "question_id": 101,
  "selected_option_id": 2
}
```

Response 200 OK :

```
{
  "message": "Answer saved",
  "data": {
    "session_id": 55,
    "question_id": 101,
    "selected_option_id": 2,
    "saved_at": "2026-03-30T09:03:45.000000Z"
  }
}
```

STEP 3 — SUBMIT ALL AND GET SCORE

```
POST /api/v1/assessments/cbt/submit
Authorization: Bearer {token}
Content-Type: application/json
```

```
{
  "session_id": 55
}
```

Response 200 OK :

```
{
  "message": "Exam submitted successfully",
  "data": {
    "session_id": 55,
    "exam_title": "Mathematics Mid-Term Exam",
    "student": { "id": 210, "name": "Tolu Adeyemi" },
    "score": 32,
    "total_marks": 40,
    "percentage": 80.0,
    "passed": true,
    "pass_mark": 20,
    "grade": "A",
    "remark": "Excellent",
    "time_taken_minutes": 45,
    "submitted_at": "2026-03-30T09:45:00.000000Z",
    "correct_answers": 16,
    "wrong_answers": 4,
    "unanswered": 0
  }
}
```

GET SESSION RESULT

```
GET /api/v1/assessments/cbt/session/55/results
Authorization: Bearer {token}
```

Response 200 OK :

```
{
  "data": {
    "session_id": 55,
    "exam": { "id": 18, "title": "Mathematics Mid-Term Exam" },
    "student": { "id": 210, "name": "Tolu Adeyemi" },
    "score": 32,
    "total_marks": 40,
    "percentage": 80.0,
    "passed": true,
    "grade": "A",
    "remark": "Excellent",
    "submitted_at": "2026-03-30T09:45:00.000000Z",
    "answers": [
      {
        "question_id": 101,
        "question_text": "What is the value of x if  $2x + 4 = 10$ ?",
        "selected_option": "3",
        "correct_option": "3",
        "is_correct": true,
        "marks_earned": 2
      }
    ]
  }
}
```

Assignment CRUD

CREATE ASSIGNMENT

```
POST /api/v1/assessments/assignments
Authorization: Bearer {token}
Content-Type: application/json

{
  "title": "Essay: Causes of World War II",
  "description": "Write a 500-word essay on the causes of World War II.",
  "subject_id": 6,
  "class_id": 9,
  "term_id": 2,
  "academic_year_id": 3,
  "due_date": "2026-04-10",
  "total_marks": 20,
  "attachment_url": null
}
```

Response 201 Created:

```
{
  "message": "Assignment created successfully",
  "data": {
    "id": 33,
    "title": "Essay: Causes of World War II",
    "subject": { "id": 6, "name": "History" },
    "class": { "id": 9, "name": "SS 2A" },
    "due_date": "2026-04-10",
    "total_marks": 20,
    "submissions_count": 0,
    "created_at": "2026-03-30T10:00:00.000000Z"
  }
}
```

LIST ASSIGNMENTS

```
GET /api/v1/assessments/assignments?class_id=9&subject_id=6&term_id=2
Authorization: Bearer {token}
```

Response 200 OK:

```
{
  "data": [
    {
      "id": 33,
      "title": "Essay: Causes of World War II",
      "subject": { "id": 6, "name": "History" },
      "class": { "id": 9, "name": "SS 2A" },
      "due_date": "2026-04-10",
      "total_marks": 20,
      "submissions_count": 18,
      "graded_count": 5,
      "created_at": "2026-03-30T10:00:00.000000Z"
    }
  ],
  "current_page": 1,
  "last_page": 1,
  "per_page": 15,
  "total": 1
}
```

STUDENT SUBMITS ASSIGNMENT

```
POST /api/v1/assessments/assignments/33/submit
Authorization: Bearer {token}
Content-Type: application/json
```

```
{
  "student_id": 210,
  "submission_text": "World War II was caused by multiple interrelated factors including the rise of fascism, the failure of appeasement...",
  "attachment_url": "https://storage.compass.net/submissions/essay-33-210.pdf"
}
```

Response 201 Created :

```
{
  "message": "Assignment submitted successfully",
  "data": {
    "id": 88,
    "assignment_id": 33,
    "student": { "id": 210, "name": "Tolu Adeyemi" },
    "submitted_at": "2026-04-09T14:22:00.000000Z",
    "status": "submitted",
    "is_late": false
  }
}
```

TEACHER GRADES SUBMISSION

PUT /api/v1/assessments/assignments/33/grade
Authorization: Bearer {token}
Content-Type: application/json

```
{
  "submission_id": 88,
  "score": 17,
  "feedback": "Good structure and solid analysis. Work on your conclusion."
}
```

Response 200 OK :

```
{
  "message": "Submission graded successfully",
  "data": {
    "submission_id": 88,
    "student": { "id": 210, "name": "Tolu Adeyemi" },
    "score": 17,
    "total_marks": 20,
    "percentage": 85.0,
    "grade": "A",
    "feedback": "Good structure and solid analysis. Work on your conclusion.",
    "graded_by": { "id": 3, "name": "Mrs. Funmilayo Adesanya" },
    "graded_at": "2026-04-11T09:00:00.000000Z"
  }
}
```

LIST SUBMISSIONS FOR AN ASSIGNMENT

GET /api/v1/assessments/assignments/33/submissions
Authorization: Bearer {token}

Response 200 OK :


```
{
  "assignment": { "id": 33, "title": "Essay: Causes of World War II", "total_marks": 20 },
  "statistics": {
    "total_students": 30,
    "submitted": 18,
    "not_submitted": 12,
    "graded": 5,
    "average_score": 14.2
  },
  "submissions": [
    {
      "id": 88,
      "student": { "id": 210, "name": "Tolu Adeyemi" },
      "submitted_at": "2026-04-09T14:22:00.000000Z",
      "score": 17,
      "status": "graded",
      "is_late": false
    }
  ]
}
```

Continuous Assessment (CA)

CREATE CA

```
POST /api/v1/assessments/continuous-assessments
Authorization: Bearer {token}
Content-Type: application/json

{
  "title": "First CA – Mathematics",
  "subject_id": 4,
  "class_id": 7,
  "term_id": 2,
  "academic_year_id": 3,
  "total_marks": 20,
  "ca_type": "test",
  "date": "2026-03-15"
}
```

Response 201 Created :

```
{
  "message": "Continuous assessment created",
  "data": {
    "id": 12,
    "title": "First CA – Mathematics",
    "subject": { "id": 4, "name": "Mathematics" },
    "class": { "id": 7, "name": "JSS 1A" },
    "total_marks": 20,
    "ca_type": "test",
    "date": "2026-03-15",
    "scores_recorded": 0
  }
}
```

BULK RECORD SCORES

POST /api/v1/assessments/continuous-assessments/12/record-scores
Authorization: Bearer {token}
Content-Type: application/json

```
{
  "scores": [
    { "student_id": 210, "score": 18 },
    { "student_id": 211, "score": 15 },
    { "student_id": 212, "score": 12 },
    { "student_id": 213, "score": 20 },
    { "student_id": 214, "score": 9 }
  ]
}
```

Response 200 OK :

```
{
  "message": "Scores recorded successfully",
  "data": {
    "ca_id": 12,
    "scores_recorded": 5,
    "average_score": 14.8,
    "highest_score": 20,
    "lowest_score": 9
  }
}
```

GET SCORES FOR A CA

GET /api/v1/assessments/continuous-assessments/12/scores
Authorization: Bearer {token}

Response 200 OK :

```
{
  "ca": {
    "id": 12,
    "title": "First CA – Mathematics",
    "total_marks": 20
  },
  "scores": [
    { "student": { "id": 210, "name": "Tolu Adeyemi" }, "score": 18, "percentage": 90.0 },
    { "student": { "id": 211, "name": "Kemi Okafor" }, "score": 15, "percentage": 75.0 }
  ],
  "statistics": {
    "average": 14.8,
    "highest": 20,
    "lowest": 9,
    "pass_rate": 80.0
  }
}
```

Psychomotor Assessment

RECORD SINGLE PSYCHOMOTOR ASSESSMENT

```
POST /api/v1/assessments/psychomotor-assessments
Authorization: Bearer {token}
Content-Type: application/json
```

```
{
  "student_id": 210,
  "term_id": 2,
  "academic_year_id": 3,
  "skill": "Handwriting",
  "rating": 4,
  "remark": "Neat and well-formed letters",
  "teacher_id": 3
}
```

Response 201 Created :

```
{
  "message": "Psychomotor assessment recorded",
  "data": {
    "id": 45,
    "student": { "id": 210, "name": "Tolu Adeyemi" },
    "skill": "Handwriting",
    "rating": 4,
    "remark": "Neat and well-formed letters",
    "term": { "id": 2, "name": "Second Term" },
    "created_at": "2026-03-30T10:00:00.000000Z"
  }
}
```

BULK RECORD FOR ENTIRE CLASS

```
POST /api/v1/assessments/psychomotor-assessments/bulk
Authorization: Bearer {token}
Content-Type: application/json
```

```
{
  "class_id": 7,
  "term_id": 2,
  "academic_year_id": 3,
  "skill": "Drawing",
  "assessments": [
    { "student_id": 210, "rating": 5, "remark": "Exceptional artistic talent" },
    { "student_id": 211, "rating": 3, "remark": "Average performance" },
    { "student_id": 212, "rating": 4, "remark": "Good creativity" }
  ]
}
```

Response 200 OK :

```
{
  "message": "Bulk psychomotor assessments recorded",
  "data": {
    "class_id": 7,
    "skill": "Drawing",
    "recorded_count": 3,
    "average_rating": 4.0
  }
}
```

Grading System

CREATE GRADING SYSTEM

```
POST /api/v1/assessments/grading-systems
Authorization: Bearer {token}
Content-Type: application/json
```

```
{
  "name": "Standard Nigerian WAEC Scale",
  "is_default": true,
  "grade_ranges": [
    { "min_score": 75, "max_score": 100, "grade": "A1", "remark": "Excellent", "points": 5 },
    { "min_score": 70, "max_score": 74, "grade": "B2", "remark": "Very Good", "points": 4 },
    { "min_score": 65, "max_score": 69, "grade": "B3", "remark": "Good", "points": 3 },
    { "min_score": 60, "max_score": 64, "grade": "C4", "remark": "Credit", "points": 3 },
    { "min_score": 55, "max_score": 59, "grade": "C5", "remark": "Credit", "points": 2 },
    { "min_score": 50, "max_score": 54, "grade": "C6", "remark": "Credit", "points": 2 },
    { "min_score": 45, "max_score": 49, "grade": "D7", "remark": "Pass", "points": 1 },
    { "min_score": 40, "max_score": 44, "grade": "E8", "remark": "Pass", "points": 1 },
    { "min_score": 0, "max_score": 39, "grade": "F9", "remark": "Fail", "points": 0 }
  ]
}
```

Response 201 Created :

```
{
  "message": "Grading system created successfully",
  "data": {
    "id": 2,
    "name": "Standard Nigerian WAEC Scale",
    "is_default": true,
    "grade_ranges_count": 9,
    "created_at": "2026-03-30T10:00:00.000000Z"
  }
}
```

CALCULATE GRADE FOR A SCORE

```
POST /api/v1/assessments/grading-systems/calculate-grade
Authorization: Bearer {token}
Content-Type: application/json

{
  "score": 72,
  "grading_system_id": 2
}
```

Response 200 OK :

```
{
  "score": 72,
  "grade": "B2",
  "remark": "Very Good",
  "points": 4,
  "grading_system": "Standard Nigerian WAEC Scale"
}
```

Result Generation

GENERATE RESULTS (TERM + CLASS)

```
POST /api/v1/assessments/results/generate
Authorization: Bearer {token}
Content-Type: application/json

{
  "term_id": 2,
  "academic_year_id": 3,
  "class_id": 7
}
```

Response 200 OK :

```
{
  "message": "Results generated successfully",
  "data": {
    "class": { "id": 7, "name": "JSS 1A" },
    "term": { "id": 2, "name": "Second Term" },
    "academic_year": { "id": 3, "name": "2025/2026" },
    "students_processed": 30,
    "subjects_included": 9,
    "generated_at": "2026-03-30T11:00:00.000000Z"
  }
}
```

GET STUDENT RESULT SHEET (FULL JSON)

```
GET /api/v1/assessments/results/student/210/2/3
Authorization: Bearer {token}
```

Response 200 OK:

```

{
  "data": {
    "student": {
      "id": 210,
      "name": "Tolu Adeyemi",
      "admission_number": "GF/2024/001",
      "class": "JSS 1A",
      "photo_url": "https://storage.compassse.net/photos/210.jpg"
    },
    "term": { "id": 2, "name": "Second Term" },
    "academic_year": { "id": 3, "name": "2025/2026" },
    "school": { "name": "Greenfield Academy", "logo_url": "... " },
    "subjects": [
      {
        "subject": { "id": 4, "name": "Mathematics" },
        "ca_score": 36,
        "exam_score": 64,
        "total_score": 100,
        "grade": "A1",
        "remark": "Excellent",
        "teacher": "Mr. Biodun Okonkwo",
        "teacher_comment": "Outstanding performance. Keep it up!"
      },
      {
        "subject": { "id": 6, "name": "History" },
        "ca_score": 28,
        "exam_score": 55,
        "total_score": 83,
        "grade": "B2",
        "remark": "Very Good",
        "teacher": "Mrs. Funmilayo Adesanya",
        "teacher_comment": "Good work. Pay attention to dates."
      }
    ],
    "summary": {
      "total_subjects": 9,
      "total_score_obtained": 720,
      "total_score_obtainable": 900,
      "average": 80.0,
      "overall_grade": "A",
      "class_position": 3,
      "class_size": 30,
      "times_present": 52,
      "times_absent": 6,
      "next_term_begins": "2026-05-06"
    },
    "psychomotor": [
      { "skill": "Handwriting", "rating": 4, "remark": "Neat" },
      { "skill": "Drawing", "rating": 5, "remark": "Excellent" }
    ],
    "teacher_comment": "Tolu is a dedicated student. Encourage her to participate more in class.",
    "principal_comment": "A very promising student. We look forward to her continued growth.",
    "status": "approved",
    "approved_by": { "id": 1, "name": "Mr. Chukwuemeka Obi" },
    "approved_at": "2026-03-28T14:00:00.000000Z"
  }
}

```

```
}  
}
```

ADD TEACHER / PRINCIPAL COMMENT

POST /api/v1/assessments/results/77/comments

Authorization: Bearer {token}

Content-Type: application/json

```
{  
  "teacher_comment": "Tolu is a dedicated student with great potential.",  
  "principal_comment": "A very promising student. We expect continued excellence."  
}
```

Response 200 OK :

```
{  
  "message": "Comments added successfully",  
  "data": {  
    "result_id": 77,  
    "teacher_comment": "Tolu is a dedicated student with great potential.",  
    "principal_comment": "A very promising student. We expect continued excellence."  
  }  
}
```

APPROVE RESULT

POST /api/v1/assessments/results/77/approve

Authorization: Bearer {token}

Response 200 OK :

```
{  
  "message": "Result approved successfully",  
  "data": {  
    "result_id": 77,  
    "status": "approved",  
    "approved_by": { "id": 1, "name": "Mr. Chukwuemeka Obi" },  
    "approved_at": "2026-03-30T14:00:00.000000Z"  
  }  
}
```

PUBLISH RESULTS


```
POST /api/v1/assessments/results/publish
Authorization: Bearer {token}
Content-Type: application/json
```

```
{
  "term_id": 2,
  "academic_year_id": 3,
  "class_id": 7,
  "notify_parents": true,
  "notification_channel": "sms"
}
```

Response 200 OK:

```
{
  "message": "Results published successfully",
  "data": {
    "published_count": 30,
    "notifications_queued": 28,
    "published_at": "2026-03-30T15:00:00.000000Z"
  }
}
```

Scoreboard

CLASS RANKINGS

```
GET /api/v1/assessments/scoreboards/class/7?term_id=2&academic_year_id=3
Authorization: Bearer {token}
```

Response 200 OK:

```
{
  "class": { "id": 7, "name": "JSS 1A" },
  "term": { "id": 2, "name": "Second Term" },
  "rankings": [
    {
      "position": 1,
      "student": { "id": 215, "name": "Amara Eze", "admission_number": "GF/2024/006" },
      "total_score": 845,
      "average": 93.9,
      "overall_grade": "A1"
    },
    {
      "position": 2,
      "student": { "id": 218, "name": "Emeka Nwosu" },
      "total_score": 830,
      "average": 92.2,
      "overall_grade": "A1"
    },
    {
      "position": 3,
      "student": { "id": 210, "name": "Tolu Adeyemi" },
      "total_score": 720,
      "average": 80.0,
      "overall_grade": "A"
    }
  ],
  "total_students": 30
}
```

TOP PERFORMERS SCHOOL-WIDE

```
GET /api/v1/assessments/scoreboards/top-performers?term_id=2&academic_year_id=3&limit=10
Authorization: Bearer {token}
```

Response 200 OK :

```
{
  "term": { "id": 2, "name": "Second Term" },
  "top_performers": [
    {
      "position": 1,
      "student": { "id": 215, "name": "Amara Eze", "class": "JSS 1A" },
      "average": 93.9,
      "overall_grade": "A1"
    }
  ],
  "total_in_school": 450
}
```

Report Card

GET REPORT CARD DATA (JSON)

```
GET /api/v1/assessments/report-cards/210/2/3
Authorization: Bearer {token}
```

Response: Same full JSON structure as the result sheet above, formatted for display in a card layout.

DOWNLOAD PDF

```
GET /api/v1/assessments/report-cards/210/2/3/pdf
Authorization: Bearer {token}
```

Response: Binary PDF file.

```
Content-Type: application/pdf
Content-Disposition: attachment; filename="report-card-Tolu-Adeyemi-Term2-2025-2026.pdf"
```

EMAIL REPORT CARD TO PARENT

```
POST /api/v1/assessments/report-cards/210/2/3/email
Authorization: Bearer {token}
Content-Type: application/json

{
  "email": "parent@example.com",
  "message": "Please find attached Tolu's second term report card."
}
```

Response 200 OK :

```
{
  "message": "Report card emailed successfully",
  "recipient": "parent@example.com",
  "status": "queued"
}
```

BULK DOWNLOAD FOR CLASS

```
POST /api/v1/assessments/report-cards/bulk-download
Authorization: Bearer {token}
Content-Type: application/json

{
  "class_id": 7,
  "term_id": 2,
  "academic_year_id": 3
}
```

Response 200 OK :

```
{
  "message": "Bulk PDF generation queued",
  "download_url": "https://storage.compass.net/bulk-reports/class-7-term-2-2025-2026.zip",
  "expires_at": "2026-03-31T15:00:00.000000Z"
}
```

Promotion

PROMOTE SINGLE STUDENT

```
POST /api/v1/assessments/promotions/promote
Authorization: Bearer {token}
Content-Type: application/json
```

```
{
  "student_id": 210,
  "from_class_id": 7,
  "to_class_id": 8,
  "academic_year_id": 3,
  "reason": "End of year promotion"
}
```

Response 200 OK :

```
{
  "message": "Student promoted successfully",
  "data": {
    "id": 101,
    "student": { "id": 210, "name": "Tolu Adeyemi" },
    "from_class": { "id": 7, "name": "JSS 1A" },
    "to_class": { "id": 8, "name": "JSS 2A" },
    "promoted_at": "2026-07-01T09:00:00.000000Z",
    "promoted_by": { "id": 1, "name": "Mr. Chukwuemeka Obi" }
  }
}
```

BULK PROMOTE CLASS

```
POST /api/v1/assessments/promotions/bulk-promote
Authorization: Bearer {token}
Content-Type: application/json
```

```
{
  "from_class_id": 7,
  "to_class_id": 8,
  "academic_year_id": 3,
  "student_ids": [210, 211, 212, 213, 214]
}
```

Response 200 OK :

```
{
  "message": "Bulk promotion completed",
  "data": {
    "promoted_count": 5,
    "failed_count": 0,
    "from_class": "JSS 1A",
    "to_class": "JSS 2A"
  }
}
```

AUTO-PROMOTE BASED ON RESULTS

POST /api/v1/assessments/promotions/auto-promote
Authorization: Bearer {token}
Content-Type: application/json

```
{
  "class_id": 7,
  "term_id": 2,
  "academic_year_id": 3,
  "pass_threshold": 40,
  "dry_run": false
}
```

Response 200 OK :

```
{
  "message": "Auto-promotion completed",
  "data": {
    "total_students": 30,
    "promoted": 26,
    "held_back": 4,
    "held_back_students": [
      { "id": 220, "name": "Kunle Bello", "average": 32.5 }
    ]
  }
}
```

Analytics

Base path: /api/v1/assessments/analytics/

Method	Endpoint	Description
GET	/school	School-wide performance summary
GET	/class/{id}	Class performance analytics
GET	/subject/{id}	Subject performance analytics
GET	/student/{id}/trend	Student performance over time
GET	/comparative	Cross-class / cross-term comparison
GET	/student/{id}/prediction	AI-based performance prediction

Question Bank

Base path: /api/v1/question-bank/

Method	Endpoint	Description
GET	/	List question banks
POST	/	Create bank
GET	/statistics	Question statistics
GET	/for-exam	Get questions suitable for an exam
GET	/ {id}	Get bank details
PUT	/ {id}	Update
DELETE	/ {id}	Delete

Frontend Integration

Tenancy

The frontend sends requests to `https://{school_subdomain}.compasse.net/api/v1/assessments/...`. The subdomain is stored in `localStorage` after login and prepended to all API calls via an Axios base URL or interceptor.

CBT Exam Player

The exam player component handles the full CBT session lifecycle:

- Start session** — POST /assessments/cbt/start with `exam_id`. Store `session_id` and `expires_at` in component state.
- Timer** — Calculate remaining time from `expires_at - now`. Display a countdown clock. Auto-submit when the timer reaches zero.
- Question navigation** — Render a numbered question palette. Answered questions are highlighted in green; unanswered in grey. Students can jump between questions freely.

4. **Per-question save** — On each option selection call `POST /assessments/cbt/submit-answer` silently in the background. Show a small “Saving...” indicator. This ensures no answers are lost on accidental refresh.
5. **Final submit** — Call `POST /assessments/cbt/submit` on explicit submit or timer expiry. Disable the submit button after the first click to prevent double submission.
6. **Result display** — If `show_result_immediately = true`, redirect to the result screen after submit and render the score, grade, and per-question breakdown from `GET /assessments/cbt/session/{id}/results`.

Result Sheet Viewer

- Fetch the full result JSON from `GET /assessments/results/student/{id}/{termId}/{yearId}`.
- Display subject scores in a table with CA, Exam, Total, Grade, and Remark columns.
- Render the psychomotor section below the subject table.
- Show teacher and principal comments at the bottom.
- If `status !== "published"`, show a greyed-out preview banner (“Result pending approval”).

Report Card PDF Download

- Call `GET /assessments/report-cards/{student}/{term}/{year}/pdf` with `responseType: 'blob'` in Axios.
- Create a temporary `<a>` element with the blob URL and trigger a download.
- Show a loading spinner during the request.

```
const response = await axios.get(
  `/assessments/report-cards/${studentId}/${termId}/${yearId}/pdf`,
  { responseType: 'blob' }
);
const url = window.URL.createObjectURL(new Blob([response.data]));
const link = document.createElement('a');
link.href = url;
link.setAttribute('download', `report-card-${studentId}.pdf`);
document.body.appendChild(link);
link.click();
link.remove();
```

Business Rules

1. **Module gate** — All endpoints require the `cbt` module to be active on the school's subscription.
2. **Session expiry** — CBT sessions expire at `started_at + duration_minutes`. Submission after expiry is rejected with HTTP 422.
3. **Auto-grade** — Objective questions (multiple_choice, true_false) are auto-graded on submit. Subjective questions require manual teacher grading.
4. **Result generation** — `POST /results/generate` aggregates all CA scores and exam scores for the given term, class, and year. Re-running it overwrites any existing draft results.

5. **Approve before publish** — Results must be in `approved` status before `POST /results/publish` succeeds.
6. **Promotion audit** — Every promotion creates a record in the `promotions` table with `promoted_by` and timestamp for audit trail. Use `DELETE /promotions/{id}` to undo.

Communication — Messaging, Notifications, SMS & Email

Base URL: `https://{subdomain}.compasse.net/api/v1/` **Auth:** `Authorization: Bearer {token}` required on all protected endpoints **Module gate:** None (messages & notifications) | `sms_integration` (SMS) | `email_integration` (Email)

Overview

The communication module provides internal messaging, push-style notifications, and outbound SMS/Email dispatch. Messages and notifications are core features available to all authenticated tenant users. SMS and Email require their respective module subscriptions. All outbound SMS and email are **queued** — the API responds immediately and delivery happens asynchronously via Laravel Horizon queue workers.

User Stories

As any user, I want to send and receive internal messages to/from other users in my school.

As an admin, I want to send a bulk SMS to all parents informing them of an event or emergency.

As a teacher, I want to notify students about an assignment deadline via in-app notification.

As an accountant, I want to email a parent a payment receipt.

Internal Messages

No module gate — available to all tenant users. **Base path:** `/api/v1/communication/messages`

Method	Endpoint	Description
GET	/messages	List messages (inbox / outbox filter)
POST	/messages	Send a message
GET	/messages/{id}	Read a message
PUT	/messages/{id}	Update message
DELETE	/messages/{id}	Delete message
PUT	/messages/{id}/read	Mark as read

Send a Message

POST /api/v1/communication/messages

Authorization: Bearer {token}

Content-Type: application/json

```
{
  "to": [3, 7, 12],
  "subject": "Parent-Teacher Meeting – Friday 4 April",
  "body": "Dear colleagues, please be reminded that the parent-teacher meeting is scheduled for Friday 4 April at 3:00 PM in the main hall. Attendance is compulsory.",
  "priority": "normal"
}
```

Response 201 Created :

```
{
  "message": "Message sent successfully",
  "data": {
    "id": 88,
    "subject": "Parent-Teacher Meeting – Friday 4 April",
    "body": "Dear colleagues, please be reminded...",
    "from": { "id": 1, "name": "Mr. Chukwuemeka Obi" },
    "recipients": [
      { "id": 3, "name": "Mrs. Funmilayo Adesanya" },
      { "id": 7, "name": "Mr. Biodun Okonkwo" },
      { "id": 12, "name": "Miss Ngozi Eze" }
    ],
    "priority": "normal",
    "sent_at": "2026-03-30T08:30:00.000000Z"
  }
}
```

List Messages (Inbox / Outbox)

GET /api/v1/communication/messages?type=inbox&per_page=20&page=1

Authorization: Bearer {token}

Query parameters:

- `type` — `inbox` (received) or `outbox` (sent). Defaults to `inbox`.
- `is_read` — `true` / `false` to filter by read status.
- `search` — Full-text search on subject and body.

Response 200 OK :

```
{
  "data": [
    {
      "id": 88,
      "subject": "Parent-Teacher Meeting – Friday 4 April",
      "body_preview": "Dear colleagues, please be reminded...",
      "from": { "id": 1, "name": "Mr. Chukwuemeka Obi" },
      "is_read": false,
      "priority": "normal",
      "sent_at": "2026-03-30T08:30:00.000000Z"
    },
    {
      "id": 85,
      "subject": "Timetable Update – Week 11",
      "body_preview": "Please note that Monday's schedule has been revised...",
      "from": { "id": 2, "name": "Mrs. Adaeze Ugwu" },
      "is_read": true,
      "priority": "high",
      "sent_at": "2026-03-28T10:00:00.000000Z"
    }
  ],
  "unread_count": 5,
  "current_page": 1,
  "last_page": 3,
  "per_page": 20,
  "total": 42
}
```

Read a Message

```
GET /api/v1/communication/messages/88
Authorization: Bearer {token}
```

Response 200 OK :

```
{
  "data": {
    "id": 88,
    "subject": "Parent-Teacher Meeting – Friday 4 April",
    "body": "Dear colleagues, please be reminded that the parent-teacher meeting is scheduled for Friday 4 April at 3:00 PM in the main hall. Attendance is compulsory.",
    "from": { "id": 1, "name": "Mr. Chukwuemeka Obi", "role": "admin" },
    "recipients": [
      { "id": 3, "name": "Mrs. Funmilayo Adesanya", "is_read": true, "read_at": "2026-03-30T09:15:00.000000Z" },
      { "id": 7, "name": "Mr. Biodun Okonkwo", "is_read": false, "read_at": null }
    ],
    "priority": "normal",
    "sent_at": "2026-03-30T08:30:00.000000Z",
    "is_read": false
  }
}
```

Mark as Read

```
PUT /api/v1/communication/messages/88/read
Authorization: Bearer {token}
```

Response 200 OK :

```
{
  "message": "Message marked as read",
  "data": {
    "id": 88,
    "is_read": true,
    "read_at": "2026-03-30T09:20:00.000000Z"
  }
}
```

Notifications

No module gate — available to all tenant users. Base path: `/api/v1/communication/notifications`

Method	Endpoint	Description
GET	/notifications	List notifications for the authenticated user
POST	/notifications	Create a notification (bulk, role-targeted)
GET	/notifications/{id}	Get a single notification
PUT	/notifications/{id}	Update notification
DELETE	/notifications/{id}	Delete notification
PUT	/notifications/{id}/read	Mark a single notification as read
PUT	/notifications/read-all	Mark all notifications as read

Create Notification (Bulk with Role Filter)

POST /api/v1/communication/notifications

Authorization: Bearer {token}

Content-Type: application/json

```
{
  "title": "Second Term Exams Begin Monday",
  "body": "Please remind your students that second term examinations begin on Monday 14 April. All students must come with their school ID.",
  "type": "announcement",
  "target_role": "teacher",
  "class_id": null,
  "action_url": "/events/13",
  "send_push": true
}
```

Request fields:

- `target_role` — `all`, `student`, `teacher`, `parent`, `admin`. Sends to every user with that role.
- `class_id` — Optionally scope to a single class. Pass `null` for school-wide.
- `action_url` — Deep link the notification to a specific screen.
- `send_push` — If `true`, also dispatches a push notification job.

Response 201 Created :

```
{
  "message": "Notification sent",
  "data": {
    "id": 204,
    "title": "Second Term Exams Begin Monday",
    "body": "Please remind your students...",
    "type": "announcement",
    "target_role": "teacher",
    "recipients_count": 28,
    "created_at": "2026-03-30T09:00:00.000000Z"
  }
}
```

List Notifications

```
GET /api/v1/communication/notifications?is_read=false&per_page=15
Authorization: Bearer {token}
```

Response 200 OK :

```
{
  "data": [
    {
      "id": 204,
      "title": "Second Term Exams Begin Monday",
      "body": "Please remind your students...",
      "type": "announcement",
      "is_read": false,
      "action_url": "/events/13",
      "created_at": "2026-03-30T09:00:00.000000Z"
    },
    {
      "id": 201,
      "title": "New Assignment: Essay on WWI",
      "body": "A new assignment has been posted for SS 2A – History.",
      "type": "assignment",
      "is_read": false,
      "action_url": "/assignments/33",
      "created_at": "2026-03-28T14:00:00.000000Z"
    }
  ],
  "unread_count": 7,
  "current_page": 1,
  "last_page": 1,
  "per_page": 15,
  "total": 7
}
```

Mark All as Read

```
PUT /api/v1/communication/notifications/read-all
Authorization: Bearer {token}
```

Response 200 OK :

```
{
  "message": "All notifications marked as read",
  "marked_count": 7
}
```

SMS

Required module: sms_integration **Base path:** /api/v1/communication/sms

Method	Endpoint	Description
POST	/sms/send	Send a single SMS
POST	/sms/bulk	Send SMS to multiple recipients
GET	/sms/logs	View SMS delivery logs

Send a Single SMS

```
POST /api/v1/communication/sms/send
```

```
Authorization: Bearer {token}
```

```
Content-Type: application/json
```

```
{
  "to": "+2348012345678",
  "message": "Dear Parent, second term fees are due by April 30th. Please visit the school portal to pay. – Greenfield Academy",
  "sender_id": "Greenfield"
}
```

Response 200 OK:

```
{
  "message": "SMS queued for delivery",
  "data": {
    "id": 512,
    "to": "+2348012345678",
    "message": "Dear Parent, second term fees are due by April 30th...",
    "sender_id": "Greenfield",
    "status": "queued",
    "provider": "termii",
    "queued_at": "2026-03-30T10:00:00.000000Z"
  }
}
```

The API returns immediately. Delivery happens via `SendSMSJob` on the `sms` queue.

Bulk SMS (Multiple Recipients)

```
POST /api/v1/communication/sms/bulk
```

```
Authorization: Bearer {token}
```

```
Content-Type: application/json
```

```
{
  "recipients": [
    { "phone": "+2348012345678", "name": "Mrs. Okafor" },
    { "phone": "+2347089001234", "name": "Mr. Adeyemi" },
    { "phone": "+2348055667788", "name": "Mrs. Eze" }
  ],
  "message": "Dear {name}, the next PTA meeting holds on Saturday 12 April at 10 AM. Please attend.",
  "sender_id": "Greenfield",
  "schedule_at": null
}
```

The `{name}` placeholder is replaced per recipient. Set `schedule_at` to an ISO datetime string to delay dispatch.

Response 200 OK :

```
{
  "message": "Bulk SMS queued for delivery",
  "data": {
    "batch_id": "sms-batch-20260330-001",
    "total_recipients": 3,
    "status": "queued",
    "estimated_cost": 3,
    "queued_at": "2026-03-30T10:05:00.000000Z"
  }
}
```

SMS Delivery Logs

```
GET /api/v1/communication/sms/logs?status=failed&date_from=2026-03-01&date_to=2026-03-31
```

```
Authorization: Bearer {token}
```

Response 200 OK :


```
{
  "data": [
    {
      "id": 510,
      "to": "+2348099887766",
      "message": "Dear Parent...",
      "status": "failed",
      "error": "Invalid phone number",
      "provider": "termii",
      "attempted_at": "2026-03-29T11:00:00.000000Z"
    }
  ],
  "current_page": 1,
  "total": 1
}
```

Supported SMS Providers

Configure via `SERVICES_SMS_PROVIDER` in `.env` :

Value	Provider
<code>twilio</code>	Twilio (international)
<code>vonage</code>	Vonage / Nexmo
<code>termii</code>	Termii (Nigeria-focused)
<code>log</code>	Log-only (development / testing)

Provider credentials in `config/services.php` :

```
# Termii
SERVICES_TERMII_KEY=your_api_key
SERVICES_SMS_SENDER_ID=Compassse

# Twilio
SERVICES_TWILIO_SID=ACxxxxxxx
SERVICES_TWILIO_TOKEN=xxxxxxx
SERVICES_TWILIO_FROM=+15551234567

# Vonage
SERVICES_VONAGE_KEY=your_key
SERVICES_VONAGE_SECRET=your_secret
SERVICES_VONAGE_FROM=Compassse
```

Email

Required module: `email_integration` **Base path:** `/api/v1/communication/email`

Method	Endpoint	Description
POST	/email/send	Send a single email
POST	/email/bulk	Send email to multiple recipients
GET	/email/logs	View email delivery logs

Send a Single Email

```
POST /api/v1/communication/email/send
```

```
Authorization: Bearer {token}
```

```
Content-Type: application/json
```

```
{
  "to": "parent@example.com",
  "cc": ["admin@greenfield.edu"],
  "bcc": [],
  "subject": "Fee Receipt – Greenfield Academy",
  "body": "Dear Mrs Okafor,\n\nPlease find attached your payment receipt for the second term fee payment of ₦85,000.\n\nThank you.\n\nGreenfield Academy Accounts",
  "attachment_url": "https://storage.compass.net/receipts/receipt-1045.pdf",
  "template": null
}
```

Response 200 OK :

```
{
  "message": "Email queued for delivery",
  "data": {
    "id": 310,
    "to": "parent@example.com",
    "cc": ["admin@greenfield.edu"],
    "subject": "Fee Receipt – Greenfield Academy",
    "status": "queued",
    "queued_at": "2026-03-30T11:00:00.000000Z"
  }
}
```

Delivery happens via `SendEmailJob` on the `emails` queue.

Bulk Email

```
POST /api/v1/communication/email/bulk
```

```
Authorization: Bearer {token}
```

```
Content-Type: application/json
```

```
{
  "recipients": [
    { "email": "parent1@example.com", "name": "Mrs Okafor" },
    { "email": "parent2@example.com", "name": "Mr Adeyemi" }
  ],
  "subject": "End of Term Report Cards Now Available",
  "body": "Dear {name},\n\nYour child's second term report card is now available on the Compasse parent portal. Log in at https://greenfield.compasse.net to view it.\n\nRegards,\nGreenfield Academy",
  "schedule_at": null
}
```

Response 200 OK:

```
{
  "message": "Bulk email queued for delivery",
  "data": {
    "batch_id": "email-batch-20260330-001",
    "total_recipients": 2,
    "status": "queued",
    "queued_at": "2026-03-30T11:05:00.000000Z"
  }
}
```

Email Delivery Logs

```
GET /api/v1/communication/email/logs?status=failed&per_page=20
```

```
Authorization: Bearer {token}
```

Response 200 OK:

```
{
  "data": [
    {
      "id": 308,
      "to": "bounce@example.com",
      "subject": "Fee Receipt",
      "status": "failed",
      "error": "550 Mailbox does not exist",
      "attempts": 3,
      "last_attempted_at": "2026-03-29T12:30:00.000000Z"
    }
  ],
  "current_page": 1,
  "total": 1
}
```

Queue Worker Setup

```
# Development (single worker covering all queues)
php artisan queue:work --queue=emails,sms,default

# Production (Laravel Horizon – recommended)
php artisan horizon
```

Failed jobs are logged via the `failed()` hook on each job class and stored in the `failed_jobs` table. Inspect them with:

```
php artisan queue:failed
php artisan queue:retry {job_id}
php artisan queue:retry all
```

Retry Policy

Both `SendSMSJob` and `SendEmailJob` follow the same retry policy:

Attempt	Delay Before Retry
1st retry	30 seconds
2nd retry	2 minutes
3rd retry	5 minutes
Permanent failure	Logged + stored in <code>failed_jobs</code> table

Maximum attempts: `3`. After all retries are exhausted the job is moved to the failed queue, an error is logged, and the delivery record is updated with `status: failed`.

Guardian Communication

Guardians have dedicated read-only endpoints to view their own notifications, messages, and payment history:

```
GET /api/v1/guardians/{id}/notifications
GET /api/v1/guardians/{id}/messages
GET /api/v1/guardians/{id}/payments
```

These return the same JSON shape as the general endpoints but scoped to the specific guardian's data.

Frontend Integration

Tenancy

All communication requests go to

`https://{school_subdomain}.compasse.net/api/v1/communication/...`. The subdomain is resolved from

`localStorage` (set at login) and injected into every Axios request via a base URL or request interceptor.

Notification Bell with Unread Count

1. On app load, call `GET /communication/notifications?is_read=false&per_page=1` and read the `unread_count` field from the response.
2. Display the count as a badge on the bell icon.
3. Poll every 60 seconds (or use WebSocket/SSE if available) to keep the count fresh.
4. On bell click, open a dropdown fetching `GET /communication/notifications?per_page=10`.
5. On notification click, navigate to `action_url` and call `PUT /communication/notifications/{id}/read`.
6. Provide a “Mark all as read” button that calls `PUT /communication/notifications/read-all` and resets the badge to zero.

```
// Example: fetch unread count on mount
const { data } = await axios.get('/communication/notifications?is_read=false&per_page=1');
setUnreadCount(data.unread_count);
```

Message Thread View

- Inbox list: `GET /communication/messages?type=inbox` — render as a list with sender name, subject preview, and relative timestamp.
- On message click: `GET /communication/messages/{id}` and call `PUT /communication/messages/{id}/read` simultaneously.
- Compose form: `POST /communication/messages`. The `to` field is a multi-select user picker populated from `GET /users`.
- Show the recipients' read receipts (name + `read_at`) on the sent message detail view.

Bulk SMS Composer for Admins

The admin bulk SMS UI should:

1. Allow selecting a target group (class, role, or custom list) from a dropdown.
2. Load phone numbers for the selected group from the relevant `GET /students` or `GET /teachers` list.
3. Show a character counter (160 chars = 1 SMS unit; beyond 160 = concatenated SMS, cost multiplies).
4. Support `{name}` merge tags; show a live preview of the personalised message for the first recipient.
5. On submit, call `POST /communication/sms/bulk`.
6. After sending, poll `GET /communication/sms/logs?batch_id={batch_id}` to show delivery status per recipient.

Attendance Management

Base URL: `https://{subdomain}.compasse.net/api/v1/` **Auth:** `Authorization: Bearer {token}`
required on all protected endpoints **Module gate:** `attendance_management`

Overview

The attendance module tracks student and staff presence using a polymorphic model — the same table records attendance for any type of person (student, teacher, staff). Attendance can be marked individually or in bulk for an entire class. Staff check-in and check-out are also handled through the same endpoint by specifying `attendanceable_type`.

User Stories

As a teacher, I want to mark daily attendance for my class, recording each student as present, absent, or late.

As a school admin, I want to view class-level and student-level attendance reports with percentages.

As a parent, I want to see my child's attendance record so I am aware of any absences.

As an admin, I want attendance to track check-in/check-out times for staff so I can generate time and payroll reports.

Attendance Model

The model uses **polymorphic association** — `attendanceable_type` + `attendanceable_id` can point to `Student`, `Teacher`, or any other staff model.

Field	Type	Description
attendanceable_id	integer	ID of the person
attendanceable_type	string	Fully qualified model class, e.g. App\Models\Student
date	date	Attendance date
status	derived	present / absent / completed / pending (never stored)
check_in_time	datetime	Datetime of check-in
check_out_time	datetime	Datetime of check-out
total_hours	float	Computed hours worked
break_duration	integer	Break time in minutes
overtime_hours	float	Hours beyond standard 8 hrs
is_late	boolean	Whether check-in was after 08:00
late_minutes	integer	Minutes late
is_absent	boolean	Explicitly marked absent
absence_reason	string	Reason if absent
is_excused	boolean	Whether the absence is excused
excuse_notes	string	Justification for excused absence
marked_by	integer	FK → User who recorded the attendance
location	string	GPS coordinates or room identifier

Status is derived (not stored):

- absent — if is_absent = true
- present — if checked in but not yet checked out
- completed — if both check_in_time and check_out_time are set
- pending — otherwise (no check-in recorded yet)

API Endpoints

Base path: /api/v1/attendance/

Method	Endpoint	Description
GET	/	List all attendance records
GET	/reports	Attendance summary reports
GET	/students	All student attendance records
GET	/teachers	All teacher attendance records
GET	/class/{class_id}	Class attendance for a specific date
GET	/student/{student_id}	Individual student attendance history
POST	/mark	Mark attendance (single or bulk)
PUT	/id	Correct an attendance record
DELETE	/id	Delete an attendance record
GET	/id	Get a single record

Complete Request / Response Examples

Bulk Mark Class Attendance (Students)

```
POST /api/v1/attendance/mark
Authorization: Bearer {token}
Content-Type: application/json
```

```
{
  "date": "2026-03-30",
  "class_id": 3,
  "records": [
    {
      "student_id": 10,
      "is_absent": false,
      "check_in_time": "2026-03-30 07:55:00"
    },
    {
      "student_id": 11,
      "is_absent": false,
      "check_in_time": "2026-03-30 08:10:00",
      "is_late": true,
      "late_minutes": 10
    },
    {
      "student_id": 12,
      "is_absent": true,
      "absence_reason": "Sick – parent notified via phone"
    },
    {
      "student_id": 13,
      "is_absent": true,
      "absence_reason": "Family bereavement",
      "is_excused": true,
      "excuse_notes": "Parent submitted a written excuse letter on 2026-03-29"
    }
  ]
}
```

Response 200 OK :

```
{
  "message": "Attendance marked successfully",
  "data": {
    "date": "2026-03-30",
    "class": { "id": 3, "name": "JSS 2B" },
    "total_students": 4,
    "present": 2,
    "absent": 2,
    "late": 1,
    "excused": 1,
    "marked_by": { "id": 5, "name": "Mrs. Ngozi Eze" },
    "records": [
      {
        "id": 1001,
        "student": { "id": 10, "name": "Emeka Nwosu" },
        "status": "present",
        "is_late": false,
        "check_in_time": "2026-03-30T07:55:00.000000Z"
      },
      {
        "id": 1002,
        "student": { "id": 11, "name": "Amaka Obi" },
        "status": "present",
        "is_late": true,
        "late_minutes": 10,
        "check_in_time": "2026-03-30T08:10:00.000000Z"
      },
      {
        "id": 1003,
        "student": { "id": 12, "name": "Dare Adeyemi" },
        "status": "absent",
        "is_excused": false,
        "absence_reason": "Sick – parent notified via phone"
      },
      {
        "id": 1004,
        "student": { "id": 13, "name": "Chioma Dike" },
        "status": "absent",
        "is_excused": true,
        "absence_reason": "Family bereavement"
      }
    ]
  }
}
```

Staff Check-In

```
POST /api/v1/attendance/mark
Authorization: Bearer {token}
Content-Type: application/json
```

```
{
  "attendanceable_type": "App\\Models\\Teacher",
  "attendanceable_id": 5,
  "date": "2026-03-30",
  "check_in_time": "2026-03-30 07:48:00",
  "location": "Main Gate - GPS: 6.5244,3.3792"
}
```

Response 201 Created :

```
{
  "message": "Check-in recorded",
  "data": {
    "id": 2055,
    "attendanceable_type": "App\\Models\\Teacher",
    "attendanceable_id": 5,
    "teacher": { "id": 5, "name": "Mr. Biodun Okonkwo" },
    "date": "2026-03-30",
    "check_in_time": "2026-03-30T07:48:00.000000Z",
    "is_late": false,
    "status": "present",
    "location": "Main Gate - GPS: 6.5244,3.3792"
  }
}
```

Staff Check-Out

```
PUT /api/v1/attendance/2055
Authorization: Bearer {token}
Content-Type: application/json
```

```
{
  "check_out_time": "2026-03-30 15:30:00",
  "break_duration": 60
}
```

Response 200 OK :

```
{
  "message": "Check-out recorded",
  "data": {
    "id": 2055,
    "teacher": { "id": 5, "name": "Mr. Biodun Okonkwo" },
    "date": "2026-03-30",
    "check_in_time": "2026-03-30T07:48:00.000000Z",
    "check_out_time": "2026-03-30T15:30:00.000000Z",
    "total_hours": 6.7,
    "break_duration": 60,
    "overtime_hours": 0,
    "status": "completed"
  }
}
```

Get Class Attendance for a Specific Date

```
GET /api/v1/attendance/class/3?date=2026-03-30
Authorization: Bearer {token}
```

Response 200 OK:

```

{
  "class": { "id": 3, "name": "JSS 2B" },
  "date": "2026-03-30",
  "summary": {
    "total_students": 32,
    "present": 28,
    "absent": 4,
    "late": 3,
    "not_marked": 0,
    "attendance_rate": 87.5
  },
  "records": [
    {
      "id": 1001,
      "student": {
        "id": 10,
        "name": "Emeka Nwosu",
        "admission_number": "GF/2024/001"
      },
      "status": "present",
      "is_late": false,
      "check_in_time": "2026-03-30T07:55:00.000000Z"
    },
    {
      "id": 1002,
      "student": {
        "id": 11,
        "name": "Amaka Obi",
        "admission_number": "GF/2024/002"
      },
      "status": "present",
      "is_late": true,
      "late_minutes": 10,
      "check_in_time": "2026-03-30T08:10:00.000000Z"
    },
    {
      "id": 1003,
      "student": {
        "id": 12,
        "name": "Dare Adeyemi",
        "admission_number": "GF/2024/003"
      },
      "status": "absent",
      "is_excused": false,
      "absence_reason": "Sick – parent notified via phone"
    }
  ]
}

```

Get Student Attendance with Date Range and Summary Stats

```

GET /api/v1/attendance/student/10?date_from=2026-01-01&date_to=2026-03-31
Authorization: Bearer {token}

```

Response 200 OK :

```
{
  "student": {
    "id": 10,
    "name": "Emeka Nwosu",
    "admission_number": "GF/2024/001",
    "class": "JSS 2B"
  },
  "date_range": {
    "from": "2026-01-01",
    "to": "2026-03-31"
  },
  "summary": {
    "total_school_days": 58,
    "present_days": 52,
    "absent_days": 6,
    "late_arrivals": 4,
    "excused_absences": 2,
    "unexcused_absences": 4,
    "attendance_percentage": 89.7
  },
  "records": [
    {
      "id": 1001,
      "date": "2026-03-30",
      "status": "present",
      "is_late": false,
      "check_in_time": "2026-03-30T07:55:00.000000Z",
      "check_out_time": null
    },
    {
      "id": 998,
      "date": "2026-03-25",
      "status": "absent",
      "is_excused": false,
      "absence_reason": "Sick – parent notified via phone"
    }
  ],
  "current_page": 1,
  "last_page": 4,
  "per_page": 15,
  "total": 58
}
```

Attendance Reports

```
GET /api/v1/attendance/reports?class_id=3&term_id=2&academic_year_id=3&type=summary
Authorization: Bearer {token}
```

Query parameters:

Parameter	Description
class_id	Filter by class
term_id	Filter by term
academic_year_id	Filter by academic year
type	summary (aggregated) or detailed (per-day records)
date_from	Start date filter
date_to	End date filter

Response 200 OK :

```
{
  "report_type": "summary",
  "class": { "id": 3, "name": "JSS 2B" },
  "term": { "id": 2, "name": "Second Term" },
  "academic_year": { "id": 3, "name": "2025/2026" },
  "school_days_in_term": 58,
  "generated_at": "2026-03-30T12:00:00.000000Z",
  "students": [
    {
      "student": { "id": 10, "name": "Emeka Nwosu", "admission_number": "GF/2024/001" },
      "present_days": 52,
      "absent_days": 6,
      "late_arrivals": 4,
      "attendance_percentage": 89.7
    },
    {
      "student": { "id": 11, "name": "Amaka Obi", "admission_number": "GF/2024/002" },
      "present_days": 55,
      "absent_days": 3,
      "late_arrivals": 2,
      "attendance_percentage": 94.8
    }
  ],
  "class_summary": {
    "average_attendance_percentage": 91.3,
    "perfect_attendance_count": 5,
    "chronic_absentees": 2
  }
}
```

Correct an Attendance Record

```
PUT /api/v1/attendance/1003
Authorization: Bearer {token}
Content-Type: application/json

{
  "is_absent": false,
  "check_in_time": "2026-03-30 08:45:00",
  "is_late": true,
  "late_minutes": 45,
  "correction_reason": "Student was present but not marked due to teacher error"
}
```

Response 200 OK :

```
{
  "message": "Attendance record updated",
  "data": {
    "id": 1003,
    "student": { "id": 12, "name": "Dare Adeyemi" },
    "date": "2026-03-30",
    "status": "present",
    "is_late": true,
    "late_minutes": 45,
    "updated_by": { "id": 1, "name": "Mr. Chukwuemeka Obi" },
    "updated_at": "2026-03-30T13:00:00.000000Z"
  }
}
```

Get All Student Attendance

```
GET /api/v1/attendance/students?class_id=3&date=2026-03-30
Authorization: Bearer {token}
```

Response 200 OK :

```
{
  "data": [
    {
      "id": 1001,
      "student": { "id": 10, "name": "Emeka Nwosu", "class": "JSS 2B" },
      "date": "2026-03-30",
      "status": "present",
      "is_late": false
    }
  ],
  "current_page": 1,
  "last_page": 3,
  "per_page": 15,
  "total": 32
}
```

Get All Teacher Attendance

```
GET /api/v1/attendance/teachers?date=2026-03-30
Authorization: Bearer {token}
```

Response 200 OK :

```
{
  "data": [
    {
      "id": 2055,
      "teacher": { "id": 5, "name": "Mr. Biodun Okonkwo", "department": "Mathematics" },
      "date": "2026-03-30",
      "status": "completed",
      "check_in_time": "2026-03-30T07:48:00.000000Z",
      "check_out_time": "2026-03-30T15:30:00.000000Z",
      "total_hours": 6.7,
      "is_late": false
    }
  ],
  "current_page": 1,
  "total": 28
}
```

Business Rules

1. **Polymorphic model** — One attendance table covers students, teachers, and any staff. Differentiate using `attendanceable_type`.
 2. **Status is derived** — Never stored; always computed from `is_absent`, `check_in_time`, and `check_out_time`.
 3. **Late detection** — `isLate()` compares `check_in_time` against expected `08:00`; `getLateMinutes()` returns the difference in minutes.
 4. **Overtime detection** — `isOvertime()` checks if `total_hours > 8` (i.e., more than a standard working day).
 5. **Attendance percentage** — Calculated as $(\text{present_days} / \text{total_school_days}) \times 100$ in the student report.
 6. **Bulk mark is idempotent** — Re-submitting attendance for the same date and class updates existing records rather than creating duplicates.
 7. **Correction audit** — Any update to a record must include `correction_reason` and stores `updated_by` for audit trail.
-

Frontend Integration

Tenancy

All attendance requests go to `https://{school_subdomain}.compasse.net/api/v1/attendance/...`. The subdomain is resolved from local state and injected via an Axios interceptor.

Daily Class Register UI

The daily register is the primary teacher-facing attendance UI:

1. **Load class list** — `GET /classes/{classId}/students` to populate the student roster for the day.
2. **Pre-fill from existing records** — Call `GET /attendance/class/{classId}?date=YYYY-MM-DD` on mount. If records exist (attendance already marked), pre-populate each student's status row.
3. **Mark each student** — Render a row per student with three toggle buttons: Present, Late, Absent. Selecting "Late" reveals a "Minutes late" input. Selecting "Absent" reveals a text field for the absence reason and an "Excused" checkbox.
4. **Submit** — On form submit call `POST /attendance/mark` with the full `records` array. Disable the submit button after the first click.
5. **Correction mode** — If attendance was already submitted today, show an "Edit" button. On edit, each row calls `PUT /attendance/{id}` individually with `correction_reason`.

Attendance Calendar Heatmap (Individual Student)

The parent and student portal shows a monthly heatmap of attendance:

1. Fetch the student's attendance records for the current academic term via `GET /attendance/student/{studentId}?date_from={termStart}&date_to={termEnd}`.
2. Map each date to a colour: green (present), orange (late), red (absent), grey (school holiday / non-school day).
3. Render a calendar grid (month view) with coloured dots on each date.
4. On date click, show a tooltip with the full record details (check-in time, reason if absent, etc.).
5. Display the summary stats (attendance %, total present, total absent) above the calendar.

```
// Example: Fetch student attendance for heatmap
const { data } = await axios.get(
  `/attendance/student/${studentId}`,
  { params: { date_from: termStart, date_to: termEnd, per_page: 0 } }
);
// data.records is an array of { date, status, is_late, ... }
// data.summary holds { attendance_percentage, present_days, absent_days, ... }
```

Events & School Calendar

Base URL: `https://{subdomain}.compasse.net/api/v1/` **Auth:** Authorization: Bearer {token}
required on all protected endpoints **Module gate:** `event_management`

Overview

The events module manages school events (sports days, open days, graduations, exams, holidays) and a structured academic calendar. Both are gated by the `event_management` module subscription. Events can target specific audiences or individual classes; calendar entries support iCal recurrence rules for recurring holidays and term dates.

User Stories

As a school admin, I want to create school events and publish them so students, parents, and staff can see what is coming up.

As a teacher, I want to see upcoming events for my class so I can plan lessons around them.

As a parent, I want to check the school calendar for holidays and exam dates.

As an admin, I want to add recurring public holidays to the calendar linked to the academic year.

Models

Event

Field	Type	Description
title	string	Event name
description	text	Full event details
event_type	enum	academic , sports , cultural , holiday , meeting , exam , other
start_date	date	Start date
end_date	date	End date (same as start_date for single-day events)
start_time	time	Start time (HH:MM), null if is_all_day
end_time	time	End time (HH:MM), null if is_all_day
location	string	Venue or room
organizer	string	Organising person or team name
target_audience	enum	all , students , staff , parents , teachers
class_id	integer	Specific class (null = school-wide)
is_all_day	boolean	Whether the event spans the whole day
status	enum	scheduled , ongoing , completed , cancelled
max_participants	integer	Capacity limit (null = unlimited)
attachments	JSON	Array of file URLs
created_by	integer	FK → User who created the event

Scope `upcoming()` — returns events where `start_date >= today` and `status != cancelled`, ordered by `start_date` ascending.

Calendar

Field	Type	Description
title	string	Entry title
description	text	Details
date	date	Start date
end_date	date	End date (optional)
type	enum	holiday , exam , event , term_start , term_end , other
color	string	Hex colour for UI display (e.g. <code>#4CAF50</code>)
is_recurring	boolean	Whether the entry repeats
recurrence_rule	string	iCal RRULE string (e.g. <code>FREQ=YEARLY</code>)
academic_year_id	integer	Linked academic year

API Endpoints

Required module: `event_management`

Events

Method	Endpoint	Description
GET	<code>/api/v1/events</code>	List events (filter: status, event_type, date range, search, class_id)
POST	<code>/api/v1/events</code>	Create an event
GET	<code>/api/v1/events/upcoming</code>	Upcoming events (paginated, limited via <code>?limit=</code>)
GET	<code>/api/v1/events/{id}</code>	Get event details
PUT	<code>/api/v1/events/{id}</code>	Update an event
DELETE	<code>/api/v1/events/{id}</code>	Delete an event

Note on routing: `/events/upcoming` is declared **before** `/events/{id}` in the route file so “upcoming” is not treated as an event ID.

Calendar

Method	Endpoint	Description
GET	<code>/api/v1/calendars</code>	List calendar entries (filter: type, academic_year_id, month+year, date range)
POST	<code>/api/v1/calendars</code>	Create a calendar entry
GET	<code>/api/v1/calendars/{id}</code>	Get a calendar entry
PUT	<code>/api/v1/calendars/{id}</code>	Update a calendar entry
DELETE	<code>/api/v1/calendars/{id}</code>	Delete a calendar entry

Complete Request / Response Examples

Create an Event (School-Wide)

```
POST /api/v1/events
Authorization: Bearer {token}
Content-Type: application/json

{
  "title": "Inter-House Sports Day",
  "description": "Annual inter-house sports competition. All houses compete in track, field, and ball events. Parents are invited.",
  "event_type": "sports",
  "start_date": "2026-05-15",
  "end_date": "2026-05-15",
  "start_time": "08:00",
  "end_time": "16:00",
  "location": "School Sports Complex",
  "organizer": "Mr. Emeka Eze (Sports HOD)",
  "target_audience": "all",
  "class_id": null,
  "is_all_day": true,
  "status": "scheduled",
  "max_participants": null,
  "attachments": [
    "https://storage.compassse.net/events/sports-day-2026-schedule.pdf"
  ]
}
```

Response 201 Created :

```
{
  "message": "Event created successfully",
  "data": {
    "id": 14,
    "title": "Inter-House Sports Day",
    "description": "Annual inter-house sports competition...",
    "event_type": "sports",
    "start_date": "2026-05-15",
    "end_date": "2026-05-15",
    "start_time": "08:00",
    "end_time": "16:00",
    "location": "School Sports Complex",
    "organizer": "Mr. Emeka Eze (Sports HOD)",
    "target_audience": "all",
    "class_id": null,
    "is_all_day": true,
    "status": "scheduled",
    "attachments": [
      "https://storage.compassse.net/events/sports-day-2026-schedule.pdf"
    ],
    "created_by": { "id": 1, "name": "Mr. Chukwuemeka Obi" },
    "created_at": "2026-03-30T09:00:00.000000Z"
  }
}
```

Create a Class-Specific Event

```
POST /api/v1/events
Authorization: Bearer {token}
Content-Type: application/json

{
  "title": "JSS 1A Science Fair Presentation",
  "description": "Students in JSS 1A will present their science fair projects to the judging panel.",
  "event_type": "academic",
  "start_date": "2026-04-22",
  "end_date": "2026-04-22",
  "start_time": "10:00",
  "end_time": "12:00",
  "location": "Science Laboratory Block B",
  "organizer": "Mr. Biodun Okonkwo",
  "target_audience": "students",
  "class_id": 7,
  "is_all_day": false,
  "status": "scheduled"
}
```

Response 201 Created :

```
{
  "message": "Event created successfully",
  "data": {
    "id": 15,
    "title": "JSS 1A Science Fair Presentation",
    "event_type": "academic",
    "start_date": "2026-04-22",
    "target_audience": "students",
    "class": { "id": 7, "name": "JSS 1A" },
    "status": "scheduled",
    "created_at": "2026-03-30T09:30:00.000000Z"
  }
}
```

List Events (with Filters)

```
GET /api/v1/events?status=scheduled&event_type=sports&date_from=2026-04-01&date_to=2026-06-30&search=sports
Authorization: Bearer {token}
```

Query parameters:

Parameter	Description
status	Filter by <code>scheduled</code> , <code>ongoing</code> , <code>completed</code> , <code>cancelled</code>
event_type	Filter by <code>academic</code> , <code>sports</code> , <code>cultural</code> , <code>holiday</code> , <code>meeting</code> , <code>exam</code> , <code>other</code>
target_audience	Filter by audience
class_id	Filter to events for a specific class (includes school-wide events)
date_from	Events on or after this date
date_to	Events on or before this date
search	Search in title and description
per_page	Results per page (default 15)

Response `200 OK` :

```
{
  "data": [
    {
      "id": 14,
      "title": "Inter-House Sports Day",
      "event_type": "sports",
      "start_date": "2026-05-15",
      "end_date": "2026-05-15",
      "location": "School Sports Complex",
      "target_audience": "all",
      "status": "scheduled",
      "is_all_day": true
    }
  ],
  "current_page": 1,
  "last_page": 1,
  "per_page": 15,
  "total": 1
}
```

Get Upcoming Events

```
GET /api/v1/events/upcoming?limit=5
Authorization: Bearer {token}
```

Response `200 OK` :


```
{
  "upcoming_events": [
    {
      "id": 12,
      "title": "Mid-Term Break",
      "event_type": "holiday",
      "start_date": "2026-04-01",
      "end_date": "2026-04-04",
      "is_all_day": true,
      "target_audience": "all"
    },
    {
      "id": 13,
      "title": "Second Term Examinations Begin",
      "event_type": "exam",
      "start_date": "2026-04-14",
      "end_date": "2026-04-25",
      "is_all_day": false,
      "start_time": "08:00",
      "target_audience": "students"
    },
    {
      "id": 14,
      "title": "Inter-House Sports Day",
      "event_type": "sports",
      "start_date": "2026-05-15",
      "is_all_day": true,
      "target_audience": "all"
    },
    {
      "id": 15,
      "title": "JSS 1A Science Fair Presentation",
      "event_type": "academic",
      "start_date": "2026-04-22",
      "class": { "id": 7, "name": "JSS 1A" },
      "target_audience": "students"
    },
    {
      "id": 16,
      "title": "PTA Annual General Meeting",
      "event_type": "meeting",
      "start_date": "2026-05-03",
      "start_time": "10:00",
      "target_audience": "parents"
    }
  ],
  "count": 5
}
```

Get Event Details

```
GET /api/v1/events/14
Authorization: Bearer {token}
```

Response 200 OK :

```
{
  "data": {
    "id": 14,
    "title": "Inter-House Sports Day",
    "description": "Annual inter-house sports competition. All houses compete in track, field,
and ball events. Parents are invited.",
    "event_type": "sports",
    "start_date": "2026-05-15",
    "end_date": "2026-05-15",
    "start_time": null,
    "end_time": null,
    "location": "School Sports Complex",
    "organizer": "Mr. Emeka Eze (Sports HOD)",
    "target_audience": "all",
    "class": null,
    "is_all_day": true,
    "status": "scheduled",
    "max_participants": null,
    "attachments": [
      "https://storage.compassse.net/events/sports-day-2026-schedule.pdf"
    ],
    "created_by": { "id": 1, "name": "Mr. Chukwuemeka Obi" },
    "created_at": "2026-03-30T09:00:00.000000Z",
    "updated_at": "2026-03-30T09:00:00.000000Z"
  }
}
```

Update an Event

```
PUT /api/v1/events/14
Authorization: Bearer {token}
Content-Type: application/json

{
  "status": "ongoing",
  "location": "Main School Field (relocated from Sports Complex)"
}
```

Response 200 OK :

```
{
  "message": "Event updated successfully",
  "data": {
    "id": 14,
    "title": "Inter-House Sports Day",
    "status": "ongoing",
    "location": "Main School Field (relocated from Sports Complex)",
    "updated_at": "2026-05-15T08:05:00.000000Z"
  }
}
```

Delete an Event

```
DELETE /api/v1/events/14
Authorization: Bearer {token}
```

Response 200 OK :

```
{
  "message": "Event deleted successfully"
}
```

Create a Calendar Entry (Holiday)

```
POST /api/v1/calendars
Authorization: Bearer {token}
Content-Type: application/json
```

```
{
  "title": "Easter Monday",
  "description": "Public holiday – school closed",
  "date": "2026-04-06",
  "end_date": "2026-04-06",
  "type": "holiday",
  "color": "#4CAF50",
  "is_recurring": true,
  "recurrence_rule": "FREQ=YEARLY",
  "academic_year_id": 3
}
```

Response 201 Created :

```
{
  "message": "Calendar entry created successfully",
  "data": {
    "id": 55,
    "title": "Easter Monday",
    "date": "2026-04-06",
    "type": "holiday",
    "color": "#4CAF50",
    "is_recurring": true,
    "recurrence_rule": "FREQ=YEARLY",
    "academic_year": { "id": 3, "name": "2025/2026" },
    "created_at": "2026-03-30T10:00:00.000000Z"
  }
}
```

Create a Calendar Entry (Term Date)

```
POST /api/v1/calendars
Authorization: Bearer {token}
Content-Type: application/json

{
  "title": "Third Term Begins",
  "description": "Students resume for the third and final term of the 2025/2026 academic year.",
  "date": "2026-05-06",
  "type": "term_start",
  "color": "#2196F3",
  "is_recurring": false,
  "recurrence_rule": null,
  "academic_year_id": 3
}
```

Response 201 Created:

```
{
  "message": "Calendar entry created successfully",
  "data": {
    "id": 56,
    "title": "Third Term Begins",
    "date": "2026-05-06",
    "type": "term_start",
    "color": "#2196F3",
    "is_recurring": false,
    "academic_year": { "id": 3, "name": "2025/2026" },
    "created_at": "2026-03-30T10:05:00.000000Z"
  }
}
```

View Calendar Entries for a Specific Month

```
GET /api/v1/calendars?month=4&year=2026
Authorization: Bearer {token}
```

Response 200 OK:

```
{
  "month": 4,
  "year": 2026,
  "entries": [
    {
      "id": 55,
      "title": "Easter Monday",
      "date": "2026-04-06",
      "type": "holiday",
      "color": "#4CAF50",
      "is_recurring": true
    },
    {
      "id": 57,
      "title": "Second Term Examinations Begin",
      "date": "2026-04-14",
      "end_date": "2026-04-25",
      "type": "exam",
      "color": "#F44336",
      "is_recurring": false
    },
    {
      "id": 58,
      "title": "Second Term Ends",
      "date": "2026-04-30",
      "type": "term_end",
      "color": "#FF9800",
      "is_recurring": false
    }
  ],
  "count": 3
}
```

List All Calendar Entries (with Filters)

```
GET /api/v1/calendars?type=holiday&academic_year_id=3&date_from=2026-01-01&date_to=2026-12-31
Authorization: Bearer {token}
```

Query parameters:

Parameter	Description
type	Filter by holiday , exam , event , term_start , term_end , other
academic_year_id	Filter by academic year
month	Month number (1–12). Combine with year for monthly view.
year	Calendar year
date_from	Entries on or after this date
date_to	Entries on or before this date

Update a Calendar Entry

```
PUT /api/v1/calendars/55
Authorization: Bearer {token}
Content-Type: application/json

{
  "color": "#8BC34A",
  "description": "Public holiday – school closed. No activities scheduled."
}
```

Response 200 OK :

```
{
  "message": "Calendar entry updated successfully",
  "data": {
    "id": 55,
    "title": "Easter Monday",
    "color": "#8BC34A",
    "description": "Public holiday – school closed. No activities scheduled.",
    "updated_at": "2026-03-30T10:15:00.000000Z"
  }
}
```

Delete a Calendar Entry

```
DELETE /api/v1/calendars/55
Authorization: Bearer {token}
```

Response 200 OK :

```
{
  "message": "Calendar entry deleted successfully"
}
```

Business Rules

1. **created_by auto-set** — Set to the authenticated user's ID on event creation; not required in the request body.
2. **Default status** — New events default to `scheduled`.
3. **Upcoming scope** — `GET /events/upcoming` uses the `scopeUpcoming()` model scope: `start_date >= today AND status != cancelled`, ordered by `start_date` ascending.
4. **Route ordering** — `/events/upcoming` must be declared before `/events/{id}` in the routes file to prevent "upcoming" being resolved as a numeric event ID.
5. **Calendar month filter** — Pass `?month=4&year=2026` to retrieve all entries for a specific month and year. The backend uses a `whereMonth` / `whereYear` query.

6. **Class-specific visibility** — Events with a `class_id` are only returned to users who belong to that class (student, class teacher). School-wide events (`class_id = null`) are visible to all.
 7. **Recurrence rule** — The `recurrence_rule` field stores a standard iCal RRULE string. The backend does not automatically expand recurrences; the frontend is responsible for rendering repeating entries from the rule.
-

Frontend Integration

Tenancy

All event and calendar requests are sent to

`https://{school_subdomain}.compasse.net/api/v1/events/...` and `.../calendars/...`. The subdomain comes from local state set at login and is injected into all Axios requests.

School Calendar Widget

The school calendar widget (monthly grid view) combines events and calendar entries into a unified display:

1. On month change, make two parallel requests:
 - `GET /events?date_from={firstDayOfMonth}&date_to={lastDayOfMonth}`
 - `GET /calendars?month={m}&year={y}`
2. Merge both response arrays and display each item on its corresponding date cell.
3. Use the calendar entry's `color` field to colour-code the dot or badge on each date.
4. Render multi-day events (where `end_date > start_date`) as a horizontal bar spanning multiple date cells.
5. On a date cell click, open a popover listing all events and calendar entries for that day.

Upcoming Events Sidebar

The sidebar widget on the dashboard or parent portal:

1. Call `GET /events/upcoming?limit=5` on page load.
2. Render a list with event title, date, and a coloured event-type badge.
3. Refresh on tab focus or on a 5-minute polling interval.
4. Each list item links to the event detail modal.

Event Detail Modal with RSVP

When a user clicks an event:

1. Fetch `GET /events/{id}` and render all fields in a modal.
2. Show title, description, date/time, location, organizer, and any attachments (as downloadable links).
3. If `target_audience` includes the current user's role, show a confirmation/RSVP button (if RSVP is enabled via `max_participants`).
4. For admin users, show Edit and Delete action buttons that call `PUT /events/{id}` and `DELETE /events/{id}`.

5. If `status === "cancelled"`, show a prominent “This event has been cancelled” banner.

```
// Example: Load upcoming events for sidebar
const { data } = await axios.get('/events/upcoming', { params: { limit: 5 } });
setSidebarEvents(data.upcoming_events);

// Example: Load monthly calendar
const [eventsRes, calendarRes] = await Promise.all([
  axios.get('/events', { params: { date_from: firstDay, date_to: lastDay } }),
  axios.get('/calendars', { params: { month: currentMonth, year: currentYear } })
]);
const allEntries = [...eventsRes.data.data, ...calendarRes.data.entries];
setCalendarEntries(allEntries);
```


API Reference

Base URL

```
https://{subdomain}.compasse.net/api/v1/
```

For super admin (no subdomain):

```
https://compasse.net/api/v1/
```

Authentication

All protected routes require:

```
Authorization: Bearer {token}
```

Obtain a token:

```
POST /api/v1/auth/login
Content-Type: application/json

{
  "email": "admin@greenfield.compasse.net",
  "password": "YourPassword"
}
```

Response:

```
{
  "token": "1|abc123...",
  "user": { "id": 1, "name": "John Doe", "role": "admin" }
}
```

For tenant routes, the subdomain in the URL resolves the tenant. Alternatively send:

```
X-Subdomain: greenfield
```

HTTP Status Codes

Code	Meaning
200	OK — successful GET/PUT
201	Created — successful POST
400	Bad Request — missing school context or invalid state
401	Unauthenticated — missing/invalid token
403	Forbidden — wrong role or module not in subscription
404	Not Found — record doesn't exist
422	Unprocessable — validation failed
429	Too Many Requests — rate limit hit
500	Server Error
503	Service Unavailable — subscription check failed

Pagination

All list endpoints return paginated responses:

```
{
  "data": [...],
  "current_page": 1,
  "last_page": 5,
  "per_page": 15,
  "total": 73,
  "next_page_url": "...",
  "prev_page_url": null
}
```

Control with `?per_page=25&page=2` .

Full Route Map

Public Routes (no auth)

```
GET /api/health
GET /api/v1/schools/by-subdomain/{subdomain}
POST /api/v1/tenants/verify
GET /api/v1/tenants/verify
GET /api/v1/schools/landing-page/templates
GET /api/v1/public/{subdomain}
```

Auth Routes (rate-limited: 5/min)

```
POST /api/v1/auth/login
POST /api/v1/auth/register
POST /api/v1/auth/forgot-password
POST /api/v1/auth/reset-password
POST /api/v1/auth/logout           [auth required]
POST /api/v1/auth/refresh          [auth required]
GET  /api/v1/auth/me               [tenant + auth]
```

Super Admin Routes [auth + role:super_admin]

```
GET|POST|PUT|DELETE /api/v1/tenants
GET                 /api/v1/tenants/{id}/stats
GET|POST|DELETE    /api/v1/schools
GET|PUT            /api/v1/admin/schools/{id}
GET               /api/v1/admin/schools/{id}/stats
GET              /api/v1/admin/schools/{id}/dashboard
POST             /api/v1/admin/schools/{id}/suspend
POST             /api/v1/admin/schools/{id}/activate
POST             /api/v1/admin/schools/{id}/send-email
POST             /api/v1/admin/schools/{id}/reset-admin-password
GET              /api/v1/admin/schools/{id}/users-count
GET              /api/v1/admin/schools/{id}/activity-logs
GET              /api/v1/dashboard/super-admin
GET              /api/v1/health/db
```

Tenant + Auth Routes (all require tenant middleware + Bearer token)

CORE — NO MODULE GATE

```
GET|PUT          /api/v1/schools/me
GET|PUT          /api/v1/schools/{id}
GET             /api/v1/schools/{id}/stats
GET             /api/v1/schools/{id}/dashboard
GET             /api/v1/schools/{id}/organogram
GET|POST|PUT|DELETE /api/v1/users
POST            /api/v1/users/{id}/activate
POST            /api/v1/users/{id}/suspend
POST            /api/v1/users/{id}/assign-role
POST            /api/v1/users/{id}/remove-role
GET             /api/v1/roles
GET             /api/v1/dashboard
GET             /api/v1/dashboard/stats
POST|DELETE     /api/v1/users/me/profile-picture
POST|DELETE     /api/v1/users/{id}/profile-picture
```

SUBSCRIPTIONS — NO MODULE GATE

```
GET    /api/v1/subscriptions/plans
GET    /api/v1/subscriptions/modules
GET    /api/v1/subscriptions/status
GET    /api/v1/subscriptions/school/modules
GET    /api/v1/subscriptions/school/limits
GET    /api/v1/subscriptions/modules/{module}/access
GET    /api/v1/subscriptions/features/{feature}/access
GET|POST|PUT|DELETE /api/v1/subscriptions
GET    /api/v1/subscriptions/{id}
POST   /api/v1/subscriptions/create
PUT    /api/v1/subscriptions/{id}/upgrade
POST   /api/v1/subscriptions/{id}/renew
DELETE /api/v1/subscriptions/{id}/cancel
```

MODULE: ACADEMIC_MANAGEMENT

```
CRUD /api/v1/academic-years
CRUD /api/v1/terms
CRUD /api/v1/departments
CRUD /api/v1/classes
GET   /api/v1/classes/{id}/students
CRUD /api/v1/subjects
CRUD /api/v1/arms
POST  /api/v1/arms/assign-to-class
POST  /api/v1/arms/remove-from-class
GET   /api/v1/arms/class/{classId}
GET   /api/v1/arms/{armId}/students
```

MODULE: STUDENT_MANAGEMENT

```
CRUD /api/v1/students
GET   /api/v1/students/{id}/attendance
GET   /api/v1/students/{id}/results
GET   /api/v1/students/{id}/assignments
GET   /api/v1/students/{id}/subjects
POST  /api/v1/students/generate-admission-number
POST  /api/v1/students/generate-credentials
CRUD /api/v1/guardians
POST  /api/v1/guardians/{id}/assign-student
DELETE /api/v1/guardians/{id}/remove-student
GET   /api/v1/guardians/{id}/students
```

MODULE: TEACHER_MANAGEMENT

```
CRUD /api/v1/teachers
GET   /api/v1/teachers/{id}/classes
GET   /api/v1/teachers/{id}/subjects
GET   /api/v1/teachers/{id}/students
```

MODULE: CBT

CRUD	/api/v1/assessments/exams
CRUD	/api/v1/assessments/assignments
GET	/api/v1/assessments/assignments/{id}/submissions
POST	/api/v1/assessments/assignments/{id}/submit
PUT	/api/v1/assessments/assignments/{id}/grade
POST	/api/v1/assessments/cbt/start
POST	/api/v1/assessments/cbt/submit
POST	/api/v1/assessments/cbt/submit-answer
GET	/api/v1/assessments/cbt/{exam}/questions
GET	/api/v1/assessments/cbt/attempts/{attempt}/status
GET	/api/v1/assessments/cbt/session/{id}/results
CRUD	/api/v1/assessments/results
POST	/api/v1/assessments/results/generate
GET	/api/v1/assessments/results/student/{id}/{termId}/{yearId}
GET	/api/v1/assessments/results/class/{classId}
POST	/api/v1/assessments/results/{id}/approve
POST	/api/v1/assessments/results/publish
CRUD	/api/v1/assessments/grading-systems
POST	/api/v1/assessments/grading-systems/calculate-grade
CRUD	/api/v1/assessments/continuous-assessments
POST	/api/v1/assessments/continuous-assessments/{id}/record-scores
GET	/api/v1/assessments/continuous-assessments/{id}/scores
CRUD	/api/v1/assessments/psychomotor-assessments
GET	/api/v1/assessments/scoreboards/class/{classId}
GET	/api/v1/assessments/scoreboards/top-performers
GET	/api/v1/assessments/scoreboards/subject/{id}/toppers
GET	/api/v1/assessments/report-cards/{student}/{term}/{year}
GET	/api/v1/assessments/report-cards/{student}/{term}/{year}/pdf
POST	/api/v1/assessments/report-cards/{student}/{term}/{year}/email
POST	/api/v1/assessments/report-cards/bulk-download
GET	/api/v1/assessments/analytics/school
GET	/api/v1/assessments/analytics/class/{id}
GET	/api/v1/assessments/analytics/student/{id}/trend
POST	/api/v1/assessments/promotions/promote
POST	/api/v1/assessments/promotions/bulk-promote
POST	/api/v1/assessments/promotions/auto-promote
POST	/api/v1/assessments/promotions/graduate
CRUD	/api/v1/quizzes
CRUD	/api/v1/grades
CRUD	/api/v1/timetable
CRUD	/api/v1/announcements
CRUD	/api/v1/question-bank

MODULE: FEE_MANAGEMENT

CRUD	/api/v1/financial/fees
POST	/api/v1/financial/fees/{id}/pay
GET	/api/v1/financial/fees/student/{id}
GET POST	/api/v1/financial/fees/structure
CRUD	/api/v1/financial/payments
GET	/api/v1/financial/payments/student/{id}
GET	/api/v1/financial/payments/receipt/{id}
CRUD	/api/v1/financial/expenses
CRUD	/api/v1/financial/payroll
GET	/api/v1/financial/payroll/{id}/pay-stub

MODULE: ATTENDANCE_MANAGEMENT

```
GET|POST|PUT|DELETE /api/v1/attendance
GET /api/v1/attendance/reports
GET /api/v1/attendance/class/{classId}
GET /api/v1/attendance/student/{studentId}
POST /api/v1/attendance/mark
```

MODULE: TRANSPORT_MANAGEMENT

```
CRUD /api/v1/transport/vehicles
CRUD /api/v1/transport/drivers
CRUD /api/v1/transport/routes
GET /api/v1/transport/routes/{id}/students
POST /api/v1/transport/routes/{id}/students
DELETE /api/v1/transport/routes/{id}/students
CRUD /api/v1/transport/secure-pickup
POST /api/v1/transport/secure-pickup/verify
```

MODULE: HOSTEL_MANAGEMENT

```
CRUD /api/v1/hostel/rooms
CRUD /api/v1/hostel/allocations
POST /api/v1/hostel/allocations/{id}/vacate
CRUD /api/v1/hostel/maintenance
```

MODULE: HEALTH_MANAGEMENT

```
CRUD /api/v1/health/records
CRUD /api/v1/health/appointments
CRUD /api/v1/health/medications
```

MODULE: INVENTORY_MANAGEMENT

```
CRUD /api/v1/inventory/categories
CRUD /api/v1/inventory/items
CRUD /api/v1/inventory/transactions
POST /api/v1/inventory/transactions/checkout
POST /api/v1/inventory/transactions/{id}/return
```

MODULE: EVENT_MANAGEMENT

```
CRUD /api/v1/events
GET /api/v1/events/upcoming
CRUD /api/v1/calendars
```

MODULE: LIVESTREAM

CRUD	/api/v1/livestreams
POST	/api/v1/livestreams/{id}/start
POST	/api/v1/livestreams/{id}/end
POST	/api/v1/livestreams/{id}/join
POST	/api/v1/livestreams/{id}/leave
GET	/api/v1/livestreams/{id}/attendance

MODULE: SMS_INTEGRATION

POST	/api/v1/communication/sms/send
------	--------------------------------

MODULE: EMAIL_INTEGRATION

POST	/api/v1/communication/email/send
------	----------------------------------

LANDING PAGE (NO MODULE GATE)

GET	/api/v1/schools/landing-page
PUT	/api/v1/schools/landing-page
POST	/api/v1/schools/landing-page/upload-asset

NO MODULE GATE

CRUD	/api/v1/communication/messages
PUT	/api/v1/communication/messages/{id}/read
CRUD	/api/v1/communication/notifications
PUT	/api/v1/communication/notifications/{id}/read
PUT	/api/v1/communication/notifications/read-all
GET	/api/v1/reports/academic
GET	/api/v1/reports/financial
GET	/api/v1/reports/attendance
GET	/api/v1/reports/performance
GET	/api/v1/reports/{type}/export
CRUD	/api/v1/staff
CRUD	/api/v1/achievements
CRUD	/api/v1/library/books
POST	/api/v1/library/borrow
POST	/api/v1/library/return
CRUD	/api/v1/houses
CRUD	/api/v1/sports/activities
GET PUT	/api/v1/settings
POST	/api/v1/bulk/students/register
POST	/api/v1/bulk/teachers/register
POST	/api/v1/bulk/attendance/mark
POST	/api/v1/bulk/notifications/send
POST	/api/v1/uploads/upload

Common Request Headers

```
Authorization: Bearer {sanctum_token}
Content-Type: application/json
Accept: application/json
X-Subdomain: {tenant_subdomain}           (alternative to using subdomain in URL)
X-School-ID: {school_id}                  (if tenant has multiple schools)
```

Validation Errors

```
HTTP 422
{
  "message": "The given data was invalid.",
  "errors": {
    "email": ["The email field is required."],
    "amount": ["The amount must be a number."]
  }
}
```

Module Denied

```
HTTP 403
{
  "error": "Module access denied",
  "message": "This school does not have access to the transport_management module. Please upgrade your subscription.",
  "module": "transport_management",
  "upgrade_required": true
}
```

Frontend Integration

Domain Architecture

Domain	Purpose
compasse.net	Company marketing website
{school}.compasse.net	School portal (tenanted)

API Base URLs

```
// src/utils/tenancy.ts
export const ROOT_DOMAIN = 'compasse.net';
export const API_BASE = `https://${ROOT_DOMAIN}/api/v1`;

// For school tenant requests (called on subdomain):
// https://greenfield.compasse.net/api/v1/...
// The Laravel tenancy middleware resolves the tenant from the subdomain.
```

Detecting Context

```
import { getSubdomain, isCompanyDomain } from '@utils/tenancy';

const subdomain = getSubdomain();
// greenfield.compasse.net → "greenfield"
// compasse.net           → null
// localhost (dev)         → process.env.VITE_DEV_SUBDOMAIN || null

if (isCompanyDomain()) {
  // Render company marketing page (compasse.net)
} else {
  // Render school portal (subdomain.compasse.net)
}
```

Local dev: set `VITE_DEV_SUBDOMAIN=greenfield` in `.env.local` to simulate a school subdomain.

Token Storage per Subdomain

```
// Each school gets its own token key, allowing multiple tabs across schools.
const key = `compasse_token_${subdomain}`;
localStorage.setItem(key, token);
const token = localStorage.getItem(key);
```

Making Authenticated Requests

```
// src/services/schoolApi.ts
export function createSchoolApi(subdomain: string) {
  const base = `https://${subdomain}.${ROOT_DOMAIN}/api/v1`;
  const headers = () => ({
    'Authorization': `Bearer ${localStorage.getItem(`compasse_token_${subdomain}`)}`,
    'Content-Type': 'application/json',
    'Accept': 'application/json',
  });

  return {
    get: (path: string) => fetch(`${base}${path}`, { headers: headers() }),
    post: (path: string, body: unknown) =>
      fetch(`${base}${path}`, { method: 'POST', headers: headers(), body: JSON.stringify(body)
    }),
    put: (path: string, body: unknown) =>
      fetch(`${base}${path}`, { method: 'PUT', headers: headers(), body: JSON.stringify(body)
    }),
    delete: (path: string) =>
      fetch(`${base}${path}`, { method: 'DELETE', headers: headers() }),
  };
}
```

Module Gating in Routes

```
// Only renders if the school's subscription includes "transport_management"
<Route
  path="/school/transport"
  element={
    <SchoolLayout requiredModule="transport_management">
      <TransportPage />
    </SchoolLayout>
  }
/>

// Only renders for specific roles
<Route
  path="/school/finance"
  element={
    <SchoolLayout requiredModule="fee_management" requiredRoles={['admin', 'accountant']}>
      <Finance />
    </SchoolLayout>
  }
/>
```

Public School Landing Page

Fetch school data without authentication:

```
// GET https://compasse.net/api/v1/public/{subdomain}
const res = await fetch(`https://compasse.net/api/v1/public/${subdomain}`);
const { school, settings, template } = await res.json();
// school: { id, name, logo_url, ... }
// settings: { hero_title, hero_subtitle, contact_email, ... }
// template: { id, name, colors, ... }
```

The `"/"` route renders `<PublicLandingPage />` which:

- Shows `<CompanyMarketingPage />` when `getSubdomain() === null` (i.e., `compasse.net`)
- Fetches and renders the school's configured landing page when a subdomain is detected

Frontend Integration Guide — Multi-Tenant Architecture

Target audience: Frontend developers building the Compasse school portal (React / Next.js / Vue / React Native) **Backend:** Laravel 12 + stancl/tenancy v3 · One database per school **Auth:** Laravel Sanctum (opaque bearer tokens per tenant)

How Multi-Tenancy Works

Each school gets its own subdomain and isolated database:

```
https://greenfield.compassse.net → Greenfield Academy (tenant DB: db_greenfield)
https://stmarys.compassse.net   → St Mary's School   (tenant DB: db_stmarys)
https://compassse.net           → Super Admin portal (central DB)
```

The backend identifies the tenant from the `Host` header subdomain. All data — users, students, teachers, tokens — lives in that tenant's database. A token from `greenfield` is completely unknown to `stmarys`.

Base URL Strategy

Option A — Subdomain URL (recommended for web)

```
https://{subdomain}.compassse.net/api/v1/{endpoint}
```

The backend auto-resolves the tenant from the subdomain.

Option B — X-Subdomain header (recommended for mobile / cross-origin)

```
https://compassse.net/api/v1/{endpoint}
X-Subdomain: greenfield
```

Use this when you cannot control the subdomain (e.g. React Native, Electron).

Option C — X-School-ID header (multi-school tenants)

Some tenants manage more than one school. After login, pass:

```
X-School-ID: 3
```

to scope requests to a specific school within the tenant.

Required Headers (all authenticated requests)

```
Authorization: Bearer {token}
Content-Type: application/json
Accept: application/json
X-Subdomain: {subdomain}           (if not using subdomain URL)
X-School-ID: {school_id}           (optional – multi-school tenants only)
```

Authentication Flow

Step 1 — Detect subdomain (web)

```
// utils/tenant.ts
export function getSubdomain(): string | null {
  const host = window.location.hostname;           // e.g. greenfield.compassse.net
  const parts = host.split('.');
  if (parts.length >= 3) return parts[0];           // "greenfield"
  return null;                                     // top-level domain – super admin or
  landing
}
```

Step 2 — Verify tenant exists

```
// Before showing login screen, verify the subdomain is a valid tenant
const subdomain = getSubdomain();

const res = await fetch(`https://compassse.net/api/v1/tenants/verify`, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ subdomain }),
});

if (!res.ok) {
  // Redirect to compassse.net – invalid subdomain
  window.location.href = 'https://compassse.net';
}
const { tenant, school } = await res.json();
// Store school info for display on login page
```

Step 3 — Login

```
const login = async (email: string, password: string) => {
  const res = await fetch(
    `https://${subdomain}.compasse.net/api/v1/auth/login`,
    {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ email, password }),
    }
  );

  if (!res.ok) throw new Error('Invalid credentials');

  const { token, user } = await res.json();

  // Store token keyed by subdomain – supports multiple schools open at once
  localStorage.setItem(`compasse_token_${subdomain}`, token);
  localStorage.setItem(`compasse_user_${subdomain}`, JSON.stringify(user));
  localStorage.setItem('compasse_active_subdomain', subdomain);

  return { token, user };
};
```

Step 4 — All subsequent requests

```
// utils/api.ts
const subdomain = localStorage.getItem('compasse_active_subdomain');
const token      = localStorage.getItem(`compasse_token_${subdomain}`);

const api = async (path: string, options: RequestInit = {}) => {
  const res = await fetch(
    `https://${subdomain}.compasse.net/api/v1/${path}`,
    {
      ...options,
      headers: {
        'Authorization': `Bearer ${token}`,
        'Content-Type': 'application/json',
        'Accept': 'application/json',
        ...(options.headers ?? {}),
      },
    },
  );

  if (res.status === 401) {
    // Token expired – redirect to login
    localStorage.removeItem(`compasse_token_${subdomain}`);
    window.location.href = '/login';
  }

  if (res.status === 403) {
    const body = await res.json();
    if (body.upgrade_required) {
      // Show subscription upgrade modal
      showUpgradeModal(body.module);
    }
    throw new Error(body.message);
  }

  return res;
};
```

Super Admin Authentication

The super admin does NOT use a subdomain. The frontend for the super admin portal lives at `https://compasse.net/admin`.

```

const superAdminLogin = async (email: string, password: string) => {
  const res = await fetch('https://compasse.net/api/v1/auth/login', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ email, password }),
  });

  const { token, user } = await res.json();
  localStorage.setItem('compasse_superadmin_token', token);
  return { token, user };
};

// All super admin API calls use this helper
const adminApi = async (path: string, options: RequestInit = {}) => {
  const token = localStorage.getItem('compasse_superadmin_token');
  return fetch(`https://compasse.net/api/v1/${path}`, {
    ...options,
    headers: {
      'Authorization': `Bearer ${token}`,
      'Content-Type': 'application/json',
      'Accept': 'application/json',
      ...(options.headers ?? {}),
    },
  });
};

```

Module-Based UI Gating

After login, fetch the school's enabled modules and cache them. Use this list to show/hide navigation items and feature sections.

```

// On login success – fetch enabled modules
const loadModules = async () => {
  const res = await api('subscriptions/school/modules');
  const { modules } = await res.json();

  // modules = ["academic_management", "student_management", "cbt", ...]
  sessionStorage.setItem(`modules_${subdomain}`, JSON.stringify(modules));
  return modules;
};

// Helper to check if a module is active
export const hasModule = (module: string): boolean => {
  const modules = JSON.parse(
    sessionStorage.getItem(`modules_${subdomain}`) ?? '[]'
  );
  return modules.includes(module);
};

// Usage in navigation
{hasModule('fee_management') && <NavItem href="/fees" label="Fees" />}
{hasModule('cbt') && <NavItem href="/assessments" label="Assessments" />}

```


Handling 403 Module Denied

```
// When backend returns 403 with upgrade_required: true
{
  "error": "Module access denied",
  "message": "This school does not have access to the transport_management module.",
  "module": "transport_management",
  "upgrade_required": true
}

// Frontend: intercept in API wrapper and show upgrade modal
const handleUpgradeRequired = (module: string) => {
  // Fetch available plans
  api('subscriptions/plans').then(r => r.json()).then(({ plans }) => {
    showModal(<UpgradeModal module={module} plans={plans} />);
  });
};
```

Pagination

All list endpoints return paginated responses. Use the standard pagination wrapper:

```
interface PaginatedResponse<T> {
  data: T[];
  current_page: number;
  last_page: number;
  per_page: number;
  total: number;
  next_page_url: string | null;
  prev_page_url: string | null;
}

// Example – paginated student list
const fetchStudents = async (page = 1, perPage = 25) => {
  const res = await api(`students?page=${page}&per_page=${perPage}`);
  return res.json() as Promise<PaginatedResponse<Student>>;
};
```

File Uploads

```
const uploadProfilePicture = async (userId: number, file: File) => {
  const formData = new FormData();
  formData.append('file', file);

  const res = await fetch(
    `https://${subdomain}.compass.net/api/v1/users/${userId}/profile-picture`,
    {
      method: 'POST',
      headers: {
        'Authorization': `Bearer ${token}`,
        'Accept': 'application/json',
        // Do NOT set Content-Type – browser sets it with boundary for multipart
      },
      body: formData,
    }
  );

  return res.json(); // { url: "https://..." }
};
```

Error Handling Reference

HTTP Status	Meaning	Frontend Action
200	OK	Process response
201	Created	Show success, refresh list
400	Bad request / missing school context	Show error message
401	Token missing or expired	Clear token, redirect to login
403 upgrade_required: true	Module not subscribed	Show upgrade modal
403 (role denied)	Insufficient permissions	Show “Access Denied”
404	Record not found	Show 404 UI
422	Validation failed	Show field-level errors from <code>errors</code> object
429	Rate limit hit	Back off and retry after <code>Retry-After</code> header
500	Server error	Show generic error, log to Sentry
503	Subscription check failed	Show “Service Unavailable” with retry

Validation error handling (422)

```
if (res.status === 422) {
  const { errors } = await res.json();
  // errors = { "email": ["The email field is required."], "amount": [...] }
  // Map to form field errors
  Object.entries(errors).forEach(([field, messages]) => {
    form.setError(field, { message: messages[0] });
  });
}
```

React Query / SWR Integration

```
// hooks/useStudents.ts
import { useQuery } from '@tanstack/react-query';

export const useStudents = (page = 1) =>
  useQuery({
    queryKey: ['students', page],
    queryFn: () => api(`students?page=${page}`).then(r => r.json()),
    staleTime: 30_000,
  });

// Cache invalidation after mutation
const queryClient = useQueryClient();
const createStudent = useMutation({
  mutationFn: (data) => api('students', { method: 'POST', body: JSON.stringify(data) }),
  onSuccess: () => queryClient.invalidateQueries({ queryKey: ['students'] }),
});
```

Real-Time Notifications (WebSocket / Polling)

Compassse uses a queue-based notification system. Frontend has two options:

Option A — Polling (simple)

```
// Poll every 30 seconds for new notifications
setInterval(async () => {
  const res = await api('communication/notifications?unread=1&per_page=5');
  const { data } = await res.json();
  setUnreadCount(data.length);
}, 30_000);
```

Option B — Laravel Echo + Pusher (real-time)

```
import Echo from 'laravel-echo';
import Pusher from 'pusher-js';

const echo = new Echo({
  broadcaster: 'pusher',
  key: process.env.NEXT_PUBLIC_PUSHER_KEY,
  cluster: process.env.NEXT_PUBLIC_PUSHER_CLUSTER,
  forceTLS: true,
  auth: { headers: { Authorization: `Bearer ${token}` } } },
});

echo.private(`school.${schoolId}`)
  .listen('.notification.created', (e) => {
    addNotification(e.notification);
    incrementBadge();
  });
```

Multi-School Support (within one tenant)

Some tenants operate multiple schools. After login, check if the user has access to multiple schools:

```
const { user } = await api('auth/me').then(r => r.json());

if (user.schools && user.schools.length > 1) {
  // Show school picker
  showSchoolSelector(user.schools);
}

// After school selection, include X-School-ID in all subsequent requests
const switchSchool = (schoolId: number) => {
  localStorage.setItem(`compasse_school_${subdomain}`, String(schoolId));
};

// In API wrapper, add school header when set
const schoolId = localStorage.getItem(`compasse_school_${subdomain}`);
if (schoolId) headers['X-School-ID'] = schoolId;
```

Landing Page (Public)

School landing pages are publicly accessible — no auth required.

```
// Fetch a school's public landing page data
const fetchLandingPage = async (subdomain: string) => {
  const res = await fetch(
    `https://compasse.net/api/v1/public/${subdomain}`,
    { headers: { 'Accept': 'application/json' } }
  );
  return res.json();
};

// Response includes:
// - school info (name, logo, contact)
// - template (id, colors, features)
// - hero (headline, image, CTA)
// - about (tagline, vision, mission)
// - social links
// - SEO metadata
```

Environment Variables (Frontend)

```
NEXT_PUBLIC_API_BASE=https://compasse.net/api/v1
NEXT_PUBLIC_SUBDOMAIN_BASE=compasse.net
NEXT_PUBLIC_PUSHER_KEY=your_pusher_key
NEXT_PUBLIC_PUSHER_CLUSTER=eu
NEXT_PUBLIC_SENTRY_DSN=https://...
```

React Native / Mobile

For mobile apps, use the `X-Subdomain` header approach — no subdomain in URL required.

```
// Secure token storage for mobile
import * as SecureStore from 'expo-secure-store';

const saveToken = async (subdomain: string, token: string) => {
  await SecureStore.setItemAsync(`compasse_token_${subdomain}`, token);
};

const getToken = async (subdomain: string) => {
  return SecureStore.getItemAsync(`compasse_token_${subdomain}`);
};

// API wrapper with X-Subdomain header
const mobileApi = async (subdomain: string, path: string, options = {}) => {
  const token = await getToken(subdomain);
  return fetch(`https://compasse.net/api/v1/${path}`, {
    ...options,
    headers: {
      'Authorization': `Bearer ${token}`,
      'X-Subdomain': subdomain,
      'Content-Type': 'application/json',
      'Accept': 'application/json',
    },
  });
};
```

TypeScript Types Reference

```
// Core types

interface User {
  id: number;
  name: string;
  email: string;
  role: string;
  roles: string[];
  profile_picture?: string;
}

interface School {
  id: number;
  name: string;
  code: string;
  logo?: string;
  email: string;
  phone: string;
  address: string;
  status: 'active' | 'suspended';
}

interface Subscription {
  id: number;
  plan_name: string;
  status: 'active' | 'trial' | 'cancelled' | 'expired';
  modules: string[];
  end_date: string;
  days_remaining: number;
}

interface ApiError {
  message: string;
  errors?: Record<string, string[]>;
  error?: string;
  module?: string;
  upgrade_required?: boolean;
}
```

School Customization — Landing Page & Branding

Base URL: `https://{subdomain}.compasse.net/api/v1/` **Public endpoints:** No auth required

Protected endpoints: Authorization: Bearer {token} + tenant context required **Module gate:** None — available to all authenticated school admins

Overview

Every school on Compasse gets a public-facing landing page. Admins can:

1. **Choose a template** from 5 predefined designs
2. **Configure branding** — colours, hero image/video, CTA button
3. **Add school information** — about text, vision, mission, contact details
4. **Set up social links** — Facebook, Twitter, Instagram, YouTube, LinkedIn
5. **Upload assets** — hero image, about image, Open Graph image
6. **Configure SEO** — page title, meta description, keywords
7. **Enable/disable features** — choose which sections appear on the landing page

All landing page settings are stored in the `settings` table under `category = 'landing_page'`. No separate migration is required.

User Stories

As a school admin, I want to choose a template for our public landing page so the school's website looks professional without needing a web developer.

As a school admin, I want to upload a hero banner image and set a tagline so that parents see the right first impression when they visit our subdomain.

As a parent, I want to visit `greenfield.compasse.net` and immediately see the school's name, logo, contact details, and a “Apply Now” button.

As a frontend developer, I want a single public API endpoint that returns all landing page data so I can render the school website with one request.

Templates

Five predefined templates are available. Templates define the layout structure, default colour palette, and which feature sections are supported.

ID	Name	Description
classic	Classic	Traditional navy + white, bulletin-board sidebar, news ticker
modern	Modern	Full-width hero, card layout, dark sticky header, stats counter
minimal	Minimal	Clean white, icon-driven feature blocks, thin typographic hierarchy
vibrant	Vibrant	Gradient header, animated counters, social media feed strip
academic	Academic	Ivory background, serif headings, crest-first branding — for universities

Each template ships with a default colour palette that is overridable per school.

API Endpoints

Public Endpoints (no auth)

Method	Endpoint	Description
GET	/api/v1/schools/landing-page/templates	List all available templates with previews
GET	/api/v1/public/{subdomain}	Get a school's full landing page data (for rendering)

Authenticated Endpoints (tenant + auth)

Method	Endpoint	Description
GET	/api/v1/schools/landing-page	Get the current school's landing page config
PUT	/api/v1/schools/landing-page	Update landing page settings (partial update supported)
POST	/api/v1/schools/landing-page/upload-asset	Upload a hero/about/OG image

Request & Response Examples

GET /api/v1/schools/landing-page/templates

No auth required.

Request:

```
GET /api/v1/schools/landing-page/templates
Accept: application/json
```

Response 200 :

```
{
  "templates": [
    {
      "id": "classic",
      "name": "Classic",
      "description": "Traditional school website look – white background, navy header, bulletin-board sidebar.",
      "preview_url": "/templates/previews/classic.png",
      "colors": {
        "primary": "#1a3a6b",
        "secondary": "#f5a623",
        "background": "#ffffff",
        "text": "#333333"
      },
      "features": ["hero_image", "news_ticker", "quick_links", "contact_form"]
    },
    {
      "id": "modern",
      "name": "Modern",
      "description": "Full-width hero, card-based layout, bold typography, and a dark sticky header.",
      "preview_url": "/templates/previews/modern.png",
      "colors": {
        "primary": "#0f172a",
        "secondary": "#6366f1",
        "background": "#f8fafc",
        "text": "#1e293b"
      },
      "features": ["hero_video", "stats_counter", "testimonials", "cta_banner"]
    },
    {
      "id": "minimal",
      "name": "Minimal",
      "description": "Clean white canvas – icon-driven feature blocks, thin typographic hierarchy.",
      "preview_url": "/templates/previews/minimal.png",
      "colors": {
        "primary": "#18181b",
        "secondary": "#22c55e",
        "background": "#ffffff",
        "text": "#3f3f46"
      },
      "features": ["hero_image", "feature_icons", "contact_form"]
    },
    {
      "id": "vibrant",
      "name": "Vibrant",
      "description": "Bright gradient header, animated counters, social media feed strip.",
      "preview_url": "/templates/previews/vibrant.png",
      "colors": {
        "primary": "#7c3aed",
        "secondary": "#f59e0b",
        "background": "#faf5ff",
        "text": "#1f1f1f"
      },
      "features": ["hero_image", "stats_counter", "news_ticker", "social_strip", "cta_banner"]
    }
  ]
}
```

```
{
  "id": "academic",
  "name": "Academic",
  "description": "Formal ivory background, serif headings, crest-first branding.",
  "preview_url": "/templates/previews/academic.png",
  "colors": {
    "primary": "#1d4ed8",
    "secondary": "#b45309",
    "background": "#ffffbe",
    "text": "#292524"
  },
  "features": ["hero_image", "quick_links", "news_section", "contact_form",
"departments_grid"]
},
{
  "count": 5
}
```

GET /api/v1/public/{subdomain}

No auth required. Used by the frontend to render the school's landing page.

Request:

```
GET /api/v1/public/greenfield
Accept: application/json
```

Response 200 :

```

{
  "school": {
    "name": "Greenfield Academy",
    "logo": "https://cdn.compassse.net/schools/1/logo.png",
    "email": "info@greenfieldacademy.edu.ng",
    "phone": "+234 801 234 5678",
    "address": "14 Education Road, Lagos, Nigeria",
    "website": "https://greenfieldacademy.edu.ng"
  },
  "template": {
    "id": "modern",
    "name": "Modern",
    "colors": {
      "primary": "#0f172a",
      "secondary": "#6366f1",
      "background": "#f8fafc",
      "text": "#1e293b"
    },
    "features": ["hero_video", "stats_counter", "testimonials", "cta_banner"]
  },
  "branding": {
    "primary_color": "#1a3a6b",
    "secondary_color": "#d4a017",
    "background_color": "#ffffff",
    "text_color": "#333333",
    "custom_css": null
  },
  "hero": {
    "headline": "Shaping Tomorrow's Leaders Today",
    "subheadline": "A Premier Secondary School in Lagos – Nurturing Excellence Since 1998",
    "image_url": "https://cdn.compassse.net/schools/1/landing/hero_image/banner.jpg",
    "video_url": null,
    "cta_text": "Apply for Admission",
    "cta_url": "https://greenfield.compassse.net/apply"
  },
  "about": {
    "tagline": "Where Excellence Meets Character",
    "text": "Greenfield Academy is committed to providing world-class education in a nurturing environment...",
    "image_url": "https://cdn.compassse.net/schools/1/landing/about_image/building.jpg",
    "vision": "To produce well-rounded, globally competitive graduates.",
    "mission": "To deliver quality education through innovative teaching and strong values."
  },
  "contact": {
    "address": "14 Education Road, Lekki Phase 1, Lagos",
    "phone": "+234 801 234 5678",
    "email": "info@greenfieldacademy.edu.ng",
    "map_embed_url": "https://maps.google.com/maps?q=Greenfield+Academy+Lagos&output=embed",
    "working_hours": "Mon–Fri: 7:30am – 3:30pm"
  },
  "social": {
    "facebook": "https://facebook.com/GreenfieldAcademyLagos",
    "twitter": "https://twitter.com/GreenfieldLagos",
    "instagram": "https://instagram.com/greenfield_academy",
    "youtube": "https://youtube.com/@GreenfieldAcademy",
    "linkedin": null
  }
}

```

```
"features": ["hero_image", "stats_counter", "news_ticker", "contact_form"],
"seo": {
  "title": "Greenfield Academy – Premier Secondary School in Lagos",
  "description": "Greenfield Academy offers world-class JSS and SSS education in Lagos,
Nigeria. Apply for admission today.",
  "keywords": "secondary school lagos, best school lagos, greenfield academy",
  "og_image": "https://cdn.compassse.net/schools/1/logo.png"
}
}
```

Response 404 :

```
{ "error": "School not found" }
```

GET /api/v1/schools/landing-page

Returns the current school's config plus all available templates (for the admin UI picker).

Request:

```
GET /api/v1/schools/landing-page
Authorization: Bearer 1|abc123...
X-Subdomain: greenfield
Accept: application/json
```

Response 200 :

```
{
  "school": {
    "id": 1,
    "name": "Greenfield Academy",
    "logo": "https://cdn.compassse.net/schools/1/logo.png",
    "email": "info@greenfieldacademy.edu.ng"
  },
  "template": {
    "id": "modern",
    "name": "Modern",
    "colors": { "primary": "#0f172a", "secondary": "#6366f1" }
  },
  "settings": {
    "template": "modern",
    "primary_color": "#1a3a6b",
    "hero_headline": "Shaping Tomorrow's Leaders Today",
    "hero_cta_text": "Apply for Admission",
    "contact_phone": "+234 801 234 5678",
    "social_facebook": "https://facebook.com/GreenfieldAcademyLagos"
  },
  "all_templates": [ ... ]
}
```

PUT /api/v1/schools/landing-page

Partial update — only send the keys you want to change.

Request:

```
PUT /api/v1/schools/landing-page
Authorization: Bearer 1|abc123...
X-Subdomain: greenfield
Content-Type: application/json
```

```
{
  "template": "modern",
  "primary_color": "#1a3a6b",
  "secondary_color": "#d4a017",
  "hero_headline": "Shaping Tomorrow's Leaders Today",
  "hero_subheadline": "A Premier Secondary School in Lagos Since 1998",
  "hero_cta_text": "Apply for Admission",
  "hero_cta_url": "https://greenfield.compass.net/apply",
  "about_tagline": "Where Excellence Meets Character",
  "about_text": "Greenfield Academy is committed to providing world-class education...",
  "vision": "To produce well-rounded, globally competitive graduates.",
  "mission": "To deliver quality education through innovative teaching and strong values.",
  "contact_phone": "+234 801 234 5678",
  "contact_email": "info@greenfieldacademy.edu.ng",
  "contact_address": "14 Education Road, Lekki Phase 1, Lagos",
  "working_hours": "Mon-Fri: 7:30am - 3:30pm",
  "map_embed_url": "https://maps.google.com/maps?q=Greenfield+Academy+Lagos&output=embed",
  "social_facebook": "https://facebook.com/GreenfieldAcademyLagos",
  "social_twitter": "https://twitter.com/GreenfieldLagos",
  "social_instagram": "https://instagram.com/greenfield_academy",
  "seo_title": "Greenfield Academy - Premier Secondary School in Lagos",
  "seo_description": "World-class JSS and SSS education in Lagos. Apply today.",
  "seo_keywords": "secondary school lagos, greenfield academy",
  "enabled_features": ["hero_image", "stats_counter", "contact_form", "news_ticker"]
}
```

Response 200 :

```
{
  "message": "Landing page updated successfully",
  "settings": {
    "template": "modern",
    "primary_color": "#1a3a6b",
    "hero_headline": "Shaping Tomorrow's Leaders Today",
    ...
  }
}
```

Validation error 422 :

```
{
  "errors": {
    "template": ["The selected template is invalid."],
    "contact_email": ["The contact email must be a valid email address."]
  }
}
```

POST /api/v1/schools/landing-page/upload-asset

Upload an image for the landing page. Returns the public URL and automatically saves it to the corresponding setting key.

Request (multipart/form-data):

```
POST /api/v1/schools/landing-page/upload-asset
Authorization: Bearer 1|abc123...
X-Subdomain: greenfield
Content-Type: multipart/form-data
```

Form fields:

Field	Type	Values	Description
file	File	jpg, jpeg, png, webp, gif (max 5MB)	The image to upload
type	string	hero_image , about_image , logo , og_image	Asset category

Response 201 :

```
{
  "message": "Asset uploaded successfully",
  "url": "https://cdn.compassse.net/schools/1/landing/hero_image/banner.jpg",
  "key": "hero_image_url"
}
```

Configurable Settings Reference

All keys accepted by `PUT /api/v1/schools/landing-page` :

Template & Colours

Key	Type	Description
template	string	Template ID: classic , modern , minimal , vibrant , academic
primary_color	string	Primary brand colour (hex, e.g. #1a3a6b)
secondary_color	string	Accent colour
background_color	string	Page background colour
text_color	string	Body text colour
custom_css	string	Raw CSS injected into <style> (max 10,000 chars)

Hero Section

Key	Type	Description
hero_headline	string	Main hero heading
hero_subheadline	string	Sub-heading below headline
hero_image_url	string	URL of the hero banner image (upload via upload-asset)
hero_video_url	string	YouTube/Vimeo embed URL for video hero
hero_cta_text	string	Call-to-action button label (e.g. “Apply Now”)
hero_cta_url	string	CTA button destination URL

About Section

Key	Type	Description
about_tagline	string	Short tagline (e.g. “Where Excellence Meets Character”)
about_text	string	Long-form about paragraph
about_image_url	string	About section image URL
vision	string	School vision statement
mission	string	School mission statement

Contact Section

Key	Type	Description
contact_address	string	Physical address shown on landing page
contact_phone	string	Contact phone number
contact_email	string	Contact email
map_embed_url	string	Google Maps embed URL
working_hours	string	Office hours text

Social Links

Key	Type
social_facebook	URL string
social_twitter	URL string
social_instagram	URL string
social_youtube	URL string
social_linkedin	URL string

SEO

Key	Type	Description
seo_title	string	<title> tag (max 255 chars)
seo_description	string	Meta description (max 500 chars)
seo_keywords	string	Meta keywords (comma-separated)
seo_og_image	string	Open Graph image URL (1200×630 recommended)

Feature Toggle

Key	Type	Description
enabled_features	array	List of feature section IDs to show on landing page

Available feature IDs by template:

Feature ID	Description	Available In
hero_image	Hero banner with image	classic, minimal, vibrant, academic
hero_video	Video background hero	modern
stats_counter	Animated counters (students, teachers, years)	modern, vibrant
news_ticker	Scrolling news/announcements strip	classic, vibrant
quick_links	Icon grid of quick navigation links	classic, academic
contact_form	Contact inquiry form	classic, minimal, academic
testimonials	Parent/alumni testimonials slider	modern
cta_banner	Full-width call-to-action banner	modern, vibrant
social_strip	Social media feed strip	vibrant
feature_icons	Icon + text feature highlights	minimal
news_section	Full news article listing	academic
departments_grid	Grid of school departments	academic

Business Rules

1. **Partial updates** — `PUT /schools/landing-page` merges with existing settings; keys not sent are unchanged
 2. **Template defaults** — if `primary_color` etc. are not set, the selected template's default colours are used at display time (in the `GET /public/{subdomain}` response)
 3. **Asset uploads** — uploaded files are stored at `schools/{id}/landing/{type}/` and served via `Storage::url()`
 4. `enabled_features` **is JSON** — sent as an array in requests, stored as JSON string, returned as array
 5. **No module gate** — landing page customisation is available to all tenant admins on any subscription plan
-

Frontend Integration

Admin UI — Landing Page Builder

```
// 1. Load available templates (once, can be cached)
const { templates } = await fetch('/api/v1/schools/landing-page/templates').then(r => r.json());

// 2. Load current settings
const { settings, template } = await api('schools/landing-page').then(r => r.json());

// 3. Render template picker with live preview
<TemplatePicker
  templates={templates}
  selected={settings.template ?? 'classic'}
  onChange={(id) => setForm({ ...form, template: id })}
/>

// 4. Colour picker for brand colours
<ColourPicker label="Primary Colour" value={form.primary_color ?? template.colors.primary}
  onChange={(v) => setForm({ ...form, primary_color: v })} />

// 5. Upload hero image
const uploadHero = async (file: File) => {
  const formData = new FormData();
  formData.append('file', file);
  formData.append('type', 'hero_image');

  const res = await fetch(`/api/v1/schools/landing-page/upload-asset`, {
    method: 'POST',
    headers: { Authorization: `Bearer ${token}` },
    body: formData,
  });
  const { url } = await res.json();
  setForm({ ...form, hero_image_url: url });
};

// 6. Save all changes
const save = async () => {
  await api('schools/landing-page', {
    method: 'PUT',
    body: JSON.stringify(form),
  });
  toast.success('Landing page updated!');
};
```

Rendering the Public Landing Page

```
// pages/[subdomain].tsx (Next.js)
export async function getServerSideProps({ params }) {
  const data = await fetch(
    `https://compasse.net/api/v1/public/${params.subdomain}`
  ).then(r => r.json());

  if (data.error) return { notFound: true };

  return { props: { landingPage: data } };
}

export default function SchoolLanding({ landingPage }) {
  const { school, template, hero, about, contact, social, seo, branding } = landingPage;

  return (
    <>
      <Head>
        <title>{seo.title}</title>
        <meta name="description" content={seo.description} />
        <meta property="og:image" content={seo.og_image} />
        {branding.custom_css && <style>{branding.custom_css}</style>}
      </Head>
      <TemplateRenderer template={template.id} data={landingPage} />
    </>
  );
}
```

SEO & Open Graph

The `GET /public/{subdomain}` response contains all SEO fields needed to populate `<head>` tags. Use the `seo_og_image` (1200×630px recommended) for social sharing previews.

```
<title>{seo.title}</title>
<meta name="description" content="{seo.description}" />
<meta name="keywords" content="{seo.keywords}" />
<meta property="og:title" content="{seo.title}" />
<meta property="og:description" content="{seo.description}" />
<meta property="og:image" content="{seo.og_image}" />
<meta property="og:url" content="https://{subdomain}.compasse.net" />
<meta name="twitter:card" content="summary_large_image" />
```



PART II

Role Dashboard API Reference

Per-role dashboard endpoints — the data each user type sees on login.

Admin Dashboard API Documentation

Complete API reference for School Admin Dashboard functionality.

Table of Contents

- 1. [Dashboard Overview](#)
 - 2. [School Management](#)
 - 3. [User Management](#)
 - 4. [Academic Setup](#)
 - 5. [Student Management](#)
 - 6. [Teacher Management](#)
 - 7. [Staff & Guardian Management](#)
 - 8. [Settings](#)
 - 9. [Subscriptions & Modules](#)
-

Authentication

All endpoints require:

```
Authorization: Bearer {token}
X-Subdomain: {school_subdomain}
Content-Type: application/json
```

Dashboard Overview

Get Admin Dashboard

Endpoint: GET /api/v1/dashboard/admin

Response (200):

```
{
  "user": {
    "id": 1,
    "name": "John Admin",
    "email": "admin@westwoodschool.com",
    "role": "school_admin"
  },
  "school": {
    "id": 1,
    "name": "Westwood School",
    "subdomain": "westwood",
    "logo": "https://...",
    "website": "https://westwoodschool.com"
  },
  "stats": {
    "total_students": 450,
    "total_teachers": 35,
    "total_staff": 20,
    "total_classes": 15,
    "active_terms": 1,
    "pending_fees": 250000
  },
  "role": "school_admin"
}
```

School Management

Get School Details

Endpoint: GET /api/v1/schools/{school_id}

Update School

Endpoint: PUT /api/v1/schools/{school_id}

Request Body:

```
{
  "name": "Westwood International School",
  "email": "info@westwoodschool.com",
  "phone": "+234803456789",
  "address": "123 Education Street, Lagos",
  "logo": "https://s3..../logo.png",
  "website": "https://westwoodschool.com",
  "motto": "Excellence in Education",
  "description": "Premier institution..."
}
```

Get School Statistics

Endpoint: GET /api/v1/schools/{school_id}/stats

Response (200):


```
{
  "students": {
    "total": 450,
    "active": 445,
    "inactive": 5,
    "by_class": [...]
  },
  "teachers": {
    "total": 35,
    "active": 34,
    "on_leave": 1
  },
  "staff": {
    "total": 20,
    "by_role": [...]
  },
  "academic": {
    "classes": 15,
    "subjects": 12,
    "arms": 30
  },
  "financial": {
    "total_fees_due": 5000000,
    "fees_paid": 4750000,
    "pending": 250000
  }
}
```

User Management

List All Users

Endpoint: GET /api/v1/users

Query Parameters:

- `role` - Filter by role
- `status` - Filter by status (active, inactive, suspended)
- `search` - Search by name/email
- `per_page` - Items per page (default: 15)

Create User

Endpoint: POST /api/v1/users

Request Body:

```
{
  "name": "Jane Doe",
  "email": "jane@westwoodschool.com",
  "password": "SecurePass123",
  "password_confirmation": "SecurePass123",
  "role": "teacher",
  "phone": "+234801234567",
  "status": "active",
  "profile_picture": "https://..."
}
```

Update User

Endpoint: PUT /api/v1/users/{id}

Activate User

Endpoint: POST /api/v1/users/{id}/activate

Suspend User

Endpoint: POST /api/v1/users/{id}/suspend

Delete User

Endpoint: DELETE /api/v1/users/{id}

Profile Picture Management

Upload Profile Picture

Endpoint: POST /api/v1/users/{id}/profile-picture

Request Body:

```
{
  "profile_picture": "https://s3.amazonaws.com/.../photo.jpg"
}
```

Response (200):

```
{
  "message": "Profile picture updated successfully",
  "user": {
    "id": 5,
    "name": "John Doe",
    "profile_picture": "https://..."
  }
}
```

Upload Own Profile Picture

Endpoint: POST /api/v1/users/me/profile-picture

Delete Profile Picture

Endpoint: DELETE /api/v1/users/{id}/profile-picture

Delete Own Profile Picture

Endpoint: DELETE /api/v1/users/me/profile-picture

Academic Setup

Academic Years

List Academic Years: GET /api/v1/academic-years

Create Academic Year:

```
POST /api/v1/academic-years
{
  "name": "2025/2026",
  "start_date": "2025-09-01",
  "end_date": "2026-07-31",
  "is_active": true
}
```

Terms

List Terms: GET /api/v1/terms

Create Term:

```
POST /api/v1/terms
{
  "academic_year_id": 1,
  "name": "First Term",
  "start_date": "2025-09-01",
  "end_date": "2025-12-15",
  "is_active": true
}
```

Departments

List Departments: GET /api/v1/departments

Create Department:

```
POST /api/v1/departments
{
  "name": "Science Department",
  "code": "SCI",
  "description": "All science subjects",
  "head_of_department_id": 5
}
```

Classes

List Classes: GET /api/v1/classes

Create Class:

```
POST /api/v1/classes
{
  "name": "JSS 1",
  "description": "Junior Secondary School 1",
  "capacity": 180,
  "class_teacher_id": 5,
  "academic_year_id": 1,
  "term_id": 1
}
```

Arms (Class Divisions)

List All Arms: GET /api/v1/arms

Create Arm:

```
POST /api/v1/arms
{
  "class_id": 1,
  "name": "A",
  "description": "JSS 1A",
  "capacity": 30,
  "class_teacher_id": 5,
  "status": "active"
}
```

Get Arms for a Class: GET /api/v1/arms/class/{class_id}

Response (200):

```
{
  "class": {
    "id": 1,
    "name": "JSS 1"
  },
  "arms": [
    {
      "id": 1,
      "name": "A",
      "capacity": 30,
      "students_count": 28,
      "stats": {
        "total_students": 28,
        "capacity_utilization": 93.33
      },
      "is_full": false,
      "available_capacity": 2
    }
  ],
  "total_arms": 6,
  "total_capacity": 180,
  "total_students": 165
}
```

Get Students in Arm: GET /api/v1/arms/{id}/students

Assign Teacher to Arm: POST /api/v1/arms/{id}/assign-teacher

```
{
  "class_teacher_id": 5
}
```

Subjects

List Subjects: GET /api/v1/subjects

Create Subject:

```
POST /api/v1/subjects
{
  "name": "Mathematics",
  "code": "MATH101",
  "description": "General Mathematics",
  "department_id": 1,
  "teacher_id": 5
}
```

Student Management

List Students

Endpoint: GET /api/v1/students

Query Parameters:

- `class_id` - Filter by class
- `arm_id` - Filter by arm
- `status` - Filter by status
- `search` - Search by name/admission number
- `per_page` - Items per page (default: 15)

Create Student

Endpoint: `POST /api/v1/students`

Request Body:

```
{
  "first_name": "John",
  "last_name": "Doe",
  "middle_name": "Smith",
  "date_of_birth": "2010-05-15",
  "gender": "male",
  "class_id": 1,
  "arm_id": 1,
  "phone": "+234801234567",
  "address": "123 Main St",
  "blood_group": "O+",
  "guardians": [
    {
      "first_name": "Jane",
      "last_name": "Doe",
      "email": "jane@example.com",
      "phone": "+234809876543",
      "relationship": "Mother",
      "is_primary": true
    }
  ]
}
```

Note: All of these are auto-generated:

- `admission_number` - Auto-generated (ADM00001)
- `email` - Auto-generated ([firstname.lastname{id}@schoolomain.com](#))
- `username` - Auto-generated
- `password` - Auto-generated (Password@123)
- `school_id` - Auto-derived from X-Subdomain

Bulk Create Students

Endpoint: `POST /api/v1/bulk/students/register`

Request Body:

```
{
  "students": [
    {
      "first_name": "Student1",
      "last_name": "Test",
      "date_of_birth": "2010-01-15",
      "gender": "male",
      "class_id": 1
    }
    // ... up to 10,000 students
  ]
}
```

Performance: Can handle 10,000+ students without timeout using optimized bulk inserts.

Get Student Details

Endpoint: GET /api/v1/students/{id}

Update Student

Endpoint: PUT /api/v1/students/{id}

Delete Student

Endpoint: DELETE /api/v1/students/{id}

Teacher Management

List Teachers

Endpoint: GET /api/v1/teachers

Create Teacher

Endpoint: POST /api/v1/teachers

Request Body:

```
{
  "first_name": "John",
  "last_name": "Smith",
  "email": "john.smith@westwoodschool.com",
  "phone": "+234801234567",
  "date_of_birth": "1985-05-15",
  "gender": "male",
  "department_id": 1,
  "qualification": "B.Sc. Mathematics",
  "specialization": "Mathematics",
  "date_of_joining": "2025-01-15"
}
```

Note: Auto-generated:

- `employee_id` - Auto-generated (TCH001)
- `email` - Can be auto-generated if not provided
- `password` - Auto-generated (Password@123)

Bulk Create Teachers

Endpoint: `POST /api/v1/bulk/teachers/register`

Get Teacher Details

Endpoint: `GET /api/v1/teachers/{id}`

Update Teacher

Endpoint: `PUT /api/v1/teachers/{id}`

Delete Teacher

Endpoint: `DELETE /api/v1/teachers/{id}`

Staff & Guardian Management

Staff

List Staff: `GET /api/v1/staff`

Create Staff: `POST /api/v1/staff`

```
{
  "first_name": "Mary",
  "last_name": "Johnson",
  "email": "mary@westwoodschooL.com",
  "phone": "+234801234567",
  "role": "accountant",
  "department": "Administration",
  "date_of_joining": "2025-01-15"
}
```

Bulk Create Staff: `POST /api/v1/bulk/staff/create`

Guardians

List Guardians: `GET /api/v1/guardians`

Create Guardian: `POST /api/v1/guardians`


```
{
  "first_name": "Jane",
  "last_name": "Doe",
  "email": "jane@example.com",
  "phone": "+234809876543",
  "address": "123 Main St",
  "occupation": "Engineer",
  "relationship": "Mother"
}
```

Bulk Create Guardians: POST /api/v1/bulk/guardians/create

Settings

Get Settings

Endpoint: GET /api/v1/settings

Update Settings

Endpoint: PUT /api/v1/settings

Request Body:

```
{
  "academic_year_format": "YYYY/YYYY",
  "attendance_grace_period": 15,
  "late_payment_fee_percentage": 5,
  "minimum_age_requirement": 4,
  "maximum_age_requirement": 18,
  "default_class_capacity": 30,
  "grading_system": "percentage"
}
```

Get School Settings

Endpoint: GET /api/v1/settings/school

Update School Settings

Endpoint: PUT /api/v1/settings/school

Subscriptions & Modules

Get Subscription Status

Endpoint: GET /api/v1/subscriptions/status

Response (200):

```
{
  "subscription": {
    "plan": "Premium",
    "start_date": "2025-01-01",
    "end_date": "2025-12-31",
    "status": "active",
    "days_remaining": 300
  },
  "modules": [
    {
      "name": "Academic Management",
      "status": "active"
    },
    {
      "name": "Student Management",
      "status": "active"
    },
    {
      "name": "CBT System",
      "status": "active"
    }
  ],
  "limits": {
    "max_students": 1000,
    "current_students": 450,
    "max_teachers": 100,
    "current_teachers": 35
  }
}
```

Get Available Plans

Endpoint: GET /api/v1/subscriptions/plans

Get Available Modules

Endpoint: GET /api/v1/subscriptions/modules

Upgrade Subscription

Endpoint: PUT /api/v1/subscriptions/{subscription_id}/upgrade

Summary

Admin Can:

- ✓ View comprehensive dashboard
- ✓ Manage school information
- ✓ Create and manage users (all roles)
- ✓ Upload/delete profile pictures
- ✓ Setup academic structure (years, terms, departments)
- ✓ Create and manage classes & arms

- ✓ Manage students (single & bulk)
 - ✓ Manage teachers (single & bulk)
 - ✓ Manage staff & guardians
 - ✓ Configure school settings
 - ✓ Manage subscriptions
 - ✓ Access all modules based on subscription
-

Last Updated: November 26, 2025

API Version: 1.0.0

Principal Dashboard API Documentation

Complete API reference for Principal/Vice Principal Dashboard functionality.

Authentication

All endpoints require:

```
Authorization: Bearer {token}
X-Subdomain: {school_subdomain}
Content-Type: application/json
```

Dashboard Overview

Get Principal Dashboard

Endpoint: GET /api/v1/dashboard/principal

Response (200):

```
{
  "user": {
    "id": 2,
    "name": "Dr. Principal",
    "email": "principal@westwoodschool.com",
    "role": "principal"
  },
  "school_overview": {
    "total_students": 1200,
    "total_teachers": 85,
    "total_staff": 45,
    "total_classes": 36,
    "student_teacher_ratio": 14.1
  },
  "academic_performance": {
    "overall_average": 76.5,
    "top_performing_class": "SS3A",
    "pass_rate": 92.3
  },
  "attendance": {
    "student_attendance_today": 95.5,
    "teacher_attendance_today": 98.2,
    "absent_students": 54,
    "absent_teachers": 2
  },
  "pending_approvals": {
    "leave_requests": 5,
    "disciplinary_cases": 3,
    "expense_approvals": 12
  },
  "recent_activities": [...],
  "role": "principal"
}
```

School Management

Get School Statistics

Endpoint: GET /api/v1/schools/statistics

Get Department Performance

Endpoint: GET /api/v1/departments/performance

Get Staff Overview

Endpoint: GET /api/v1/staff/overview

Approval Management

Get Pending Approvals

Endpoint: GET /api/v1/approvals/pending

Approve Leave Request

Endpoint: POST /api/v1/approvals/leave/{id}/approve

Approve Expense

Endpoint: POST /api/v1/approvals/expense/{id}/approve

Academic Oversight

Get Class Performance Report

Endpoint: GET /api/v1/academic/performance/classes

Get Teacher Performance

Endpoint: GET /api/v1/academic/teachers/performance

Get Exam Analysis

Endpoint: GET /api/v1/academic/exams/analysis

Disciplinary Management

Get Disciplinary Cases

Endpoint: GET /api/v1/discipline/cases

Review Disciplinary Case

Endpoint: POST /api/v1/discipline/cases/{id}/review

Communication

Send School-Wide Announcement

Endpoint: POST /api/v1/announcements/school-wide

Send Message to Department

Endpoint: POST /api/v1/messages/department/{id}

Summary

Principal Can:

- ✓ View comprehensive school overview
 - ✓ Monitor academic performance
 - ✓ Track attendance (students & staff)
 - ✓ Approve leave requests and expenses
 - ✓ Review disciplinary cases
 - ✓ Monitor department performance
 - ✓ Send school-wide communications
 - ✓ Access all reports and analytics
-

Last Updated: November 26, 2025

API Version: 1.0.0

HOD (Head of Department) Dashboard API Documentation

Complete API reference for HOD Dashboard functionality.

Authentication

All endpoints require:

```
Authorization: Bearer {token}
X-Subdomain: {school_subdomain}
Content-Type: application/json
```

Dashboard Overview

Get HOD Dashboard

Endpoint: GET /api/v1/dashboard/hod

Response (200):

```
{
  "user": {
    "id": 15,
    "name": "Dr. HOD Science",
    "email": "hod.science@westwoodschoool.com",
    "role": "hod"
  },
  "department": {
    "id": 1,
    "name": "Science Department",
    "total_teachers": 12,
    "total_subjects": 6,
    "total_students": 450
  },
  "stats": {
    "department_average": 78.5,
    "teachers_present_today": 11,
    "subjects_taught": 6,
    "pending_approvals": 3
  },
  "teacher_performance": [...],
  "subject_statistics": [...],
  "recent_activities": [...],
  "role": "hod"
}
```

Department Management

Get Department Teachers

Endpoint: GET /api/v1/departments/{id}/teachers

Get Department Subjects

Endpoint: GET /api/v1/departments/{id}/subjects

Get Department Performance

Endpoint: GET /api/v1/departments/{id}/performance

Teacher Management

Assign Teacher to Subject

Endpoint: POST /api/v1/departments/assign-teacher

Monitor Teacher Attendance

Endpoint: GET /api/v1/departments/{id}/teacher-attendance

Review Teacher Performance

Endpoint: GET /api/v1/teachers/{id}/performance-review

Subject Management

Get Subject Performance

Endpoint: GET /api/v1/subjects/{id}/performance

Review Curriculum Progress

Endpoint: GET /api/v1/subjects/{id}/curriculum-progress

Summary

HOD Can:

- ✔ Manage department teachers
- ✔ Monitor subject performance
- ✔ Track teacher attendance
- ✔ Review curriculum progress

- ✔ Approve departmental requests
 - ✔ Generate department reports
-

Last Updated: November 26, 2025

API Version: 1.0.0

Teacher Dashboard API Documentation

Complete API reference for Teacher Dashboard functionality.

Table of Contents

1. [Dashboard Overview](#)
 2. [My Classes & Students](#)
 3. [Exams & CBT](#)
 4. [Question Bank](#)
 5. [Grading & Results](#)
 6. [Assignments](#)
 7. [Attendance](#)
 8. [Timetable](#)
 9. [Communication](#)
 10. [Profile Management](#)
-

Authentication

All endpoints require:

```
Authorization: Bearer {token}
X-Subdomain: {school_subdomain}
Content-Type: application/json
```

Dashboard Overview

Get Teacher Dashboard

Endpoint: GET `/api/v1/dashboard/teacher`

Response (200):

```
{
  "user": {
    "id": 5,
    "name": "Mr. Johnson",
    "email": "johnson@westwoodschoool.com",
    "role": "teacher",
    "profile_picture": "https://..."
  },
  "teacher": {
    "id": 1,
    "employee_id": "TCH001",
    "department_id": 2,
    "qualification": "B.Sc. Mathematics",
    "specialization": "Mathematics"
  },
  "stats": {
    "my_classes": 3,
    "my_subjects": 2,
    "my_students": 90,
    "pending_assignments": 5,
    "upcoming_exams": 2
  },
  "role": "teacher"
}
```

My Classes & Students

Get My Classes

Endpoint: GET /api/v1/classes?class_teacher_id={my_teacher_id}

Response (200):

```
[
  {
    "id": 1,
    "name": "JSS 1A",
    "capacity": 30,
    "students_count": 28,
    "class_teacher": {
      "id": 1,
      "name": "Mr. Johnson"
    },
    "arms": [...]
  }
]
```

Get My Students

Endpoint: GET /api/v1/students?class_id={my_class_id}

Get Student Details

Endpoint: GET /api/v1/students/{id}

Response (200):

```
{
  "id": 10,
  "admission_number": "ADM001",
  "first_name": "John",
  "last_name": "Doe",
  "class": {
    "id": 1,
    "name": "JSS 1A"
  },
  "performance": {
    "average_score": 78.5,
    "attendance_rate": 95,
    "rank": 5
  },
  "guardians": [...]
}
```

Get My Subjects

Endpoint: GET /api/v1/subjects?teacher_id={my_teacher_id}

Response (200):

```
[
  {
    "id": 1,
    "name": "Mathematics",
    "code": "MATH101",
    "classes": [
      {"id": 1, "name": "JSS 1A"},
      {"id": 2, "name": "JSS 1B"}
    ],
    "students_count": 60
  }
]
```

Exams & CBT

Create Exam

Endpoint: POST /api/v1/assessments/exams

Request Body:

```
{
  "title": "Mathematics Mid-Term Exam",
  "subject_id": 1,
  "class_id": 1,
  "exam_type": "mid_term",
  "start_date": "2025-12-01 09:00:00",
  "end_date": "2025-12-01 11:00:00",
  "duration_minutes": 120,
  "total_marks": 100,
  "passing_marks": 40,
  "instructions": "Answer all questions",
  "is_cbt": true,
  "cbt_settings": {
    "shuffle_questions": true,
    "shuffle_options": true,
    "show_results_immediately": false,
    "allow_review": true
  }
}
```

Response (201):

```
{
  "message": "Exam created successfully",
  "exam": {
    "id": 1,
    "title": "Mathematics Mid-Term Exam",
    "exam_code": "EXAM2025001",
    "subject": {...},
    "class": {...},
    "created_by": 5,
    "status": "scheduled"
  }
}
```

List My Exams

Endpoint: GET /api/v1/assessments/exams?teacher_id={my_id}

Query Parameters:

- subject_id - Filter by subject
- class_id - Filter by class
- status - Filter by status (scheduled, ongoing, completed)
- exam_type - Filter by type

Get Exam Details

Endpoint: GET /api/v1/assessments/exams/{id}

Update Exam

Endpoint: PUT /api/v1/assessments/exams/{id}

Delete Exam

Endpoint: DELETE /api/v1/assessments/exams/{id}

Question Bank

Add Question

Endpoint: POST /api/v1/question-bank

Request Body (Multiple Choice):

```
{
  "subject_id": 1,
  "class_id": 1,
  "term_id": 1,
  "academic_year_id": 1,
  "question_type": "multiple_choice",
  "question": "What is 2 + 2?",
  "options": [
    {"key": "A", "value": "3"},
    {"key": "B", "value": "4"},
    {"key": "C", "value": "5"},
    {"key": "D", "value": "6"}
  ],
  "correct_answer": ["B"],
  "explanation": "2 + 2 equals 4",
  "difficulty": "easy",
  "marks": 2,
  "tags": ["arithmetic", "addition"],
  "topic": "Basic Arithmetic"
}
```

Request Body (True/False):

```
{
  "question_type": "true_false",
  "question": "The earth is flat",
  "correct_answer": ["false"],
  "explanation": "The earth is spherical"
}
```

Request Body (Short Answer):

```
{
  "question_type": "short_answer",
  "question": "What is the capital of Nigeria?",
  "correct_answer": ["Abuja"],
  "marks": 3
}
```

Request Body (Essay):

```
{
  "question_type": "essay",
  "question": "Discuss the causes of World War II",
  "marks": 20,
  "rubric": "Introduction (5 marks), Body (10 marks), Conclusion (5 marks)"
}
```

Bulk Add Questions

Endpoint: POST /api/v1/bulk/questions/create

Request Body:

```
{
  "questions": [
    {
      "subject_id": 1,
      "class_id": 1,
      "term_id": 1,
      "academic_year_id": 1,
      "question_type": "multiple_choice",
      "question": "Question 1",
      "options": [...],
      "correct_answer": ["A"]
    }
    // ... up to 10,000 questions
  ]
}
```

Performance: Optimized for bulk inserts, can handle 10,000+ questions.

Get Questions for Exam

Endpoint: GET /api/v1/question-bank/for-exam

Query Parameters:

```
?subject_id=1&class_id=1&term_id=1&academic_year_id=1&difficulty=medium&count=30
```

Response (200):

```
{
  "total_available": 150,
  "returned": 30,
  "questions": [...]
}
```

List My Questions

Endpoint: GET /api/v1/question-bank?created_by={my_id}

Query Parameters:

- `subject_id` - Filter by subject

- `class_id` - Filter by class
- `difficulty` - Filter by difficulty (easy, medium, hard)
- `question_type` - Filter by type
- `tags` - Filter by tags
- `per_page` - Items per page

Update Question

Endpoint: `PUT /api/v1/question-bank/{id}`

Delete Question

Endpoint: `DELETE /api/v1/question-bank/{id}`

Duplicate Question

Endpoint: `POST /api/v1/question-bank/{id}/duplicate`

Response (201):

```
{
  "message": "Question duplicated successfully",
  "question": {...}
}
```

Grading & Results

Record Student Result

Endpoint: `POST /api/v1/assessments/results`

Request Body:

```
{
  "student_id": 10,
  "exam_id": 1,
  "subject_id": 1,
  "marks_obtained": 85,
  "total_marks": 100,
  "grade": "A",
  "remarks": "Excellent performance"
}
```

Bulk Update Results

Endpoint: `POST /api/v1/bulk/results/update`

Request Body:

```
{
  "results": [
    {
      "student_id": 10,
      "exam_id": 1,
      "subject_id": 1,
      "marks_obtained": 85,
      "total_marks": 100,
      "grade": "A"
    }
    // ... up to 1000 results
  ]
}
```

Response (200):

```
{
  "success": true,
  "summary": {
    "total": 30,
    "created": 30,
    "failed": 0
  }
}
```

View Results

Endpoint: GET /api/v1/assessments/results?exam_id={exam_id}

Get Class Performance

Endpoint: GET /api/v1/grades/class/{class_id}

Response (200):

```
{
  "class": {...},
  "statistics": {
    "total_students": 30,
    "average_score": 75.5,
    "highest_score": 95,
    "lowest_score": 45,
    "pass_rate": 85
  },
  "grades": [
    {
      "student": {...},
      "marks": 85,
      "grade": "A",
      "position": 2
    }
  ]
}
```

Assignments

Create Assignment

Endpoint: POST /api/v1/assessments/assignments

Request Body:

```
{
  "title": "Algebra Homework",
  "description": "Solve problems 1-10",
  "subject_id": 1,
  "class_id": 1,
  "due_date": "2025-12-10 23:59:59",
  "total_marks": 20,
  "instructions": "Show all working",
  "attachments": [
    {"name": "questions.pdf", "url": "https://..."}
  ]
}
```

List My Assignments

Endpoint: GET /api/v1/assessments/assignments?teacher_id={my_id}

Get Assignment Submissions

Endpoint: GET /api/v1/assessments/assignments/{id}/submissions

Response (200):

```
{
  "assignment": {...},
  "submissions": [
    {
      "id": 1,
      "student": {...},
      "submitted_at": "2025-12-09 15:30:00",
      "status": "submitted",
      "attachments": [...],
      "marks": null,
      "feedback": null
    }
  ],
  "statistics": {
    "total_students": 30,
    "submitted": 25,
    "pending": 5,
    "late": 2,
    "graded": 10
  }
}
```

Grade Assignment

Endpoint: POST

/api/v1/assessments/assignments/{assignment_id}/submissions/{submission_id}/grade

Request Body:

```
{
  "marks": 18,
  "feedback": "Well done! Minor errors in question 5.",
  "status": "graded"
}
```

Attendance

Mark Class Attendance

Endpoint: POST /api/v1/attendance/mark

Request Body:

```
{
  "class_id": 1,
  "date": "2025-11-26",
  "attendance": [
    {
      "student_id": 10,
      "status": "present",
      "check_in_time": "08:00:00"
    },
    {
      "student_id": 11,
      "status": "absent",
      "notes": "Sick"
    },
    {
      "student_id": 12,
      "status": "late",
      "check_in_time": "08:30:00"
    }
  ]
}
```

Bulk Mark Attendance

Endpoint: POST /api/v1/bulk/attendance/mark

```
{
  "attendance_records": [
    {
      "attendanceable_id": 10,
      "attendanceable_type": "student",
      "date": "2025-11-26",
      "status": "present",
      "check_in_time": "08:00:00"
    }
    // ... up to 1000 records
  ]
}
```

Get Class Attendance

Endpoint: GET /api/v1/attendance/class/{class_id}?date=2025-11-26

Get Student Attendance

Endpoint: GET /api/v1/attendance/student/{student_id}

Attendance Reports

Endpoint: GET /api/v1/attendance/reports?class_id=1&from=2025-11-01&to=2025-11-30

Timetable

Get My Timetable

Endpoint: GET /api/v1/timetable/teacher/{my_teacher_id}

Response (200):

```
{
  "teacher": {...},
  "timetable": [
    {
      "day": "Monday",
      "periods": [
        {
          "period": 1,
          "time": "08:00-09:00",
          "subject": "Mathematics",
          "class": "JSS 1A",
          "room": "Room 101"
        },
        {
          "period": 2,
          "time": "09:00-10:00",
          "subject": "Mathematics",
          "class": "JSS 1B",
          "room": "Room 101"
        }
      ]
    }
  ]
}
```

Communication

Send Announcement

Endpoint: POST /api/v1/announcements

Request Body:

```
{
  "title": "Important Notice",
  "content": "Class will start 30 minutes early tomorrow",
  "target_audience": "students",
  "class_id": 1,
  "priority": "high",
  "expires_at": "2025-12-10 23:59:59"
}
```

Send Message to Students

Endpoint: POST /api/v1/messages/send

Request Body:

```
{
  "recipients": [10, 11, 12],
  "recipient_type": "students",
  "subject": "Assignment Reminder",
  "message": "Please submit your homework by Friday",
  "priority": "normal"
}
```

Profile Management

Update My Profile

Endpoint: PUT /api/v1/teachers/me

Request Body:

```
{
  "phone": "+234801234567",
  "address": "New address",
  "qualification": "M.Sc. Mathematics",
  "specialization": "Advanced Mathematics"
}
```

Upload Profile Picture

Endpoint: POST /api/v1/users/me/profile-picture

Request Body:

```
{
  "profile_picture": "https://s3.amazonaws.com/.../photo.jpg"
}
```

Steps:

1. Get presigned URL: GET /api/v1/uploads/presigned-urls?
type=profile_picture&entity_type=teacher&entity_id={my_id}
2. Upload file to S3 using presigned URL
3. Update profile with S3 URL

Delete Profile Picture

Endpoint: DELETE /api/v1/users/me/profile-picture

Summary

Teacher Can:

- ✓ View personalized dashboard with stats
 - ✓ Create and manage exams (traditional & CBT)
 - ✓ Add questions to question bank (single & bulk up to 10,000)
 - ✓ Grade and record student results (single & bulk)
 - ✓ Create and grade assignments
 - ✓ Mark attendance (single & bulk)
 - ✓ View student performance and analytics
 - ✓ Send messages and announcements
 - ✓ View teaching timetable
 - ✓ Manage profile and upload profile picture
-

Last Updated: November 26, 2025

API Version: 1.0.0

Student Dashboard API Documentation

Complete API reference for Student Dashboard functionality.

Table of Contents

1. [Dashboard Overview](#)
 2. [My Profile](#)
 3. [My Classes & Subjects](#)
 4. [Assignments](#)
 5. [Exams & CBT](#)
 6. [My Grades & Results](#)
 7. [My Attendance](#)
 8. [My Timetable](#)
 9. [My Guardians](#)
 10. [Communication](#)
 11. [Profile Management](#)
-

Authentication

All endpoints require:

```
Authorization: Bearer {token}
X-Subdomain: {school_subdomain}
Content-Type: application/json
```

Dashboard Overview

Get Student Dashboard

Endpoint: `GET /api/v1/dashboard/student`

Response (200):

```
{
  "user": {
    "id": 10,
    "name": "John Doe",
    "email": "john.doe1@westwoodschool.com",
    "role": "student",
    "profile_picture": "https://..."
  },
  "student": {
    "id": 1,
    "admission_number": "ADM001",
    "first_name": "John",
    "last_name": "Doe",
    "class": {
      "id": 1,
      "name": "JSS 1A"
    },
    "arm": {
      "id": 1,
      "name": "A"
    }
  },
  "stats": {
    "my_class": {
      "id": 1,
      "name": "JSS 1A",
      "class_teacher": "Mr. Johnson"
    },
    "my_subjects": 8,
    "pending_assignments": 3,
    "upcoming_exams": 2,
    "recent_grades": [
      {
        "subject": "Mathematics",
        "score": 85,
        "grade": "A"
      }
    ],
    "attendance_rate": 95.5
  },
  "role": "student"
}
```

My Profile

Get My Profile

Endpoint: GET /api/v1/students/me

Response (200):

```
{
  "id": 1,
  "admission_number": "ADM001",
  "first_name": "John",
  "last_name": "Doe",
  "middle_name": "Smith",
  "email": "john.doe1@westwoodschool.com",
  "phone": "+234801234567",
  "date_of_birth": "2010-05-15",
  "gender": "male",
  "address": "123 Main St",
  "blood_group": "O+",
  "class": {
    "id": 1,
    "name": "JSS 1A",
    "class_teacher": {...}
  },
  "arm": {
    "id": 1,
    "name": "A"
  },
  "guardians": [
    {
      "id": 1,
      "name": "Jane Doe",
      "email": "jane@example.com",
      "phone": "+234809876543",
      "relationship": "Mother",
      "is_primary": true
    }
  ],
  "performance_summary": {
    "average_score": 78.5,
    "rank": 5,
    "total_students": 30,
    "attendance_rate": 95.5
  }
}
```

Update My Profile

Endpoint: PUT /api/v1/students/me

Request Body:

```
{
  "phone": "+234801234567",
  "address": "New address",
  "emergency_contact": "+234809876543"
}
```

Note: Students can only update limited fields (phone, address, emergency contact).

My Classes & Subjects

Get My Class

Endpoint: GET /api/v1/classes/my-class

Response (200):

```
{
  "id": 1,
  "name": "JSS 1A",
  "description": "Junior Secondary School 1A",
  "capacity": 30,
  "students_count": 28,
  "class_teacher": {
    "id": 1,
    "name": "Mr. Johnson",
    "email": "johnson@westwoodschoool.com",
    "phone": "+234801234567"
  },
  "subjects": [
    {
      "id": 1,
      "name": "Mathematics",
      "code": "MATH101",
      "teacher": {...}
    }
  ]
}
```

Get My Subjects

Endpoint: GET /api/v1/subjects/my-subjects

Response (200):

```
[
  {
    "id": 1,
    "name": "Mathematics",
    "code": "MATH101",
    "description": "General Mathematics",
    "teacher": {
      "name": "Mr. Johnson",
      "email": "johnson@westwoodschoool.com",
      "phone": "+234801234567"
    },
    "my_performance": {
      "average_score": 82.5,
      "total_assignments": 5,
      "completed_assignments": 5,
      "total_exams": 2,
      "exam_average": 85
    }
  }
]
```

Assignments

Get My Assignments

Endpoint: GET /api/v1/assessments/assignments/my-assignments

Query Parameters:

- `status` - Filter by status (pending, submitted, graded, late)
- `subject_id` - Filter by subject
- `per_page` - Items per page (default: 15)

Response (200):

```

{
  "assignments": [
    {
      "id": 1,
      "title": "Algebra Homework",
      "description": "Solve problems 1-10",
      "subject": {
        "id": 1,
        "name": "Mathematics"
      },
      "teacher": {
        "name": "Mr. Johnson"
      },
      "due_date": "2025-12-10 23:59:59",
      "total_marks": 20,
      "status": "pending",
      "submission": null,
      "attachments": [
        {
          "name": "questions.pdf",
          "url": "https://..."
        }
      ]
    },
    {
      "id": 2,
      "title": "Essay on Climate Change",
      "subject": {
        "id": 2,
        "name": "English"
      },
      "due_date": "2025-12-05 23:59:59",
      "total_marks": 30,
      "status": "graded",
      "submission": {
        "submitted_at": "2025-12-04 15:30:00",
        "marks": 25,
        "grade": "B+",
        "feedback": "Well written! Great analysis."
      }
    }
  ],
  "summary": {
    "total": 10,
    "pending": 3,
    "submitted": 5,
    "graded": 2,
    "late": 0
  }
}

```

Get Assignment Details

Endpoint: GET /api/v1/assessments/assignments/{id}

Submit Assignment

Endpoint: POST /api/v1/assessments/assignments/{id}/submit

Request Body:

```
{
  "content": "My answer here...",
  "attachments": [
    {"name": "homework.pdf", "url": "https://..."}
  ]
}
```

Response (201):

```
{
  "message": "Assignment submitted successfully",
  "submission": {
    "id": 1,
    "assignment_id": 1,
    "student_id": 10,
    "submitted_at": "2025-12-09 15:30:00",
    "status": "submitted",
    "content": "My answer...",
    "attachments": [...]
  }
}
```

Exams & CBT

Get My Exams

Endpoint: GET /api/v1/assessments/exams/my-exams

Query Parameters:

- `status` - Filter by status (upcoming, ongoing, completed)
- `subject_id` - Filter by subject
- `per_page` - Items per page

Response (200):

```

{
  "exams": [
    {
      "id": 1,
      "title": "Mathematics Mid-Term",
      "exam_code": "EXAM2025001",
      "subject": {
        "id": 1,
        "name": "Mathematics"
      },
      "exam_type": "mid_term",
      "start_date": "2025-12-01 09:00:00",
      "end_date": "2025-12-01 11:00:00",
      "duration_minutes": 120,
      "total_marks": 100,
      "is_cbt": true,
      "status": "upcoming",
      "my_result": null
    },
    {
      "id": 2,
      "title": "English Language Test",
      "subject": {
        "id": 2,
        "name": "English"
      },
      "start_date": "2025-11-20 09:00:00",
      "status": "completed",
      "my_result": {
        "marks": 85,
        "grade": "A",
        "position": 2,
        "total_students": 30
      }
    }
  ]
}

```

Start CBT Exam

Endpoint: POST /api/v1/assessments/cbt/{exam_id}/start

Response (201):


```
{
  "message": "Exam session started",
  "session": {
    "id": "sess_12345",
    "exam": {
      "id": 1,
      "title": "Mathematics Mid-Term",
      "duration_minutes": 120,
      "total_marks": 100
    },
    "started_at": "2025-12-01 09:00:00",
    "ends_at": "2025-12-01 11:00:00",
    "time_remaining": 7200
  }
}
```

Get CBT Questions

Endpoint: GET /api/v1/assessments/cbt/{exam_id}/questions

Response (200):

```
{
  "session_id": "sess_12345",
  "exam": {
    "id": 1,
    "title": "Mathematics Mid-Term",
    "total_marks": 100
  },
  "questions": [
    {
      "id": 1,
      "question": "What is 2 + 2?",
      "question_type": "multiple_choice",
      "options": [
        {"key": "A", "value": "3"},
        {"key": "B", "value": "4"},
        {"key": "C", "value": "5"},
        {"key": "D", "value": "6"}
      ],
      "marks": 2
    },
    {
      "id": 2,
      "question": "The earth is flat",
      "question_type": "true_false",
      "options": [
        {"key": "A", "value": "True"},
        {"key": "B", "value": "False"}
      ],
      "marks": 1
    }
  ],
  "total_questions": 50,
  "time_remaining": 7140
}
```

Submit CBT Answers

Endpoint: POST /api/v1/assessments/cbt/submit

Request Body:

```
{
  "session_id": "sess_12345",
  "exam_id": 1,
  "answers": [
    {
      "question_id": 1,
      "answer": ["B"]
    },
    {
      "question_id": 2,
      "answer": ["false"]
    }
  ]
}
```

Response (200):

```
{
  "message": "Exam submitted successfully",
  "result": {
    "exam": {
      "id": 1,
      "title": "Mathematics Mid-Term"
    },
    "total_questions": 50,
    "answered": 48,
    "unanswered": 2,
    "score": 85,
    "total_marks": 100,
    "percentage": 85,
    "grade": "A",
    "status": "graded",
    "position": 2,
    "total_students": 30
  }
}
```

View Exam Result

Endpoint: GET /api/v1/assessments/exams/{id}/my-result

My Grades & Results

Get All My Grades

Endpoint: GET /api/v1/grades/student/me

Query Parameters:

- `subject_id` - Filter by subject
- `term_id` - Filter by term
- `academic_year_id` - Filter by academic year
- `exam_type` - Filter by exam type

Response (200):

```
{
  "student": {
    "id": 1,
    "name": "John Doe",
    "admission_number": "ADM001",
    "class": "JSS 1A"
  },
  "grades": [
    {
      "subject": "Mathematics",
      "exam_type": "Mid-Term",
      "marks": 85,
      "total_marks": 100,
      "percentage": 85,
      "grade": "A",
      "position": 2,
      "class_average": 72,
      "remarks": "Excellent performance"
    },
    {
      "subject": "English",
      "exam_type": "Mid-Term",
      "marks": 78,
      "total_marks": 100,
      "percentage": 78,
      "grade": "B+",
      "position": 5,
      "class_average": 70
    }
  ],
  "summary": {
    "overall_average": 78.5,
    "overall_grade": "B+",
    "overall_position": 5,
    "total_students": 30,
    "total_subjects": 8
  }
}
```

Get Subject Performance

Endpoint: `GET /api/v1/grades/student/me/subject/{subject_id}`

Response (200):

```
{
  "subject": {
    "id": 1,
    "name": "Mathematics"
  },
  "performance": [
    {
      "exam": "CA1",
      "score": 18,
      "total": 20,
      "percentage": 90,
      "grade": "A"
    },
    {
      "exam": "CA2",
      "score": 17,
      "total": 20,
      "percentage": 85,
      "grade": "A"
    },
    {
      "exam": "Mid-Term",
      "score": 85,
      "total": 100,
      "percentage": 85,
      "grade": "A"
    }
  ],
  "average": 86.67,
  "trend": "improving",
  "class_rank": 2,
  "highest_score": 92,
  "lowest_score": 85
}
```

My Attendance

Get My Attendance

Endpoint: GET /api/v1/attendance/student/me

Query Parameters:

- from - Start date (YYYY-MM-DD)
- to - End date (YYYY-MM-DD)
- per_page - Items per page

Response (200):

```
{
  "student": {
    "id": 1,
    "name": "John Doe",
    "admission_number": "ADM001",
    "class": "JSS 1A"
  },
  "attendance": [
    {
      "date": "2025-11-26",
      "day": "Wednesday",
      "status": "present",
      "check_in_time": "08:00:00"
    },
    {
      "date": "2025-11-25",
      "day": "Tuesday",
      "status": "present",
      "check_in_time": "07:55:00"
    },
    {
      "date": "2025-11-24",
      "day": "Monday",
      "status": "late",
      "check_in_time": "08:30:00"
    },
    {
      "date": "2025-11-23",
      "day": "Sunday",
      "status": "absent",
      "notes": "Sick"
    }
  ],
  "summary": {
    "total_days": 20,
    "present": 18,
    "absent": 1,
    "late": 1,
    "attendance_rate": 95,
    "punctuality_rate": 90
  }
}
```

My Timetable

Get My Timetable

Endpoint: GET /api/v1/timetable/student/me

Response (200):

```
{
  "student": {
    "id": 1,
    "name": "John Doe",
    "class": "JSS 1A"
  },
  "timetable": [
    {
      "day": "Monday",
      "periods": [
        {
          "period": 1,
          "time": "08:00-09:00",
          "subject": "Mathematics",
          "teacher": "Mr. Johnson",
          "room": "Room 101"
        },
        {
          "period": 2,
          "time": "09:00-10:00",
          "subject": "English",
          "teacher": "Mrs. Smith",
          "room": "Room 102"
        },
        {
          "period": 3,
          "time": "10:00-11:00",
          "subject": "Break",
          "teacher": null,
          "room": null
        }
      ]
    }
  ]
}
```

My Guardians

Get My Guardians

Endpoint: GET /api/v1/students/me/guardians

Response (200):

```
{
  "student": {
    "id": 1,
    "name": "John Doe"
  },
  "guardians": [
    {
      "id": 1,
      "first_name": "Jane",
      "last_name": "Doe",
      "email": "jane@example.com",
      "phone": "+234809876543",
      "address": "123 Main St",
      "occupation": "Engineer",
      "relationship": "Mother",
      "is_primary": true
    },
    {
      "id": 2,
      "first_name": "James",
      "last_name": "Doe",
      "email": "james@example.com",
      "phone": "+234807654321",
      "relationship": "Father",
      "is_primary": false
    }
  ]
}
```

Communication

Get My Announcements

Endpoint: GET /api/v1/announcements/my-announcements

Response (200):

```
{
  "announcements": [
    {
      "id": 1,
      "title": "Important Notice",
      "content": "Class will start 30 minutes early tomorrow",
      "priority": "high",
      "created_by": "Mr. Johnson",
      "created_at": "2025-11-26 10:00:00",
      "expires_at": "2025-11-27 23:59:59"
    }
  ]
}
```

Get My Messages

Endpoint: GET /api/v1/messages/my-messages

Profile Management

Upload Profile Picture

Endpoint: POST /api/v1/users/me/profile-picture

Request Body:

```
{
  "profile_picture": "https://s3.amazonaws.com/.../photo.jpg"
}
```

Steps:

1. Get presigned URL: GET /api/v1/uploads/presigned-urls?type=profile_picture&entity_type=student&entity_id={my_id}
2. Upload file to S3 using presigned URL
3. Update profile with S3 URL

Delete Profile Picture

Endpoint: DELETE /api/v1/users/me/profile-picture

Summary

Student Can:

- ✓ View personalized dashboard with stats
 - ✓ View and update profile (limited fields)
 - ✓ View class, subjects, and classmates
 - ✓ View and submit assignments
 - ✓ Take CBT exams online
 - ✓ View exam results and grades
 - ✓ Track attendance history
 - ✓ View class timetable
 - ✓ View guardian information
 - ✓ Receive announcements and messages
 - ✓ Upload and manage profile picture
-

Last Updated: November 26, 2025

API Version: 1.0.0

Parent/Guardian Dashboard API Documentation

Complete API reference for Parent/Guardian Dashboard functionality.

Table of Contents

1. [Dashboard Overview](#)
 2. [My Children](#)
 3. [Child Performance](#)
 4. [Attendance Monitoring](#)
 5. [Assignments & Homework](#)
 6. [Exam Results](#)
 7. [Fee Management](#)
 8. [Communication](#)
 9. [Profile Management](#)
-

Authentication

All endpoints require:

```
Authorization: Bearer {token}
X-Subdomain: {school_subdomain}
Content-Type: application/json
```

Dashboard Overview

Get Parent Dashboard

Endpoint: GET /api/v1/dashboard/parent

Response (200):

```
{
  "user": {
    "id": 15,
    "name": "Mrs. Jane Doe",
    "email": "jane@example.com",
    "role": "guardian",
    "profile_picture": "https://..."
  },
  "guardian": {
    "id": 1,
    "first_name": "Jane",
    "last_name": "Doe",
    "phone": "+234809876543",
    "occupation": "Engineer",
    "relationship": "Mother"
  },
  "children": [
    {
      "id": 1,
      "name": "John Doe",
      "admission_number": "ADM001",
      "class": "JSS 1A",
      "profile_picture": "https://...",
      "stats": {
        "average_score": 78.5,
        "attendance_rate": 95.5,
        "rank": 5,
        "pending_assignments": 3
      }
    },
    {
      "id": 2,
      "name": "Mary Doe",
      "admission_number": "ADM002",
      "class": "JSS 3B",
      "profile_picture": "https://...",
      "stats": {
        "average_score": 85.2,
        "attendance_rate": 98.0,
        "rank": 2,
        "pending_assignments": 1
      }
    }
  ],
  "stats": {
    "total_children": 2,
    "upcoming_events": 3,
    "pending_fees": 50000,
    "unread_messages": 2
  },
  "role": "parent"
}
```

My Children

Get All My Children

Endpoint: GET /api/v1/guardians/me/children

Response (200):

```
{
  "guardian": {
    "id": 1,
    "name": "Jane Doe"
  },
  "children": [
    {
      "id": 1,
      "admission_number": "ADM001",
      "first_name": "John",
      "last_name": "Doe",
      "email": "john.doe1@westwoodschoo1.com",
      "class": {
        "id": 1,
        "name": "JSS 1A",
        "class_teacher": "Mr. Johnson"
      },
      "arm": {
        "id": 1,
        "name": "A"
      },
      "performance_summary": {
        "average_score": 78.5,
        "rank": 5,
        "total_students": 30,
        "attendance_rate": 95.5
      }
    }
  ]
}
```

Get Child Details

Endpoint: GET /api/v1/students/{child_id}

Response (200):

```
{
  "id": 1,
  "admission_number": "ADM001",
  "first_name": "John",
  "last_name": "Doe",
  "email": "john.doe1@westwoodschoo1.com",
  "phone": "+234801234567",
  "class": {...},
  "arm": {...},
  "guardians": [...],
  "performance_summary": {
    "average_score": 78.5,
    "rank": 5,
    "attendance_rate": 95.5
  }
}
```

Child Performance

Get Child's Grades

Endpoint: GET /api/v1/grades/student/{child_id}

Query Parameters:

- `subject_id` - Filter by subject
- `term_id` - Filter by term
- `academic_year_id` - Filter by academic year

Response (200):

```

{
  "student": {
    "id": 1,
    "name": "John Doe",
    "admission_number": "ADM001",
    "class": "JSS 1A"
  },
  "grades": [
    {
      "subject": "Mathematics",
      "exam_type": "Mid-Term",
      "marks": 85,
      "total_marks": 100,
      "percentage": 85,
      "grade": "A",
      "position": 2,
      "class_average": 72,
      "teacher_remarks": "Excellent performance"
    },
    {
      "subject": "English",
      "exam_type": "Mid-Term",
      "marks": 78,
      "total_marks": 100,
      "percentage": 78,
      "grade": "B+",
      "position": 5,
      "class_average": 70
    }
  ],
  "summary": {
    "overall_average": 78.5,
    "overall_grade": "B+",
    "overall_position": 5,
    "total_students": 30,
    "total_subjects": 8
  },
  "term": {
    "id": 1,
    "name": "First Term",
    "academic_year": "2025/2026"
  }
}

```

Get Subject Performance

Endpoint: GET /api/v1/grades/student/{child_id}/subject/{subject_id}

Response (200):

```
{
  "student": {
    "id": 1,
    "name": "John Doe"
  },
  "subject": {
    "id": 1,
    "name": "Mathematics",
    "teacher": "Mr. Johnson"
  },
  "performance": [
    {
      "exam": "CA1",
      "date": "2025-10-15",
      "score": 18,
      "total": 20,
      "percentage": 90,
      "grade": "A"
    },
    {
      "exam": "CA2",
      "date": "2025-11-10",
      "score": 17,
      "total": 20,
      "percentage": 85,
      "grade": "A"
    },
    {
      "exam": "Mid-Term",
      "date": "2025-12-01",
      "score": 85,
      "total": 100,
      "percentage": 85,
      "grade": "A"
    }
  ],
  "statistics": {
    "average": 86.67,
    "trend": "improving",
    "class_rank": 2,
    "highest_score": 92,
    "lowest_score": 85
  }
}
```

Attendance Monitoring

Get Child's Attendance

Endpoint: GET /api/v1/attendance/student/{child_id}

Query Parameters:

- `from` - Start date (YYYY-MM-DD)

- to - End date (YYYY-MM-DD)
- per_page - Items per page

Response (200):

```
{
  "student": {
    "id": 1,
    "name": "John Doe",
    "admission_number": "ADM001",
    "class": "JSS 1A"
  },
  "attendance": [
    {
      "date": "2025-11-26",
      "day": "Wednesday",
      "status": "present",
      "check_in_time": "08:00:00"
    },
    {
      "date": "2025-11-25",
      "day": "Tuesday",
      "status": "present",
      "check_in_time": "07:55:00"
    },
    {
      "date": "2025-11-24",
      "day": "Monday",
      "status": "late",
      "check_in_time": "08:30:00",
      "notes": "Traffic delay"
    },
    {
      "date": "2025-11-23",
      "day": "Sunday",
      "status": "absent",
      "notes": "Sick",
      "excused": true
    }
  ],
  "summary": {
    "total_days": 20,
    "present": 18,
    "absent": 1,
    "late": 1,
    "excused_absences": 1,
    "attendance_rate": 95,
    "punctuality_rate": 90
  },
  "period": {
    "from": "2025-11-01",
    "to": "2025-11-26"
  }
}
```

Assignments & Homework

Get Child's Assignments

Endpoint: GET /api/v1/assessments/assignments/student/{child_id}

Query Parameters:

- `status` - Filter by status (pending, submitted, graded, late)
- `subject_id` - Filter by subject
- `per_page` - Items per page

Response (200):


```
{
  "student": {
    "id": 1,
    "name": "John Doe",
    "class": "JSS 1A"
  },
  "assignments": [
    {
      "id": 1,
      "title": "Algebra Homework",
      "description": "Solve problems 1-10",
      "subject": "Mathematics",
      "teacher": "Mr. Johnson",
      "due_date": "2025-12-10 23:59:59",
      "total_marks": 20,
      "status": "pending",
      "submission": null,
      "days_remaining": 14
    },
    {
      "id": 2,
      "title": "Essay on Climate Change",
      "subject": "English",
      "teacher": "Mrs. Smith",
      "due_date": "2025-12-05 23:59:59",
      "total_marks": 30,
      "status": "graded",
      "submission": {
        "submitted_at": "2025-12-04 15:30:00",
        "marks": 25,
        "grade": "B+",
        "feedback": "Well written! Great analysis.",
        "on_time": true
      }
    }
  ],
  "summary": {
    "total": 10,
    "pending": 3,
    "submitted": 5,
    "graded": 2,
    "late": 0,
    "average_score": 85
  }
}
```

Get Assignment Details

Endpoint: GET /api/v1/assessments/assignments/{assignment_id}/student/{child_id}

Exam Results

Get Child's Exams

Endpoint: GET /api/v1/assessments/exams/student/{child_id}

Query Parameters:

- `status` - Filter by status (upcoming, completed)
- `subject_id` - Filter by subject
- `term_id` - Filter by term

Response (200):

```
{
  "student": {
    "id": 1,
    "name": "John Doe",
    "class": "JSS 1A"
  },
  "exams": [
    {
      "id": 1,
      "title": "Mathematics Mid-Term",
      "subject": "Mathematics",
      "exam_type": "mid_term",
      "date": "2025-12-01",
      "start_time": "09:00:00",
      "end_time": "11:00:00",
      "total_marks": 100,
      "is_cbt": true,
      "status": "upcoming"
    },
    {
      "id": 2,
      "title": "English Language Test",
      "subject": "English",
      "date": "2025-11-20",
      "total_marks": 100,
      "status": "completed",
      "result": {
        "marks": 85,
        "percentage": 85,
        "grade": "A",
        "position": 2,
        "total_students": 30,
        "class_average": 72,
        "teacher_remarks": "Excellent work"
      }
    }
  ]
}
```

Get Exam Result

Endpoint: GET /api/v1/assessments/exams/{exam_id}/student/{child_id}

Fee Management

Get Child's Fee Status

Endpoint: GET /api/v1/financial/fees/student/{child_id}

Response (200):

```
{
  "student": {
    "id": 1,
    "name": "John Doe",
    "admission_number": "ADM001",
    "class": "JSS 1A"
  },
  "fees": [
    {
      "id": 1,
      "name": "Tuition Fee",
      "amount": 150000,
      "paid": 150000,
      "balance": 0,
      "status": "paid",
      "due_date": "2025-10-01",
      "paid_date": "2025-09-25"
    },
    {
      "id": 2,
      "name": "Second Term Fee",
      "amount": 150000,
      "paid": 100000,
      "balance": 50000,
      "status": "partial",
      "due_date": "2026-01-15"
    },
    {
      "id": 3,
      "name": "Extra-Curricular Activities",
      "amount": 25000,
      "paid": 0,
      "balance": 25000,
      "status": "unpaid",
      "due_date": "2025-12-15"
    }
  ],
  "summary": {
    "total_fees": 325000,
    "total_paid": 250000,
    "total_balance": 75000,
    "next_due_date": "2025-12-15"
  }
}
```

Get Payment History

Endpoint: GET /api/v1/financial/payments/student/{child_id}

Response (200):

```
{
  "student": {
    "id": 1,
    "name": "John Doe"
  },
  "payments": [
    {
      "id": 1,
      "amount": 150000,
      "payment_method": "bank_transfer",
      "reference": "PAY2025001",
      "paid_date": "2025-09-25",
      "fee": "Tuition Fee",
      "receipt_url": "https://..."
    },
    {
      "id": 2,
      "amount": 100000,
      "payment_method": "card",
      "reference": "PAY2025045",
      "paid_date": "2025-11-10",
      "fee": "Second Term Fee (Partial)",
      "receipt_url": "https://..."
    }
  ],
  "total_paid": 250000
}
```

Make Payment

Endpoint: POST /api/v1/financial/fees/{fee_id}/pay

Request Body:

```
{
  "student_id": 1,
  "amount": 50000,
  "payment_method": "card",
  "card_details": {
    "number": "4111111111111111",
    "exp_month": "12",
    "exp_year": "2026",
    "cvv": "123"
  }
}
```

Communication

Get My Messages

Endpoint: GET /api/v1/messages/my-messages

Response (200):

```
{
  "messages": [
    {
      "id": 1,
      "from": "Mr. Johnson",
      "role": "teacher",
      "subject": "Assignment Reminder",
      "message": "Please remind John to submit the homework",
      "sent_at": "2025-11-25 10:00:00",
      "read": false,
      "regarding_student": "John Doe"
    }
  ],
  "unread_count": 1
}
```

Send Message to Teacher

Endpoint: POST /api/v1/messages/send

Request Body:

```
{
  "recipient_id": 5,
  "recipient_type": "teacher",
  "subject": "Request for Meeting",
  "message": "I would like to discuss John's progress",
  "regarding_student_id": 1
}
```

Get Announcements

Endpoint: GET /api/v1/announcements/for-parents

Response (200):

```
{
  "announcements": [
    {
      "id": 1,
      "title": "Parent-Teacher Meeting",
      "content": "All parents are invited to the meeting on Dec 15",
      "priority": "high",
      "created_at": "2025-11-20 09:00:00",
      "expires_at": "2025-12-15 23:59:59"
    }
  ]
}
```

Profile Management

Get My Profile

Endpoint: GET /api/v1/guardians/me

Response (200):

```
{
  "id": 1,
  "first_name": "Jane",
  "last_name": "Doe",
  "email": "jane@example.com",
  "phone": "+234809876543",
  "address": "123 Main St",
  "occupation": "Engineer",
  "relationship": "Mother",
  "is_primary": true,
  "children": [...]
}
```

Update My Profile

Endpoint: PUT /api/v1/guardians/me

Request Body:

```
{
  "phone": "+234809876543",
  "address": "New address",
  "occupation": "Senior Engineer"
}
```

Upload Profile Picture

Endpoint: POST /api/v1/users/me/profile-picture

Request Body:

```
{  
  "profile_picture": "https://s3.amazonaws.com/.../photo.jpg"  
}
```

Delete Profile Picture

Endpoint: DELETE /api/v1/users/me/profile-picture

Summary

Parent/Guardian Can:

- ✓ View personalized dashboard with all children's stats
 - ✓ Monitor all children's academic performance
 - ✓ Track attendance and punctuality
 - ✓ View assignments and homework status
 - ✓ Access exam results and report cards
 - ✓ View and pay school fees
 - ✓ Check payment history
 - ✓ Communicate with teachers and school
 - ✓ Receive important announcements
 - ✓ Update profile and upload profile picture
-

Last Updated: November 26, 2025

API Version: 1.0.0

Finance/Accountant Dashboard API Documentation

Complete API reference for Finance/Accountant Dashboard functionality.

Table of Contents

1. [Dashboard Overview](#)
 2. [Fee Management](#)
 3. [Payment Processing](#)
 4. [Expense Management](#)
 5. [Payroll Management](#)
 6. [Financial Reports](#)
 7. [Budget Management](#)
 8. [Invoice Generation](#)
-

Authentication

All endpoints require:

```
Authorization: Bearer {token}
X-Subdomain: {school_subdomain}
Content-Type: application/json
```

Dashboard Overview

Get Finance Dashboard

Endpoint: `GET /api/v1/dashboard/finance`

Response (200):


```
{
  "user": {
    "id": 20,
    "name": "Mrs. Finance Officer",
    "email": "finance@westwoodschoool.com",
    "role": "accountant"
  },
  "stats": {
    "total_revenue": {
      "today": 500000,
      "this_month": 15000000,
      "this_term": 45000000,
      "this_year": 120000000
    },
    "pending_fees": {
      "amount": 25000000,
      "students": 150
    },
    "expenses": {
      "today": 200000,
      "this_month": 5000000,
      "this_term": 15000000
    },
    "payroll": {
      "pending": 3000000,
      "paid_this_month": 12000000
    },
    "outstanding_invoices": 45,
    "overdue_payments": 28,
    "profit_margin": 65.5
  },
  "recent_transactions": [...],
  "pending_approvals": [...],
  "role": "accountant"
}
```

Fee Management

Get Fee Structure

Endpoint: GET /api/v1/financial/fees/structure

Create Fee Structure

Endpoint: POST /api/v1/financial/fees/structure

Request Body:

```
{
  "name": "First Term Fees",
  "academic_year_id": 1,
  "term_id": 1,
  "class_id": 1,
  "components": [
    {
      "name": "Tuition Fee",
      "amount": 150000,
      "is_mandatory": true
    },
    {
      "name": "Development Levy",
      "amount": 25000,
      "is_mandatory": true
    },
    {
      "name": "Extra-Curricular",
      "amount": 15000,
      "is_mandatory": false
    }
  ],
  "total_amount": 190000,
  "due_date": "2025-10-15"
}
```

Get Student Fee Status

Endpoint: GET /api/v1/financial/fees/student/{student_id}

Get Outstanding Fees by Class

Endpoint: GET /api/v1/financial/fees/outstanding?class_id=1

Send Fee Reminder

Endpoint: POST /api/v1/financial/fees/send-reminder

Payment Processing

Record Payment

Endpoint: POST /api/v1/financial/payments

Request Body:

```
{
  "student_id": 10,
  "amount": 150000,
  "payment_method": "bank_transfer",
  "reference": "TRX12345",
  "payment_date": "2025-11-26",
  "payer_name": "Jane Doe",
  "description": "First Term Fees - Partial Payment",
  "fee_components": [
    {
      "fee_id": 1,
      "amount": 150000
    }
  ]
}
```

Get Payment History

Endpoint: GET /api/v1/financial/payments

Query Parameters:

- `student_id` - Filter by student
- `class_id` - Filter by class
- `payment_method` - Filter by method
- `from` - Start date
- `to` - End date
- `per_page` - Items per page

Generate Receipt

Endpoint: GET /api/v1/financial/payments/{id}/receipt

Get Daily Collections

Endpoint: GET /api/v1/financial/payments/daily-collections?date=2025-11-26

Expense Management

Record Expense

Endpoint: POST /api/v1/financial/expenses

Request Body:

```
{
  "category": "utilities",
  "description": "Electricity Bill – November",
  "amount": 250000,
  "expense_date": "2025-11-26",
  "vendor": "PHCN",
  "payment_method": "bank_transfer",
  "reference": "EXP12345",
  "attachments": [
    {
      "name": "bill.pdf",
      "url": "https://..."
    }
  ]
}
```

Get Expense Categories

Endpoint: GET /api/v1/financial/expenses/categories

Response (200):

```
{
  "categories": [
    {
      "id": 1,
      "name": "Utilities",
      "budget": 3000000,
      "spent": 2500000,
      "remaining": 500000
    },
    {
      "id": 2,
      "name": "Salaries",
      "budget": 15000000,
      "spent": 12000000,
      "remaining": 3000000
    },
    {
      "id": 3,
      "name": "Maintenance",
      "budget": 2000000,
      "spent": 1500000,
      "remaining": 500000
    }
  ]
}
```

Get Expense Report

Endpoint: GET /api/v1/financial/expenses/report?from=2025-11-01&to=2025-11-30

Payroll Management

Create Payroll

Endpoint: POST /api/v1/financial/payroll

Request Body:

```
{
  "month": "November",
  "year": 2025,
  "employees": [
    {
      "employee_id": 5,
      "basic_salary": 300000,
      "allowances": {
        "housing": 100000,
        "transport": 50000
      },
      "deductions": {
        "tax": 45000,
        "pension": 30000
      },
      "net_salary": 375000
    }
  ]
}
```

Get Payroll History

Endpoint: GET /api/v1/financial/payroll?month=11&year=2025

Generate Payslip

Endpoint: GET /api/v1/financial/payroll/{id}/payslip/{employee_id}

Process Bulk Salary Payment

Endpoint: POST /api/v1/financial/payroll/{id}/process

Financial Reports

Income Statement

Endpoint: GET /api/v1/financial/reports/income-statement

Query Parameters:

- from - Start date
- to - End date
- format - pdf, excel, json

Response (200):

```
{
  "period": {
    "from": "2025-11-01",
    "to": "2025-11-30"
  },
  "income": {
    "tuition_fees": 15000000,
    "other_fees": 2000000,
    "total": 17000000
  },
  "expenses": {
    "salaries": 12000000,
    "utilities": 500000,
    "maintenance": 300000,
    "other": 200000,
    "total": 13000000
  },
  "net_profit": 4000000,
  "profit_margin": 23.5
}
```

Cash Flow Statement

Endpoint: GET /api/v1/financial/reports/cash-flow

Balance Sheet

Endpoint: GET /api/v1/financial/reports/balance-sheet

Fee Collection Report

Endpoint: GET /api/v1/financial/reports/fee-collections

Expense Analysis

Endpoint: GET /api/v1/financial/reports/expense-analysis

Budget Management

Create Budget

Endpoint: POST /api/v1/financial/budgets

Request Body:

```
{
  "name": "2025/2026 Academic Year Budget",
  "academic_year_id": 1,
  "categories": [
    {
      "name": "Salaries",
      "allocated_amount": 180000000,
      "priority": "high"
    },
    {
      "name": "Infrastructure",
      "allocated_amount": 50000000,
      "priority": "medium"
    },
    {
      "name": "Equipment",
      "allocated_amount": 20000000,
      "priority": "low"
    }
  ],
  "total_budget": 250000000
}
```

Get Budget vs Actual

Endpoint: GET /api/v1/financial/budgets/{id}/vs-actual

Update Budget

Endpoint: PUT /api/v1/financial/budgets/{id}

Invoice Generation

Create Invoice

Endpoint: POST /api/v1/financial/invoices

Request Body:

```
{
  "student_id": 10,
  "invoice_number": "INV2025001",
  "issue_date": "2025-11-26",
  "due_date": "2025-12-26",
  "items": [
    {
      "description": "Tuition Fee – First Term",
      "amount": 150000
    },
    {
      "description": "Development Levy",
      "amount": 25000
    }
  ],
  "total": 175000,
  "tax": 0,
  "grand_total": 175000
}
```

Send Invoice

Endpoint: POST /api/v1/financial/invoices/{id}/send

Get Outstanding Invoices

Endpoint: GET /api/v1/financial/invoices/outstanding

Mark Invoice as Paid

Endpoint: POST /api/v1/financial/invoices/{id}/mark-paid

Summary

Finance/Accountant Can:

- ✓ View comprehensive financial dashboard
 - ✓ Manage fee structures
 - ✓ Process payments and generate receipts
 - ✓ Track and categorize expenses
 - ✓ Manage payroll and generate payslips
 - ✓ Generate financial reports (P&L, Cash Flow, Balance Sheet)
 - ✓ Create and monitor budgets
 - ✓ Generate and send invoices
 - ✓ Track outstanding fees
 - ✓ Send payment reminders
 - ✓ Analyze financial performance
-

Last Updated: November 26, 2025

API Version: 1.0.0

Librarian Dashboard API Documentation

Complete API reference for Librarian Dashboard functionality.

Authentication

All endpoints require:

```
Authorization: Bearer {token}
X-Subdomain: {school_subdomain}
Content-Type: application/json
```

Dashboard Overview

Get Librarian Dashboard

Endpoint: GET /api/v1/dashboard/librarian

Response (200):

```
{
  "user": {
    "id": 25,
    "name": "Mrs. Librarian",
    "email": "librarian@westwoodschool.com",
    "role": "librarian"
  },
  "stats": {
    "total_books": 5000,
    "available_books": 4200,
    "borrowed_books": 800,
    "overdue_books": 45,
    "total_members": 650,
    "active_members": 420,
    "books_added_this_month": 50,
    "popular_categories": [...]
  },
  "recent_borrows": [...],
  "overdue_list": [...],
  "pending_requests": [...],
  "role": "librarian"
}
```

Book Management

Add New Book

Endpoint: POST /api/v1/library/books

Request Body:

```
{
  "title": "Things Fall Apart",
  "author": "Chinua Achebe",
  "isbn": "978-0-385-47454-2",
  "category_id": 1,
  "publisher": "Heinemann",
  "publication_year": 1958,
  "copies": 10,
  "shelf_location": "A-12",
  "description": "Classic Nigerian literature"
}
```

Get All Books

Endpoint: GET /api/v1/library/books

Query Parameters:

- category_id - Filter by category
- status - available, borrowed, reserved
- search - Search title, author, ISBN

Borrow Book

Endpoint: POST /api/v1/library/books/{id}/borrow

Request Body:

```
{
  "student_id": 10,
  "due_date": "2025-12-10"
}
```

Return Book

Endpoint: POST /api/v1/library/books/{id}/return

Request Body:

```
{
  "borrow_id": 123,
  "condition": "good",
  "fine": 0
}
```

Get Overdue Books

Endpoint: GET /api/v1/library/books/overdue

Get Popular Books

Endpoint: GET /api/v1/library/books/popular?limit=10

Member Management

Get Borrowing History

Endpoint: GET /api/v1/library/members/{student_id}/history

Get Active Borrows

Endpoint: GET /api/v1/library/members/{student_id}/active-borrows

Block/Unblock Member

Endpoint: POST /api/v1/library/members/{student_id}/block

Reports

Monthly Report

Endpoint: GET /api/v1/library/reports/monthly?month=11&year=2025

Most Borrowed Books

Endpoint: GET /api/v1/library/reports/most-borrowed

Fine Collection Report

Endpoint: GET /api/v1/library/reports/fines

Summary

Librarian Can:

- ✓ Manage book inventory
 - ✓ Process book borrowing and returns
 - ✓ Track overdue books and fines
 - ✓ Manage library members
 - ✓ Generate library reports
 - ✓ Monitor popular books
-

Last Updated: November 26, 2025

API Version: 1.0.0

Nurse/Medical Dashboard API Documentation

Complete API reference for Nurse/Medical Dashboard functionality.

Authentication

All endpoints require:

```
Authorization: Bearer {token}
X-Subdomain: {school_subdomain}
Content-Type: application/json
```

Dashboard Overview

Get Nurse Dashboard

Endpoint: GET /api/v1/dashboard/nurse

Response (200):

```
{
  "user": {
    "id": 35,
    "name": "Nurse Mary",
    "email": "nurse@westwoodschool.com",
    "role": "nurse"
  },
  "stats": {
    "clinic_visits_today": 12,
    "students_with_chronic_conditions": 45,
    "medications_due_today": 8,
    "pending_vaccinations": 15,
    "first_aid_cases_this_week": 25,
    "medical_supplies_low": 3
  },
  "today_appointments": [...],
  "medication_schedule": [...],
  "recent_cases": [...],
  "role": "nurse"
}
```

Medical Records

Add Medical Record

Endpoint: POST /api/v1/health/records

Request Body:

```
{
  "student_id": 10,
  "visit_date": "2025-11-26",
  "complaint": "Headache and fever",
  "diagnosis": "Common cold",
  "treatment": "Paracetamol prescribed",
  "temperature": 37.8,
  "blood_pressure": "120/80",
  "notes": "Rest advised"
}
```

Get Student Medical History

Endpoint: GET /api/v1/health/records/student/{student_id}

Get Students with Chronic Conditions

Endpoint: GET /api/v1/health/students/chronic-conditions

Medication Management

Schedule Medication

Endpoint: POST /api/v1/health/medications

Get Today's Medication Schedule

Endpoint: GET /api/v1/health/medications/today

Record Medication Given

Endpoint: POST /api/v1/health/medications/{id}/administer

Vaccination Management

Record Vaccination

Endpoint: POST /api/v1/health/vaccinations

Get Vaccination Schedule

Endpoint: GET /api/v1/health/vaccinations/schedule

Medical Supplies

Get Inventory

Endpoint: GET /api/v1/health/supplies

Record Supply Usage

Endpoint: POST /api/v1/health/supplies/use

Request Supplies

Endpoint: POST /api/v1/health/supplies/request

Summary

Nurse Can:

- ✓ Manage clinic visits and medical records
 - ✓ Track chronic conditions
 - ✓ Schedule and administer medications
 - ✓ Manage vaccination records
 - ✓ Monitor medical supplies
 - ✓ Generate health reports
-

Last Updated: November 26, 2025

API Version: 1.0.0

Driver Dashboard API Documentation

Complete API reference for Driver Dashboard functionality.

Authentication

All endpoints require:

```
Authorization: Bearer {token}
X-Subdomain: {school_subdomain}
Content-Type: application/json
```

Dashboard Overview

Get Driver Dashboard

Endpoint: GET /api/v1/dashboard/driver

Response (200):

```
{
  "user": {
    "id": 30,
    "name": "Mr. Driver",
    "email": "driver@westwoodschoo.com",
    "role": "driver"
  },
  "vehicle": {
    "id": 1,
    "name": "School Bus 1",
    "plate_number": "ABC-123-XY",
    "capacity": 40,
    "status": "active"
  },
  "route": {
    "id": 1,
    "name": "Route A - Ikeja",
    "students_count": 35,
    "pickup_points": 8
  },
  "stats": {
    "today_trips": 2,
    "students_today": 35,
    "total_trips_this_month": 44,
    "pending_maintenance": false,
    "fuel_status": "75%"
  },
  "today_schedule": [...],
  "students_list": [...],
  "role": "driver"
}
```

My Route Management

Get My Route

Endpoint: GET /api/v1/transport/drivers/me/route

Get Students on My Route

Endpoint: GET /api/v1/transport/drivers/me/students

Mark Student Pickup

Endpoint: POST /api/v1/transport/attendance/pickup

Request Body:

```
{
  "student_id": 10,
  "pickup_point_id": 3,
  "pickup_time": "07:30:00",
  "status": "picked_up"
}
```

Mark Student Dropoff

Endpoint: POST /api/v1/transport/attendance/dropoff

Trip Management

Start Trip

Endpoint: POST /api/v1/transport/trips/start

End Trip

Endpoint: POST /api/v1/transport/trips/end

Get Trip History

Endpoint: GET /api/v1/transport/drivers/me/trips

Vehicle Management

Get My Vehicle Status

Endpoint: GET /api/v1/transport/drivers/me/vehicle

Report Vehicle Issue

Endpoint: POST /api/v1/transport/vehicles/{id}/report-issue

Get Maintenance Schedule

Endpoint: GET /api/v1/transport/drivers/me/maintenance

Summary

Driver Can:

- ✔ View assigned route and students
- ✔ Track daily schedule
- ✔ Mark student pickup/dropoff
- ✔ Manage trip logs

- ✓ Report vehicle issues
 - ✓ View maintenance schedule
-

Last Updated: November 26, 2025

API Version: 1.0.0

Security Dashboard API Documentation

Complete API reference for Security Dashboard functionality.

Authentication

All endpoints require:

```
Authorization: Bearer {token}
X-Subdomain: {school_subdomain}
Content-Type: application/json
```

Dashboard Overview

Get Security Dashboard

Endpoint: GET /api/v1/dashboard/security

Response (200):

```
{
  "user": {
    "id": 40,
    "name": "Mr. Security",
    "email": "security@westwoodschool.com",
    "role": "security"
  },
  "stats": {
    "visitors_today": 15,
    "vehicles_in_campus": 25,
    "gate_passes_issued": 8,
    "incidents_this_week": 2,
    "patrol_checkpoints": 12,
    "cctv_cameras_active": 28,
    "cctv_cameras_inactive": 2
  },
  "current_visitors": [...],
  "recent_incidents": [...],
  "patrol_schedule": [...],
  "role": "security"
}
```

Visitor Management

Register Visitor

Endpoint: POST /api/v1/security/visitors

Request Body:

```
{
  "name": "John Visitor",
  "phone": "+234801234567",
  "purpose": "Meeting with Principal",
  "person_to_see": "Dr. Principal",
  "id_type": "driver_license",
  "id_number": "DL12345",
  "vehicle_number": "ABC-123-XY",
  "entry_time": "2025-11-26 10:00:00"
}
```

Get Active Visitors

Endpoint: GET /api/v1/security/visitors/active

Check Out Visitor

Endpoint: POST /api/v1/security/visitors/{id}/checkout

Gate Pass Management

Issue Gate Pass

Endpoint: POST /api/v1/security/gate-passes

Request Body:

```
{
  "student_id": 10,
  "reason": "Medical appointment",
  "exit_time": "2025-11-26 12:00:00",
  "expected_return": "2025-11-26 14:00:00",
  "guardian_phone": "+234809876543"
}
```

Verify Gate Pass

Endpoint: GET /api/v1/security/gate-passes/{code}/verify

Incident Management

Report Incident

Endpoint: POST /api/v1/security/incidents

Request Body:

```
{
  "type": "theft",
  "severity": "medium",
  "location": "Library",
  "description": "Missing laptop reported",
  "reported_time": "2025-11-26 15:00:00",
  "witnesses": ["Student A", "Teacher B"],
  "action_taken": "CCTV footage reviewed"
}
```

Get Incident History

Endpoint: GET /api/v1/security/incidents

Vehicle Management

Log Vehicle Entry

Endpoint: POST /api/v1/security/vehicles/entry

Log Vehicle Exit

Endpoint: POST /api/v1/security/vehicles/exit

Get Vehicles on Campus

Endpoint: GET /api/v1/security/vehicles/on-campus

Patrol Management

Record Patrol Check

Endpoint: POST /api/v1/security/patrols/check

Get Patrol Schedule

Endpoint: GET /api/v1/security/patrols/schedule

Lost and Found

Register Lost Item

Endpoint: POST /api/v1/security/lost-and-found

Get Lost Items

Endpoint: GET /api/v1/security/lost-and-found

Mark Item as Found

Endpoint: POST /api/v1/security/lost-and-found/{id}/found

Summary

Security Can:

- ✓ Register and manage visitors
 - ✓ Issue and verify gate passes
 - ✓ Report and track incidents
 - ✓ Log vehicle entry/exit
 - ✓ Conduct security patrols
 - ✓ Manage lost and found items
 - ✓ Monitor CCTV systems
-

Last Updated: November 26, 2025

API Version: 1.0.0